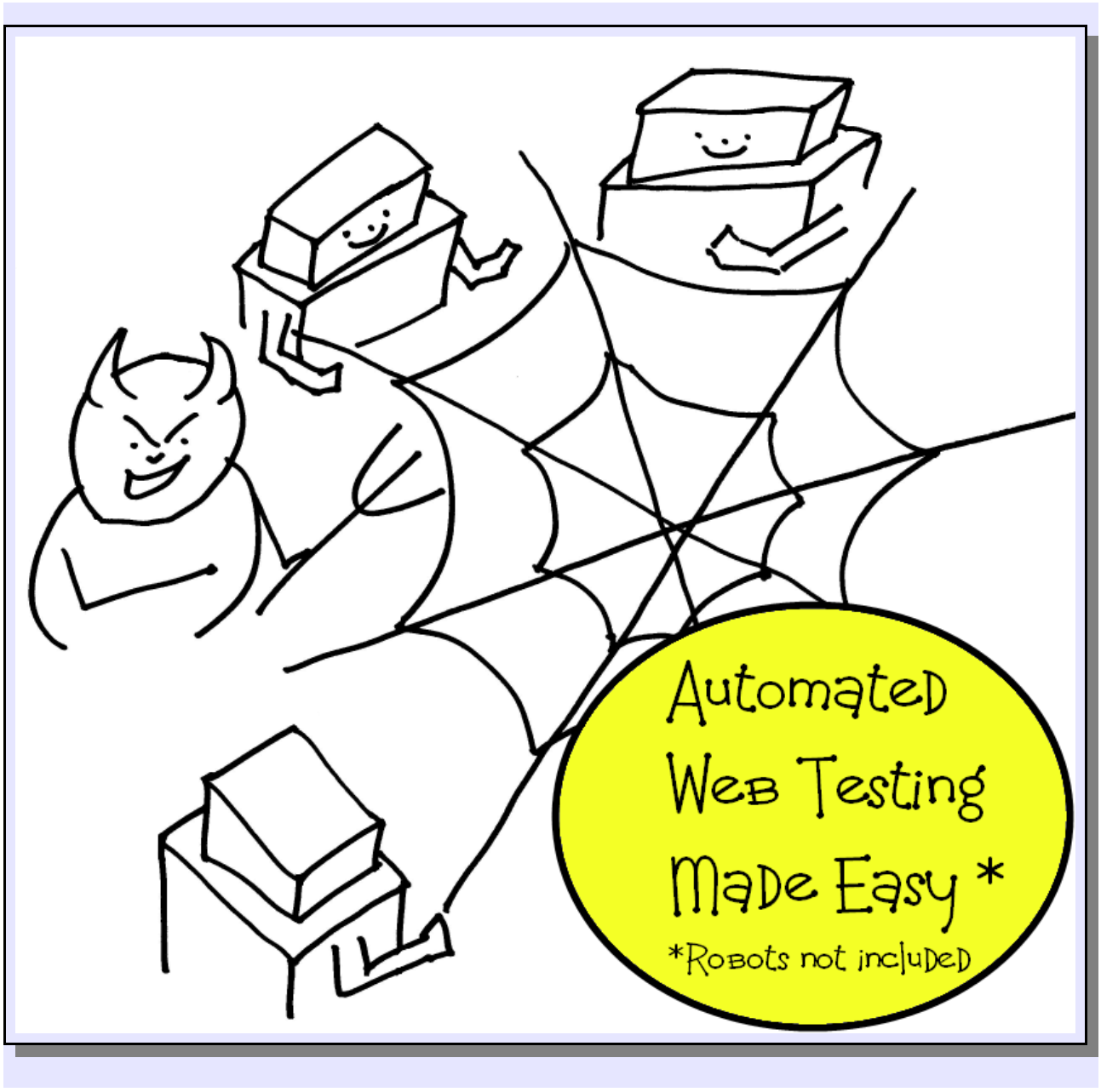
Selenium Simplified

A tutorial guide to using the Selenium API in Java with JUnit

Alan Richardson



Published by Compendium Developments

Copyright © 2010 by Alan Richardson

The right of Alan Richardson to be identified as the author of this work has been asserted by him in accordance with the Copyright, Design and Patents Act 1988.

The views expressed in this book are those of the author.

First published in Great Britain in 2010 by:

Compendium Developments

http://www.compendiumdev.co.uk/selenium

contact details:

alan@compendiumdev.co.uk

Author's Software Testing Blog: **www.eviltester.com**

Compendium Developments: **www.compendiumdev.co.uk**

Every effort has been made to ensure that the information contained in this book is accurate at the time of going to press, and the publishers and author cannot accept any responsibility for any errors or omissions, however caused. No responsibility for loss or damage occasioned by any person acting, or refraining from action, as a result of the material in this publication can be accepted by the editor, the publisher or the author.

Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Design and Patents Act 1988; this publication may only be reproduced, stored or transmitted, in any form or by any means, with the prior permission of the publishers, or in the case of reprographic reproduction in accordance with the terms and licenses issued by the Copyright Licensing Agency, 90 Tottenham Court Road, London, W1T 4LP. Enquiries concerning reproduction outside these terms should be sent to the publishers.

Version @ 02/03/2011 2nd Printing

e-book ISBN : 978-0-9567332-0-7

paperback book ISBN : 978-0-9567332-1-4

Table of Contents

Introduction....9
Getting the Source-code....9
Windows, Linux or Mac....10
Acknowledgments....10
Chapter 1:Getting Started With The Tools....11
What Tools?....11
Selenium IDE....11
Java....11
Eclipse....12
Selenium-RC....12
Chapter 2:Selenium IDE Basics....13
Install Firefox....13
Install Selenium-IDE....13
Capture Play Back – Recording a Script with the IDE....14
Command, Target and Value....17
Save The Test....17
Further reading....17
Chapter 3:Install Java....19
Chapter 4:Install and Run Selenium-RC....20
Step One - Download & Install Selenium-RC....20
Overview of the Contents of the Selenium Archive....21
Step Two - Run Selenium-RC....22
Step Three – Stop the Selenium Server....23
From the command line....23
From a browser URL....24
Running an IDE Generated Test in Different Browsers....24
End Notes....26
Chapter 5:The Eclipse IDE....27
First - Install Eclipse....27
Second - Run Eclipse....28
Third - create a new Java project....31
Chapter 6:Create a JUnit test Using the JUnit export from Selenium-IDE....33
Introduction....33
Export an IDE script as a JUnit test....33
Load an existing script into the IDE....33
Change the format of the script to Java....34
Create a New Class in Eclipse....35
Copy & Paste the code from Selenium IDE into Eclipse....38
Resolve Import Errors....40
Add JUnit to the build path....42
Remove the unused import....44
Run the JUnit test....45
Allow it through the Firewall....45
Seeing the test running....46
Check the results....47
It went too quickly!....48

Run the test in debug mode....49
Create and Import some more IDE Converted tests....52
Exercises....55
Chapter 7:The Annotated Generated Test....56
Use Attach Source to see the Selenium Driver Source Code....56
Why should we do this?....59
MyFirstSeleniumTests.java Annotated....59
A little more about selenium-java-client-driver.jar....63
A little about JUnit....65
A look at SeleneseTestCase.java....65
Use the SeleneseTestCase Functionality....67
Using the IDE to add asserts and verifies....67
First record the setup steps....67
Add the Verify Statements....68
Add the Asserts....70
What is the difference between an Assert and a Verify?....71
How to find line numbers in Eclipse?....73
Creating Screenshots on Failure with SeleneseTestCase....74
Chapter 8:Let's get coding....77
A brief pause, because you have already learned some Java....77
What we will now learn....77
Create a New Test From Scratch....77
Our First Test with Annotations....78
Back to Our Selenium Test....80
Making Something Happen....82
Your Final Test Code....83
The DefaultSelenium constructor explained....84
Retrospectives....85
Well Done....85
Why did you do that?....85
Run Selenium-RC in a DOS window?....85
Start it easily....85
See the logging messages....85
Find The Window....86
Stop and Restart Selenium Easily....86
Final Notes....86
Back to Eclipse....87
Removing the Source Code Association....87
Chapter 9:Chapter - What if it all goes wrong?....88
Check that Selenium Server is running....88
Close any Selenium-RC windows....89
Stop and restart the Selenium-RC server....89
Remove any Java threads from the Debug perspective....89
Close any blocked browser windows....90
Firefox Update Dialog....90
Summary....91
Chapter 10:Essential Firefox Add-Ons....92
What is Firebug?....92
What is XPather?....92
Basic Firebug Usage....93

Basic XPather Usage....95
Optimise XPath with XPather....96
Chapter 11:First Steps with Test Automation Thinking....99
Let's Automate Search....99
Start writing some tests....99
Make Selenium Type In Some Text....100
Start with a copy and Paste....100
Fix the duplicate method name....101
Add a Selenium Command....102
The “type” method....103
Find the locator with FireBug....103
A brief introduction to XPath....105
Add the locator into the code....106
Check that it works....106
Now make it Click on the Search Button....107
Selenium “Click” Method....107
Locate the button with FireBug....107
Create an XPath so that selenium can locate it....108
Add the XPath details to our source code....108
One last thing to do....108
Now check that it works....108
Quick Summary....108
But is it a test if we don't check any results?....109
But we can't control the data....109
Automate the Acceptance Criteria....110
Acceptance Criteria: Selenium Remote Control homepage listed in returned results....110
Look for text in the page....111
Use assertions to make the test fail on error....112
Debugging a Failing Test....113
Our final test with assertions....114
Scan the page source code....115
Look for a specific URL in the page....116
Most Robust Method....119
Acceptance Criteria: Page title has “Selenium-RC” in it....120
Acceptance Criteria: Search box on the page has Selenium-RC in it....120
One test to rule them all....120
Summary....121
Chapter 12:First Steps with Automation Refactored....123
Introducing Refactoring....123
Automatically refactoring the start up code into a new method....124
Automatically refactor the tear down code into a new method....126
JUnit @Before and @After 128
Refactoring Plan and Analysis....128
Remove Parameter from stopSeleniumAndCloseBrowser....129
Move setup code into a new method....130
Remove the setup code from each @Test method....131
Annotate stopSeleniumAndCloseBrowser with @After....131
Remove the tear down code from each @Test method....131
The Refactored Code....133
@BeforeClass & @AfterClass....134

Our refactored code....135
Automatically create JUnit Test Structure....136
Summary....138
Chapter 13:How do I upgrade my Selenium Server when a new version comes out?....139
First Download the New Version....139
Configure Eclipse Project....139
Remember to re-attach the source....141
Chapter 14:Chapter - Basic HTML Theory....142
Page Structure....142
Elements & Tags....142
Attributes....143
Expanded....143
Chapter 15:Basic XPath Theory....144
XPath Expressions....144
Node Types....144
Selections....145
Predicates....145
Advanced....146
Combining Match Queries....146
Wild Card Matches....146
Boolean Operators....147
XPath Functions....147
XPath optimisation....148
Use the ID....148
Use the attributes....148
Start at the first unique element....149
Selenium XPath Usage....149
Chapter 16:Basic CSS Selector Theory....150
Firefinder for Firebug....150
CSS Selector Expressions....151
Selenium & CSS....151
Selections....152
Direct Descendents and Absolute Paths....152
Sub Elements and Relative Paths....153
Attribute and Index matching....153
Attribute Matching....153
Special attribute selectors....153
Indexed Matching....154
Advanced....155
Combining Matches....155
Wild Card Matches....155
Attribute Substring matching....155
Boolean Operators....155
Sibling Combinators....156
Useful Links....156
Chapter 17:Learning The Selenium API....157
Chapter 18:Testing HTML Forms....158
An HTML Form....159
Submit a Form with Default Values....159
Commands Used....162

Submit Form without clicking button....162
Amend a Text, Password, TextArea or File Field....162
Text Fields....162
Type vs TypeKeys....163
Password Fields....163
TextArea Fields....163
File Field....164
Attachfile command....165
Amending Checkboxes and Radio Items....165
Click....165
Check 166
Uncheck....167
Amending Multiple Select Values....168
Amending Dropdown Elements....169
Amending Hidden Field Values....170
Checking the Values of the Input Fields....171
Text, Password, TextArea, File....171
CheckBox & Radio Items....171
Multiple Select & DropDown....172
getValue....173
isSomethingSelected....173
getSelectedId & getSelectedIds....173
getSelectedIndex & getSelectedIndexes....173
getSelectedLabel & getSelectedLabels....174
getSelectedValue & getSelectedValues....174
getSelectOptions....175
Put It All Together....175
Chapter 19:Testing Static HTML....177
A Basic Static HTML Test....177
getTitle – Check the page title....179
getText – Check the username....180
isTextPresent – check the 'heading'....182
isElementPresent – check comments and filename....182
getXPathCount – count the checkboxes entered....183
click – navigate to the form....185
The rest of the test....185
The full test and summary....186
getAttribute....187
assignId....187
getBodyText & getHtmlSource....188
goBack....189
highlight....189
Chapter 20:Using JavaScript with Selenium....191
A Basic JavaScript Enabled Page....191
The Class....192
FireEvent, isAlertPresent, and getAlert....193
getEval....194
RunScript....194
AddScript....195
Chapter 21:Start Selenium Programmatically....196

Add Selenium Server to the project....196
Start it in the code....197
The Start Up routine explored....198
BindException Handling....198
Stop Existing Server....199
Custom Remote Control Configurations....200
Chapter 22:Running Tests Outside Eclipse....202
What is Ant?....202
What is Hudson?....202
What is Subversion?....202
Chapter 23:Using Ant to Run Selenium Tests....203
Install Ant....203
Install Java JDK....203
Update the Environment Variables and Path....204
Check that Ant works....206
Check that Java works....206
Create a Lib Folder....207
Amend Eclipse Project Properties....207
Create an initial build.xml file....207
Running the Ant file....211
Add the Tests which require no server....213
Refactor The Build File....214
Summary....217
Additional Reading....217
Chapter 24:Using Hudson to Run Ant....219
Introduction....219
Install Hudson....219
Using Hudson without Version Control....222
Configure the Hudson Project....224
Scheduling the build automatically....227
Chapter 25:Adding Subversion to the mix....228
Chapter 26:Take Stock of Where we are....229
Plan Of Action....229
Chapter 27:Evolving A Selenium Manager Class....232
Chapter 28:Create Our First Page Objects....239
Create a SearchPageTests class....239
Add the SeleniumManager 240
Convert startSeleniumAndSearchForSeleniumRC....241
typeInASearchTermAndAssertThatHomePageTextIsPresent....245
typeInASearchTermAndAssertThatHomePageURLExists....246
typeInASearchTermAndCheckPageTitleHasSearchTermInIt....247
typeInASearchTermAndCheckSearchInputHasSearchTermInIt....247
After all that 248
SearchPageTests.java....248
SearchPage.java....249
Final Notes....249
Chapter 29:More Page Object Creation....250
The Tests....250
HTML_form_tests.java....250
test_submit_form_with_default_values 251

test_submit_form_without_clicking_submit 251
test_submit_form_with_new_username 252
test_submit_form_with_new_password....252
test_submit_form_with_new_password....252
test_submit_form_with_filename....252
test_submit_form_with_click_check_and_radio....253
test_submit_form_with_multiple_select_values....253
test_submit_form_with_dropdown_values....254
test_submit_form_with_hidden_field....254
test_check_text_entered_values....254
test_check_checkbox_values....255
test_check_radio_values....256
test_randomly_select_value_from_dropdown....257
Static_HTML_tests.java....258
JavaScript_With_Selenium_Tests.java....263
The Page Objects....264
BasicHTMLForm.java....264
Possible refactorings....266
HTMLFormResultsPage.java....268
Discussion of getFilename....269
BasicAjaxPage.java....270
Chapter 30:Page Object Models Summary....271
What is a Page Object Model?....271
Grow an abstraction layer....271
Refactor the Page Objects internally....271
Create Domain Level Abstractions....272
Lessons learned....272
Related Study....273
Chapter 31:Data Driven Tests in JUnit....274
Basic Data Driven Testing....274
JUnit Parameterized Class Runner And Constructor....275
Data supplied by an @Parameters annotated method....275
A Data Driven Test....276
Running a Data Driven Test....276
The full code for the test....277
Multiple Tests....278
Reading Data From Tab Delimited Files....278
General Hints....279
Chapter 32:Screen Capture on Failure....281
Chapter 33:Run the tests on multiple browsers....286
Browser Codes....286
Property Files....287
System Properties....289
Chapter 34:JavaScript And Dynamic HTML Testing Redux....291
Example Dynamic HTML Page....291
First Model the behaviour in the test....292
Code the check in the page object model....292
First understand the page....292
Summary....293
Chapter 35:Cookie Handling....294

Cookies in the example application....294
How to see cookies on a web page?....294
Firefox by Default....294
Firecookie - Firebug Plugin....297
Selenium Cookie Commands....298
Example Code For Cookie Testing....299
Basic Class....299
Check Cookie Creation using deleteAllVisibleCookies and isCookiePresent....299
Get Cookie values using getCookie and getCookieByName....300
getCookie....300
getCookieByName....301
deleteCookie....302
createCookie....303
Chapter 36:The Future of Selenium....305
If you want to experiment with 2.0....306
Chapter 37:Structuring the tests and code....308
Java project folder and package structures....308
Changing what goes into Version Control....311
Configuring Source-code Folders in Eclipse....311
Add libraries....313
Re-organise the Classes and Packages....313
Share the Server among tests and create a base test class....315
Share the Server using a Singleton....315
Create a Base Class....316
Get Ant Working....318
Make sure all tests are valid tests....320
Create end to end tests where we assert after executing functionality....320
Have all tests pass on multiple browsers....320
Locators as constants or methods....321
Locators as constants....321
Locators as methods....322
Get Hudson Working....323
Speed up the test execution with a suite....327
Running Different Suites for Different Browsers....331
Nested Suites....331
Browser Specific Groups....332
Create a conditional Ant File....333
Get Selenium 2.0 working....334
Ways of using what we have built....335
Exercises For the reader:....336
Chapter 38:In Closing....337
Additional Reading....337
Appendix A - Selenium API MindMap....339
Appendix B - Playing along at home....340

Introduction

For some people, seeing some Java code for Selenium-RC on the web, does not help as they don't know how to use it. They don't know:

- where to put the code,
- what a package is,
- how to run the code,
- how to install an IDE

So in this guide I present, in simple steps and supported by screens hots, the fundamental knowledge required to program Selenium tests in Java.

Once you have the basics, we move on to exploring common Selenium API commands and how to automate HTML pages, Forms and JavaScript enabled web applications.

I assume no previous knowledge of Java, or Automated Testing.

This book covers Selenium 1.0.3 the mature and stable version of Selenium. The Selenium 1.0.3 branch is likely to remain stable and static. The newer version of Selenium – Selenium 2.0 is still undergoing heavy development, so we cover this in the "The Future of Selenium" Chapter towards the end of the book.

Be assured that Selenium 1.0.3 is more than capable of automating your web applications. Indeed Selenium 2.0 supports the API used in Selenium 1.0.x, so everything you learn here, you can use in Selenium 2.0 when it matures.

I have chosen Java as the programming language because it is the language that the Selenium Server is written in (so by understanding Java you can understand Selenium more readily). It is also mature and very well supported by IDEs (Integrated Development Environments) which make coding simple for a beginner.

Getting the Source-code

This book takes a tutorial approach. So we present a lot of source-code. To save you having to type it all out – although there are benefits in doing this. You can download the source-code from:

- http://www.compendiumdev.co.uk/selenium/InitialSeleniumTests_JUNIT.zip
- http://www.compendiumdev.co.uk/selenium/finalChapterSource.zip

Follow the instructions in "Appendix B - Playing along at home" on page 340 to install the source-code.

This way, if you are following the explanations and typing in the code, should you get stuck, you can compare your source-code to that in the 'official' archive.

For Chapter 37 "Structuring the tests and code" on page 308, where we make our testing 'production' quality, the source-code can be downloaded from the public subversion repository

- http://svn2.xp-dev.com/svn/seleniumsimplified

This is the same code provided in finalChapterSource.zip listed above.

This is hosted on a free subversion server provided by http://www.xp-dev.com/

Windows, Linux or Mac

This book takes a Windows centric view of the world. So the main text assumes you are working on a Windows system and all screen shots have been taken on a Windows system.

This was purely to limit the install explanations for software. All the tools listed are available for Linux and Mac systems: Eclipse, Firefox, the various Firefox plug-ins, Selenium.

If you visit the various tool websites and follow the install instructions for your particular platform then you should still be able to follow along with the text and instructions.

Acknowledgments

No book has the exclusive influence of the Author. Normally in this section the author would give thanks to the editors, publishers and technical reviewers.

No technical reviewers were involved in this book. This book was forged through the usage of real readers. Readers like yourself that started with little knowledge of how to write program at all. Readers who can now use both Java and Selenium to include automation in their testing process.

In this case I give thanks to my early readers and guinea pigs.

This book grew from a 2 year period of working on an almost daily basis with Selenium on production test automation. So I need to thank the Channel 4 test team that worked with me during that time as we experimented with different ways of automating. Extra special thanks go to Tony Bruce and Michael Kiely, who participated in my Selenium training sessions based on this book. Their feedback led to immediate improvements and scope changes.

And a big thank you to the people who bought the beta e-book version. The early feedback and questions as they learned Selenium working through the book helped me tremendously. I was amazed at the speed of progress they made learning Selenium and found it hard to work through the later, more advanced, sections of the book fast enough to satisfy their learning needs. From this group I need to specifically thank those readers who submitted questions and comments that directly impacted the structure and content of this book: Alex Crookes, William Dutton, Gopi Ganesasundaram, Jay Gehlot, Ard-Jan Glas, Sunny Jain, Anthony Kearns, Felipe Knorr Kuhn, Richard Li, Darren Louie, Michael McCollum, Krishnan Mahadevan, Kaushik Mahata, Nga Nguyen, Dennis Pham, Mahmoud Passikhani, Adrian Rapan, Doug Seward, Rajat Sud, Manupriya Vijay.

And most importantly...

Love and special thanks to my family:

Billie and Keeran

Chapter 1: Getting Started With The Tools

What Tools?

This chapter will introduce you to the tools we are about to install and use:

- Selenium IDE
- Java
- Eclipse
- Selenium-RC

Each tool has links to the official documentation so that you can learn more from the most authoritative sources.

Selenium IDE

The Selenium IDE is an add-on to Firefox.

At a basic level it allows us to:

- Record user actions when browsing in Firefox
- Replay recorded scripts
- Convert recorded scripts into programming languages such as Java, Ruby, and more
- Add verification and synchronisation steps to the script during the recording process

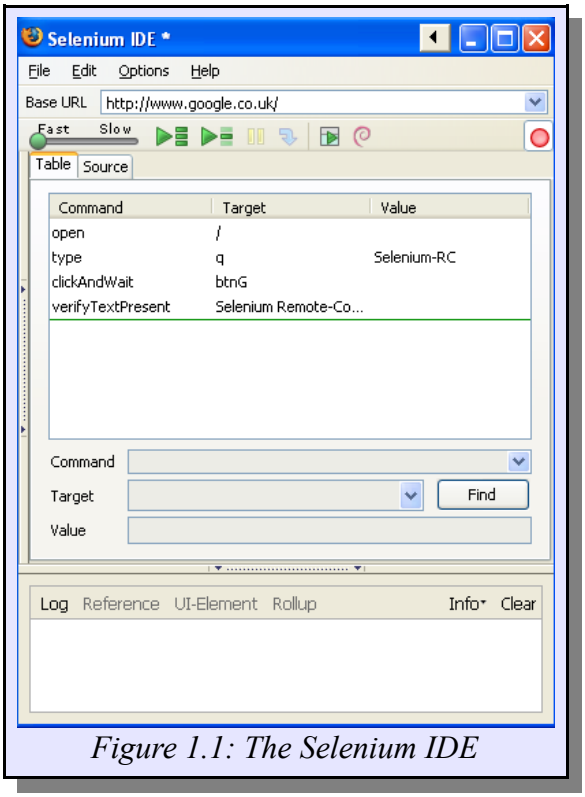


Figure 1.1: The Selenium IDE

The IDE provides excellent support for writing automated test scripts in Selenium and gets better with every release. In this text we will focus on its use as an aid to writing automated test scripts in Java, rather than as a test tool in its own right.

For more information on Selenium IDE visit:

- Official documentation - http://seleniumhq.org/docs/03_selenium_ide.html
- Official Selenium IDE site - http://seleniumhq.org/projects/ide/

Java

Java is a programming language created by Sun. The Java that we will be using in this text is very simple. If your work environment uses a different language e.g. Ruby, .Net, Python. Chances are that you will be able to use Selenium-RC in that language.

For more information on Java visit:

- Official Java Home page - http://www.java.com
- Official developers page - http://java.sun.com
- Java Community page - http://theserverside.com

Eclipse

Eclipse is a free and open-source IDE which supports many programming languages.

Out of the box it provides many useful support aids for beginning java programmers:

- wizards for code creation
- code completion
- automatically fix common coding errors

It is also simple to install, incredibly popular among developers, and has a lot of additional plugins that you can use to increase your productivity.

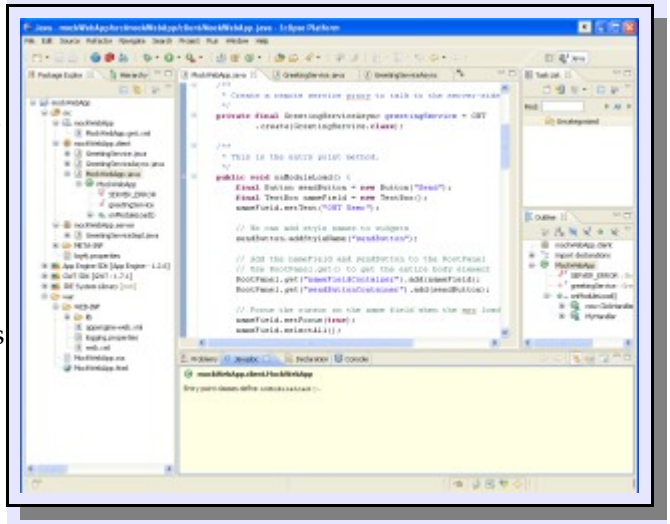


Figure 1.2: The Eclipse IDE

For more information on Eclipse visit:

- Official Home Page - http://www.eclipse.org
- Free Video Tutorials - http://eclipsetutorial.sourceforge.net/index.html

Selenium-RC

Selenium Remote Control is the server version of Selenium. You write your tests using a programming language and client library. Your tests issue commands which the client library sends to the server. The server then 'runs' your actions for you in the browser and reports the results back to your client.

Using Selenium-RC allows you to write Automated tests in any supported programming language.

Tests written in this way allow you to use standard programming practices to make them easy to maintain, robust and easy to collaborate on as a team.

For more information on Selenium-RC visit:

- Official Home Page - http://seleniumhq.org/projects/remote-control/
- Official Documentation - http://seleniumhq.org/docs/05_selenium_rc.html

Chapter 2: Selenium IDE Basics

Install Firefox

Because the Selenium IDE is a Firefox add-on we first have to download and install Firefox.

Visit http://www.mozilla.com/firefox and download and install the most recent version of Firefox.



Figure 2.1: Download and Install Firefox

Install Selenium-IDE

We download and install the Selenium-IDE from within Firefox itself.

So start the Firefox browser.

Visit `http://seleniumhq.org/download/` and click on the Download link for the Selenium IDE.



Figure 2.2 : Download page and link for the IDE

Follow the series of dialogs that walk you through the install process, as illustrated below.

First you have to acknowledge that the add-on is safe to install.

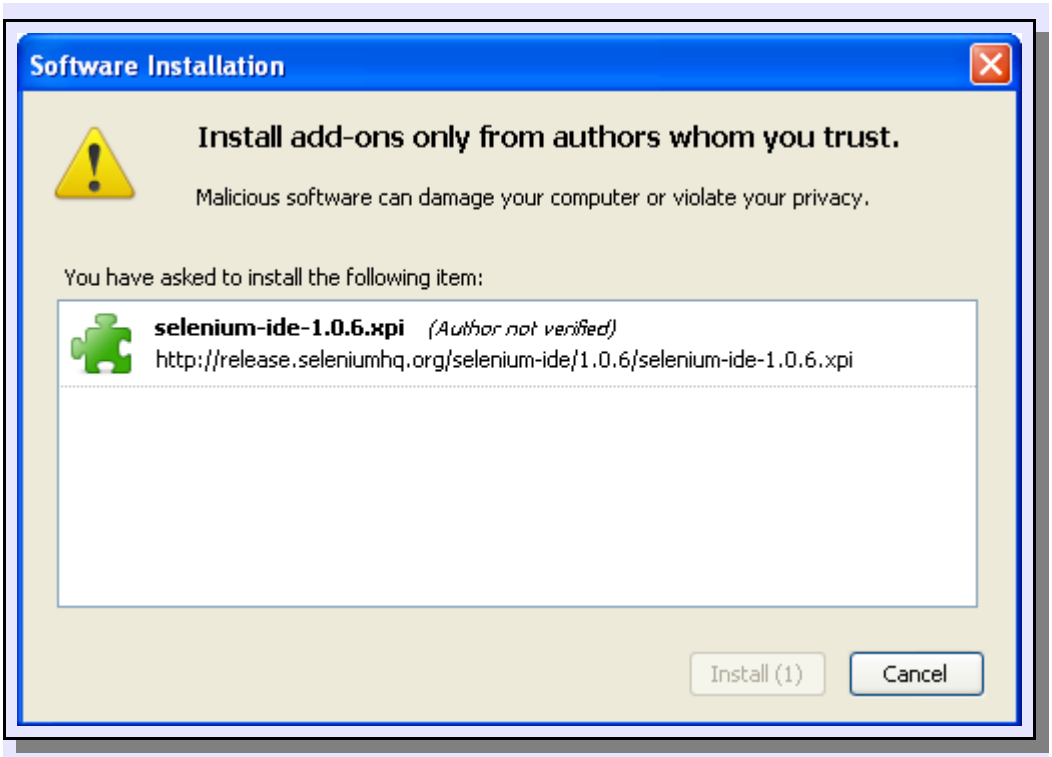


Figure 2.3 : Trust the Install of the Selenium IDE

Then after it has installed you will be prompted to restart Firefox, which you do by clicking the "Restart Firefox" button on the final Add-ons dialog.

When Firefox restarts, you should now have a new entry in your Tools menu for the Selenium-IDE.

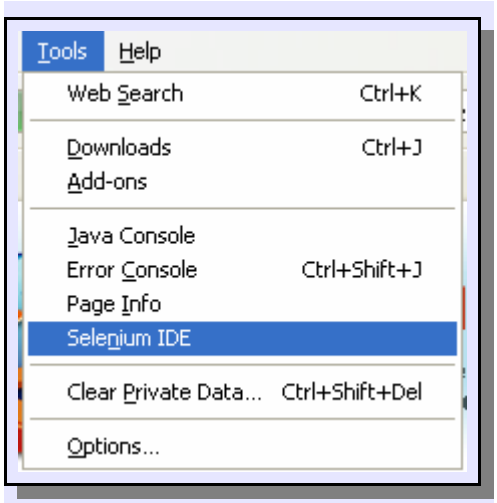


Figure 2.4: Start The Selenium-IDE

Capture Play Back – Recording a Script with the IDE

The Selenium IDE supports capture playback of test scripts by recording the actions you take when browsing web sites, and then replaying these in the browser.

So start the IDE by clicking on the "Selenium IDE" entry in the Tools Menu.

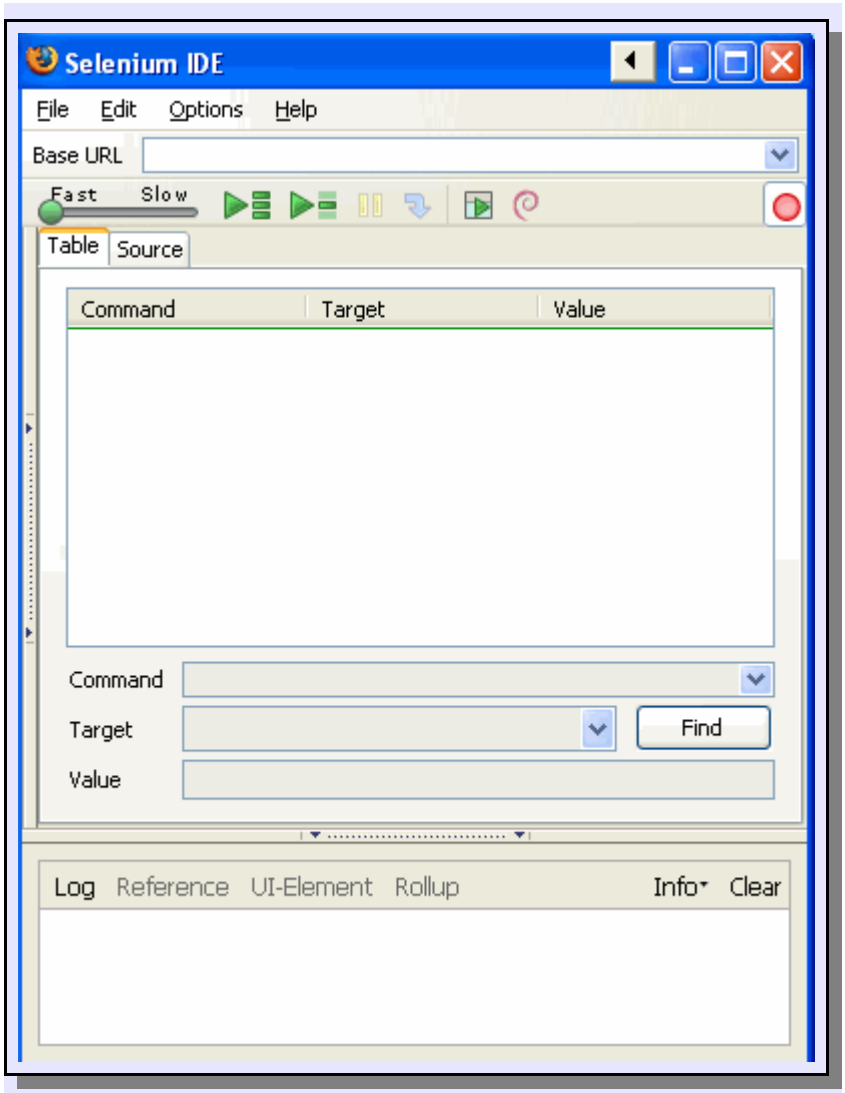


Figure 2.5 : Basic Selenium IDE

Hover the mouse over the recording button to check the recording status. If it is not recording then press the button to start recording your actions in Firefox.

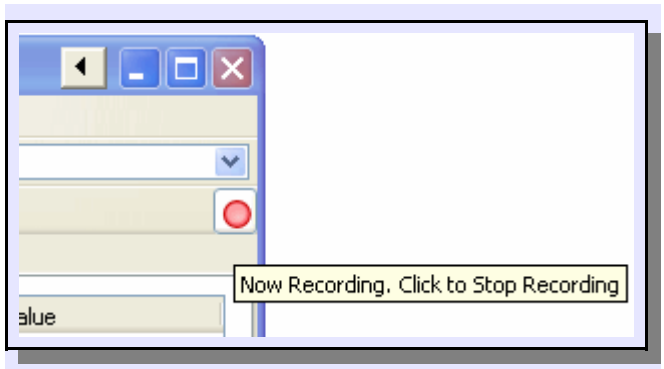


Figure 2.6: Now Recording in the IDE

Now perform the following actions in Firefox:

1. type http://www.google.com in the browser's URL field to visit Google.

2. Perform a search for “Selenium-RC”

You should see that the IDE has tracked your actions and set the “Base URL” field and created the necessary commands to repeat that script in Firefox.

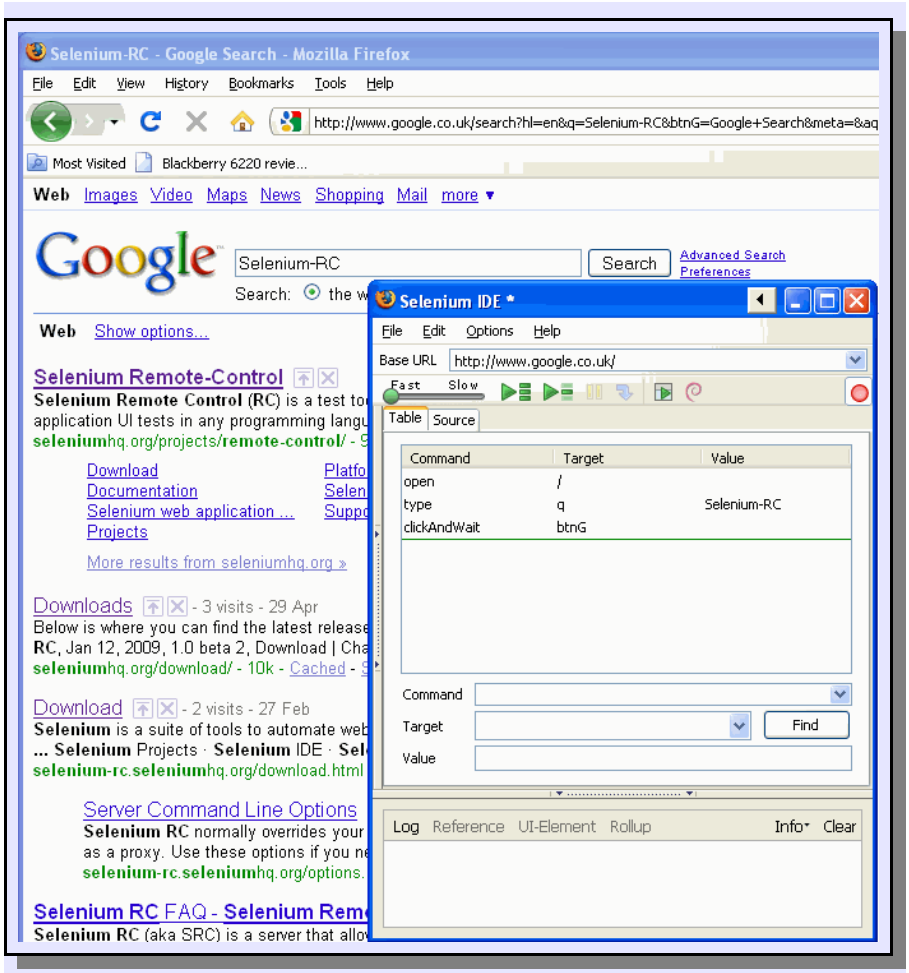


Figure 2.7: A recorded Selenese Script

Press the red recording button to stop the IDE recording any more actions. Then press the “Play Current Test Case” button to have the IDE replay your actions in the browser.



Figure 2.8: Play

You should have seen your actions repeated very quickly in Firefox. If so, then congratulations, you just recorded your first automated test script through the IDE.

Command, Target and Value

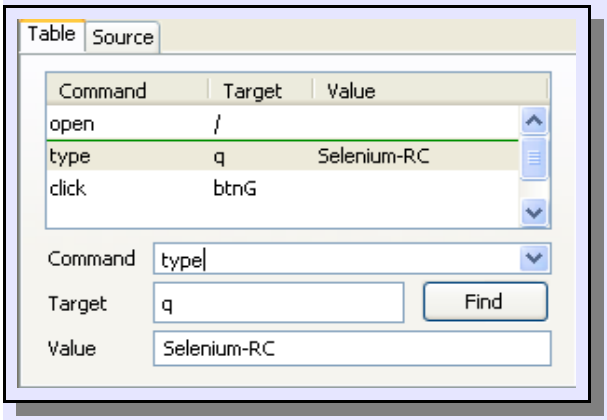


Figure 2.9: Edit any of the commands by selecting them in the table

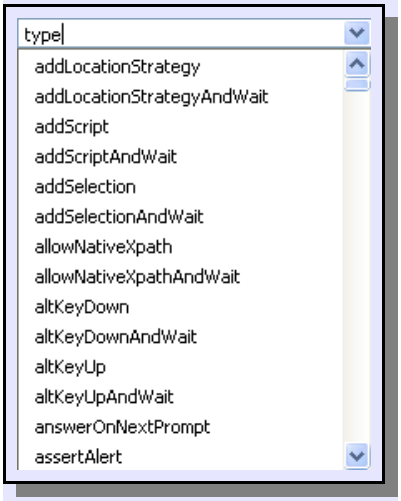


Figure 2.10: Select Any of the Commands from the Command DropDown list

You can edit any of the recorded commands in the IDE.

Simply click on any of the commands in the table and you will see that the Command, Target and Value fields are filled in with the values on the table.

You can then use the drop down for "Command" to choose any of the commands available. When you do so, you will also see documentation about that command in the Reference pane.

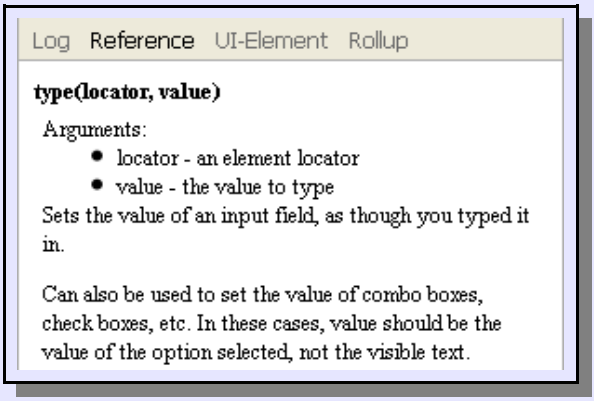


Figure 2.11: Learn what commands do in the Reference pane

Save The Test

Save this test to the disk for future use (in "Chapter 4 Install and Run Selenium-RC").

Use the "File \ Save Test Case As ..." menu and save it somewhere you can find it again. I named mine "google_for_selenium_rc.htm".

Further reading

I won't really be covering Selenium-IDE in any depth in this book. So if you want more information you should read the following links:

- official documentation - http://seleniumhq.org/docs/03_selenium_ide.html
- Adam Goucher's posts on Selenium IDE - http://adam.goucher.ca/?s=Selenium+IDE
    - particularly the plugin posts - http://adam.goucher.ca/?s=The+Selenium-IDE+1.x+plugin+API
- Selenium IDE Extensions - http://wiki.openqa.org/display/SIDE/Contributed+Extensions+and+Formats
- The Firefox plugins search may reveal some plugins for Selenium IDE https://addons.mozilla.org/en-US/firefox/search/?q=selenium
- A lot of video tutorials for Selenium IDE get added to You Tube - http://www.youtube.com/results?search_query=selenium+ide

Chapter 3: Install Java

You can check if you have Java installed by:

- visiting http://java.com and using the "Do I have Java?" check http://java.com/en/download/installed.jsp
- running the command "java -version" from the command line

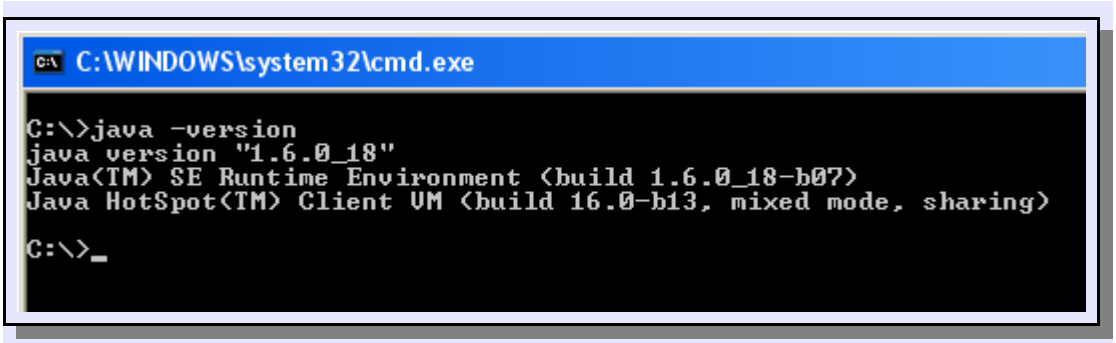


Figure 3.1 : Running "java -version" from the command line

If you don't have Java installed then visit http://java.com and follow the Download links to install it on your machine.

Chapter 4: Install and Run Selenium-RC

Before we can actually write any tests, we install Selenium-RC. "Install" is really the wrong word when all we will do is download and extract the Selenium-RC archive to a directory.

Step One - Download & Install Selenium-RC

Download Selenium RC from the Selenium download page. (http://seleniumhq.org/download/)



Figure 4.1 : Download Selenium RC from SeleniumHQ

I saved it to c:\selenium

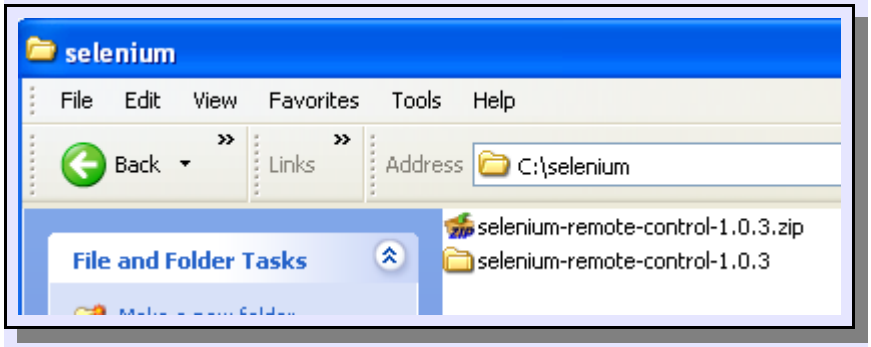


Figure 4.2 : Unarchive the saved distribution into a directory

Then I unarchived this into a folder of the same name as the the archive using an unarchiver (I use IZArc, available from http://www.izarc.org/).

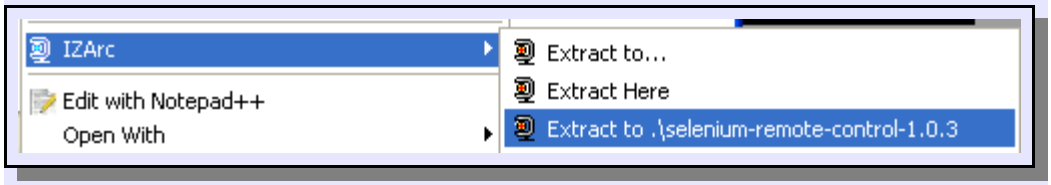


Figure 4.3 : Extracting to a folder with same name as archive

So now I have a folder with the Selenium files:

- C:\selenium\selenium-remote-control-1.0.3

Overview of the Contents of the Selenium Archive

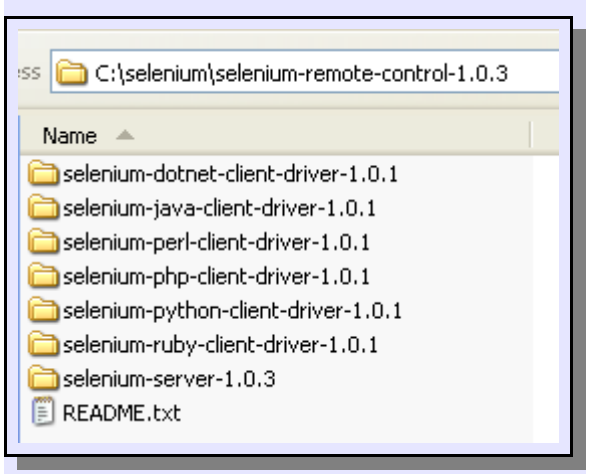


Figure 4.4 : Contents of Selenium RC

There are two main folder types here:

- specific language client drivers
- selenium server

Each of the supported languages has a client driver. You can see from the list of folders that the Selenium distribution supports:

- Microsoft .Net
- Java
- Perl
- PHP
- Python
- Ruby

The client drivers are what we use in our tests to communicate with Selenium-RC. We use the client drivers because our tests act as clients, which communicate with the server, in order to execute commands through the browser.

The client drivers contain the source code, tests, documentation and libraries that we need to work with the Selenium tests.

The selenium server directory has the documentation, source code, tests and executables for working with the Selenium server.

We will examine the contents of the archive in more detail in future sections but this high level overview should give you enough of a basic understanding to let us continue with the tutorial.

Step Two - Run Selenium-RC

I have the "Open Command Window Here" Microsoft Powertoy for XP installed. I recommend you install this very helpful tool, if you use XP. In Vista this functionality is built in and you can browse to the folder in Windows Explorer, hold down the Shift key, right-click on the folder and choose "Open Command Window Here".

- http://download.microsoft.com/download/whistler/Install/2/WXP/EN-US/CmdHerePowertoySetup.exe
- http://www.microsoft.com/windowsxp/Downloads/powertoys/Xppowertoys.mspx

> **Note:**
>
> **If you don't have "open command window here" installed then you need to get a command window open and type:**
>
> **cd c:\selenium\selenium-remote-control-1.0.3\selenium-server-1.0.3**
>
> **Hint: Try pressing 'tab' as you do this to automatically complete folder details.**

So I right click on the "Selenium-server-1.0.3" folder and choose "Open Command Window Here" from the context menu.

> **Note:**
>
> **I also use the open-source Console (http://sourceforge.net/projects/console/) to help me as I can have multiple console windows open in a single application.**

I give the command window a new title so I can find it easily by issuing the command "Title SELENIUM SERVER".

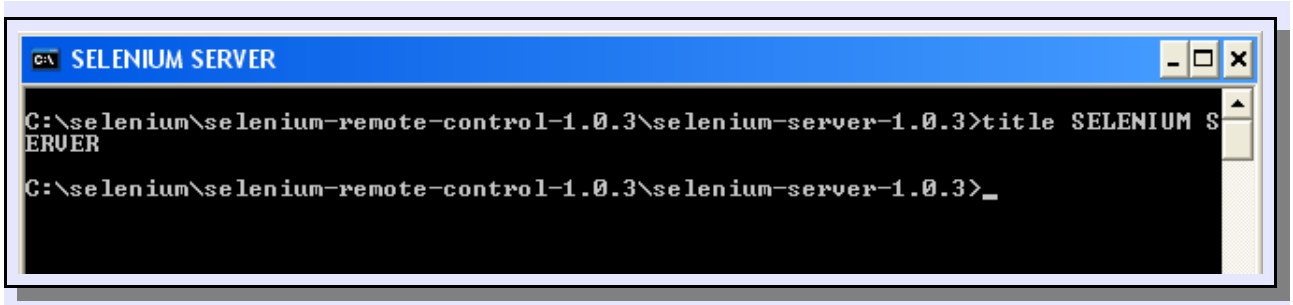


Figure 4.5 : An easy to identify Selenium Server command window

This makes it much easier to find when I Alt-Tab or look for it on the taskbar.

Now we run the selenium server by typing the command "java -jar selenium-server.jar"

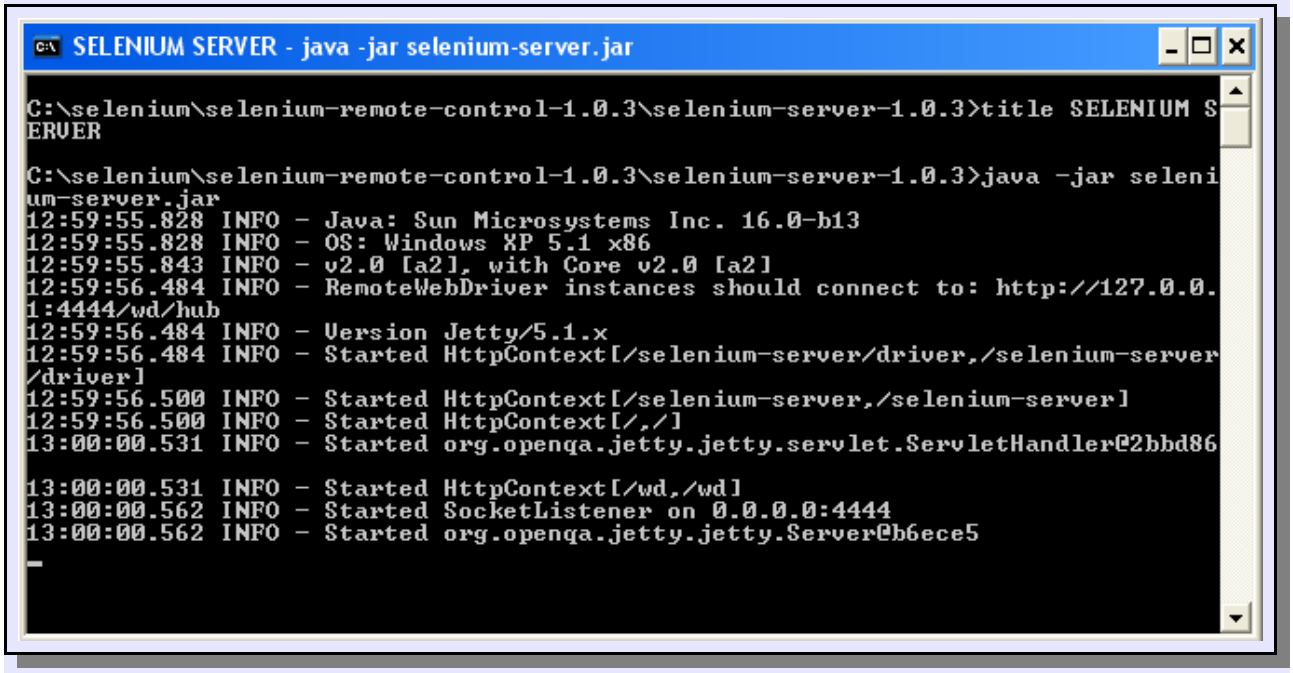


Figure 4.6 : Start up messages from the Selenium Server

If you didn't see a screen like the above then you may need to install java.

Done - you have just installed and started the Selenium-RC server.

Step Three – Stop the Selenium Server

From the command line

Since you have started the Selenium server in a DOS window you can simply use "Ctrl+c" to stop it.

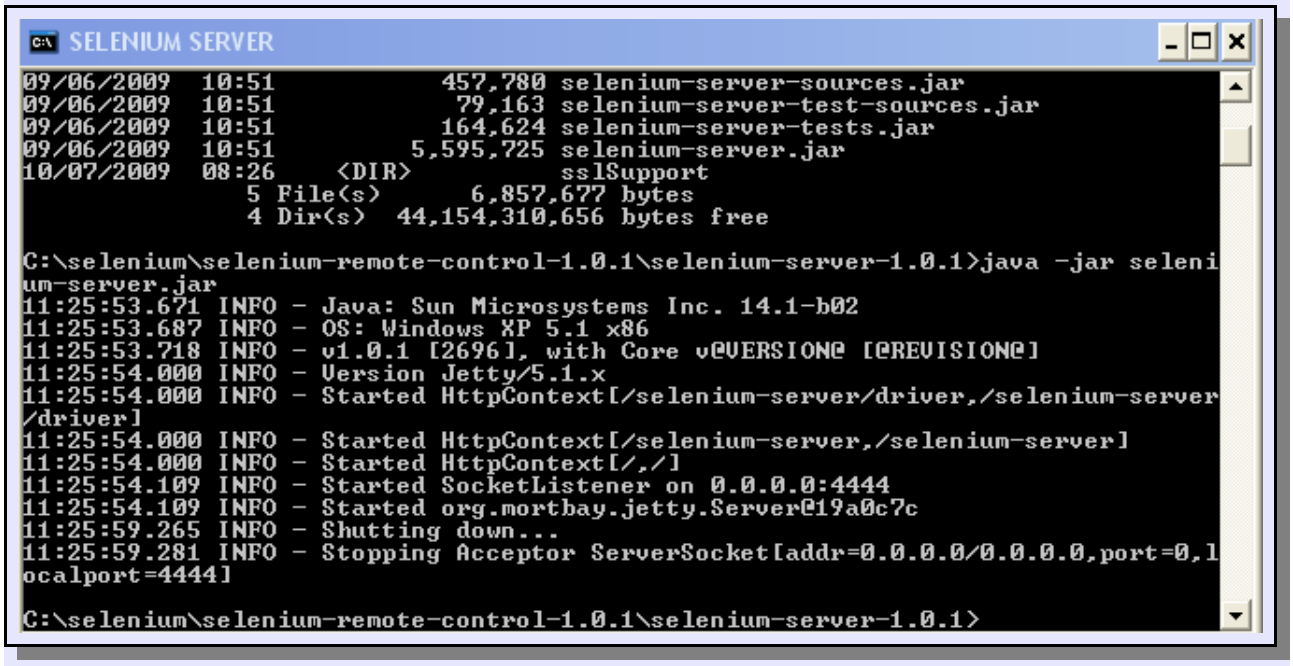


Figure 4.7 : After Ctrl+c the server shuts down

From a browser URL

Since selenium server operates a web server under Jetty it can receive commands via http through browsers. Issue the following to shutdown the server:

- http://localhost:4444/selenium-server/driver/?cmd=shutDownSeleniumServer

If you wanted to leave the server running and just close down the current Selenium session, you could issue the following command in a browser.

- http://localhost:4444/selenium-server/driver/?cmd=shutDown

Running an IDE Generated Test in Different Browsers

Although the IDE script we recorded in Chapter 2 was recorded in Firefox, and only Firefox has the IDE, we can run IDE recorded scripts in other browsers. This requires the use of Suites, and Selenium-RC.

> **Note: I do not recommend this approach to using Selenium. I only include it for completeness.**
>
> **I have had issues getting this to work with Internet Explorer on Selenium 1.0.3 using *iexplore, *iehta, or *piiexplore. Other browser codes seem to work, but when using IE8 the Selenium server seems to throw an HTML suite exception. If this happens to you, and you consider it important, then you might need to drop down to an earlier version of Selenium-RC. Stick with the book though and it shouldn't be an issue as you will never use this method to run the tests.**
>
> **http://groups.google.com/group/selenium-users/browse_thread/thread/8450ee42417806d0/3f32cadf651e57f8**

You create a suite by creating an HTML file with links to the tests you want to create.

e.g suite.html

```
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>My Test Suite</title>
</head>
<body>
    <table cellpadding="1" cellspacing="1" border="1">
        <thead>
            <tr><td>A Suite of Selenese Tests</td></tr>
        </thead>
    <tbody>
        <tr><td><a href="google_for_selenium_rc.htm">Google For Selenium
RC</a></td></tr>
        <tr><td><a href="google_for_testing.htm">Google For Testing</a></td></tr>
        <tr><td><a href="visit_google_videos.htm">Visit Google Videos</a></td></tr>
    </tbody>
    </table>
</body>
</html>
```

For the above example you would have to have three recorded IDE scripts and they would all be saved in the same directory as the suite file.

You don't really need the header:

```
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>My Test Suite</title>
</head>
```

or the cell padding *cellpadding="1" cellspacing="1" border="1"*

```
<html>
<body>
    <table>
        <thead>
            <tr><td>A Suite of Selenese Tests</td></tr>
        </thead>
    <tbody>
        <tr><td><a href="google_for_selenium_rc.htm">Google For Selenium
RC</a></td></tr>
        <tr><td><a href="google_for_testing.htm">Google For Testing</a></td></tr>
        <tr><td><a href="visit_google_videos.htm">Visit Google Videos</a></td></tr>
    </tbody>
    </table>
</body>
</html>
```

But since will typically create a test by copy and pasting, making it well typed and formatted may stand you in good stead for future versions of Selenium.)

Since you have Selenium-RC installed you can use the following command line syntax to run a suite of Selenese scripts in other browsers.

```
java -jar selenium-server.jar -htmlSuite "<insert_browser_name_here"
"<insert_root_domain_here" "<insert_full_path_to_suite_here>"
"<insert_a_full_path_to_store_reports_here"
```

e.g. To run it in Google Chrome

```
java -jar selenium-server.jar -htmlSuite "*googlechrome" "http://www.google.com"
"c:\ide_scripts\suite.htm" "c:\ide_scripts\results.htm"
```

To execute the above command you would need to run the command from the directory where selenium-server.jar is located, and your suite would need to be stored in c:\ide_scripts

I have found that on windows I need to store the suite in the same folder as the test scripts and that I have to pass the full path for the suite file.

When you run the above command you will see the Selenese suite runner with the suite of tests in the right and it will run through and report the results.

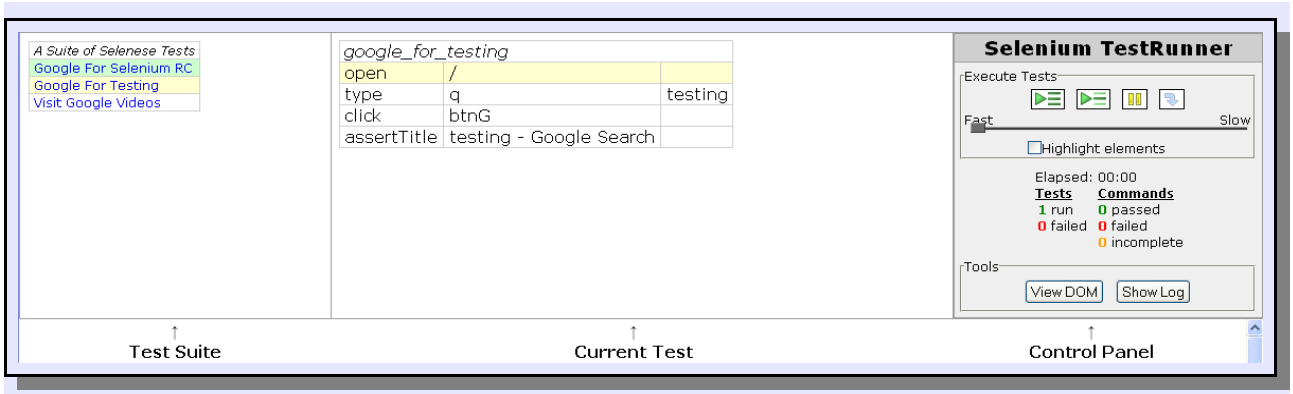
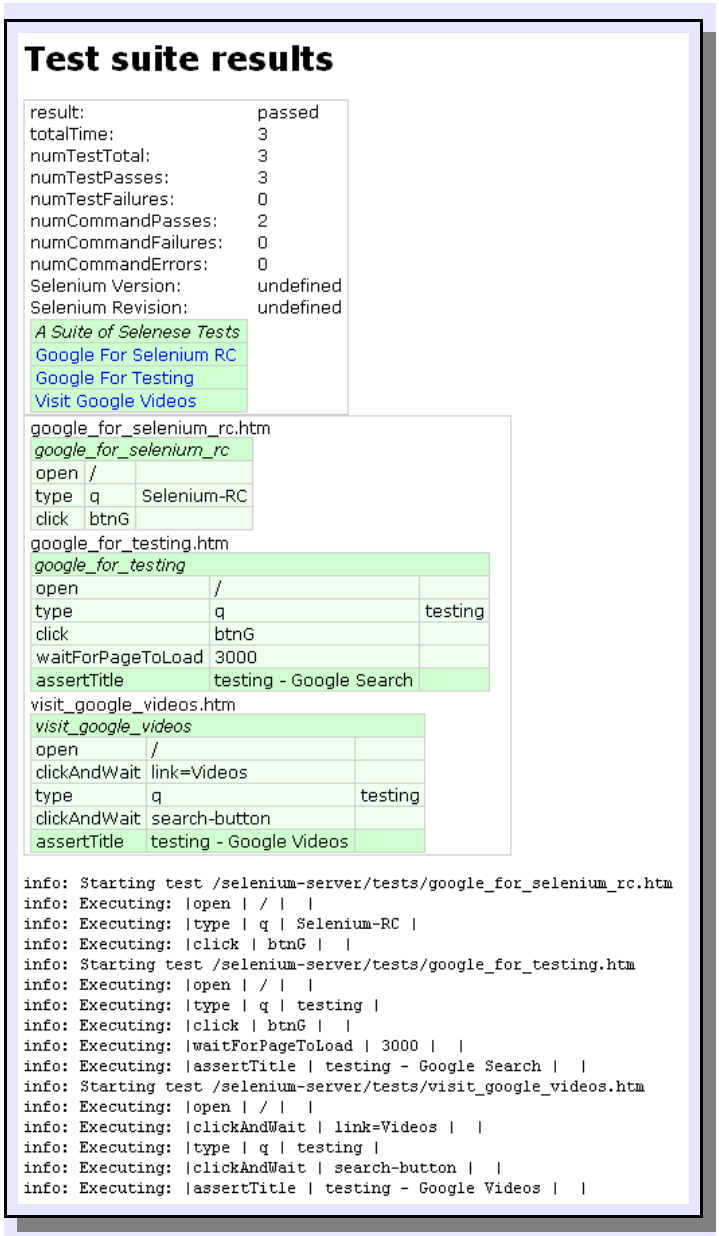


Figure 4.8: Running a suite of tests

Running the tests will result in the generation of an HTML file with the results using the path that you used in the command line.



Test suite results

result:	passed
totalTime:	3
numTestTotal:	3
numTestPasses:	3
numTestFailures:	0
numCommandPasses:	2
numCommandFailures:	0
numCommandErrors:	0
Selenium Version:	undefined
Selenium Revision:	undefined

A Suite of Selenese Tests
Google For Selenium RC
Google For Testing
Visit Google Videos

google_for_selenium_rc.htm

google_for_selenium_rc		
open	/	
type	q	Selenium-RC
click	btnG	

google_for_testing.htm

google_for_testing		
open	/	
type	q	testing
click	btnG	
waitForPageToLoad	3000	
assertTitle	testing - Google Search	

visit_google_videos.htm

visit_google_videos		
open	/	
clickAndWait	link=Videos	
type	q	testing
clickAndWait	search-button	
assertTitle	testing - Google Videos	

```
info: Starting test /selenium-server/tests/google_for_selenium_rc.htm
info: Executing: |open | / |  |
info: Executing: |type | q | Selenium-RC |
info: Executing: |click | btnG |  |
info: Starting test /selenium-server/tests/google_for_testing.htm
info: Executing: |open | / |  |
info: Executing: |type | q | testing |
info: Executing: |click | btnG |  |
info: Executing: |waitForPageToLoad | 3000 |  |
info: Executing: |assertTitle | testing - Google Search |  |
info: Starting test /selenium-server/tests/visit_google_videos.htm
info: Executing: |open | / |  |
info: Executing: |clickAndWait | link=Videos |  |
info: Executing: |type | q | testing |
info: Executing: |clickAndWait | search-button |  |
info: Executing: |assertTitle | testing - Google Videos |  |
```

Figure 4.9: Results of running a suite

Try this after following the instructions on installing Java and Selenium-RC.

End Notes

There are a lot more functions available in the IDE and you could use it as a tool to manage an entire suite of test cases. But we are aiming to manage our test cases as code and in later sections we use the IDE to create our basic test cases and as a support tool for our automation.

Chapter 5: The Eclipse IDE

First - Install Eclipse

Visit http://www.eclipse.org/ and click on the big download button.



Figure 5.1: Download Eclipse

Then download the "Eclipse IDE for Java Developers"

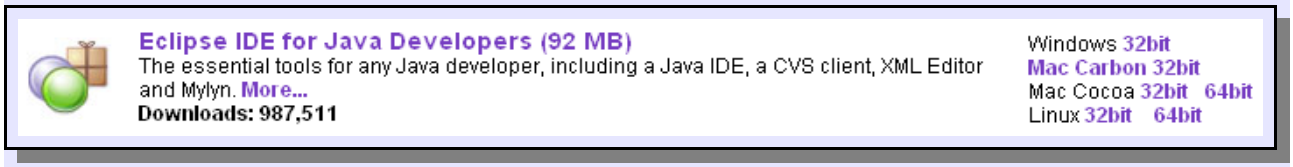


Figure 5.2: Download the appropriate IDE

Select a mirror from the list.



Figure 5.3: Choose the location to download from

Download the zip file and save it to a folder.

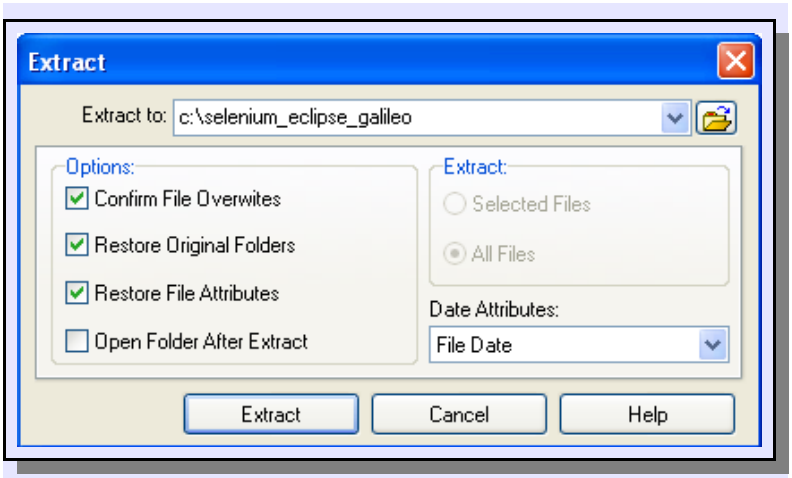


Figure 5.4: Extract Eclipse to the folder of your choice

Then unarchive this into a folder using your favourite zip unarchiver. I right click on the file and choose the unarchive option from the context menu to extract the contents of the archive to a folder – I chose "c:\selenium_eclipse_galileo" (at the time of writing the current version of Eclipse was named "Galileo"). If the version you installed has a different name then don't worry. Eclipse versions are backwardly compatible and improve with each release, so use the most up to date version of Eclipse.

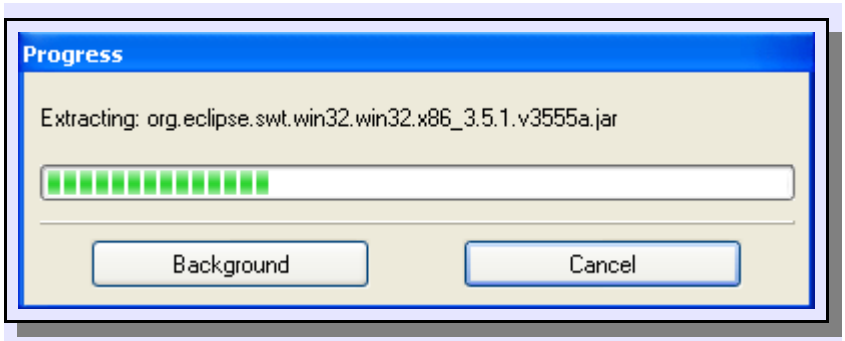


Figure 5.5: Extracting Eclipse

Second - Run Eclipse

Run Eclipse by double clicking on "eclipse.exe" in "c:\selenium_eclipse_galileo\eclipse".

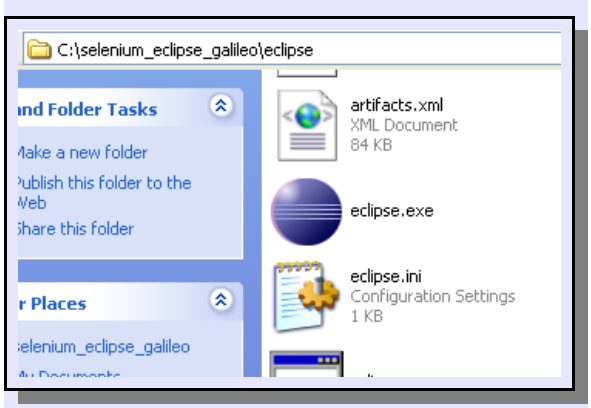


Figure 5.6: Run eclipse.exe

You will see the splash screen for Eclipse.



Figure 5.7: Eclipse Splash Screen

Choose a 'workspace' - where your project files will be saved. You can easily change this at a later date so choose a location that seems sensible for you. I have put it on the c:\ drive for clarity in the explanatory text and screen shots.

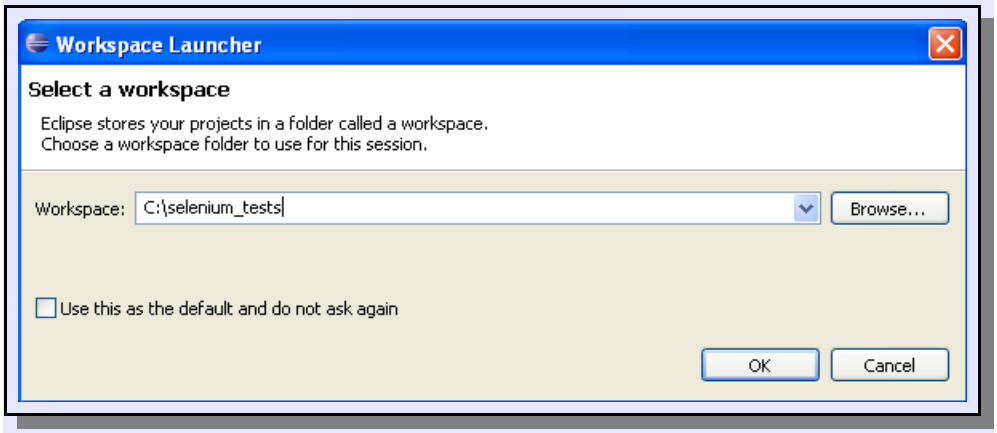


Figure 5.8: Choose a location where Eclipse will save your source code

And then you will see the Eclipse "Welcome" screen, which I always find a bit confusing.

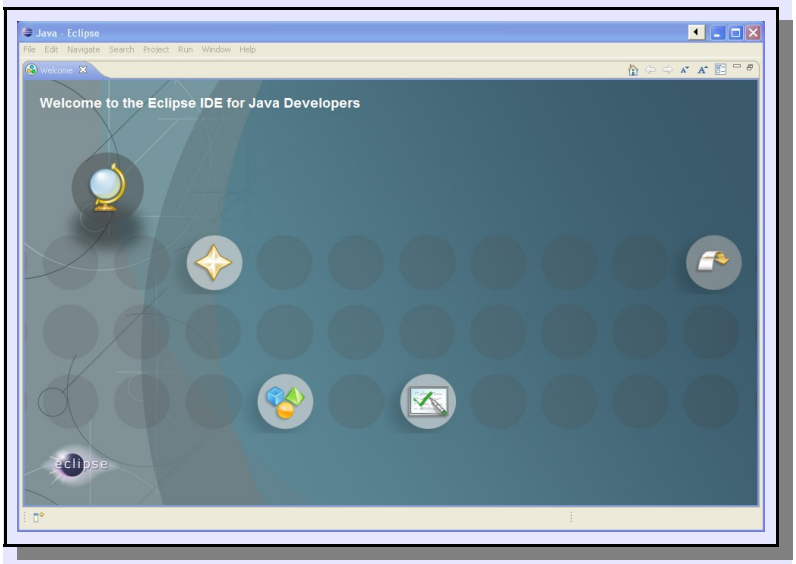


Figure 5.9: Eclipse Welcome Screen

You want to click on the curved arrow to the right which displays the text “Workbench” “Goto the Workbench” when you hover the mouse over it.

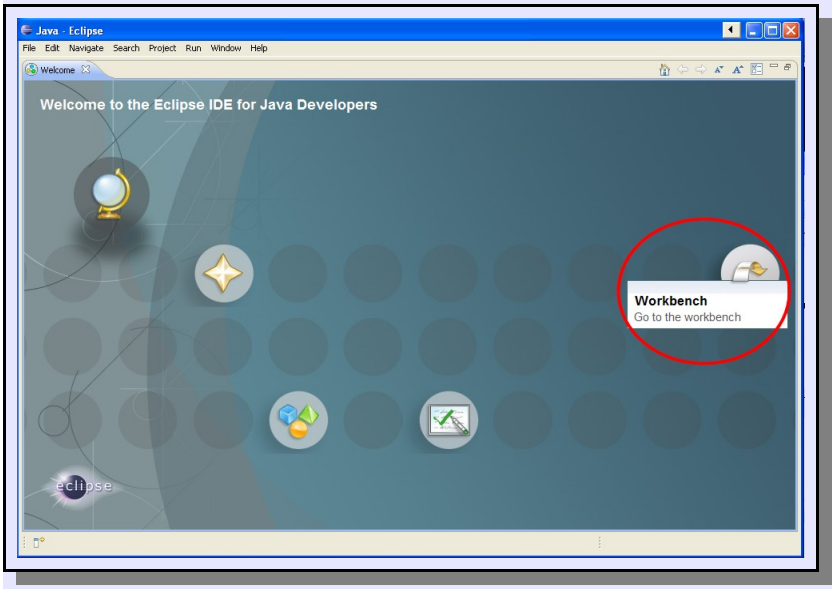


Figure 5.10: Goto Workbench Location on Welcome Screen

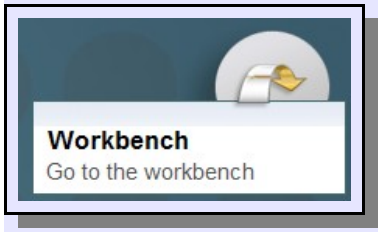


Figure 5.11: Goto Workbench

And then you will see the Eclipse workbench screen that will become very familiar and you will grow to know and love.

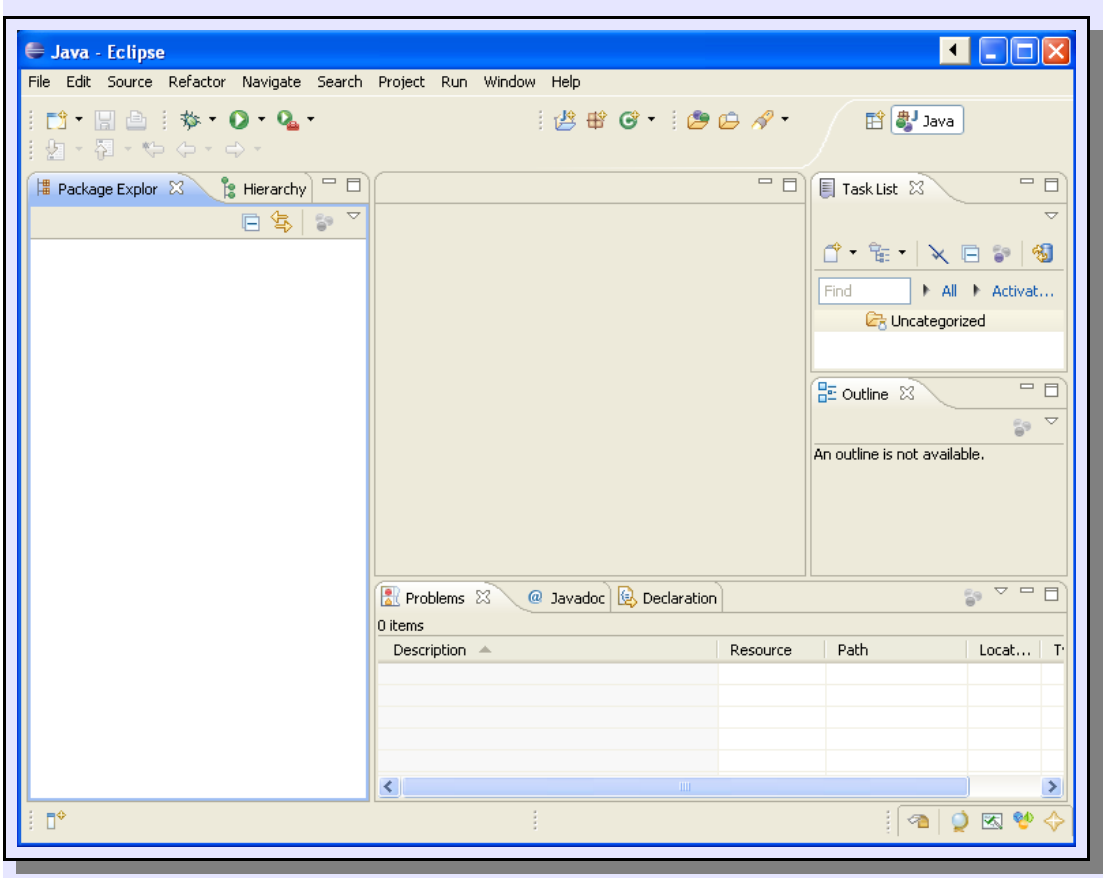


Figure 5.12: The Eclipse Workbench Screen

Third - create a new Java project

We are going to create tests in Java so to do that we need a Java project to store them in.

Start by using the file menu in Eclipse to create a new Java project.

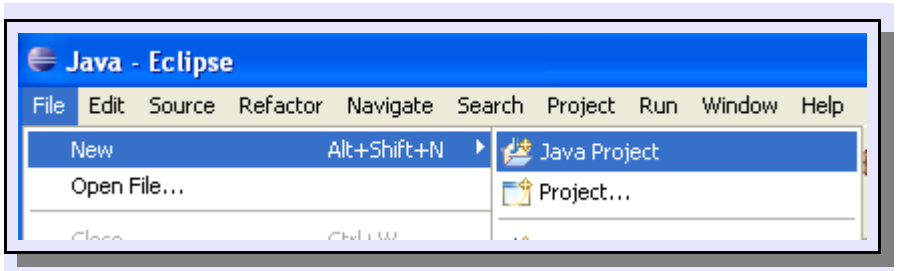


Figure 5.13: Create a New Java Project from the File Menu

Type in the name of your new project - I called mine "InitialSeleniumTests", just leave everything else on the dialog with the default entries and click 'finish'.

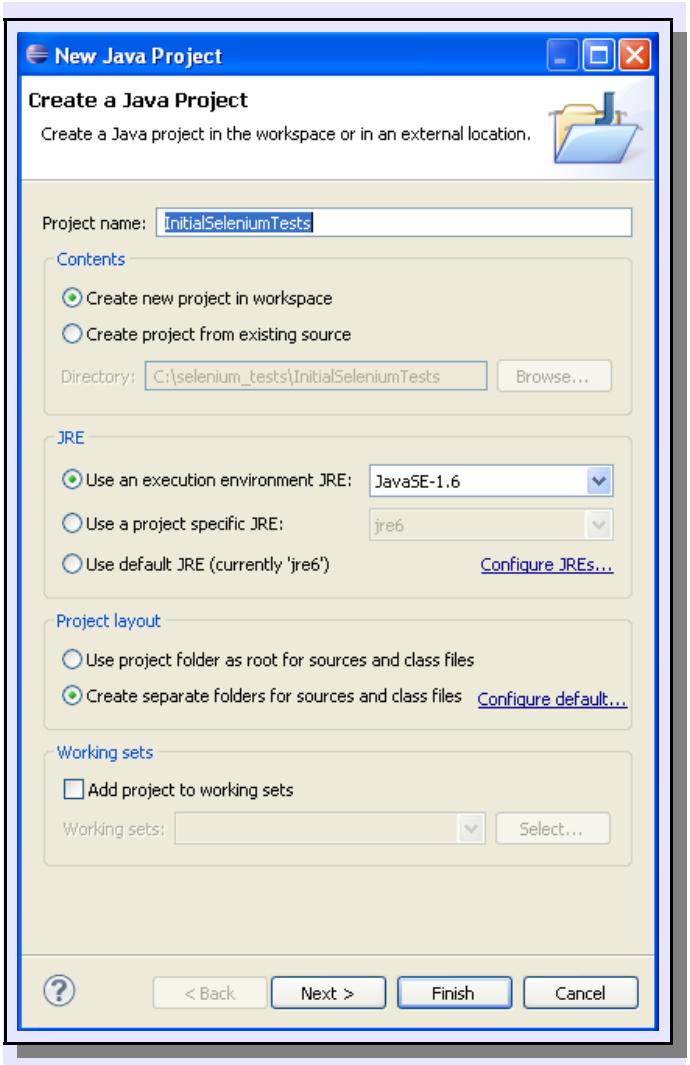


Figure 5.14: Project Named Initial Selenium Tests

This will create a folder in your workspace directory called “InitialSeleniumTests” which has the basic folders that you need when working with Java in Eclipse.

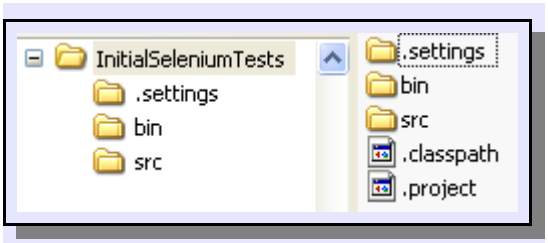


Figure 5.15: Automatically Created Files

In Eclipse, in the Package Explorer tab on the left, you will see a tree with a root node named "InitialSeleniumTests", this is your project.

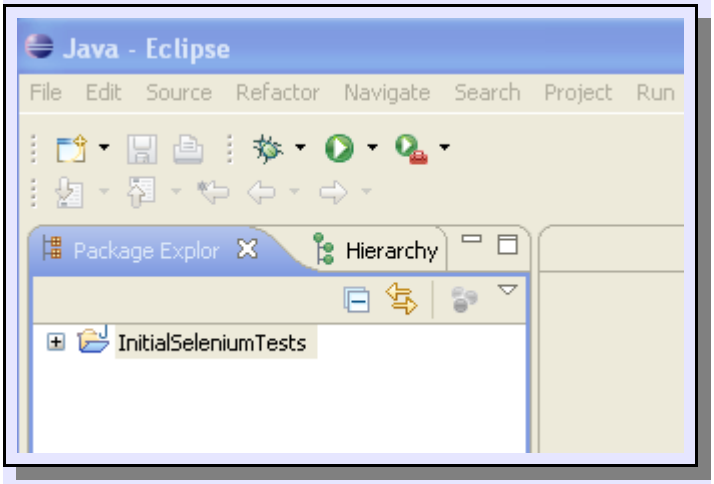


Figure 5.16: Default package view after creation

Now you are ready to start creating Selenium tests in Eclipse. We still have a few changes to make in the Eclipse configuration, but we will make those changes in the context of writing tests so that you will see and understand why we make them.

Chapter 6: Create a JUnit test Using the JUnit export from Selenium-IDE

Introduction

In this section we are going to:

- export an IDE script as a Java JUnit test
- get the exported test running in Eclipse

Export an IDE script as a JUnit test

We have already done this in the preceding section. So in the IDE load the test that you saved "google_for_selenium_rc.htm" or follow the instructions in the section "Capture Play Back – Recording a Script with the IDE" to create a test script.

Load an existing script into the IDE

Start up the Firefox browser. And click on "Selenium IDE" from the Tools menu to display the Selenium IDE.

In the IDE select "File\Open..."

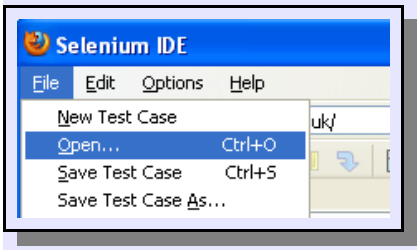


Figure 6.1: select "File\Open..."

Then navigate to the test case and Open it in the IDE. You should see the test loaded into the IDE in the table view.

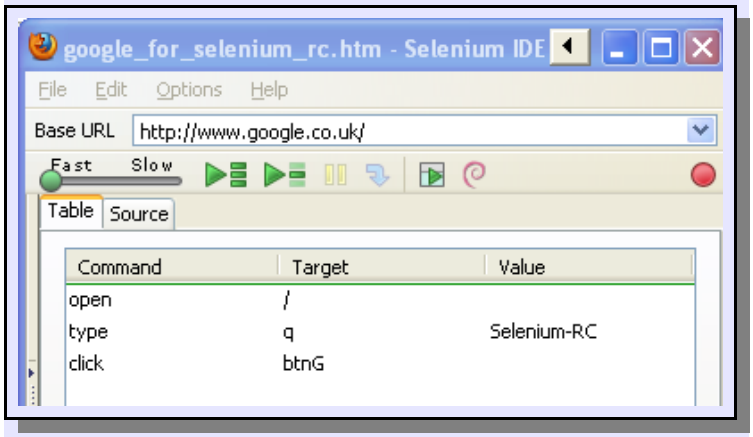


Figure 6.2: Test in table view

Change the format of the script to Java

We need to convert this test into a Java test and we do this by changing the format of the test from the options menu.

If you open the options menu from the IDE then you will see that the current format of the recorded test script is "HTML". This is the default recording and playback option for the IDE using an Action Word scripting language called "Selenese".

So select "Java (JUnit) – Selenium RC" from the "Format" menu.

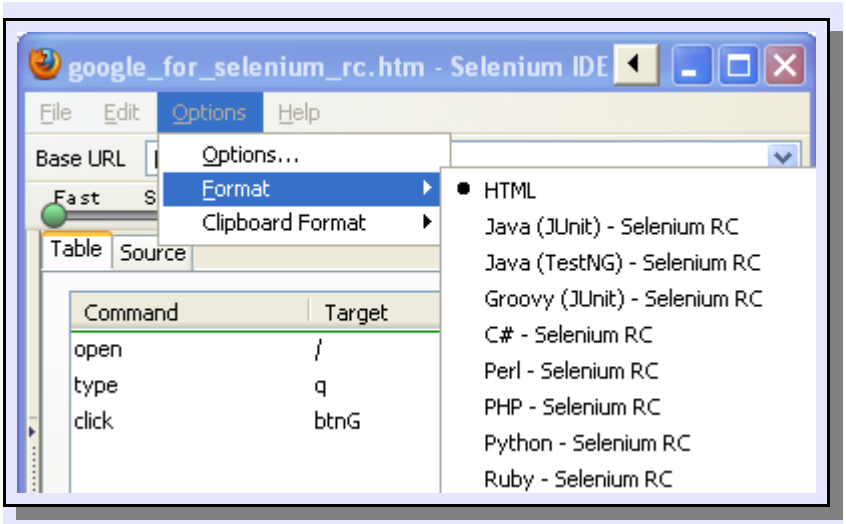


Figure 6.3: Test script formatting options

You should now see the "Source" tab selected and some Java source code displayed.

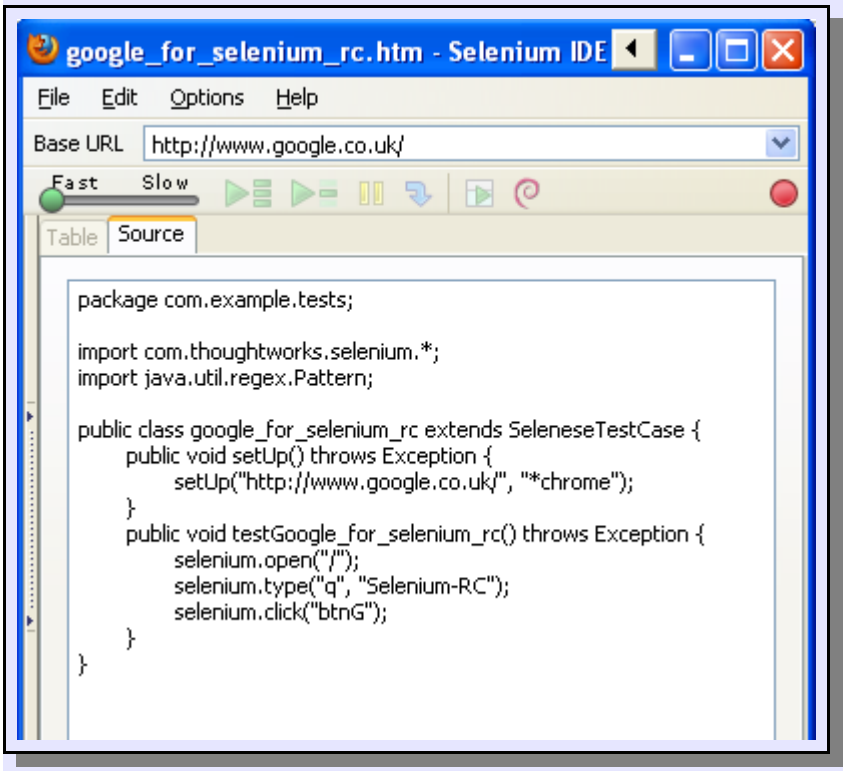


Figure 6.4: Test script displayed as a JUnit test

You don't need to understand this test code yet. We will explain it as we go through this section. But rest assure it will make sense at the end of this chapter.

So the first thing we are going to do is create a new class to store our tests in.

NOTE:

Depending on the order that you did actions in the Selenium-IDE you might find that the base URL in the sourcecode view is not www.google.co.uk, but is instead

```
setUp("http://change-this-to-the-site-you-are-testing/", "*firefox");
```

If this happens, all you have to do is edit the code after you have pasted it into the Eclipse IDE so that it has the base URL that you are working with. e.g.

```
setUp("http://www.google.co.uk/", "*firefox");
```

Create a New Class in Eclipse

In Java all of our 'tests' are going to be 'methods' of a Java Class.

Java is an Object Oriented programming language so all of the code is built from classes. And the only way to have the program do something is to execute a method of one of the classes.

You don't need to understand this yet and we will revist this in a later section to explain the basics of classes, methods, objects, packages and other terminology you will encounter.

So, start Eclipse (If you haven't already got it running), and use the Menu system: File \ New \ Class

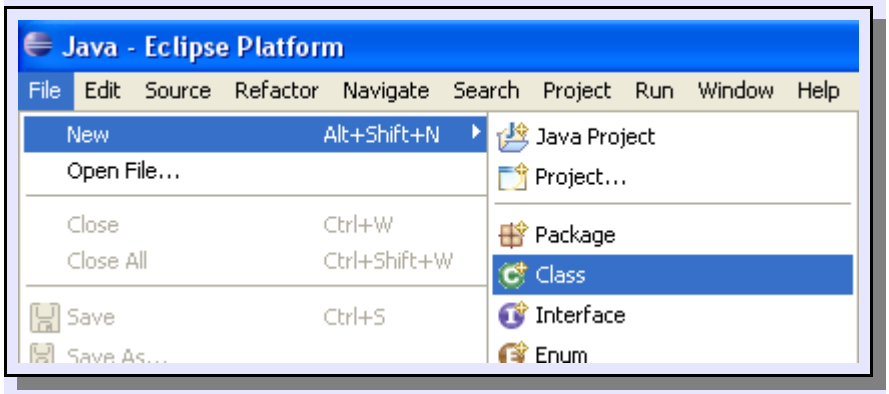


Figure 6.5: Start creating a New Class

The Eclipse "New Java Class" wizard will appear and we can use this to create our basic class - don't worry if you don't understand all the items in the dialog, just follow the instructions, you will understand everything there eventually as you become more experienced with Java. Right now, we just want to create a simple class called “MyFirstSeleniumTests”.

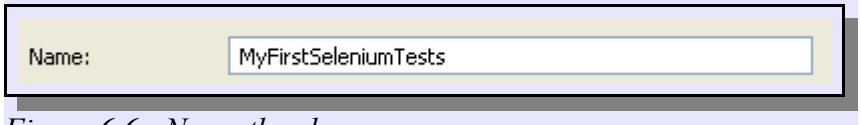


Figure 6.6: Name the class

So in the name field type "MyFirstSeleniumTests" - if you choose a different name then note that you you should start the class name with an uppercase letter.

After typing in the name, did you notice that Eclipse showed you a warning message saying "The use of the default package is discouraged". Packages are how we organise our Java code so that we can reuse it and manage it effectively on a large project.

Eclipse provides plenty of warnings to help you code. This warning means that we should really type in a package name - and we will.

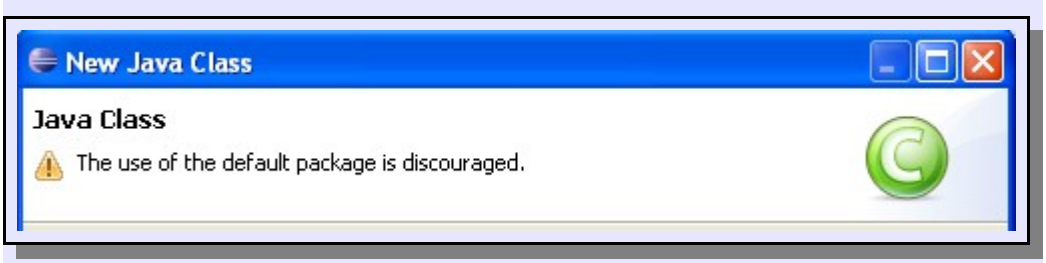


Figure 6.7: Warning to choose a package

Sun provide some guidance on package names (see links below), I will use "com.eviltester.seleniumtutorials" and type that into the "Package" field.

- http://java.sun.com/docs/codeconv/html/CodeConventions.doc8.html
- http://java.sun.com/docs/books/jls/third_edition/html/packages.html#7.7

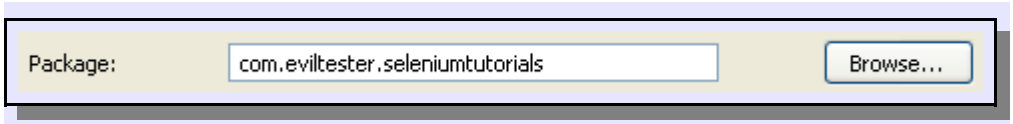


Figure 6.8: Enter a package name

The rest of the dialog can be left with the default values provided by the wizard.

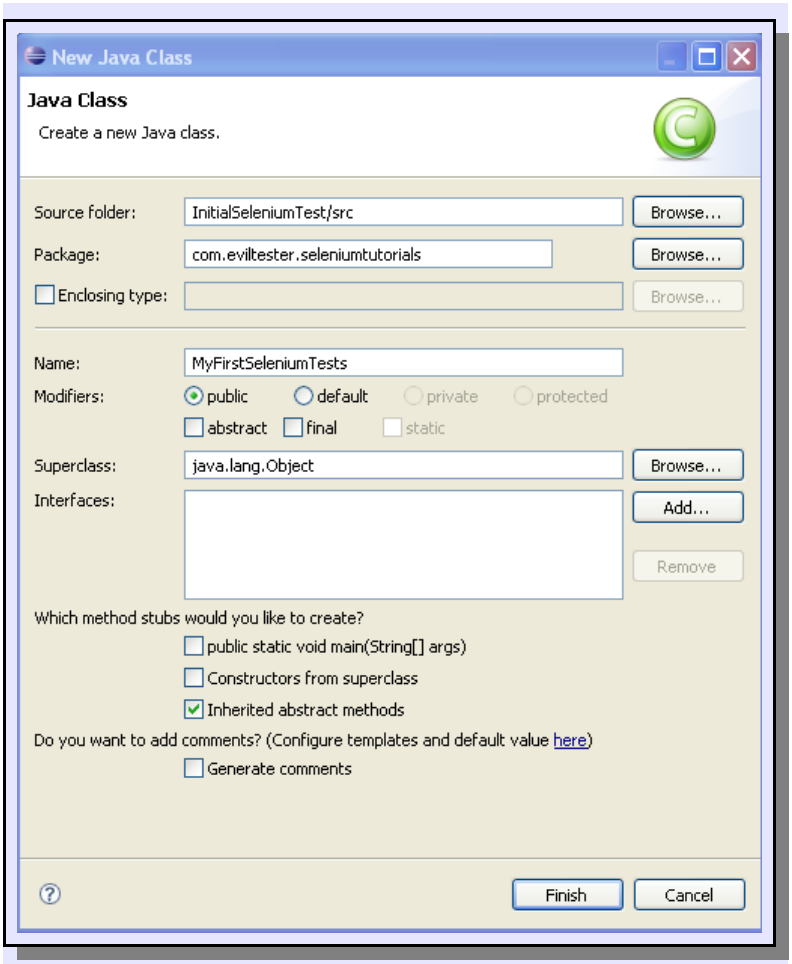


Figure 6.9: New Java Class wizard

And then press finish.

Eclipse will have created an empty class and a package for you.

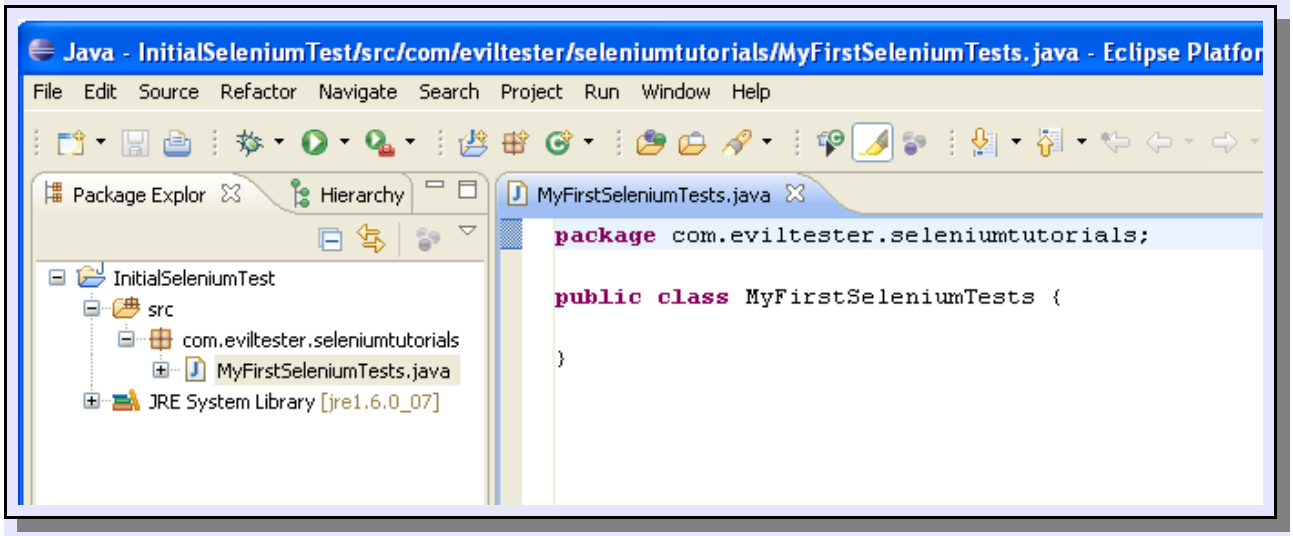


Figure 6.10: Created Class in Eclipse

Now we just have to add the code from the IDE.

Copy & Paste the code from Selenium IDE into Eclipse

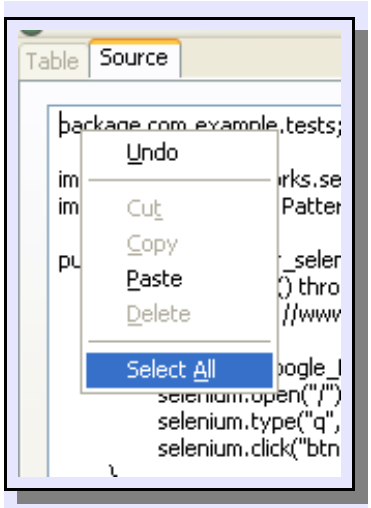


Figure 6.11: Select All

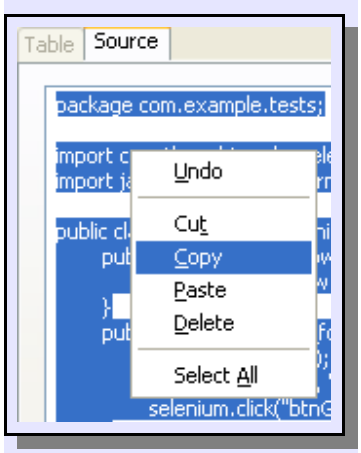


Figure 6.12: And Copy from Context Menu

Copy the code from the IDE into the clipboard.

Either by using the right click context menu to do a “select all” followed by a “copy”, or by clicking on the source code and using the keyboard and pressing Ctrl+a followed by Ctrl+c.

We can't just paste the Selenium IDE code into Eclipse because we need some of the code that the New Class Wizard created for us.

So paste the Selenium IDE code at the end of the MyFirstSeleniumTests.java code so you end up with a code listing like the one below.

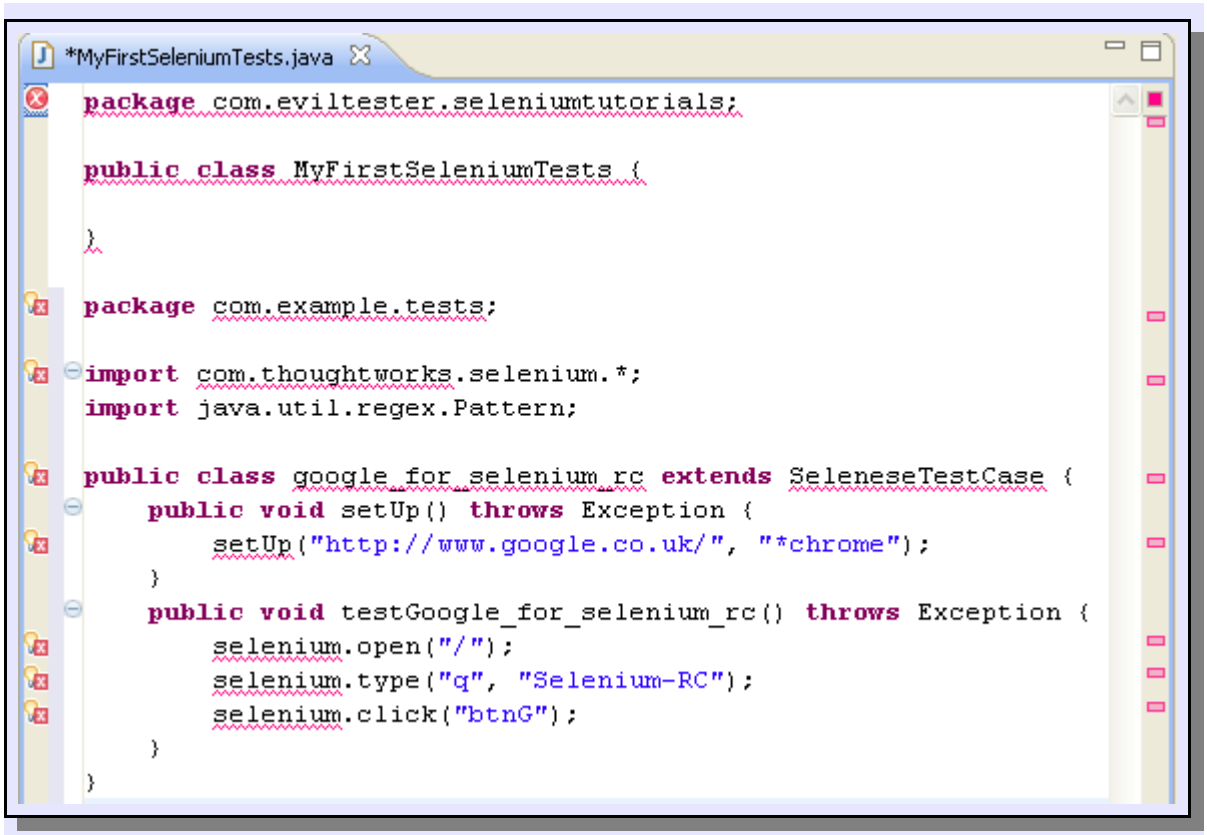


Figure 6.13: Copy and Pasted Class with lots of warning errors

As you can see there are a lot of Eclipse error symbols on the left hand side of the screen. This is because we have written invalid Java with two package declarations and two class declarations. So we will tidy that up:

- remove the line "package com.example.tests;"
  - We can only have one package declaration in each class, and it is at the top of the file.
- copy MyFirstSeleniumTests and overwrite google_for_selenium_rc
- then remove the declaration of MyFirstSeleniumTests that the wizard generated ie. "public class MyFirstSEleniumTests { }"
  - We only want one class declaration per file.

This should give you a listing like the following, which although valid Java is not yet correct because we need to make a few more changes to make it work in Eclipse:

```
package com.eviltester.seleniumtutorials;

import com.thoughtworks.selenium.*;
import java.util.regex.Pattern;

public class MyFirstSeleniumTests extends SeleneseTestCase {
	public void setUp() throws Exception {
		setUp("http://www.google.co.uk/", "*firefox");
	}
	public void testGoogle_for_selenium_rc() throws Exception {
		selenium.open("/");
		selenium.type("q", "Selenium-RC");
		selenium.click("btnG");
	}
}
```

Resolve Import Errors

The first thing we need to resolve is the error on the first import statement.

```
import com.thoughtworks.selenium.*;
```

Import statements are how we tell Java about the methods and classes we are using from external packages.

If we hover over the red cross at the side of this line then Eclipse will tell us what the problem is.



Figure 6.14: Eclipse Explains the Error

In this case it is "The import com.thoughtworks cannot be resolved" which means that Eclipse does not know where to find this package because we haven't told Eclipse where the selenium library is yet.

We can fix this error automatically by clicking on the red cross to the left of the source code line.

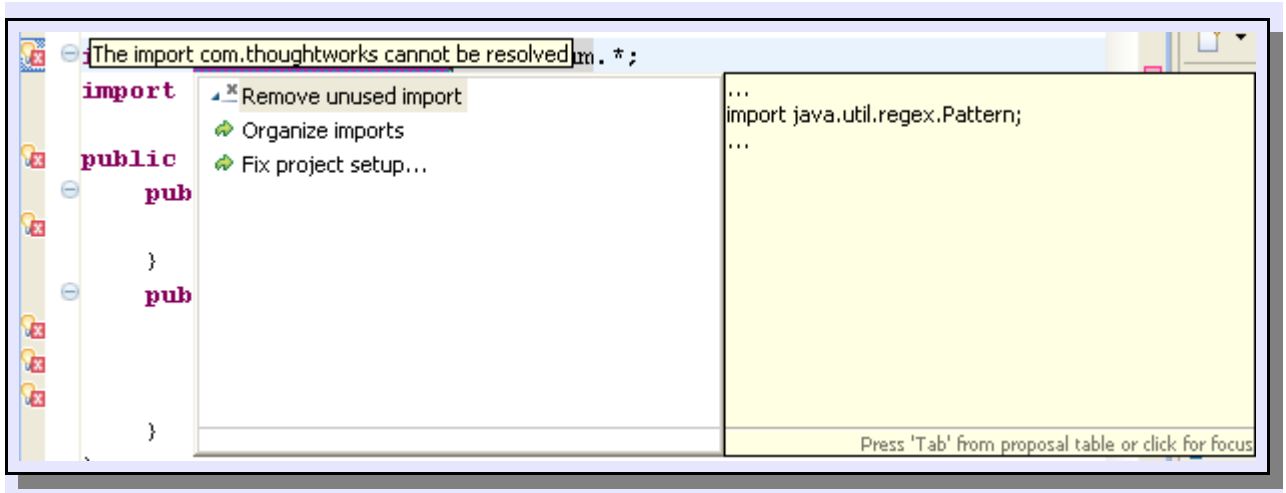


Figure 6.15: Try to Automatically fix the error

Double click on the "Fix project setup..." option and Eclipse will tell us that it doesn't know how to do that automatically.

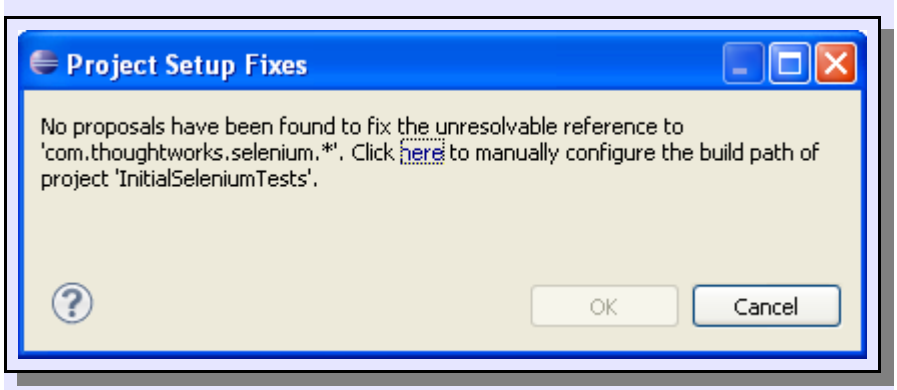


Figure 6.16: Manually fix the error

So we accept Eclipse's offer to allow us to manually configure the build path for the project. By clicking on the “here” in the dialog text “click here to manually configure the build path of project 'InitialSeleniumTests'.”

The 'build path' is how java knows what external libraries are used in this project. Java searches through those libraries looking at the different packages to find the classes and methods that we want to re-use from those libraries.

When we click 'here', we will see a subset of the properties of the project we are working on. We want to add a new library so click on the “Libraries” tab.

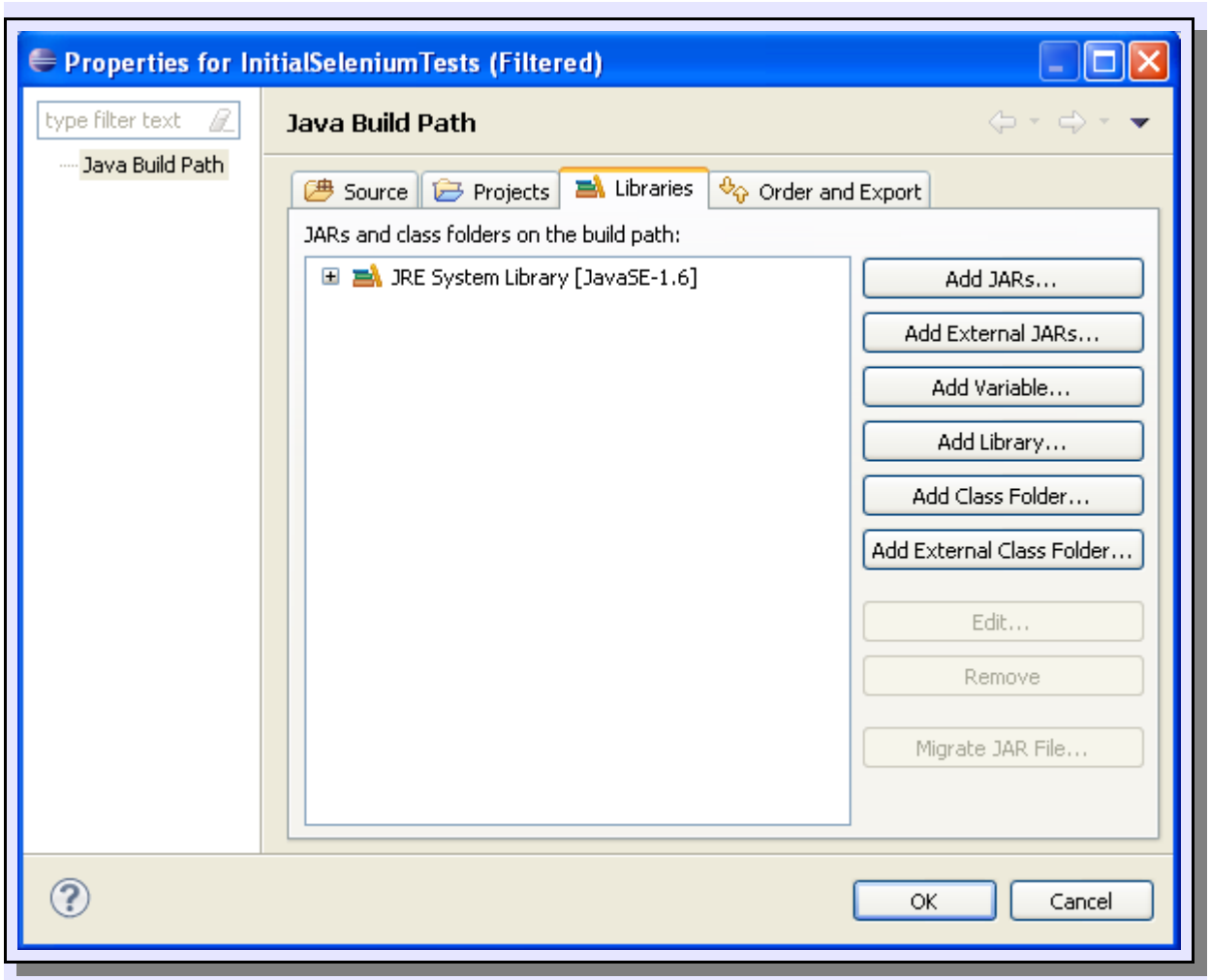


Figure 6.17: Basic properties for the default project

We want to amend the Java Build Path by pressing the [Add External JARs...] button.
Navigate to your Selenium directory, then into the 'java client directory'

C:\selenium\selenium-remote-control-1.0.3\selenium-java-client-driver-1.0.1

And select:

- selenium-java-client-driver.jar, and

Upon clicking [open] from the file dialog, That file should be displayed in the Libraries list now.

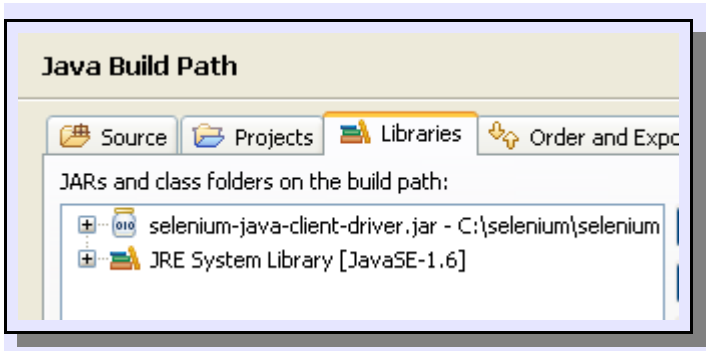


Figure 6.18: Client driver added to libraries

Click [OK] to exit the dialog.

The display of the test should now look better, with only one error symbol displayed by Eclipse.

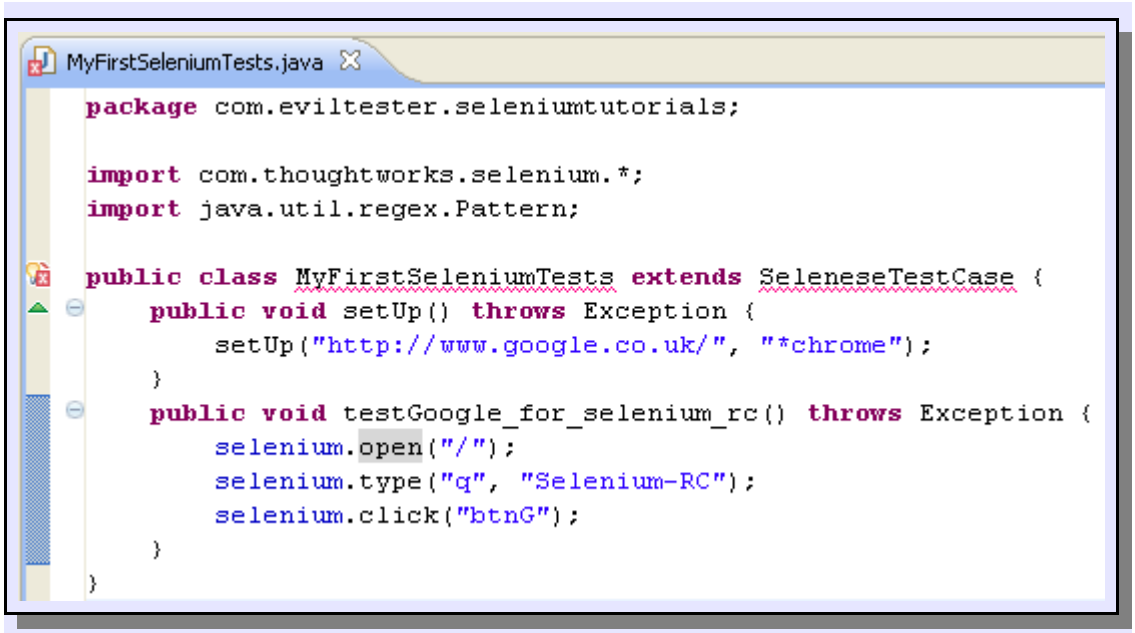


Figure 6.19: Only one error left

Add JUnit to the build path

As before, hover over the symbol to see what errors Eclipse reports.

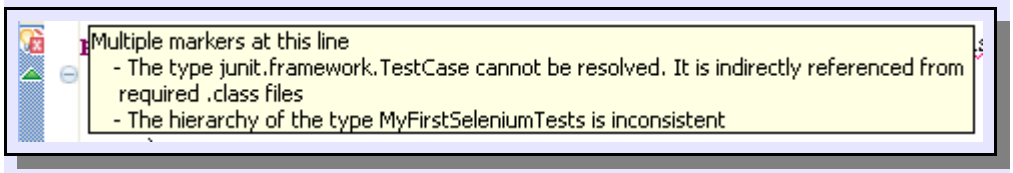


Figure 6.20: JUnit TestCase not defined

In the first error Eclipse reports that we haven't added JUnit to the build path.

Click on the error symbol and choose “Configure build path...” from the list of options.

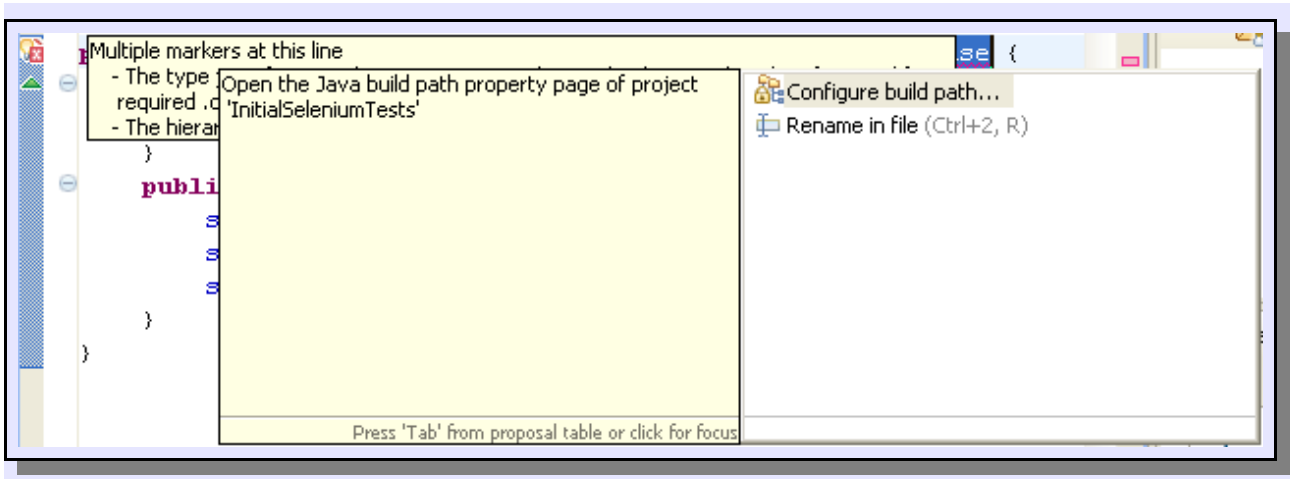


Figure 6.21: Configure build path for JUnit

This time we are going to choose the [Add Library...] button because Eclipse is distributed with JUnit. From the list of libraries presented, choose JUnit and click [Next >].

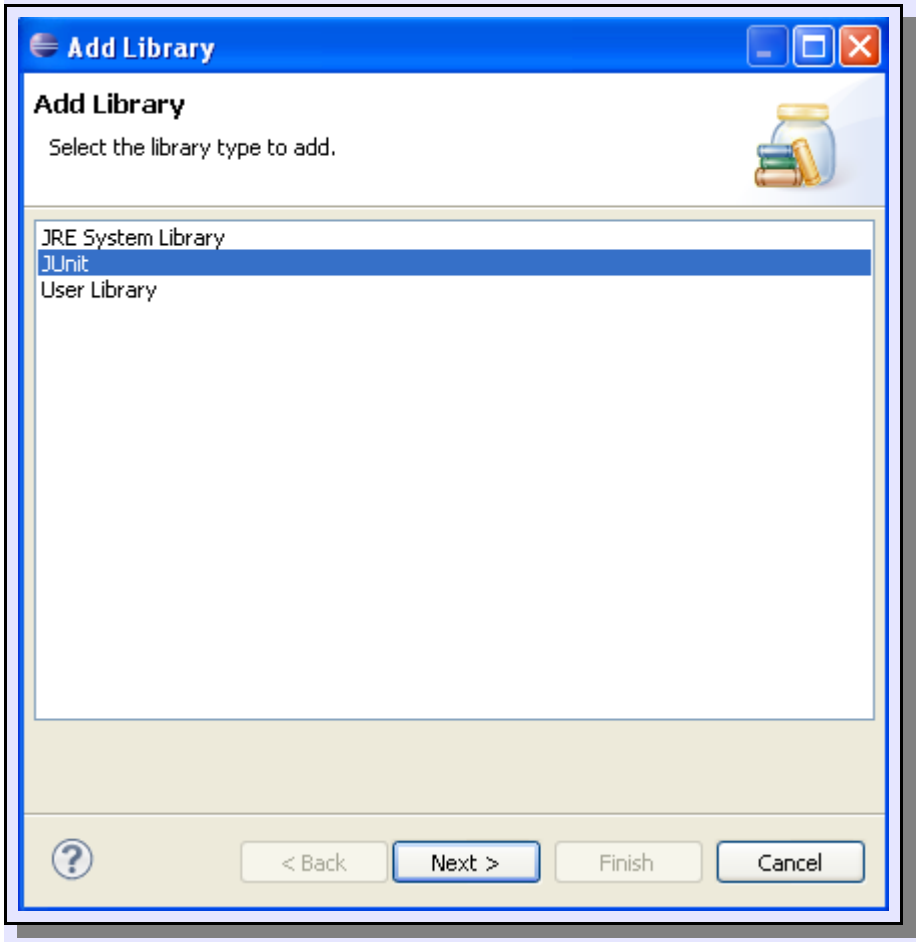


Figure 6.22: JUnit distributed as a standard library with Eclipse

From the list of JUnit library versions, choose “JUnit 4” and click [Finish].

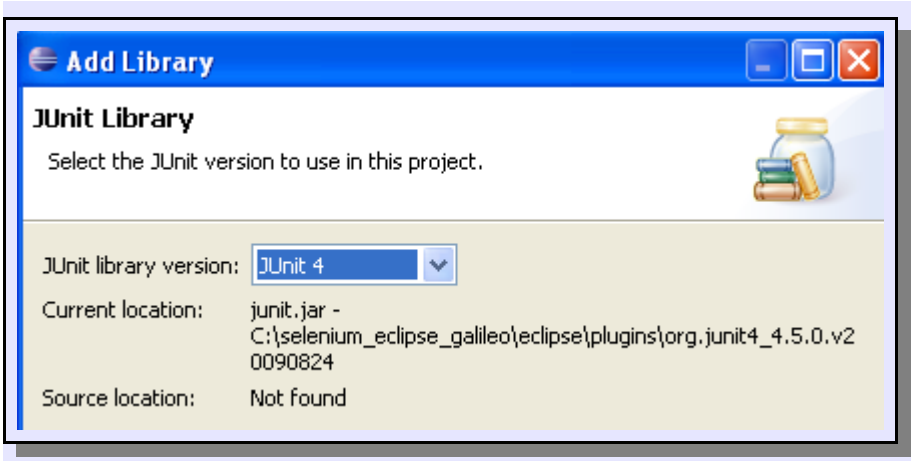


Figure Choose JUnit 4 from the drop down

Click [OK] on the properties dialog and you should see your test with a single warning symbol next to the "import java.util.regex.Pattern;" line.

Remove the unused import

If we hover over this warning then we will see Eclipse report that the imported classes are never used.



Figure 6.23: IDE creates unnecessary code

So we can safely use the option provided when we click on the warning symbol to remove the unused import.



Figure 6.24: Remove the line throwing a warning

And having done all this, our code should be clean and ready to run:

```
package com.eviltester.seleniumtutorials;
import com.thoughtworks.selenium.*;

public class MyFirstSeleniumTests extends SeleneseTestCase {
```

```
        public void setUp() throws Exception {
                setUp("http://www.google.co.uk/", "*firefox");
        }
        public void testNew() throws Exception {
                selenium.open("/");
                selenium.type("q", "Selenium-RC");
                selenium.click("btnG");
        }
}
```

Run the JUnit test

We run this by right clicking on the "MyFirstSeleniumTests.java" in the Package Explorer.

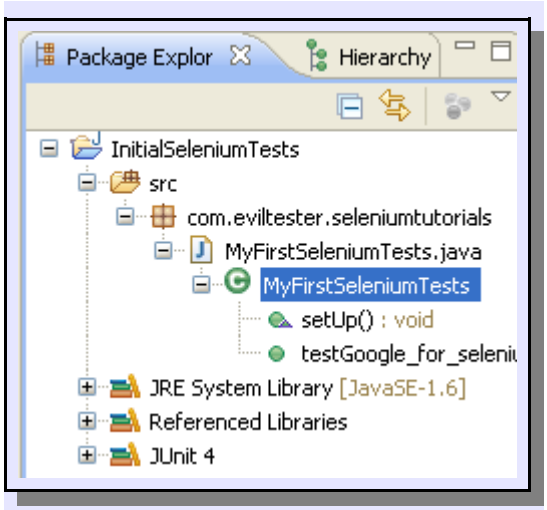


Figure 6.25: The Test in the Package Explorer

And selecting "Run as > Junit test".

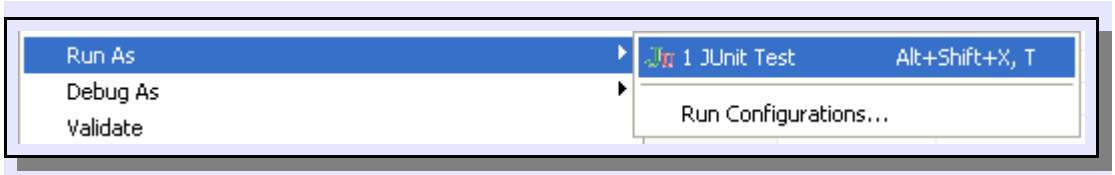


Figure 6.26: Run the test as a JUnit test

Try running it, remember first of all to start the Selenium-RC server as explained in earlier sections.

If you don't then Selenium client will display an error in the Eclipse console, asking if you have started the server.

Allow it through the Firewall

If you encounter any Firewall prompts when you run it then you should allow Eclipse or java access through your Firewall.

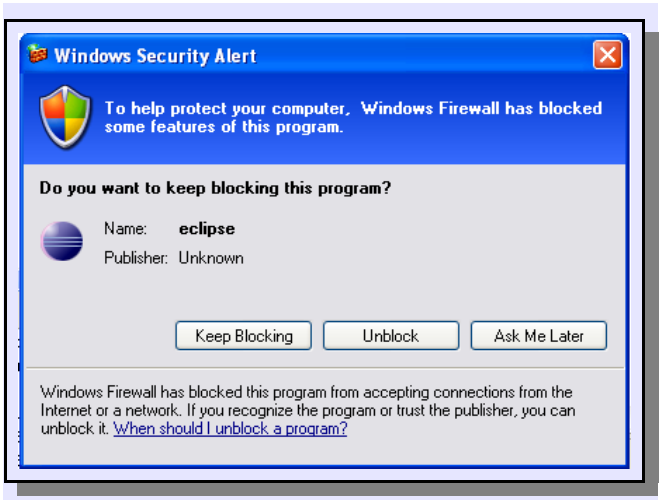


Figure 6.27: Allow Eclipse through the firewall

Seeing the test running

You should see the Selenium window appear. And underneath it the Google window.

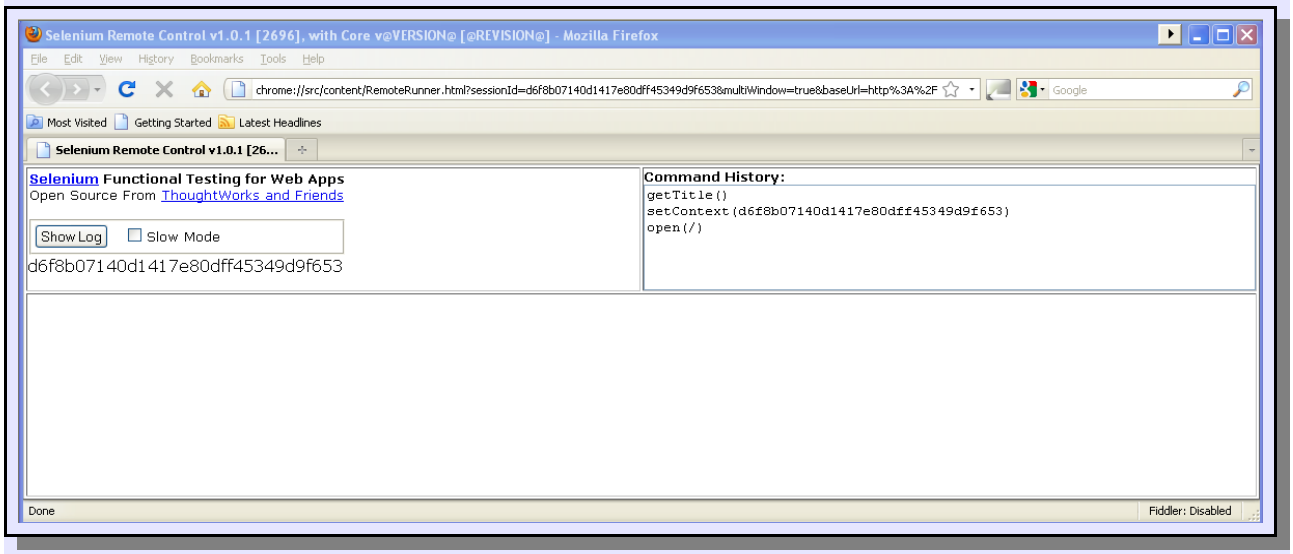


Figure 6.28: The Selenium window shows the commands and any errors that occur.

> **NOTE:**
>
> **If you didn't see this then you may have firewall problems (you could check by disabling the firewall while you try to run the test and see if that helps - if it does then you may need to do one or more of the following: open port 4444, add Eclipse, java and the browser to your exclusions list).**

In the Google window you can see the actual test execution happening. Of course since it is a small test it may have run too quickly for you to see the details. So we will cover that in the next major section after you check the results.

You should also have seen a bunch of messages appear in your console where you are running

Selenium Server.

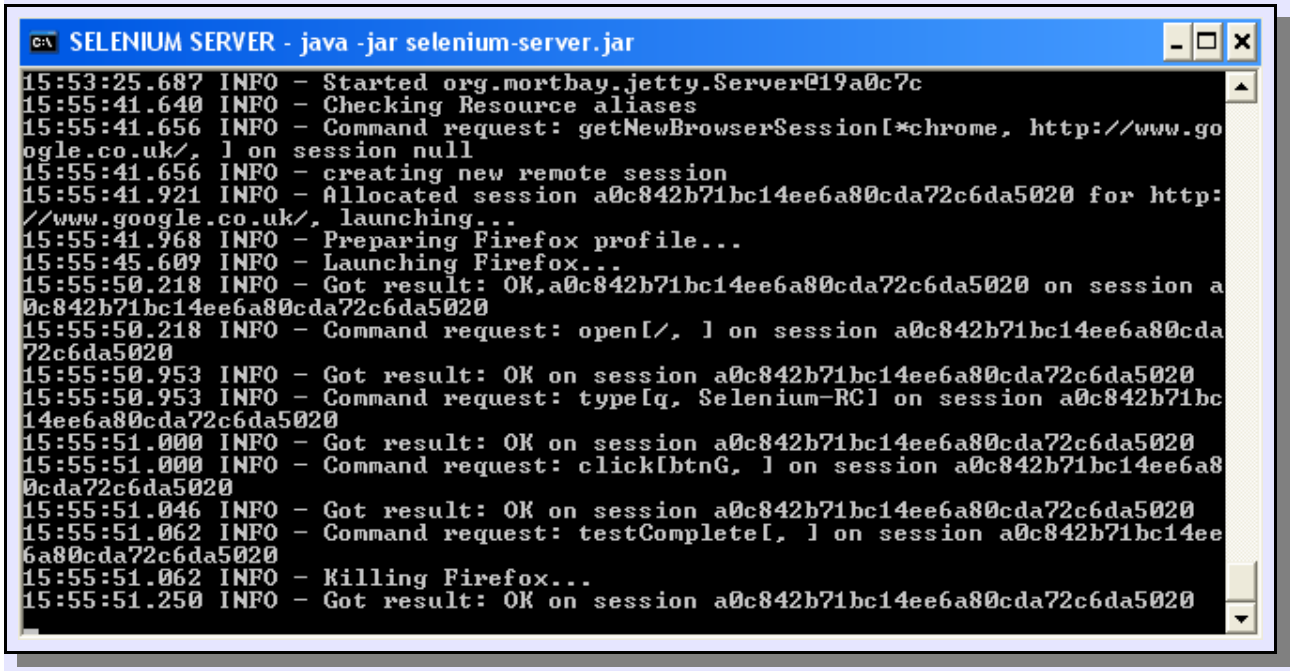


Figure 6.29: Messages from Selenium Server

From the console window you can gain a little insight into how Selenium works. Because we are using Firefox, Selenium creates a new Firefox profile for executing the test. This means that if we have any plugins etc in Firefox then they won't be available to the Selenium test. You can also see each statement in our test “open”, “type”, “click” sent through to the server as a 'Command Request' and Selenium-RC execute it within a session and return an OK result to the client.

The console can prove invaluable when investigating Selenium errors so read the output during your testing to help you continue to learn.

Check the results

After the run you should notice that the JUnit tab is now selected on the right.

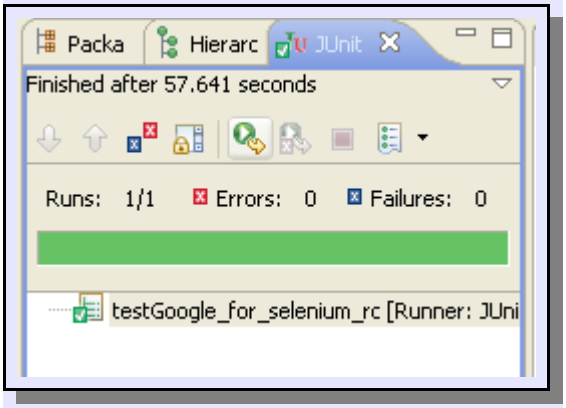


Figure 6.30: JUnit tab after a successful run

This reports that we ran 1 test, with no errors. And the green bar means that all our tests passed. If

the test had failed then the bar would be red and we would see the failing tests with a red cross next to them.

It went too quickly!

OK, we will now use the debugger to run the test slowly and see it in action.

Set a break point by right clicking on the left bar in Eclipse beside your source code on the line with the setUp command.

```
setUp("http://www.google.co.uk/", "*firefox");
```

Choose "Toggle Breakpoint" from the displayed popup menu.

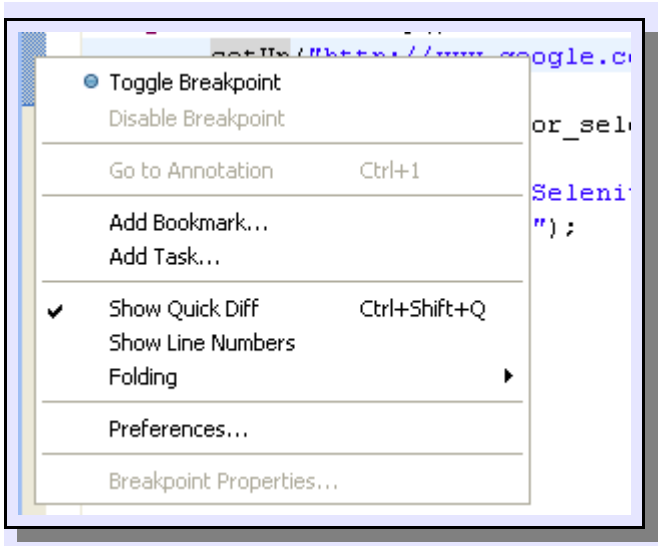


Figure 6.31: Toggle Breakpoint in the context menu

If you now see a round blue circle in the margin next to that line then it means you have set a breakpoint.



Figure 6.32: A line in Eclipse with a breakpoint set

The breakpoint means that when we run this test in debug mode. The execution of the test will pause when we hit that line and we can manually control the execution of the test.

Add another breakpoint to the line of code that reads:

```
selenium.open("/");
```

So you should see two breakpoints set in your code.

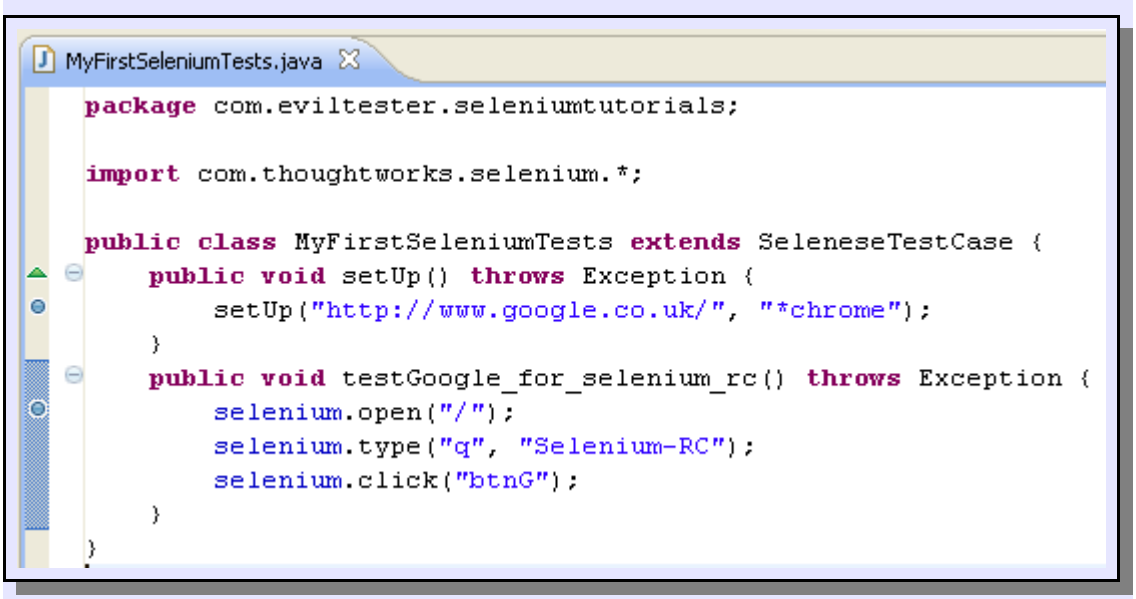


Figure 6.33: Two breakpoints visible in left info bar

Run the test in debug mode

Now we want to run the test in debug mode.

Switch back to the Package Explorer view by clicking on the Package Explorer tab.

And instead of right clicking on the test class and choosing "Run As\ JUnit Test" we want to choose "Debug As \ JUnit Test". Or select the test class name in the Package Explorer and press Alt+Shift+d, followed by 't'.

The first thing you may notice is a "Confirm Perspective Switch" dialog asking if you want to launch the Debug view. Click the "[x] remember my decision" option and choose [Yes].

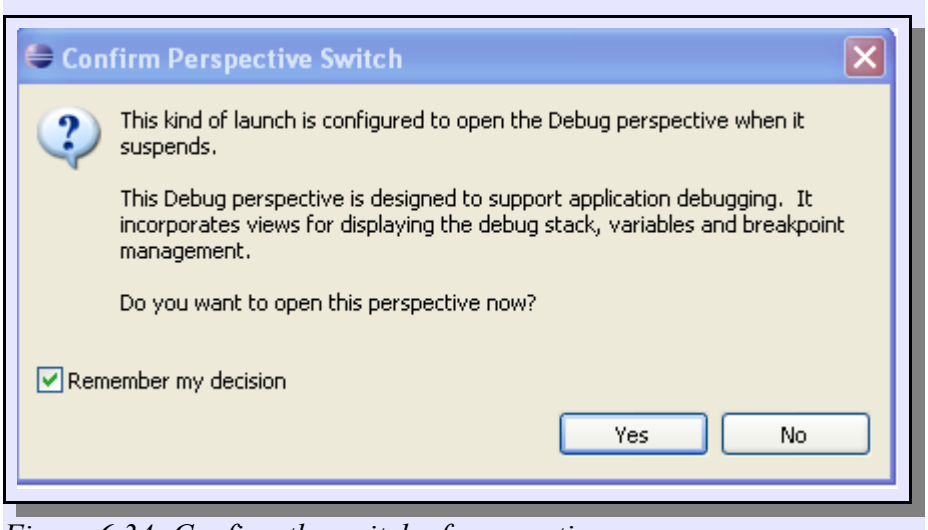


Figure 6.34: Confirm the switch of perspective

You should then find yourself in a new window layout of Eclipse. The Debug perspective.

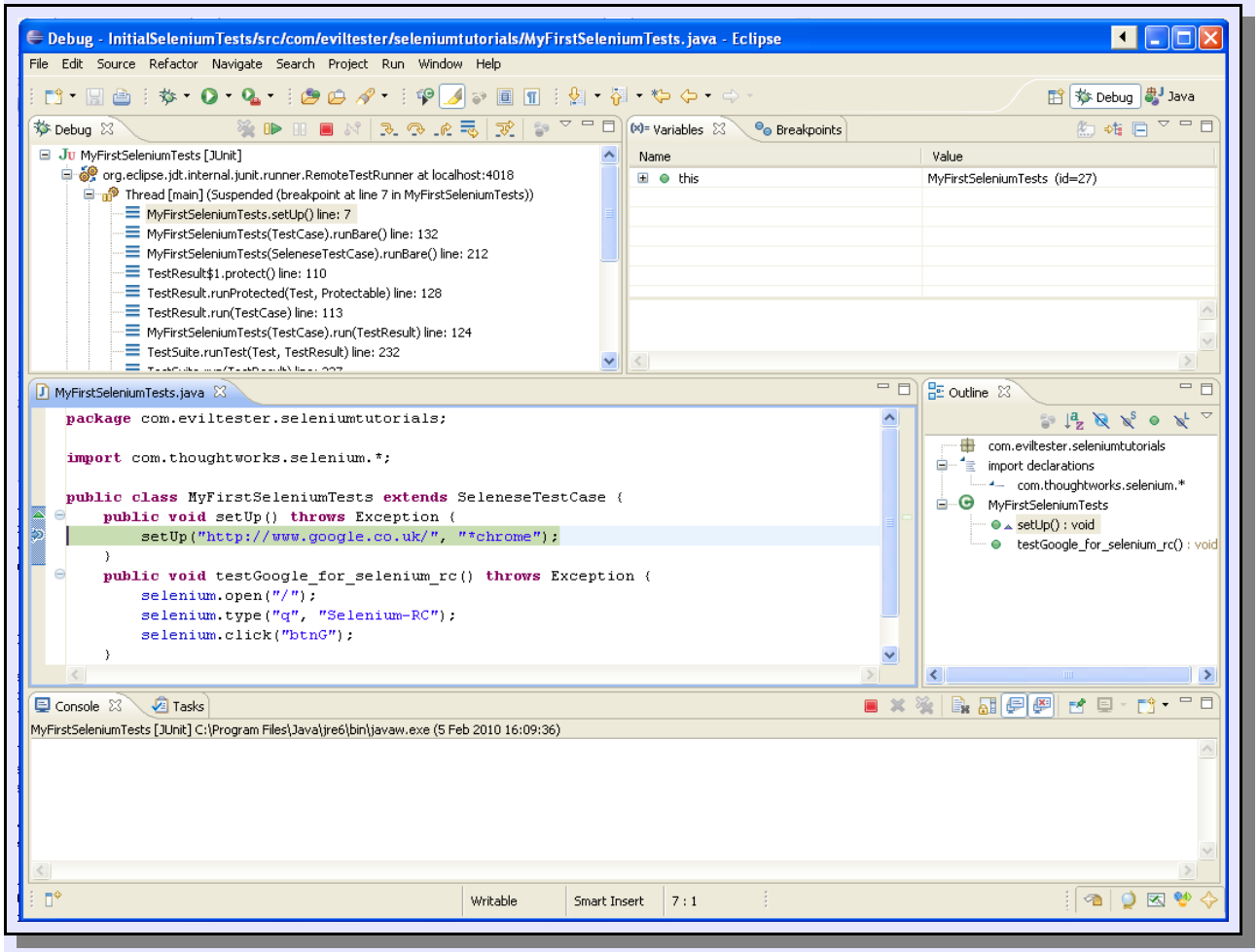


Figure 6.35: The Debug Perspective

We will use the toolbar to navigate through the code.



Figure 6.36: The debug toolbar

Each of the buttons has a shortcut key equivalent. Hover the mouse over the icon to learn what function each one has. We will only use two in this tutorial:

-  Step Over (F6)
-  Resume (F8)

Your test should have stopped executing on the **setUp("http://www.google.co.uk/", "*firefox");** line. And this should be highlighted green.

This line has not executed yet.

This code was provided by the selenium-server jar in the SeleneseTestCase class. We will look into the class in more detail later, so for now all you really need to know is that this will setup our Selenium session and start the browser for us. We will step over this line by pressing F6.

"Step Over" means that it will execute in the debugger but we will not debug through all the lines

of code that were written to make 'setup' happen.

When we step over the code, either by pressing the F6 key, or clicking on the icon, the "Selenium Remote Control" window should have been displayed. Eclipse now highlights the closing brace in the setUp method in green because this is the next line to be executed.

If you notice the URL in the execution browser you can see that it hasn't visited google yet and still has a selenium-server command listed:

- http://localhost:4444/selenium-server/core/Blank.html?start=true

Press "Step Over (F6)" again and you should see the Google homepage appear in your "Selenium Remote Control" window.

Press "Step Over (F6)" again and you will see a Class File Editor showing TestCase.class. This is a JUnit class and we haven't associated any source code. This is not an error. We can still debug through the test. This is why we set the second breakpoint.

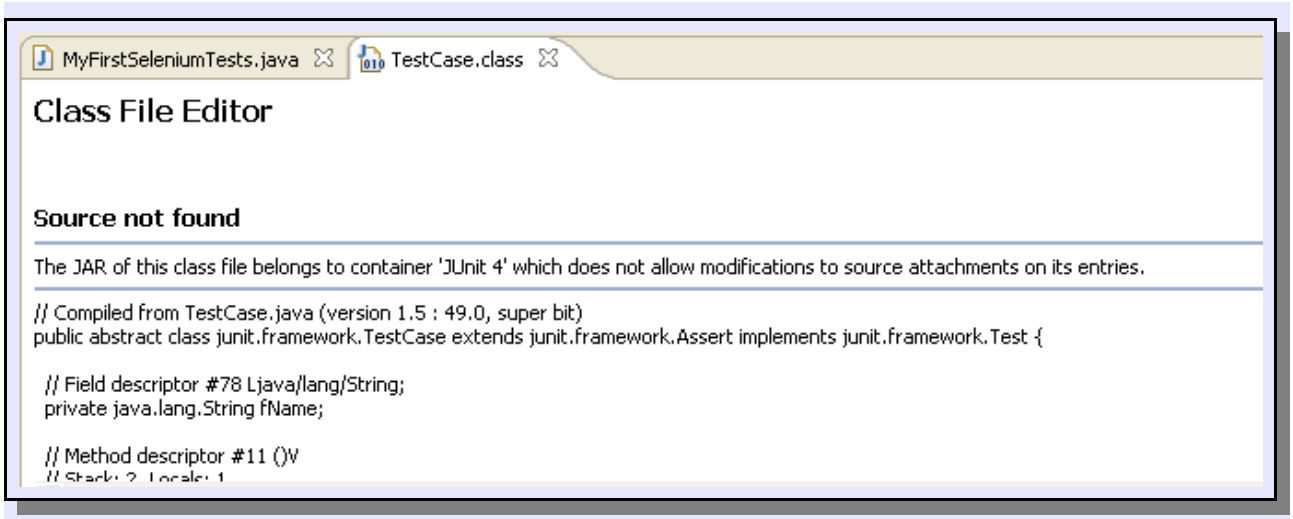


Figure 6.37: JUnit TestCase class code not found

So click the resume button or press F8. And this will execute the code to get to the next line that we are interested in seeing execute under the debugger.

Now you should see the selenium.open("/"); line selected in green.

Press "Step Over (F6)" and you should see Selenium execute that command and open Google in the window below.

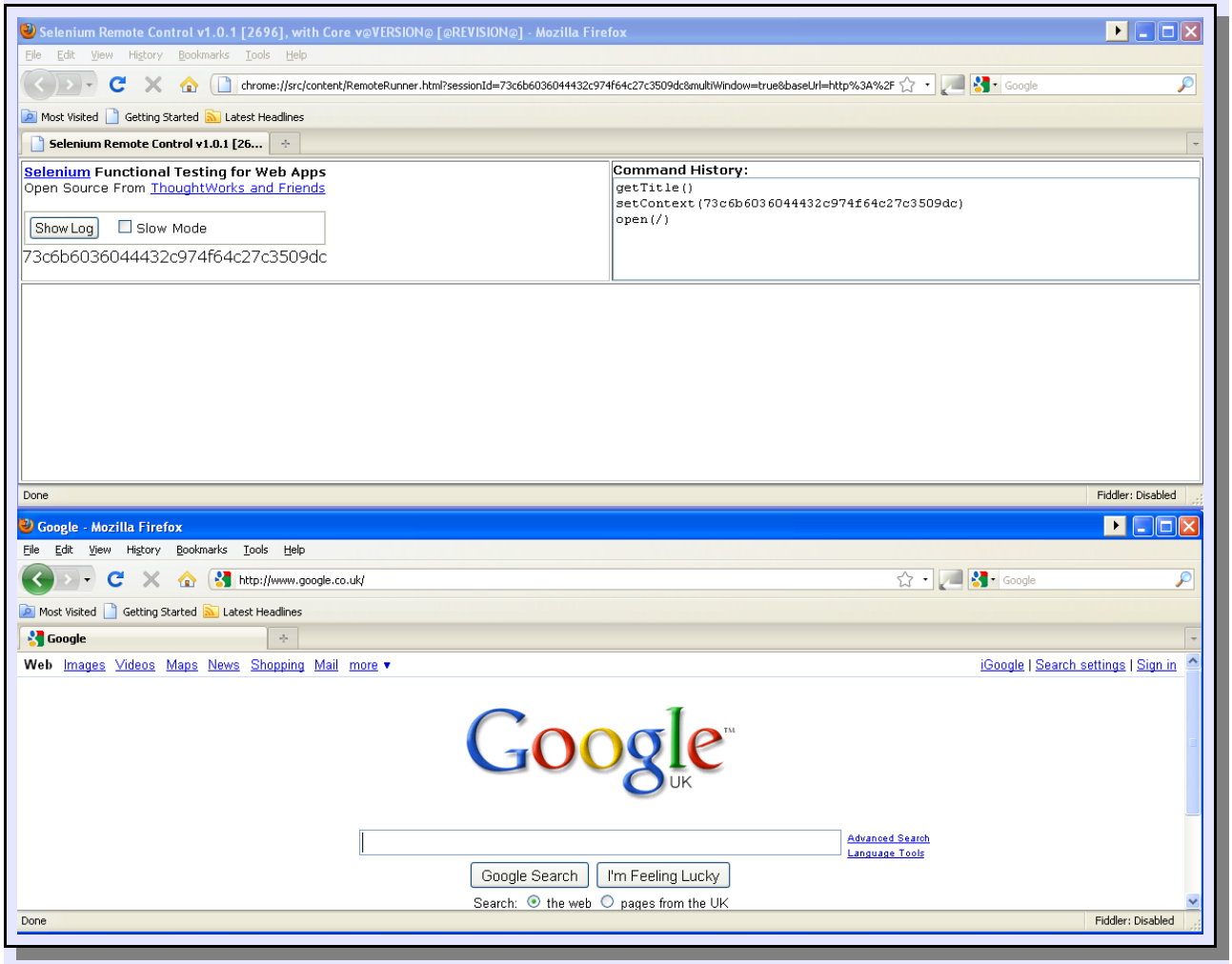


Figure 6.38: The test running

Keep pressing "Step Over (F6)" to watch the test execute and when happy that you have seen enough press “Resume (F8)” to skip over the rest of the test code to finish the test.

Click the [Java] perspective button in the top right to go back to your more familiar Package Explorer view.



Figure 6.39: Java Perspective Button

Create and Import some more IDE Converted tests

Just to make sure that we know how to import tests and you can see how easy it will be in the future now that we have done the hard work of setting up the build path.

We will now record and import another test into this class.

Start up Firefox and the Selenium IDE.

Make sure record is on.



Figure 6.40: Record is on

Make sure Options\Format is set to HTML.

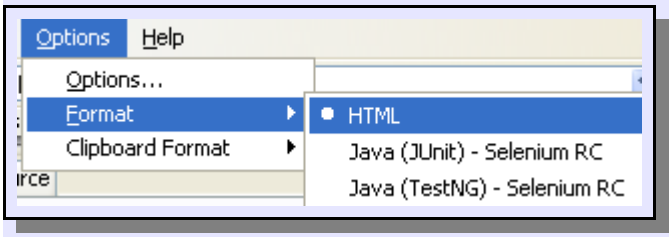


Figure 6.41: Format set to HTML

And that you have the Table view showing.

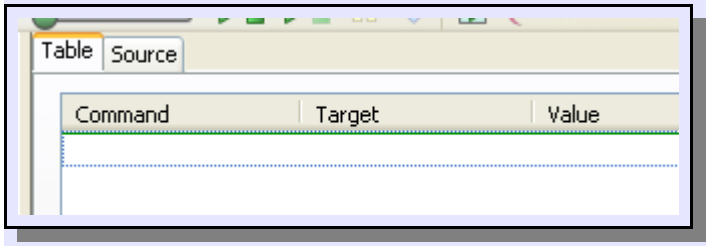


Figure 6.42: Table view of Selenese HTML format

This time record actions where you:

- type in "evil tester" to the search box
- click on "Images"
- and then perform a Google Search by pressing the "Search Images" button.

You should have a test like the following:

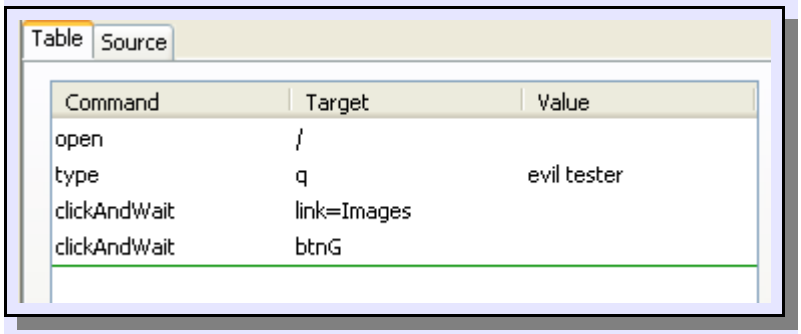


Figure 6.42: Recorded Script in Selenese Table view

Run the test again to make sure you have created something repeatable by clicking the Play button.

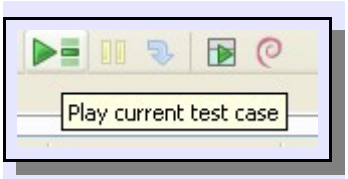


Figure 6.43: Play button on left

And if that worked, then we can change the format to "Java (JUnit) – Selenium RC" and import it into Eclipse.

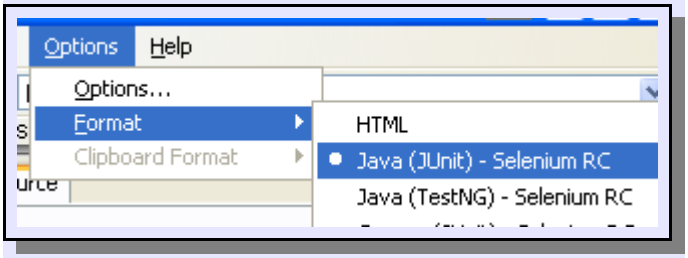


Figure 6.44: Select Java (JUnit) format

But this time, all we want to copy into Eclipse is the 'test' code i.e.

```
public void testUntitled() throws Exception {
        selenium.open("/");
        selenium.type("q", "evil tester");
        selenium.click("link=Images");
        selenium.waitForPageToLoad("30000");
        selenium.click("btnG");
        selenium.waitForPageToLoad("30000");
}
```

Paste that code into Eclipse beneath the existing test and before the closing brace "}" of the class.

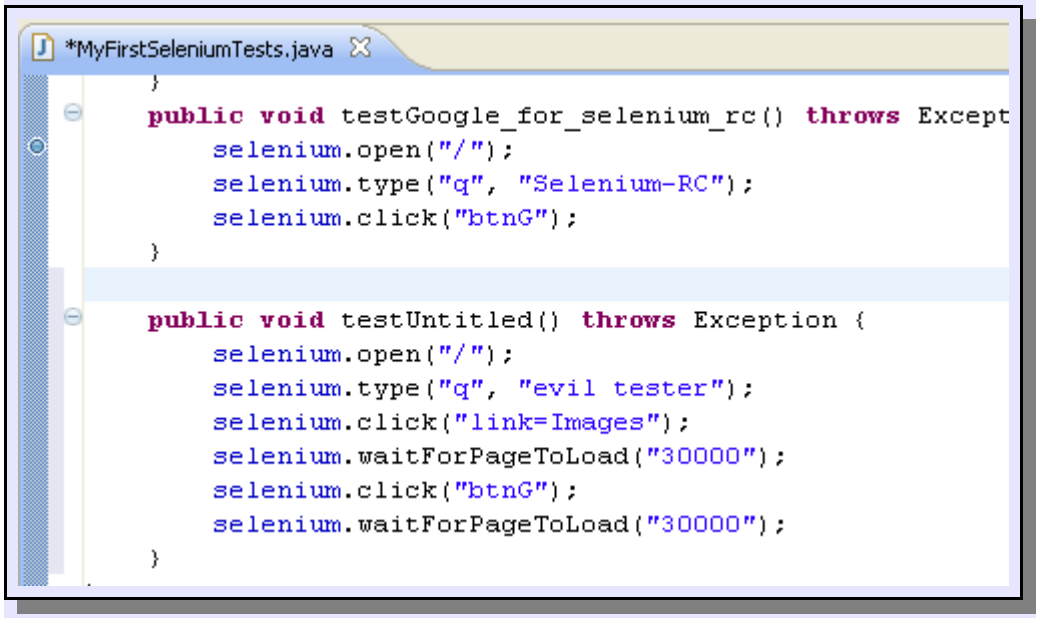


Figure 6.45: Just the method code pasted into Eclipse

Then rename the test. i.e. Change “testUntitled” to “testGoogleForEvilTesterImages”

```
public void testGoogleForEvilTesterImages() throws Exception
    selenium.open("/");
```

Figure 6.46: The renamed test in Eclipse

Click on the class and choose “run \ JUnit test”, then Eclipse will run all the tests in that class.

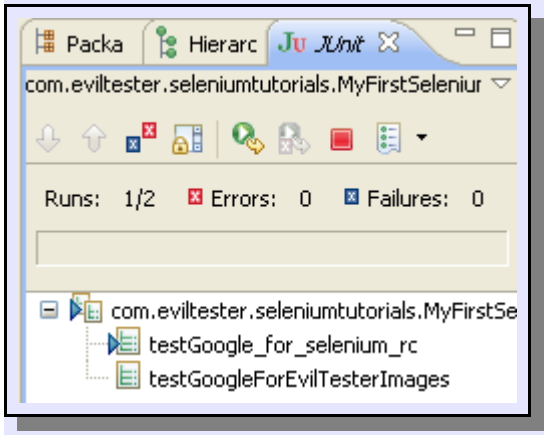


Figure 6.47: JUnit running both tests

You should now have enough information to record your own tests and add them into either the existing class, or create new classes to house them.

Hopefully you can see that we can easily use the IDE to generate the basics for simple tests. In a later chapter we will see how to identify the locators for the .type and .click methods, and how you can learn to write tests like this by hand.

Learning to write these tests by hand is important because you well learn how the application hangs together and to write robust tests.

For quickly generating basic tests, copying code in from the IDE can be very useful.

In practice I use the IDE to help capture certain actions that I want to do, but may be having difficulty figuring out the exact command sequence, or frames that controls are embedded within.

Exercises

Instructions: Create a new class called MyFirstSetOfExercises. This is where you will create some more tests from the IDE and paste them in. You should try and create all tests in the IDE and then simply change the format to JUnit and paste them into Eclipse.

Exercise:

1. search for an image, assert title of page,
2. add an assert that a specific image is present
3. change the value of the search to see the test fail
4. use verifys instead of asserts

Chapter 7: The Annotated Generated Test

Creating the test from the IDE was simple, but there is still quite a lot to learn about the automatically generated test, as it hides a lot of complexity from us that we need to understand to get the best from Selenium.

As we look at the generated test we will learn a lot about Java, Object Oriented programming and how Selenium works. We will put that to good use in the next chapter where we create tests without using the SeleneseTestCase class and in this way we will take complete control of our Selenium test cases.

Use Attach Source to see the Selenium Driver Source Code

The first thing we will do is configure Eclipse to let us see the source code for the Selenium Driver.

So in Eclipse, highlight the text "SeleneseTestCase" (you can do this by double clicking on the text).

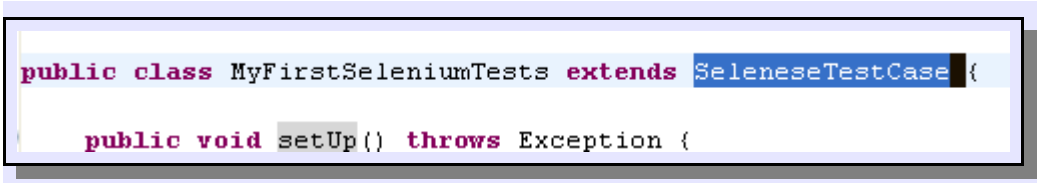


Figure 7.1: Highlighted SeleneseTestCase

Then either:

- right click and choose "Open Declaration" from the context menu or
- press F3 – the shortcut key to Open Declaration

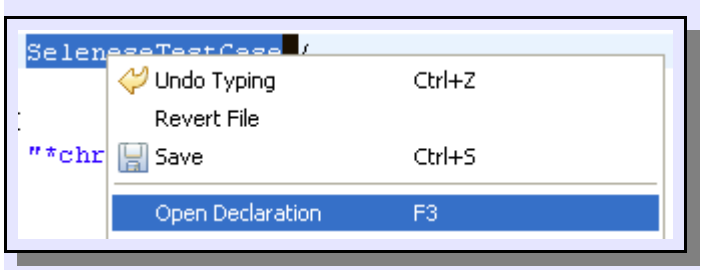


Figure 7.2: Open Declaration from Context Menu

"Open Declaration" is a very useful function in Eclipse to take you to the implementation source code for the source item you selected. In this case we want to see the source code for the Class declaration of SeleneseTestCase.

You should see a Class File Editor window, with the message "Source not found". This is because we haven't told Eclipse where the source code for the Selenium Java Driver resides on the disk.

We can easily fix this by pressing the [Attach Source...] button and navigating to the selenium-java-client-driver-sources.jar.

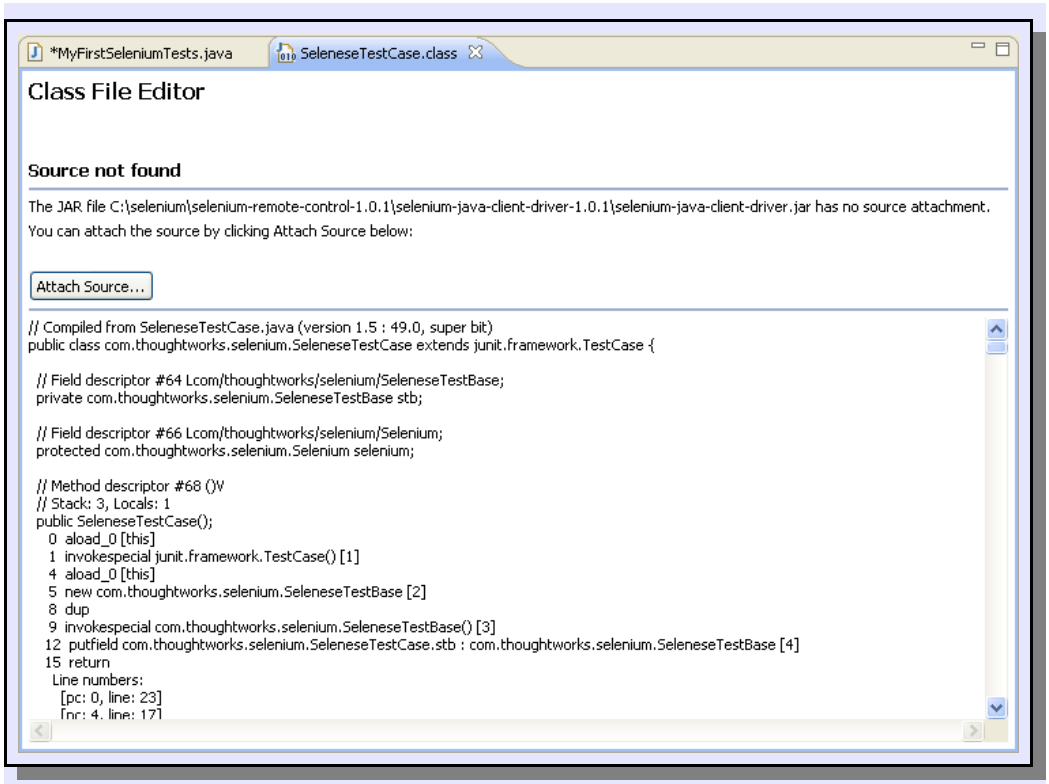


Figure 7.3: Missing Source Attachment

Click the [Attach Source...] button. And the "Source Attachment Configuration" dialog appears.

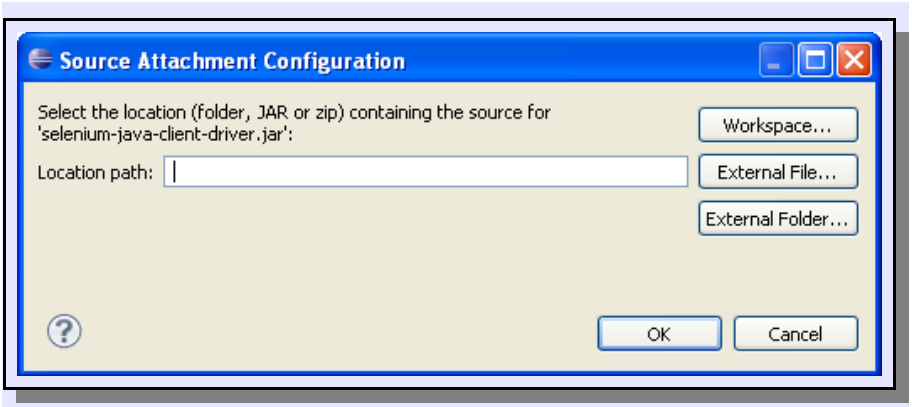


Figure 7.4: Chose a source Jar file to associate with this class

We want to choose a location path by clicking on the [External File...] button.

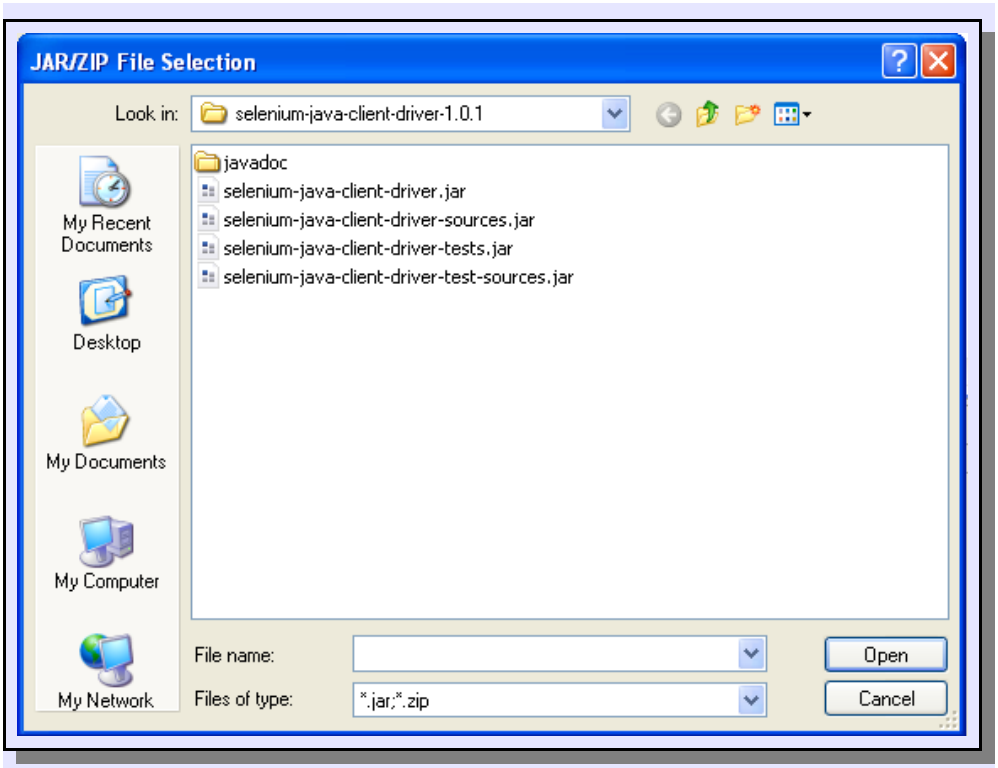


Figure 7.5: The Client Driver folder

Using the File Selection dialog that appears. We want to navigate to the directory where we extracted the Selenium Java Client Driver when we installed Selenium.

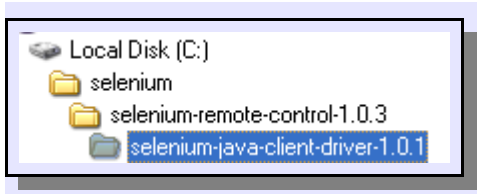


Figure 7.6 : Selenium Java Client Driver folder

In my case this was C:\selenium\selenium-remote-control-1.0.3\selenium-java-client-driver-1.0.1

The sources for the client are stored in "selenium-java-client-driver-sources.jar", so click on this file to attach it.

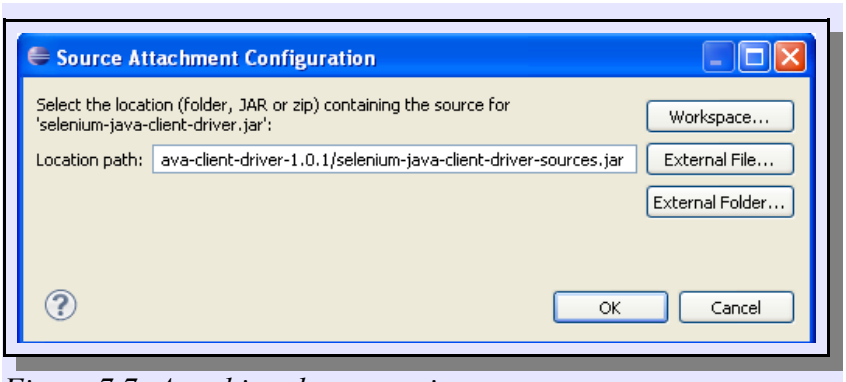


Figure 7.7: Attaching the source jar

Then click OK, and you should see the display change from the "Class File Editor" to a normal source code editor, the only difference being that the source code is not editable because you are using compiled source in a .jar file, rather than a text file of source code.

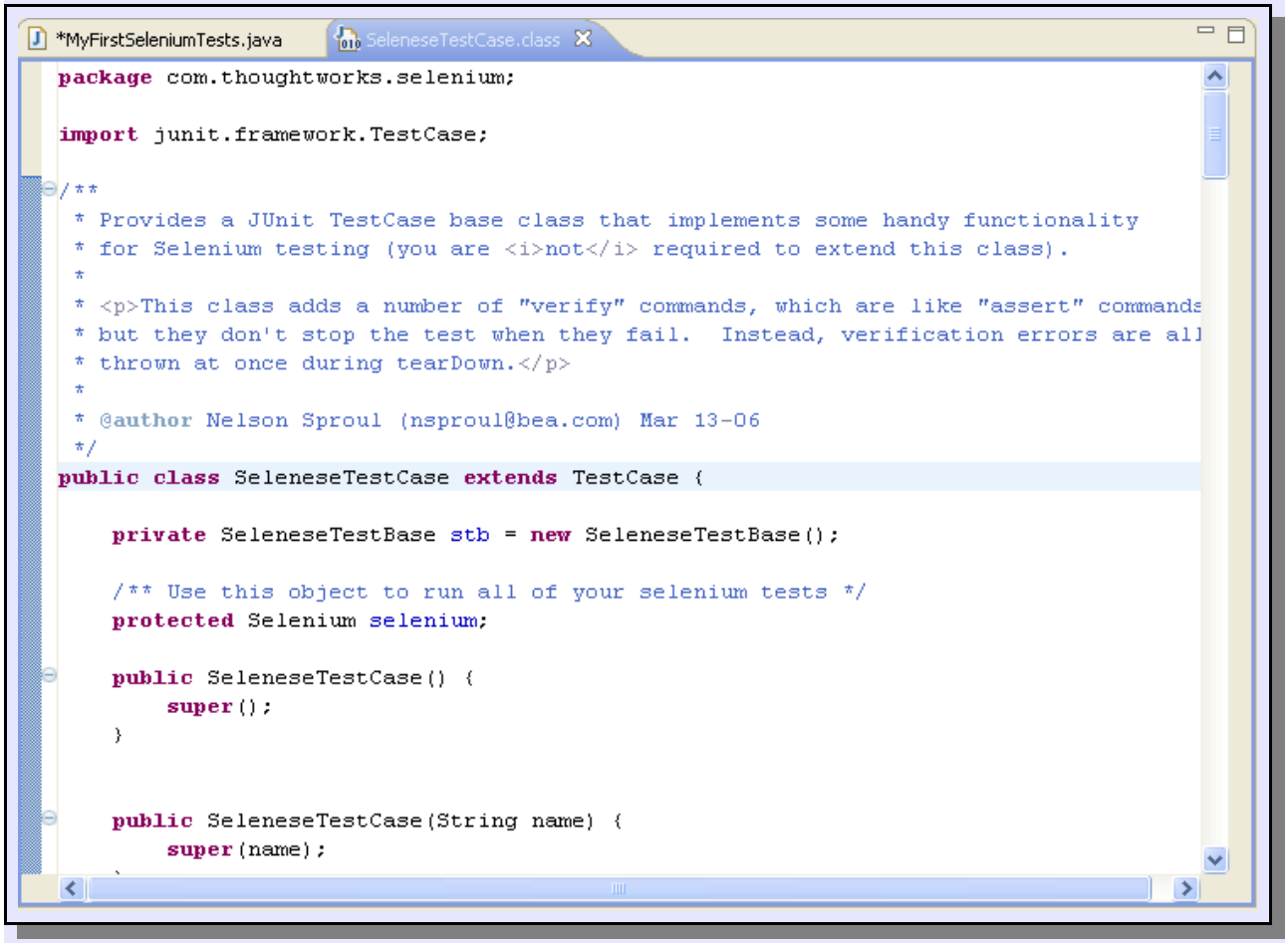


Figure 7.8: Source Code can now be displayed

Why should we do this?

With Eclipse setup like this, we can start to understand the code that we are using. It is very important to understand as much as possible about the libraries and classes that you are using in your testing. You can start to understand the limitations or restrictions of these classes so you know what you have to build yourself, and what comes for free in the short term to help you get started. If you are just learning programming then you have a whole set of relevant source code to study to look at how other people have used and built the tools. Reading source code is one of the easiest ways for a novice programmer to advance their programming skills.

MyFirstSeleniumTests.java Annotated

Note: for a good presentation of Java terminology read the official Java documentation on "Learning the Java Language". In this text we will only provide an introduction to the basic concepts.

http://java.sun.com/docs/books/tutorial/java/index.html

First, lets start with our tests. We will work through on a line by line basis to understand the code, and drill down into lower levels of the code as necessary.

MyFirstSeleniumTests.java | SeleneseTestCase.class

```
1  package com.eviltester.seleniumtutorials;
2
3  import com.thoughtworks.selenium.*;
4
5  public class MyFirstSeleniumTests extends SeleneseTestCase {
6
7      public void setUp() throws Exception {
8          setUp("http://www.google.co.uk/", "*chrome");
9      }
10
11     public void testGoogle_for_selenium_rc() throws Exception {
12         selenium.open("/");
13         selenium.type("q", "Selenium-RC");
14         selenium.click("btnG");
15     }
16
17     public void testGoogleForEvilTesterImages() throws Exception {
18         selenium.open("/");
19         selenium.type("q", "evil tester");
20         selenium.click("link=Images");
21         selenium.waitForPageToLoad("30000");
22         selenium.click("btnG");
23         selenium.waitForPageToLoad("30000");
24     }
25 }
```

Figure 7.9: The listing with line numbers to aid detailed explanation

```
1 : package com.eviltester.seleniumtutorials;
```

- First we put the "package" declaration. The package declaration is the first statement in the source code file. If we don't add a package statement then the class goes into a default "unnamed" package. Packages are a way of grouping classes logically as multiple classes in different files can all be declared as belonging to the same package. Using packages allows us to 'import' them later for reuse in different classes, packages and applications.
- The package declaration terminates with a ";". All statements in Java must end with a ";" as the programmer must tell the compiler where the statement ends.
- Packages are covered in the Java documentation http://java.sun.com/docs/books/tutorial/java/package/index.html

```
3 : import com.thoughtworks.selenium.*;
```

- Because we want to use classes in the Selenium Java Client Driver, we tell Java to look in the com.thoughtworks.selenium package to find the classes we are using .e.g. SeleneseTestCase. We could miss this line out, but then we would have to fully qualify our use of SeleneseTestCase in line 5 to read "`extends`

`com.thoughtworks.selenium.SeleneseTestCase"` by using import statements we can keep our code cleaner and easier to read.

- Import statements in Java Documentation http://java.sun.com/docs/books/tutorial/java/package/usepkgs.html

```
5 : public class MyFirstSeleniumTests extends SeleneseTestCase {
```

- For our test code, you can think of Java Classes as collections of tests and helper code to make our tests work. This isn't a particularly good definition in terms of understanding Object Oriented Programming. But in terms of the way that we will use them for our automated tests, this definition will get you started.
- Our test classes are declared as public so that they can be run as part of our test suites. And this particular class "extends" SeleneseTestCase. Object Oriented Programming has a concept of 'inheritance' where a class can inherit methods and variables from other classes. This allows reuse of code. So by extending the SeleneseTestCase distributed as part of the Selenium Java Client Driver, we get access to various source code to help us write our tests. We will explore this class in more detail later.
- This class declaration line ends with "{". Scope in Java is declared explicitly by putting everything within a pair of "{" and "}". So the scope of the class, or everything declared as part of this class, is contained between the opening "{" on line 5 and the closing "}" on line 25
- Classes in Java http://java.sun.com/docs/books/tutorial/java/javaOO/index.html
- Inheritance (extends) in Java http://java.sun.com/docs/books/tutorial/java/concepts/inheritance.html

```
7 : public void setUp() throws Exception {
```

- On line 7 we declare a new method called setUp and we are telling Java that under some circumstances it may throw an exception. This means that if something goes wrong in the setup then an exception might be generated rather than the application handling all the problems itself. We will cover exceptions in a later section. But be assured that if you missed this out, Eclipse would prompt you that it was missing.
- You can read about exceptions in the official Java documentation http://java.sun.com/docs/books/tutorial/essential/exceptions/
- Again the scope of the method is delimited by "{" and "}" so this method is all the code in lines 7,8 and 9

```
8 :     setUp("http://www.google.co.uk/", "*firefox");
```

- Line 8 'calls' the method setUp that is declared in the SeleneseTestCase. We will look at this method in more detail in the next section to learn what it does, but as a summary. This method contains all the code for connecting to the Selenium-RC server, starting the Selenium session and starting the "*firefox" browser which is the Selenium browser code for Firefox.
- There are a number of browser codes in Selenium: e.g. "*firefox", "*iexplore", "*firefox"

and they all start the browsers in different modes. See Chapter 33 “Run the tests on multiple browsers” section “Browser Codes” on page 286.

- setUp is also a standard method of JUnit, which we are using as our test control framework. This method runs before each of the tests. So when we run MyFirstSeleniumTests.java as JUnit tests, what actually gets executed is the following:
  1. setUp
  2. testGoogle_for_selenium_rc
  3. setUp
  4. testGoogleForEvilTesterImages
- You can verify this for yourself by putting a breakpoint on lines 8, 12 and 18 and running the test in debug mode. And at each breakpoint 'resuming' the test. Note: there is no guarantee that the test* methods will be run in that order so you may see testGoogleForEvilTesterImages be executed first, but each test* method will be preceded by a setUp method.

```
9 : }
```

- Line 9 ends the scope of the setUp method started on line 7

```
11 :  public void testGoogle_for_selenium_rc() throws Exception {
```

- The easy way of making sure that a class method is run as a JUnit test, is to begin the name with 'test'. So this is why we prefixed our method name with test. We will cover more advanced ways of using JUnit later, but for now, simply prefix your test method names with test and they will run as a JUnit test.
- We declare test methods as “public void” so that the JUnit test runner can execute them.
- And because we are not 'handling' any exceptions that can be thrown by the selenium commands in this method we declare that we may throw an exception.

```
12 :   selenium.open("/");
```

- This is our first selenium command in the test. Here we are calling the 'open' method on an object called 'selenium'. Now, nowhere in our code did we declare an object called selenium. We inherited this from the SeleneseTestCase class, and it was instantiated in the setUp method. So we will learn more about this object in the next section.
- This is why we start by using the SeleneseTestCase class, as some of the basic setup work is done for us.
- The open method takes a parameter of the page name to open. Because in the setUp method we put in a fully qualified URL e.g. “http://www.google.co.uk/” if we pass in a relative URL in the open method, Selenium knows that we want to prefix this with the base URL so we are actually opening the home page of Google.

```
13 :   selenium.type("q", "Selenium-RC");
14 :   selenium.click("btnG");
15 : }
```

- I'm going to start explaining blocks of text now, otherwise I'll end up repeating myself, because we have covered all the complicated parts already.
- Again, on line 13, we use a selenium method. This time the 'type' method. This takes two

parameters. The first is a locator, the second is the text to type. So we are telling selenium to type “Selenium-RC” into the field named “q”.

- On line 14, we use a selenium method 'click' which simulates a click on the button named 'btnG'
- On line 15 we end the scope of the test method

```
17 : public void testGoogleForEvilTesterImages() throws Exception {
18 :    selenium.open("/");
19 :    selenium.type("q", "evil tester");
20 :    selenium.click("link=Images");
21 :    selenium.waitForPageToLoad("30000");
22 :    selenium.click("btnG");
23 :    selenium.waitForPageToLoad("30000");
24 : }
25 : }
```

- There is very little new in this section that we haven't covered.
- On line 20 we prefix the selector to look for a 'link' with the text “Images”
- On lines 21 and 23 we use a selenium method 'waitForPageToLoad' this takes a single parameter which is a timeout value in milliseconds. Which means that selenium will wait for the page to load and if it hasn't loaded within 30 seconds then it will throw an exception and the test will fail.
- Line 24 ends the scope of the test method
- Line 25 ends the scope of the class

Hopefully you have a better idea about what all the lines of code in the test do, so you are a step closer to learning Java. You will learn more Java throughout this book, but the Java tutorials on the Sun website are a particularly good free online reference.

I included links to relevant portions in the text above but the main web page is http://java.sun.com/docs/books/tutorial/

In this book I will not overload you with the 'exact' definitions of all the pieces of Java and Object Orientation that we cover. I will provide enough information for you to understand the tests you are writing and to help you think about what you are doing. Hopefully the information here will allow you to approach the more detailed explanations in other programming books more easily and understand their more detailed presentation.

A little more about selenium-java-client-driver.jar

The SeleneseTestCase that we are about to look at is contained in the selenium-java-client-driver.jar that we added as a library to our project.

You can easily view the contents of that .jar file (.jar = Java Archive) from your package explorer in Eclipse.

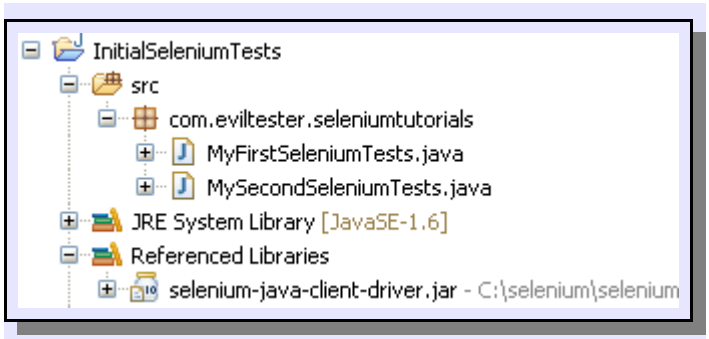


Figure 7.10: The Client Jar in the Package explorer

You can expand the contents of the jar so that they are displayed in the package explorer.

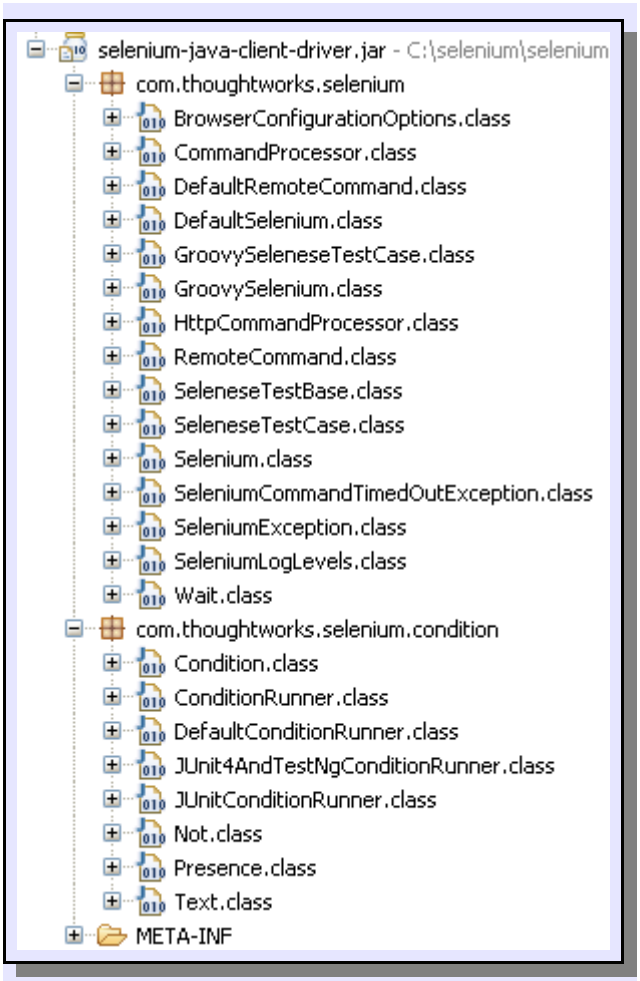


Figure 7.11: Client Driver Expanded

Here you can see a whole bunch of classes. We will discuss some of these naturally as we walk through the document. But all of these classes are available for you to use in your tests, when you look at your test needs and identify good reasons to use them.

As you get more experienced with Java, you might want to dip back in to the source code here as you will start to get a better idea of how Selenium works.

A little about JUnit

JUnit (http://www.junit.org) is a unit test framework for Java. It provides a convenient way of creating 'tests', running them and having the results tracked in a report. It is also a very simple framework to learn.

We are using it in this text because it is easy to use in Eclipse – it comes as part of the default install. And Selenium-IDE supports it as one of the exporting options.

At the moment we are using it in its most basic form. Where we extend the TestCase class (Actually SeleneseTestCase extends the TestCase class for us and we extend SeleneseTestCase) and name our methods as test* (where * is the descriptive name of the test).

In later sections we will use it in its newer form with Java annotations, but we will wait for you to get a bit more experience first.

The JUnit Eclipse plugin provides a convenient graphical test runner to show the tests running.

A look at SeleneseTestCase.java

SeleneseTestCase.java is provided as part of the selenium-java-client-driver.jar. It was contributed by Nelson Sproul and provides a quick way of accessing JUnit functionality by extending the JUnit TestCase class, provides a wrapper for helper methods in the SeleneseTestBase class and provides a Selenium object which we can use in our tests.

In this section we will learn a little about the make up of this class.

```
1  package com.thoughtworks.selenium;
2
3  import junit.framework.TestCase;
4
5  /**
6   * Provides a JUnit TestCase base class that implements some handy functionality
7   * for Selenium testing (you are <i>not</i> required to extend this class).
8   *
9   * <p>This class adds a number of "verify" commands, which are like "assert" commands,
10  * but they don't stop the test when they fail.  Instead, verification errors are all
11  * thrown at once during tearDown.</p>
12  *
13  * @author Nelson Sproul (nsproul@bea.com) Mar 13-06
14  */
15 public class SeleneseTestCase extends TestCase {
16
17     private SeleneseTestBase stb = new SeleneseTestBase();
18
19     /** Use this object to run all of your selenium tests */
20     protected Selenium selenium;
21
22     public SeleneseTestCase() {
23         super();
24     }
```

Figure 7.12: The first few lines of SeleneseTestCase

From the listing you can see that on line 3 we import the TestCase class from JUnit. And on line 15 the SeleneseTestCase class extends this TestCase.

This is the original way of using JUnit. Version 4 of JUnit, which we associated with our project has other ways of creating tests and we shall look at these later. The SeleneseTestCase class is our easy way of taking Selenium-IDE tests and moving them into Eclipse.

On line 17 the SeleneseTestCase class creates private instance of Type SeleneseTestBase called stb. By making it private, only the methods in the SeleneseTestCase class itself can reference it. The vast bulk of this SeleneseTestCase consists of wrapper methods which call the SeleneseTestBase class.

The SeleneseTestClass is an abstraction layer to hide implementation details from you so that you can get on with the business of writing tests. We are looking at it to learn a bit more about what we are using.

On line 20 there is a definition of a 'protected' variable called *selenium* which will allow us access to an object of type Selenium. The Selenium class, also contained in the selenium-java-client-driver.jar, and is the main class we care about. By making the variable 'protected' it can be accessed by any method inside SeleneseTestCase, and any class in the com.thoughtworks.selenium package, and also any class that extends SeleneseTestCase. Since our test case extends SeleneseTestCase, this explains why we can use the selenium variable directly in our tests without us declaring it.

There is a good discussion of access level modifiers (private, public, protected) on the main Java documentation site. http://java.sun.com/docs/books/tutorial/java/javaOO/accesscontrol.html

Line 22 has the 'constructor' for the SeleneseTestCase class. The constructor is called automatically when a New instance of the class is created by by the JUnit TestRunner. And because our test case class is really a SeleneseTestCase, the constructor on SeleneseTestCase is called.

"super();" means that the constructor of the super class (the class we extended) is called, in this case the TestCase from JUnit.

In our test (MyFirstSeleniumTests), on line 8 we call a setUp() method:

```
5  public class MyFirstSeleniumTests extends SeleneseTestCase {
6
7      public void setUp() throws Exception {
8          setUp("http://www.google.co.uk/", "*chrome");
9      }
```

Figure 7.13: The setUp method of our test

Now you can see where it comes from. We are actually calling the setUp method in SeleneseTestCase.java

```
77     public void setUp(String url, String browserString) throws Exception {
78         stb.setUp(url, browserString);
79         selenium = stb.selenium;
80     }
```

Figure 7.14: The setUp method in SeleneseTestCase

This setUp method delegates to the SeleniumTestBase setUp method. And this ultimately creates a DefaultSelenium object which gets assigned to the "selenium" variable which we can then use.

Another important method to look at here is the tearDown() method.

```
82      /** checks for verification errors and stops the browser */
83      public void tearDown() throws Exception {
84          stb.tearDown();
85      }
```

Figure 7.15: The tearDown method

The tearDown method is inherited from the JUnit test case and overridden by SeleneseTestCase. It runs after each test method, but since we didn't see it in our MyFirstSeleniumTests we were not aware that it was being executed.

Line 84 delegates to the tearDown method in the SeleniumBaseClass. At its simplest, this stops the selenium session for us.

Use the SeleneseTestCase Functionality

In this section we will reinforce our understanding of the SeleneseTestCase by using some of the helper functionality by:

- adding some asserts
- adding some verifications
- capturing a screenshot when a test fails

Using the IDE to add asserts and verifies

When we record a test using the Selenium-IDE, it adds 'actions' for us, but it does not add any asserts or verifies. These we have to add manually, either during the recording of the test (which is easier), or after recording has finished.

In this section we will look at how to add asserts and verifies when recording.

First record the setup steps

Start recording in the IDE. Record a script where we go to Google, type Selenium-RC, and then press the search button.

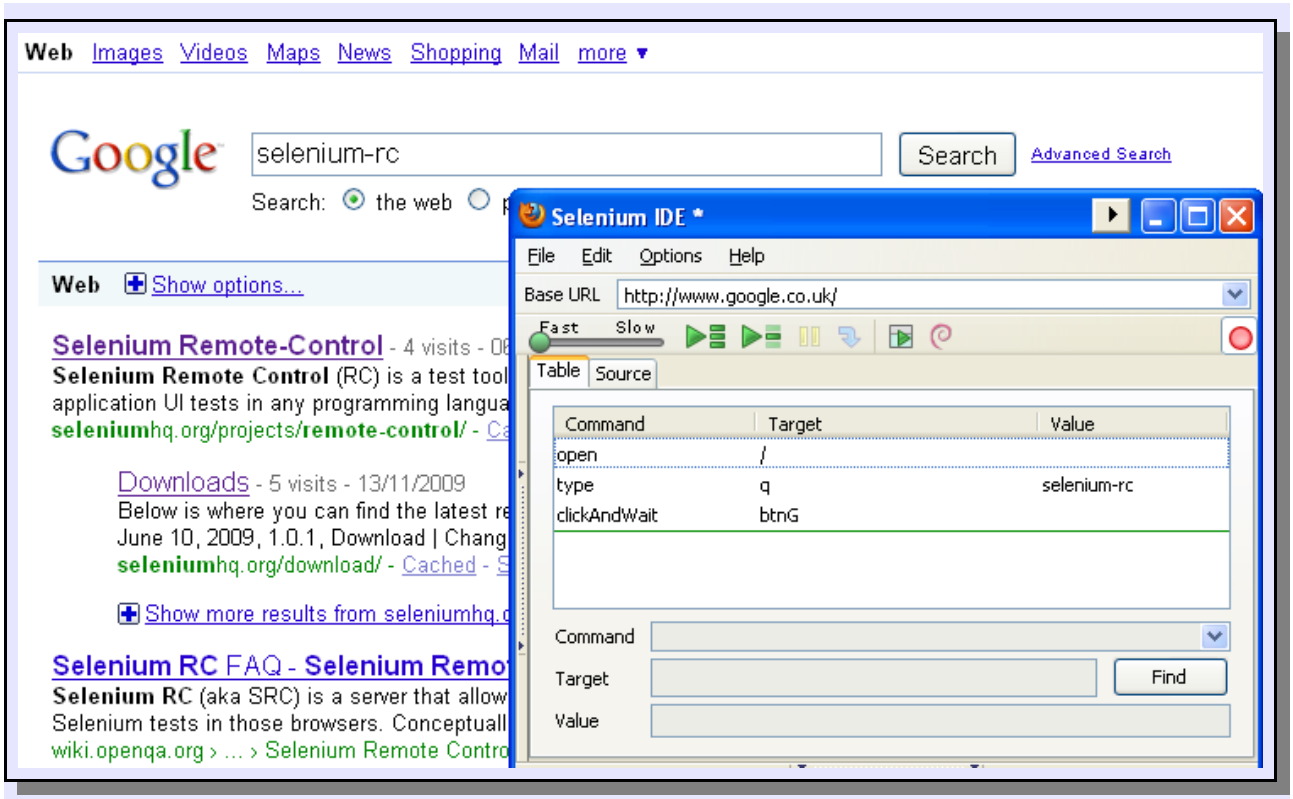


Figure 7.16: Basic recording in the IDE

Now that we have done our basic setup we can add some verify statements.

Since Google display our search term as part of the title, and in the search box, we can add checks for both of these things.

Add the Verify Statements

To add a check for the title, right click on the html page in Firefox. At the bottom of the contextual menu I can see the Selenium-IDE additions. These are populated by some common choices. Both assertTitle and verifyTitle methods, and these are pre-populated with the title of the page, so I don't have to do any work.

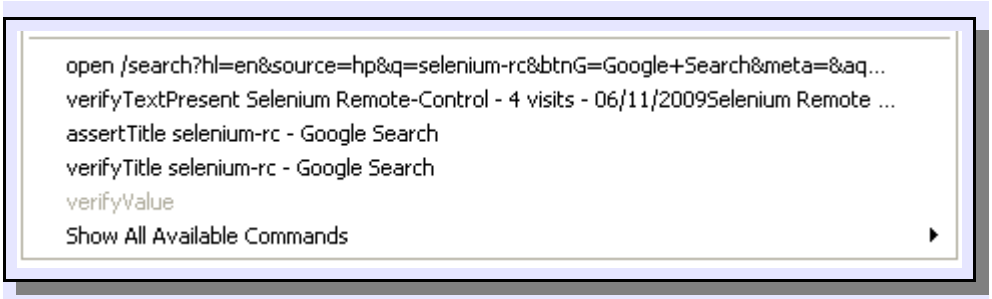


Figure 7.17: Context Menu for Selenium IDE

Select the "verifyTitle" entry and a new line appears in the test in the IDE.

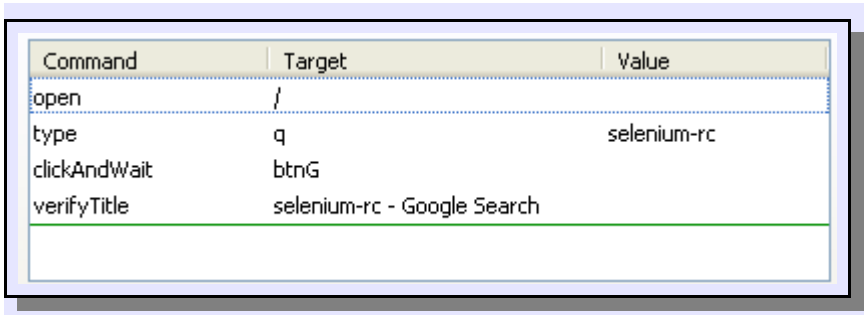


Figure 7.18: Selenese Script with verifyTitle

If the option we wanted does not appear, or if you are curious as to what you can 'contextually' do, given what you clicked on with Selenium, then use the "Show All Available Commands" submenu:

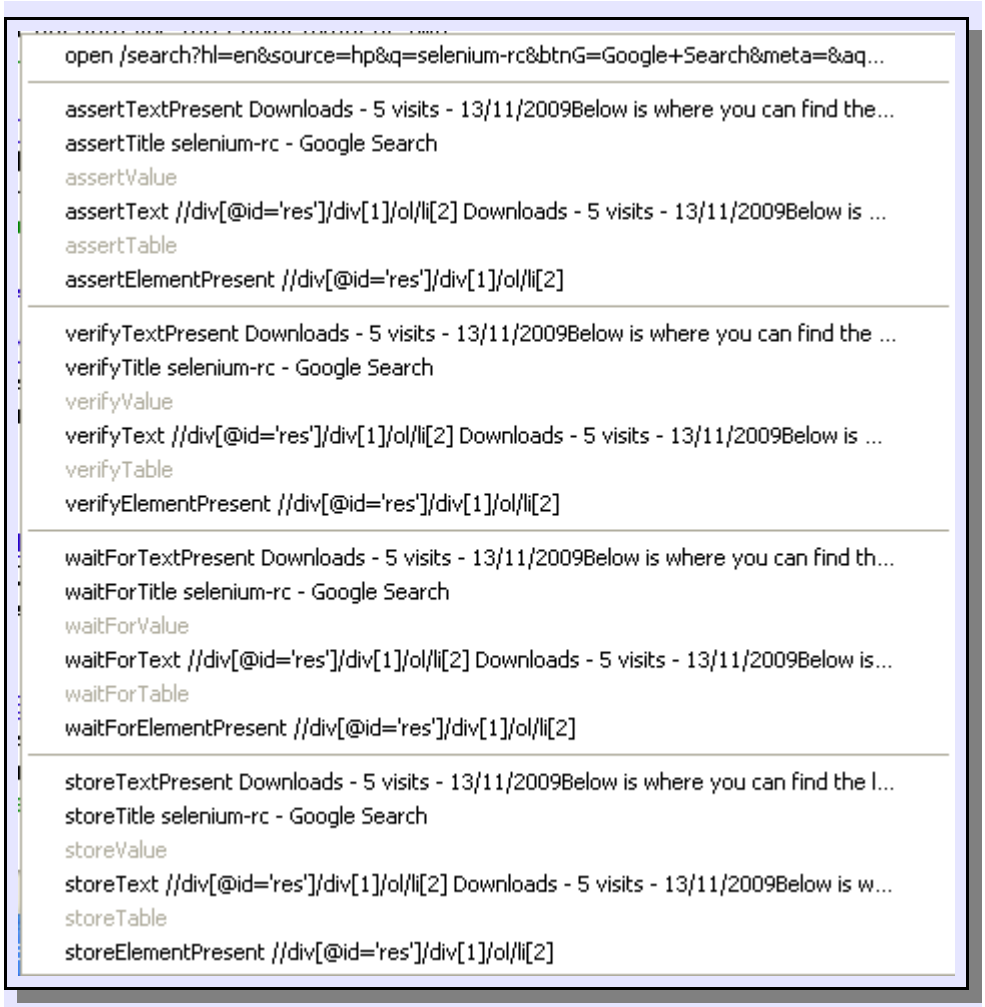


Figure 7.19: The full contextual list provided by the Selenium IDE

This submenu shows all the contextual commands available. Clicking any of them will add them to the IDE.

To add a check for the text in the search input field, right click on the search input field and choose "verifyValue q selenium-rc".

If you run that test in the IDE then both those verifications will pass.

Save that test as “verifys_for_selenium_rc.htm” (PS: I know that is not how you spell verifies, but if there were a plural for verify I'm sure it would be verifys)

Add the Asserts

To create a similar test which 'asserts' for those two values we could:

- rerecord the test from scratch and choose assertTitle and assertValue instead, and then save the test.
- delete the verify statements, by clicking on the line and choosing delete from the context menu (or pressing 'Delete') and adding asserts from the Selenium right click context menu in Firefox. And then “save as” a different test file.

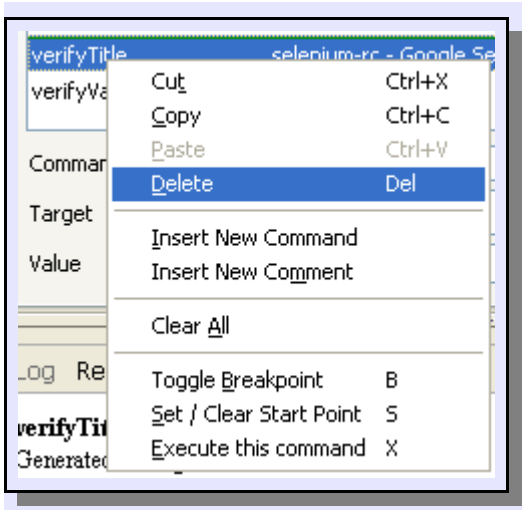


Figure 7.20: Delete a row in the table

- amend the values in situ, by clicking on the line in the table and selecting a new “command” from the drop down, and then “save as” to a different file name.

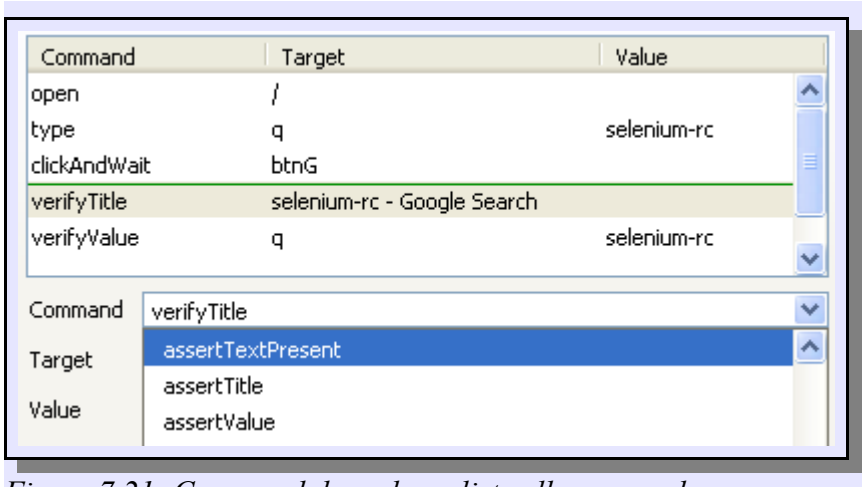


Figure 7.21: Command drop-down lists all commands

When I did this, I found it faster to amend the commands in situ and then save the test as a different file name.

What is the difference between an Assert and a Verify?

Selenium IDE supports two types of 'checks':

- assert – which stop the test on the failure of the first assertions
- verify – where the verify statement checks the results and reports an error but the test continues to the end

As an example compare two tests which I created in the IDE. I have changed the search from "selenium-rc" to "wrongvalue" so that the tests will fail when run.

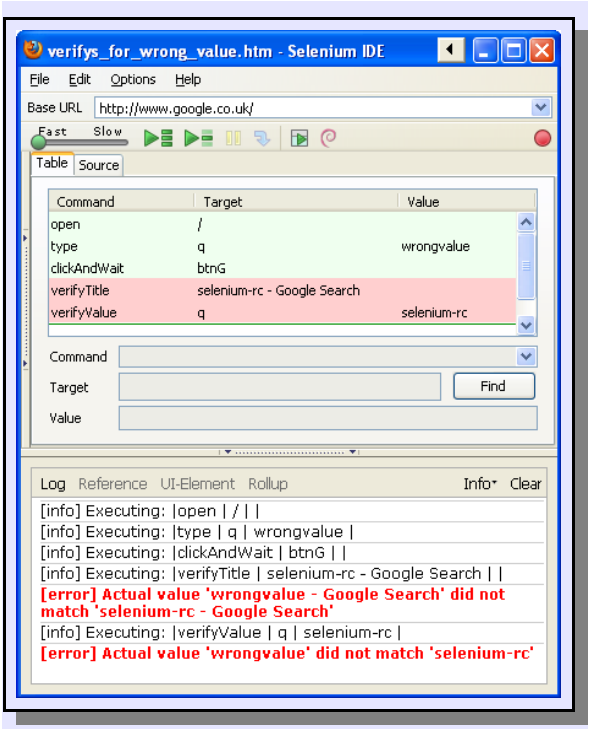


Figure 7.22: Verify methods allow test to continue after a failure

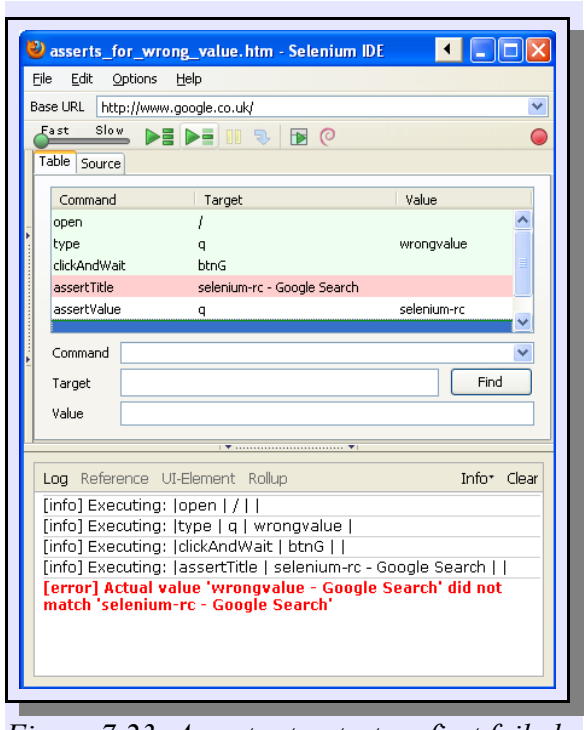


Figure 7.23: Asserts stop test on first failed assertion

Both tests run essentially the same script where they type "wrongvalue" into the google search box, and then check for the title of the page and the value in the search box.

The first test, using the verifyTitle and verifyValue methods, runs to the end of the test and shows 2 errors in the log and on the script table view. The second test which uses assertTitle and assertValue, stops on the first error assertTitle, and does not check the value in the field.

Verify methods can be useful if you want to create a test that, regardless of the outcome of previous checks, conducts a check so can be useful where you do a lot of setup steps and then perform a whole series of checks.

Asserts are useful if you don't chain checks, and instead need a test to have passed before moving on to the next step.

When these tests are exported to the IDE we get the following JUnit tests.

```
public void testVerifys_for_wrong_value() throws Exception {
        selenium.open("/");
        selenium.type("q", "wrongvalue");
        selenium.click("btnG");
        selenium.waitForPageToLoad("30000");
```

```
        verifyEquals("selenium-rc - Google Search", selenium.getTitle());
        verifyEquals("selenium-rc", selenium.getValue("q"));
}

public void testAsserts_for_wrong_value() throws Exception {
        selenium.open("/");
        selenium.type("q", "wrongvalue");
        selenium.click("btnG");
        selenium.waitForPageToLoad("30000");
        assertEquals("selenium-rc - Google Search", selenium.getTitle());
        assertEquals("selenium-rc", selenium.getValue("q"));
}
```

There are some subtle differences to be aware of when debugging a verify failure compared to an assert failure.

With an assert failure, the Failure trace pane of JUnit will display a single assertion stack trace. And a double click on the test in the JUnit opens the code in the editor at the failed assertion line.

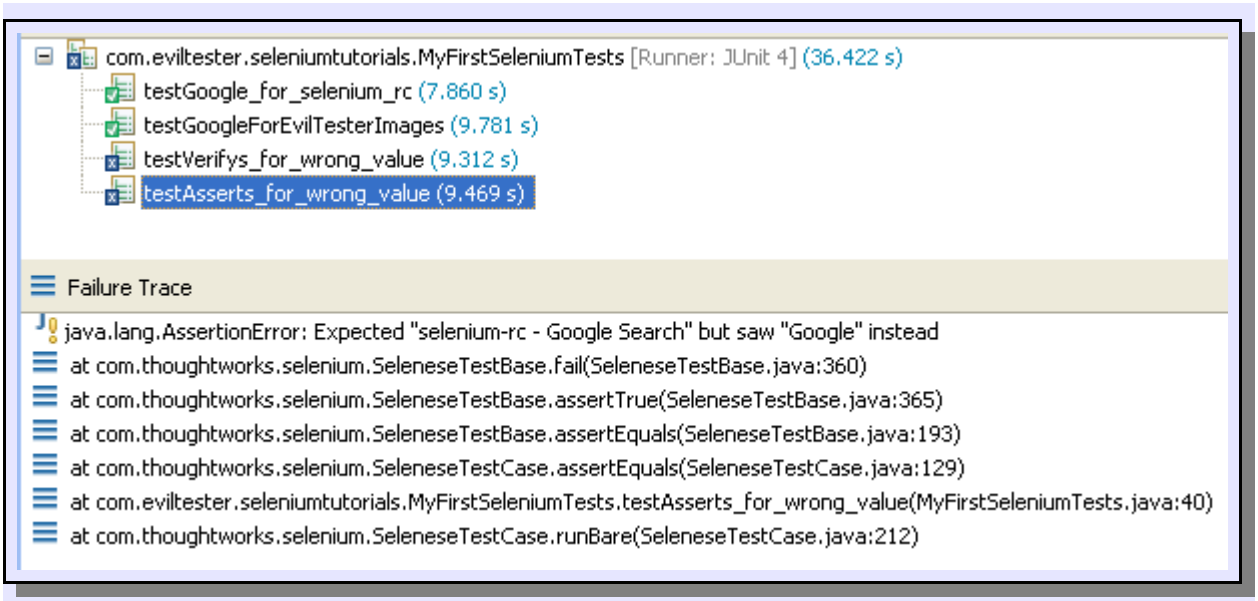


Figure 7.24: Failure Trace for an assertion

With a verify failure, the failure trace contains all the assertions that failed.

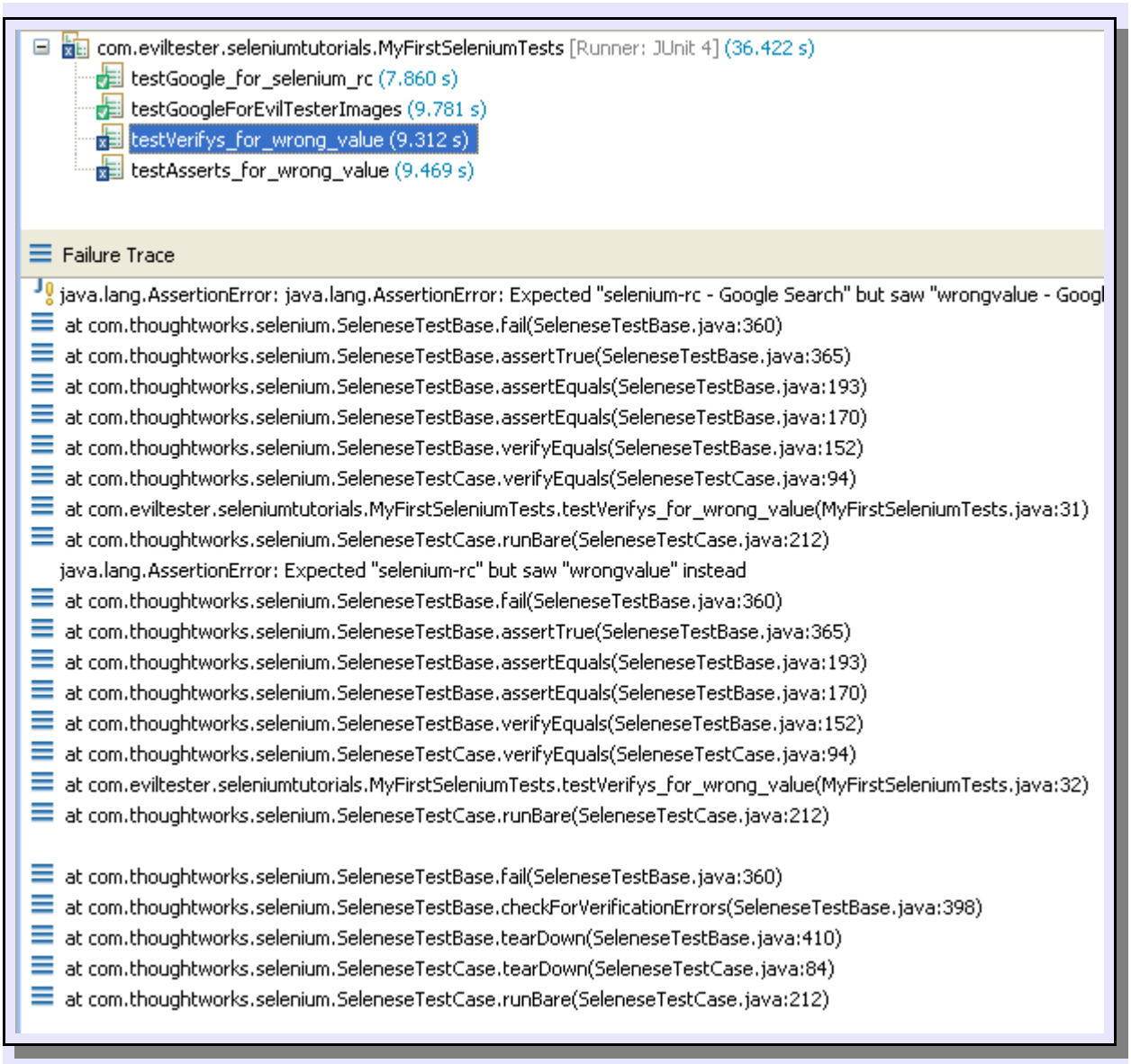


Figure 7.25: Failure trace for a Verify

If you double click on the test you will be taken to the first verify that failed. And to identify the other verify lines you will have to look through the Failure Trace for each of the lines that start

```
java.lang.AssertionError: java.lang.AssertionError: Expected ...
```

You should also look for the line in the assertion that tells you which line the assertion happened on. This is the line that reports an error at the level of the test class you are working with e.g. MyFirstSeleniumTests and then you can see that it happened on line 32. So you look at that line in the code to see the verification statement that failed.

```
At  com.eviltester.seleniumtutorials.
MyFirstSeleniumTests.testVerifys_for_wrong_value(MyFirstSeleniumTests.java:32)
```

How to find line numbers in Eclipse?

To help you find the line number you can:

- switch on line numbering by right clicking on the left margin of the code window (where you set breakpoints) and select “Show line numbers”

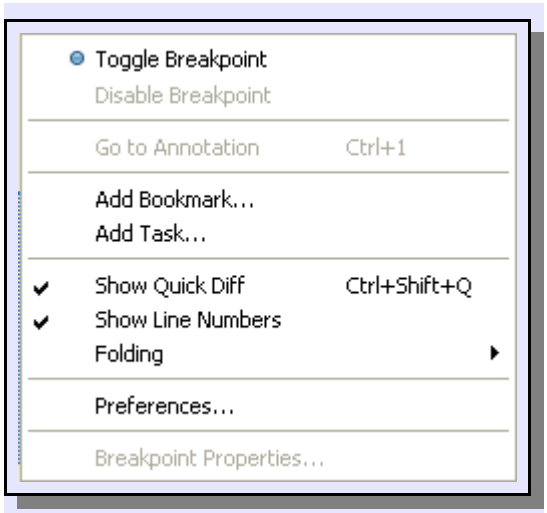


Figure 7.26: Switch on line numbers from context menu

- press Ctrl+L or choose "Navigate \ Goto Line" from the menu

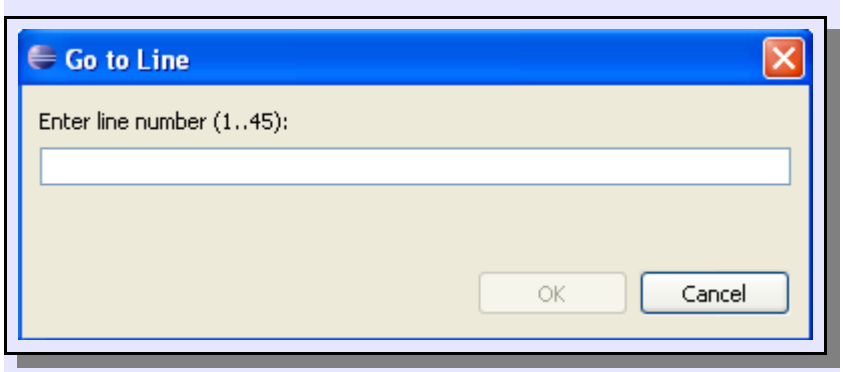


Figure 7.27: Ctrl+L to show "Go to Line" dialog

- Navigate around the code window using the "line : char" indicator on the bottom status bar of the Eclipse window

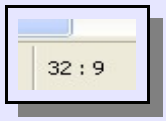


Figure 7.28: The line Character Indicator in bottom status bar

Creating Screenshots on Failure with SeleneseTestCase

Selenium has the ability to create screenshots when running tests with the following methods:

- captureScreenshot
- captureEntirePageScreenshot

The SeleneseTestCase class that we are using, has the ability to capture screenshots using the captureScreenshot method as part of the default test runner.

Before we can capture a screenshot we have to call the method:

```
setCaptureScreenShotOnFailure(true);
```

It is tempting to think that we would just add this line, inside the test that we wanted to capture the screenshot, or possibly in the setUp method so that it applies to all tests. Unfortunately that isn't the case. We have to call this method in the constructor for the test case class.

A constructor in Java is the method that is called when an object of the class is instantiated. So in our case, when the JUnit testrunner creates an object of Type MyFirstSeleniumTests.

But we don't currently have a constructor. Normally you won't need to create a constructor for your test classes, but to use the screenshot functionality, this is what it looks like.

```
public MySeleniumTestWithScreenshot(){
        setCaptureScreenShotOnFailure(true);
}
```

The constructor goes directly after the class declaration. It is always named the same as the class and has no return value declaration.

```
public class MySeleniumTestWithScreenshot extends SeleneseTestCase {

    public MySeleniumTestWithScreenshot(){
        setCaptureScreenShotOnFailure(true);
    }

    public void setUp() throws Exception {
        setUp("http://www.google.co.uk/", "*chrome");
    }
```

Figure 7.29: Add a constructor to create screenshots

Only tests that fail through asserts will actually trigger a screenshot.

Screenshots will be named with the name of the test. And they will be saved in the same directory that you started the selenium server in, because we are calling a command in the selenium server to capture the screen and it doesn't know where our tests are running from.

Try this with your test cases.

- Create the constructor
- Add the setCaptureScreenShotOnFailure method to the constructor
- change one of the tests with asserts so that it will fail the assert
- you should see a message in the console when you run the test e.g. "`Saved screenshot testAsserts_for_wrong_value.png`"
- after the test has failed, have a look in the directory where you ran the selenium-RC server from and you should see a screenshot of the screen when the test was running. If you have a multi-monitor setup then it will take a screenshot of the windows displayed by the primary monitor.

This default setup which comes with SeleneseTestCase is fun to experiment with but not particularly flexible, so we will look at ways of extending this in a later section.

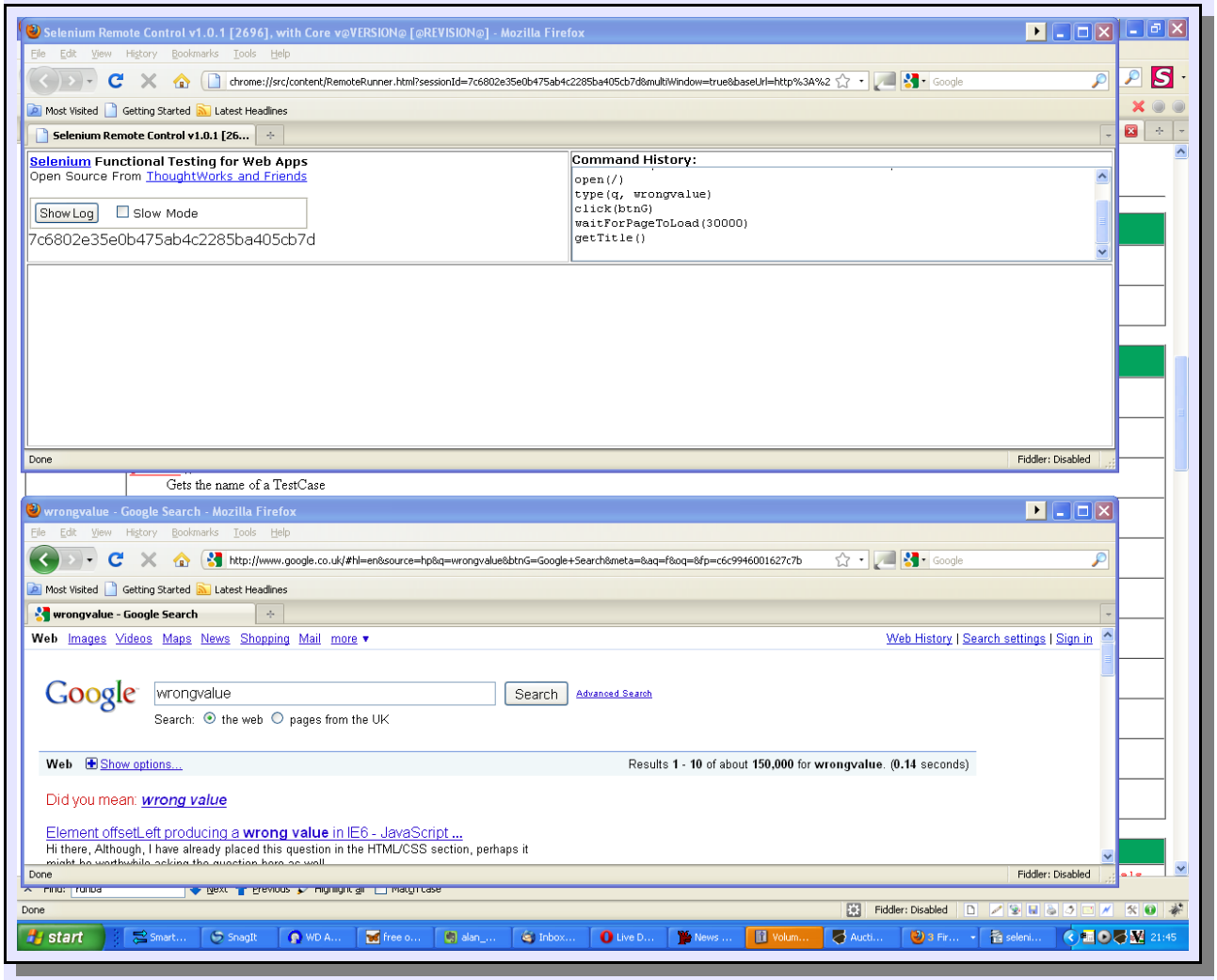


Figure 7.30: A screenshot captured by Selenium when a test assertTitle failed

Chapter 8: Let's get coding

The plan here is to start to take control over our test cases. Now that we have learned about the SeleneseTestCase class we will start learning to work with selenium without these abstraction layers. This will help us understand how to really code with Selenium. You will then be in a position to decide for yourself if you want to use the abstraction layers that come with the selenium java client driver, or build your own.

In this section you will learn how to use the tools the way that I think you will choose to use them by default.

A brief pause, because you have already learned some Java

You have already learned some Java, and some Eclipse tips.

You already know:

- how to recognise an error in Eclipse
- that Eclipse does a lot of error checking for us
- that Eclipse can fix many errors, so left click on that red [x] when it appears - this helps you learn about fixing coding errors in Java quite quickly

You have also learned:

- Java Classes start with an uppercase letter
- Java Classes are logically organised into packages
- Java Class methods become tests in JUnit by putting 'test' at the start of the method name
- Java Class Libraries need adding to our Class path so we can use them
- We need to 'import' Classes from Libraries to use them in our code

What we will now learn

- How to use JUnit annotations
- The DefaultSelenium class
- Starting and stopping the connection to Selenium server

Create a New Test From Scratch

Now create a new Java class and call this “MySecondSeleniumTests”.

> **Note: If we first select “MyFirstSeleniumTests.java” in the Package Explorer, and then do “File \ New \ Class” then you will notice that Eclipse adds the package name into the “New Java Class” wizard for us as it assumes we want to keep the new class in the same logical hierarchy as the one we selected.**

After using the “New Java Class” wizard we are going to create a set of test methods from scratch. So starting with the generated code, we will create a test method within this.

```
package com.eviltester.seleniumtutorials;
public class MySecondSeleniumTests {
}
```

```
MySecondSeleniumTests.java
1  package com.eviltester.seleniumtutorials;
2
3  public class MySecondSeleniumTests {
4
5  }
6
```

Figure 8.1: Basic declaration for the test

Our First Test with Annotations

We write our 'tests' as class methods.

So add a new method called initialiseSeleniumRCServer to your class:

```
MySecondSeleniumTests.java
1  package com.eviltester.seleniumtutorials;
2
3  public class MySecondSeleniumTests {
4
5      public void initialiseSeleniumRCServer(){
6
7      }
8  }
```

Figure 8.2: Added a method to the test

Simply adding a new method does not make it a test.

We will use JUnit annotations to turn our method into a test.

The SeleneseTestCase class hid a lot of details about JUnit usage from us, so you haven't seen any annotations yet.

Java supports annotations (http://en.wikipedia.org/wiki/Java_annotation) to help semantically markup our code. So turn your method into a test by adding an @Test annotation as follows:

```
3  public class MySecondSeleniumTests {
4
5      @Test
6      public void initialiseSeleniumRCServer(){
7
8      }
9  }
```

Figure 8.3: Add an @Test annotation to the code

Now you will have noticed that Eclipse has marked that line as an error

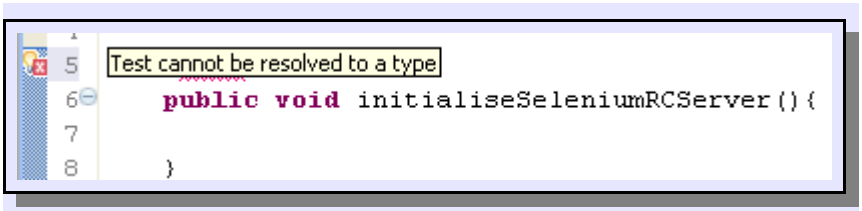


Figure 8.4: Eclipse identified error

By hovering the mouse over the red [x] on the left, Eclipse tells us that "Test cannot be resolved to a type", which you can probably guess means that we haven't told Eclipse about JUnit annotations yet.

If I click on the red [x], then Eclipse will prompt me with ways of potentially resolving this issue:

- Import 'Test' (org.junit)
- Create annotation 'Test'
- etc.



Figure 8.5: Resolve error by clicking on error icon in left border

I want to select “Import 'Test' (org.junit)” because I need to add an import statement for the annotation from JUnit.

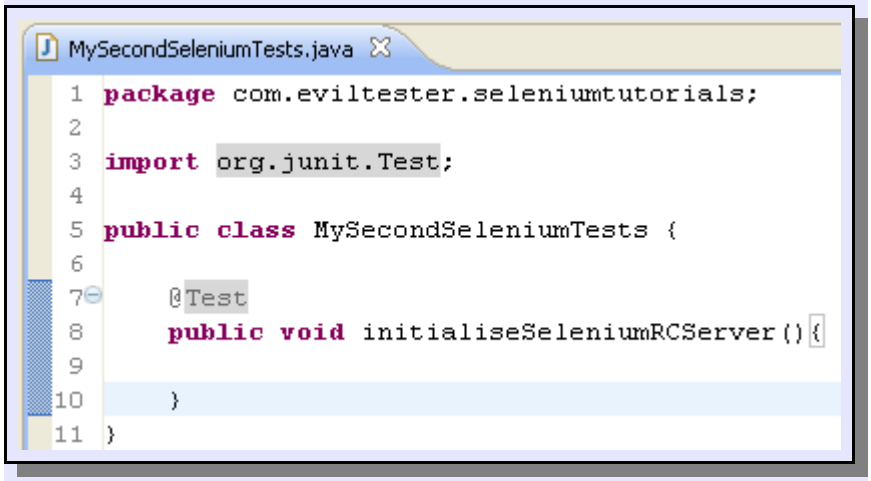


Figure 8.6: added import statement to resolve error

Since I know I'm likely to need more classes and annotations from JUnit, I'm going to pre-empt the fixing of other errors by creating an import statement that matches everything from org.junit i.e.

```
Import org.junit.*;
```

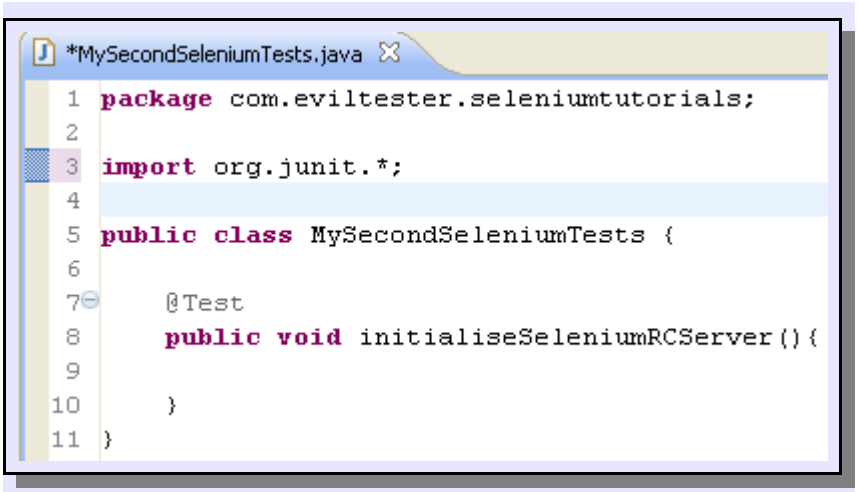


Figure 8.7: Wild card matching import statement

Note:

By using the @Test annotation, our test methods no longer have to start with the word "test" so we can give them any valid Java method name. This can help make your naming more expressive.

Back to Our Selenium Test

OK, so back to Selenium.

Add the line (in bold below) to your test:

```
@Test
public void initialiseSeleniumRCServer(){
        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                        "http://www.google.com");
}
```

That will create a new Selenium object so we can connect to the Selenium-RC server.

But, as before we have a red [x] so Eclipse informs us that we have an error here.

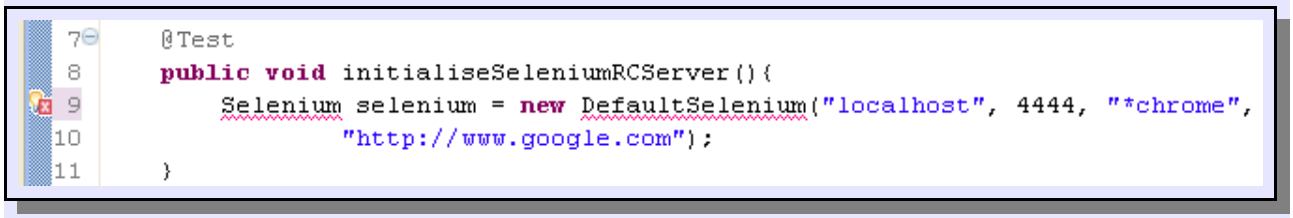


Figure 8.8: Eclipse reported an error on Selenium

By hovering on the red [x], Eclipse tells us that it doesn't know anything about Selenium with the error "Selenium cannot be resolved to a type".

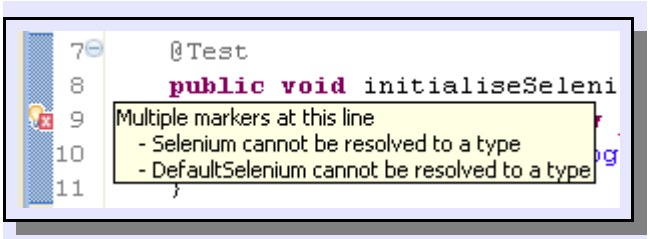


Figure 8.9: Have not defined Selenium

Eclipse can fix that for us when we left click on the red [x] and select "Import Selenium (com.thoughtworks.selenium)" from the suggestion popup.

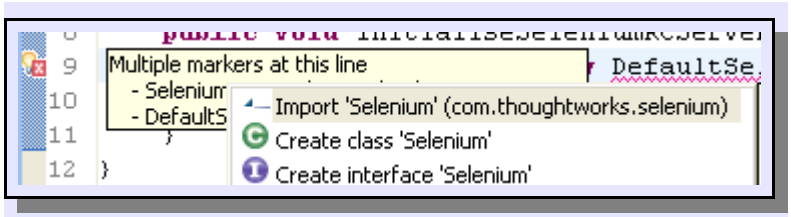


Figure 8.10: Automatically add an import

Eclipse adds the Selenium Import for us.

```
import com.thoughtworks.selenium.Selenium;
```

But this hasn't fully imported all we need. Eclipse is still telling us that it doesn't know about DefaultSelenium.

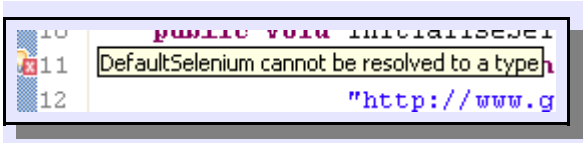


Figure 8.11: DefaultSelenium not defined

But rather than importing DefaultSelenium, we may as well do the same regex matching that we did for JUnit and change our selenium import to

```
import com.thoughtworks.selenium.*;
```

This should remove any future Selenium imports for us as well.

The SeleneseTestCase class hid all JUnit and Selenium imports from us by acting as a wrapper around JUnit and Selenium.

So now our code looks like:

```
package com.eviltester.seleniumtutorials;

import org.junit.*;
import com.thoughtworks.selenium.*;

public class MySecondSeleniumTests {

	@Test
	public void initialiseSeleniumRCServer(){
		Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
				"http://www.google.com");
	}
}
```

Notice that Eclipse has displayed a warning for us.

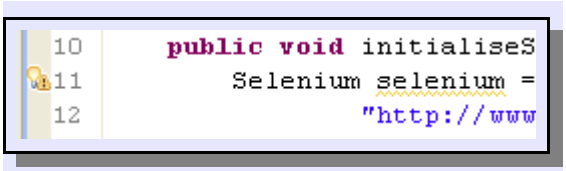


Figure 8.12: Eclipse warning indicator

And if we hover on the yellow [!] then we will learn that "The local variable selenium is never read".

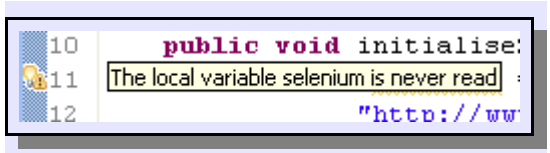


Figure 8.13: Unread local variable

Since we have just started writing our test, that seems fine - if this happens later then we may decide that we don't need the local variable and delete it from our code.

Making Something Happen

Now we are going to start using the local selenium object that we created so add “selenium.” as the next line in your test method:

```
@Test
public void initialiseSeleniumRCServer(){
        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                        "http://www.google.com");
        selenium.

}
```

When you typed "selenium." in the above code, you should have noticed a few things, first a red error cross appeared on the line (because we haven't finished the line yet) and a pop up menu appeared.

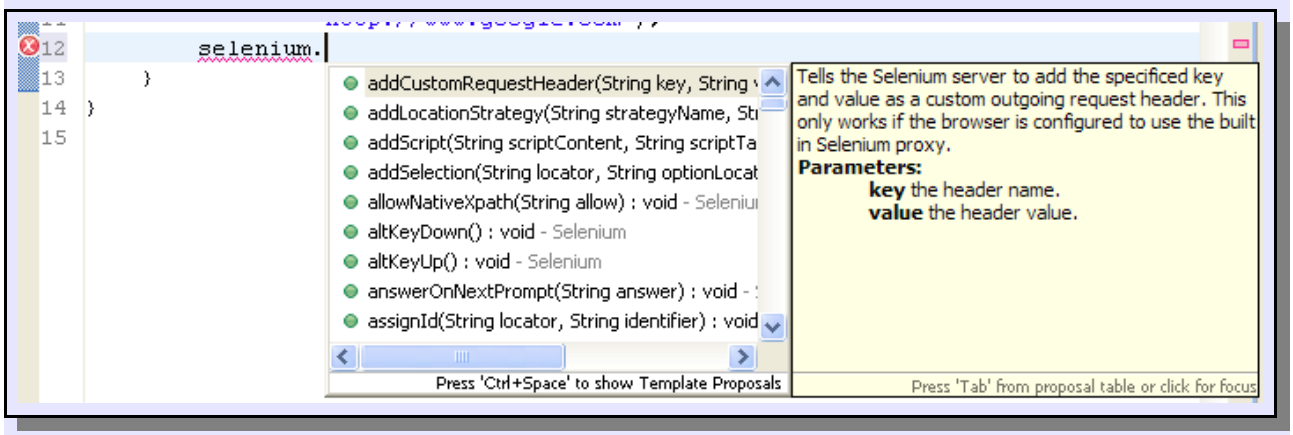


Figure 8.14: Code Completion Dialog

This popup menu is the "code completion" dialog and will help you learn the range of Selenium commands available to you. This IDE support is one reason for introducing you to Selenium with

Java and Eclipse. Auto completion can be started manually by pressing Ctrl+space and Eclipse will respond with a contextual list of actions or commands based on the text immediately preceding the point you pressed Ctrl+Space.

For now we can ignore this and just type the following code.

```
selenium.start();
selenium.open("http://www.google.com");
selenium.stop();
```

Save the file. The red [x] should have gone away and you now have a test that does something.

Specifically, we have now created a test which instantiates a Selenium object, starts up the session, opens Firefox to visit www.google.com, and stops the session.

Run it and see what it does.

Your Final Test Code

One thing we want to do to the test code before we finish up is to make sure it closes everything that it opened.

Currently it doesn't.

Our basic hierarchy at the moment is:

- start
    - open
- stop

What we really need is to close everything that we open.

- start
    - open
    - close
- stop

So we should add the *selenium.close();* statement into the source code, immediately after the *selenium.open(...);*

selenium.close will close the Selenium Remote Control window and make sure we leave the test system in the same state we found it and not leave any loose ends that might cause us problems the next time we run the test.

Hopefully if you followed all the instructions then your code looks like the code below:

```
package com.eviltester.seleniumtutorials;

import org.junit.*;
import com.thoughtworks.selenium.*;

public class MySecondSeleniumTests {

	@Test
	public void initialiseSeleniumRCServer(){
		Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
```

```
                                "http://www.google.com");
                selenium.start();
                selenium.open("http://www.google.com");
                selenium.close();
                selenium.stop();
        }
}
```

The DefaultSelenium constructor explained

```
Selenium selenium =
       new DefaultSelenium("localhost", 4444, "*firefox", "http://www.google.com");
```

The constructor for Selenium has a few key points that you really need to understand.

- “localhost”
  - We are running the Selenium-RC server from a separate DOS window. So the Selenium-RC server is a separate application running on our machine. “localhost” is the convenient hostname to refer to “this computer”. We could use this computers IP address or machine name but localhost is a convenient short cut.
  - If our Selenium-RC server was running on another machine, or a virtual machine, we would replace “localhost” with the hostname of that computer. Either its machine name or its IP address. We can get this information by running “ipconfig” from the command line – to get the IP address. Running “ipconfig /all” will give more information about the computer – including its “Host Name”.
- “4444”
  - This is the port that Selenium started on. We didn't supply a port name when we ran “java -jar selenium-server.jar” so it used the default of 4444.
  - We could have started Selenium-RC using any port we wanted by affixing a “-port” parameter e.g. “java -jar selenium-server.jar -port 8888”
  - If we get the port wrong then we won't be able to connect to the Selenium-RC server process.
- “*firefox”
  - This is the parameter which tells Selenium-RC which browser to use. I have told it to use Firefox.
  - A longer discussion of browser codes appears in Chapter 33 “Run the tests on multiple browsers“ on page 286. But if you want to use Internet Explorer use “*iexplore”.
- "http://www.google.com"
  - This is the parameter which contains the base URL of the site we are testing.

Retrospectives

Well Done

So if you followed the tutorial this far, you managed to write your first test from scratch (admittedly it didn't do much) and we covered a lot, so a quick recap of what we covered:

- How to recognise an error in Eclipse
- Eclipse does a lot of error checking for us
- Eclipse can fix many errors, so left click on that red [x] when it appears - this helps you learn about fixing coding errors in Java quite quickly
- Java Classes start with an uppercase letter
- Java Classes live in packages
- Java Class Libraries need adding to our Class path so we can use them
- We need to 'import' Classes from Libraries to use them in our code
- We add external Java Libraries to the project properties
- To connect to the Selenium server we initialise a DefaultSelenium object, then start the session, issue our selenium commands (e.g. open), then close the session
- How to switch between Java and Debug perspectives
- The Selenium Server displays a lot of logging information in the DOS Window

I think you have most of the infrastructure elements that you need to start writing tests, and you can probably start to follow some of the other Selenium tutorials that are out there on the web now, without too much effort.

Why did you do that?

In this section I will explain the reasons behind some of the decisions made in the chapter.

Run Selenium-RC in a DOS window?

I run Selenium-RC in a DOS window so that I can:

- Start it easily
- See the logging messages
- Find the window – changing the title of the DOS window means it appears on my tool bar easily
- Stop and Restart Selenium Easily

Start it easily

You saw how easy it was to start “java -jar selenium-server.jar”, but Selenium-RC also has a bunch of command line parameters. We will discuss these later when we need them.

See the logging messages

I like to view the logging messages from Selenium-RC. Sometimes I think the test might have frozen, but a quick glance at the DOS window and I can see that it is processing.

Find The Window

When you change the title of the window you can find it in the Windows task bar much more easily or through your Alt+Tab switching. This allows you to easily view the logging messages that Selenium-RC displays.

Stop and Restart Selenium Easily

If I am in the DOS window then I can Ctrl+C to stop the selenium server.

And I can easily restart it by typing in "java -jar selenium-server.jar" again or by pressing the up key – up and down cycle through the command history in DOS.

Final Notes

I want to make it clear -

- you really don't need to know much Java to write automated tests.
- you will learn more Java as and when you need it.
- Eclipse will help you identify the mistakes you make with Java.
- The programmers you work with will happily help you if you are seriously trying to learn.
- Many good books and resources exist to help you learn Java.

Some external links which might help you understand Java:

- Java Basics
    - http://www.perfectknowledgedb.com/Tutorials/H2R/main.htm
- Official Sun Java Tutorials
    - http://java.sun.com/docs/books/tutorial/java/index.html
- Thinking in Java 3rd Edition – online edition of a great Java book
    - http://www.mindview.net/Books/TIJ/
- JavaFAQ.nu – examples and tips
    - http://www.javafaq.nu/
- Learn About Java
    - http://java.about.com/
- List of Java tutorials from ProgrammingTutorials.com
    - http://www.programmingtutorials.com/java.aspx
- Javalessons.com
    - http://javalessons.com/
- Java Coffee Break.com
    - http://www.javacoffeebreak.com/

Back to Eclipse

Removing the Source Code Association

If you ever find that you want to remove the source code association then: from the Package explorer, right click on selenium-java-client-driver.jar, select “properties” from the popup menu.

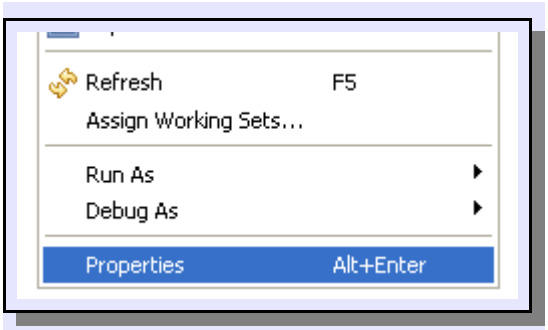


Figure 8.15:Properties menu item from Package Explorer tab

And then delete the text in the “Location Path” field, for the “Java Source Attachment” entry and then click “OK”

Chapter 9: Chapter - What if it all goes wrong?

Since you are working through these tutorial sections on your own, and things can go wrong. I will try and cover some basic troubleshooting strategies here.

- Check that Selenium Server is running
- Close Any Selenium-RC windows
- Stop and restart the Selenium-RC server
- Remove any Java threads from the Debug perspective
- Check for any blocked browser windows

Check that Selenium Server is running

This is pretty simple, but sometimes we forget to start the Selenium Server.

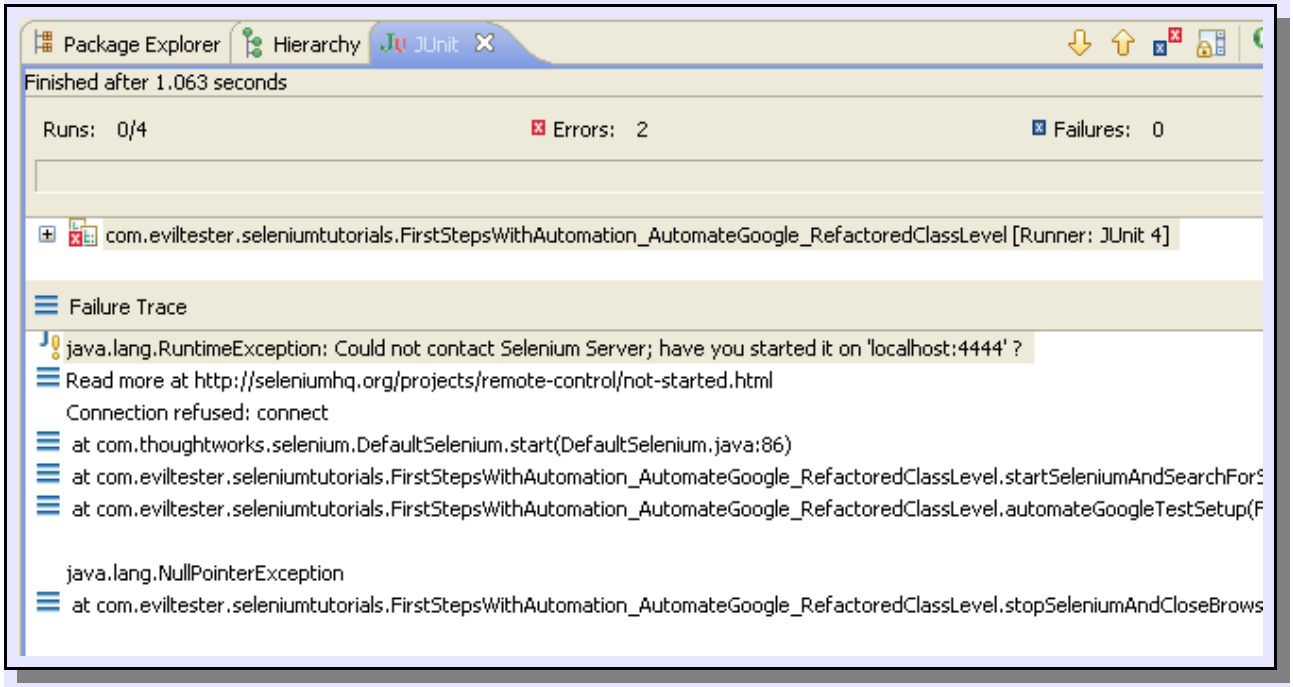


Figure 9.1: JUnit test runner informing us Selenium may not have been started

If this happens then in your JUnit test runner window you'll probably see a message like "`java.lang.RuntimeException: Could not contact Selenium Server; have you started it?`"

This is pretty easy to resolve. Just follow the instructions we provided earlier to start up the Selenium server.

- Open a DOS window and change directory to the location of your selenium-server.jar
- run the server by typing "java -jar selenium-server.jar"

Close any Selenium-RC windows

Check to see if you have any Selenium-RC browser windows open. To do this, just check the browser windows from the taskbar and close any that you don't want.

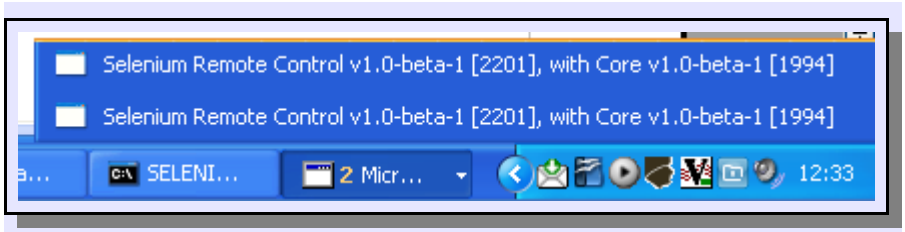


Figure 9.2: Selenium-RC windows

And if you do, then close them.

This will make sure that nothing is blocking the Selenium server from creating a session window for you.

Stop and restart the Selenium-RC server

Sometimes the Selenium server gets 'stuck'. So, go to your DOS window and press Ctrl+C to stop the server running, and then start it up again, either by retyping "java -jar selenium-server.jar" or using the up arrow to cycle through the DOS history list to re-run the command.

If that doesn't help and you really think it is a selenium server problem then try closing the DOS window completely and then run the server from scratch in a new DOS window.

You can also issue the HTTP command to stop the server. Open a browser and visit the following URL to stop the selenium server:

```
http://localhost:4444/selenium-server/driver/?cmd=shutDownSeleniumServer
```

Remove any Java threads from the Debug perspective

If you have done a bit of work in Eclipse, running, debugging and stopping tests then you may find a few threads running or stopped in the Debug perspective.

Click on the Debug perspective button if the perspective is not open.

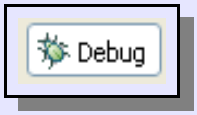


Figure 9.3: Debug Perspective Button

NOTE:

If the button is not present then select "Window > Open Perspective > Other..." from the menu, and then select "Debug" from the list of perspectives.

You can see the list of threads and sessions on the left of the screen.

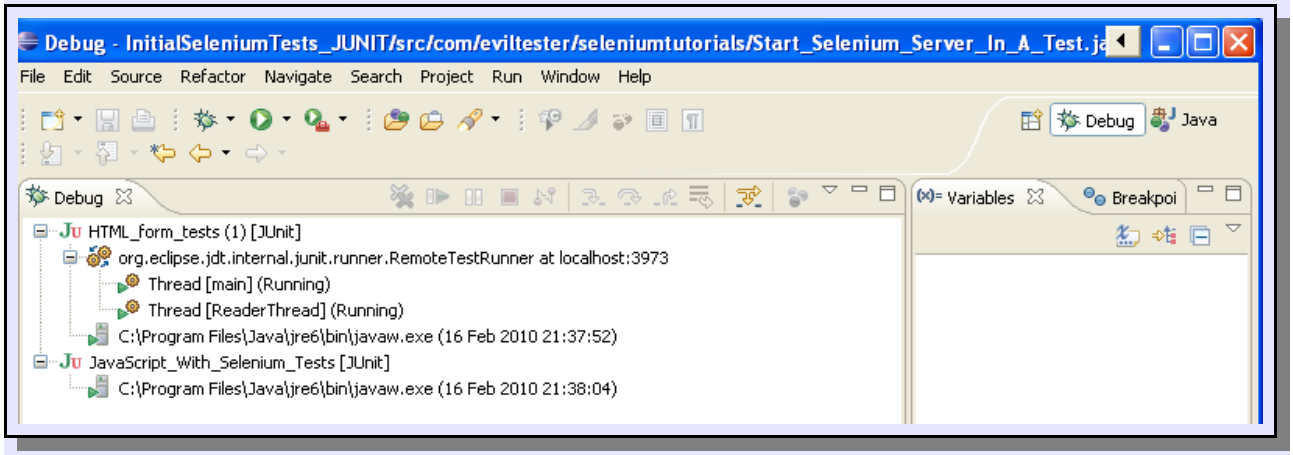


Figure 9.4: Debug view showing multiple sessions

Right click on the sessions and choose the termination options. Use "Terminate/Disconnect All" if the option is available. Otherwise just "Terminate and Remove" or simply "Terminate".

Use the "Remove All Terminated" if available and keep going until your left window is empty of any sessions. This will prevent any rogue connections of sessions interfering with what you are trying to do."

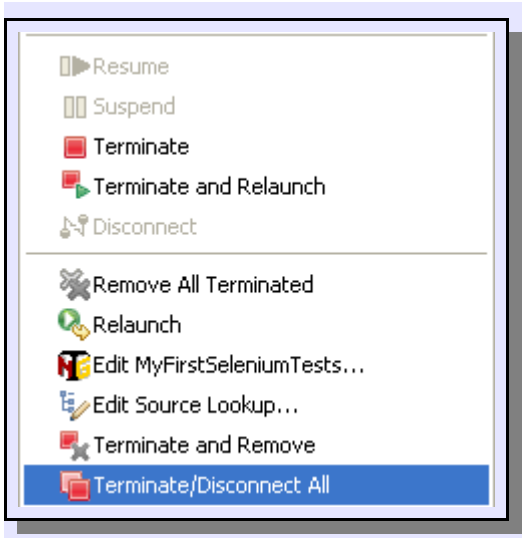


Figure 9.5: Terminate all sessions

Close any blocked browser windows

By blocked browser window I mean browser windows that may have any pop up dialogs shown, and which can't close because of it.

You may have to Alt+Tab through the windows in order to show this.

Or close down any browser windows that you have been using.

Firefox Update Dialog

If you run a test and are using any of the Firefox browser codes then you might be unlucky enough to see the Firefox updates dialog.

This will prevent your tests from running. So close down all instances of Firefox, check the task manager to ensure you got them all.

Then start up Firefox manually to trigger the update process and then your tests should run without incident.

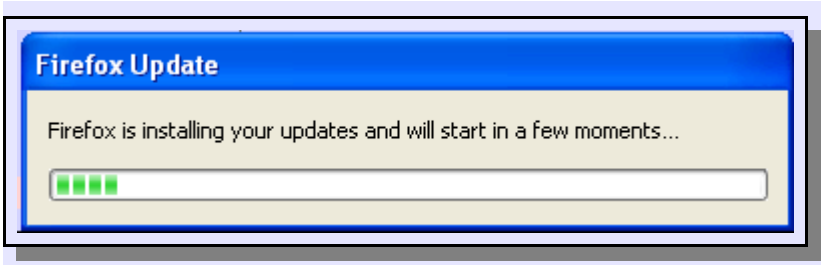


Figure 9.6 : The deadly Firefox Update dialog

Summary

I aim to get back to a clean stable state when trying to get Selenium to work again, so I tend to do the following:

1. stop the server
2. terminate any debug sessions/threads
3. remove all debug sessions/threads
4. close any Selenium-RC browser windows
5. close any blocked browser windows
6. restart the selenium session

If the above steps fail then it may be that your test code is in error so compare it carefully with the code provided in this tutorial.

Hopefully this checklist can help you get back and running if something goes wrong and you can't figure out why Selenium isn't working.

Chapter 10: Essential Firefox Add-Ons

In this section we will look at other free Firefox add-ons that we will use to help us with our Selenium test construction:

- Firebug
- XPather

What is Firebug?

The firebug add-on lets us more easily inspect the web page to:

- explore the DOM,
- the element names and Ids,
- to see the Ajax calls being made and understand the functional flow of the web page

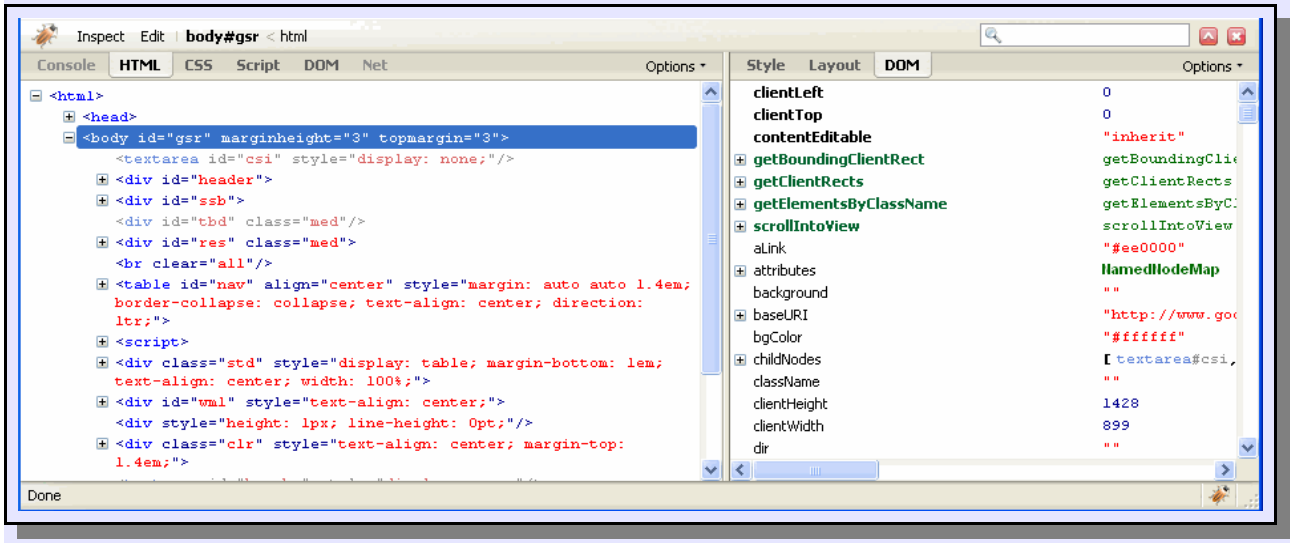


Figure 10.1: Firebug HTML view

What is XPather?

The XPather add-on lets us easily generate and experiment with the XPath statements that we will use to traverse and manipulate the web application in our tests.

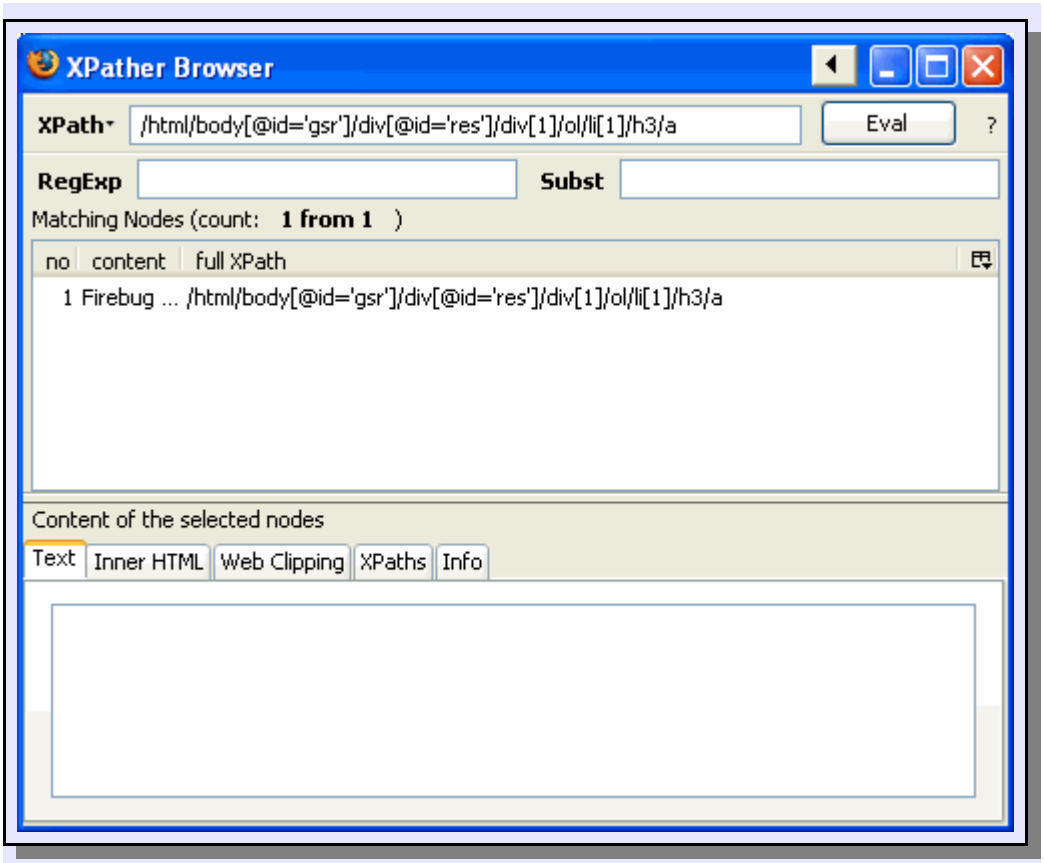


Figure 10.2 : XPather showing a matched XPath statement

Basic Firebug Usage

Either search for it on the Firefox add-on directory (https://addons.mozilla.org/) or visit http://www.getfirebug.com/ to install Firebug. As with all Firefox add-ons this will require a browser restart.

Once installed you can access firebug from the Tools menu:

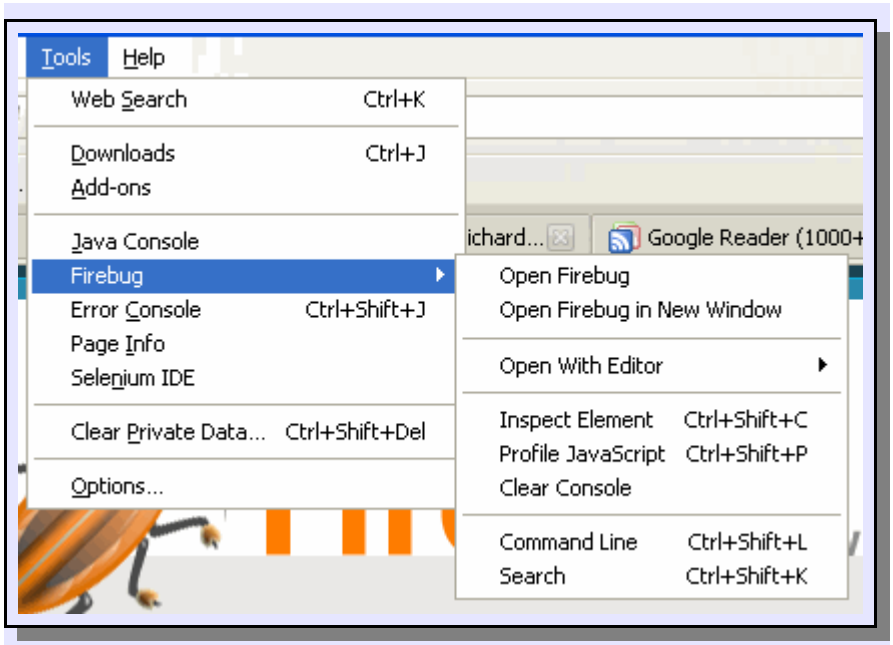


Figure 10.3 : Open Firebug from the tools menu

Firebug can also be displayed by clicking on the Firebug icon in the bottom right of the Firefox window.

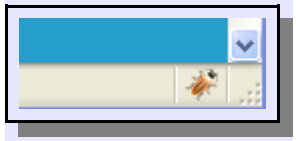


Figure 10.4 : Firebug Icon

When using firebug to help me with my Selenium testing I typically use the new "Inspect Element" option that you have when right clicking on an item in a web page.

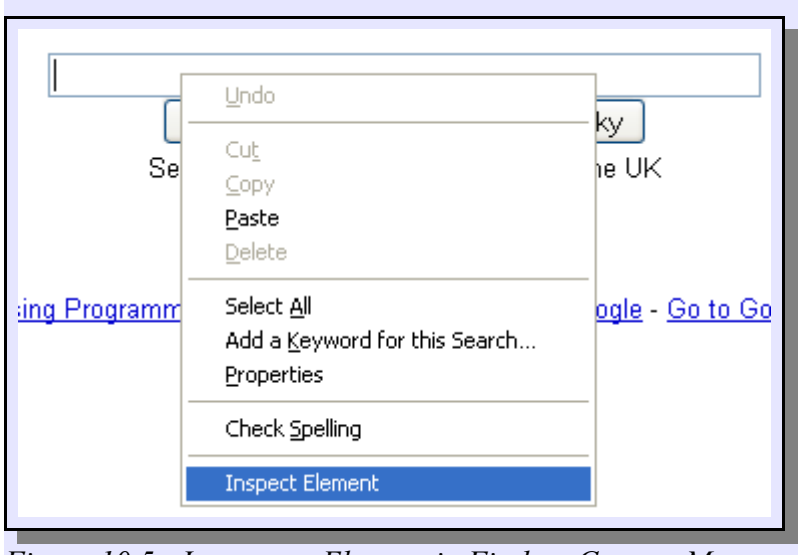


Figure 10.5 : Inspect an Element in Firebug Context Menu

If I use Google as an example then I can get the details of the Google input field by hovering the mouse over the input field. Then right clicking and selecting "Inspect Element".

This will open firebug in the "HTML view" and allow me to view the details of the HTML element

```
<td nowrap="" align="center">
    <input type="hidden" value="en" name="hl"/>
    <input value="" title="Google
    Search" size="55" name="q" maxlength="2048" autocomplete="off"/>
    <br/>
    <input type="submit" value="Google Search" name="btnG"/>
    <input type="submit" value="I'm Feeling Lucky" name="btnI"/>
</td>
```

Figure 10.6 : Code for inspected Element

From this I can easily see the identifying attributes and surrounding elements that I can use in my Selenium tests.

Experiment with the Firebug tool as it has a lot of very useful information and editing facilities that can help with your testing.

Basic XPather Usage

In our earlier tests we constructed some simple XPath statements to access the elements on the page. The XPather plugin will help us write those by constructing XPath for the selected elements automatically and allowing us to edit and manually optimise them.

First we have to install XPather. Either search for it on the Firefox add-on directory (https://addons.mozilla.org/) or go directly to the XPather home page http://xpath.alephzarro.com.

Once installed you have another new entry on the right click context menu from within Firefox: "Show in XPather".

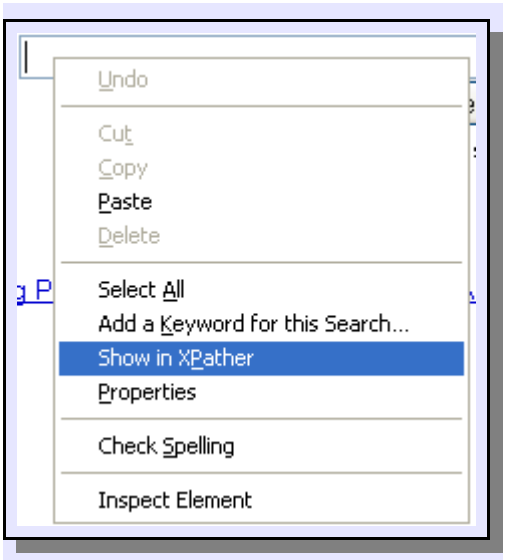


Figure 10.7 : Start XPather

So, again using Google as our example, if I select the search box on the front page and choose to "Show in XPather" I will see the XPather dialog with an example XPath for reaching that element.

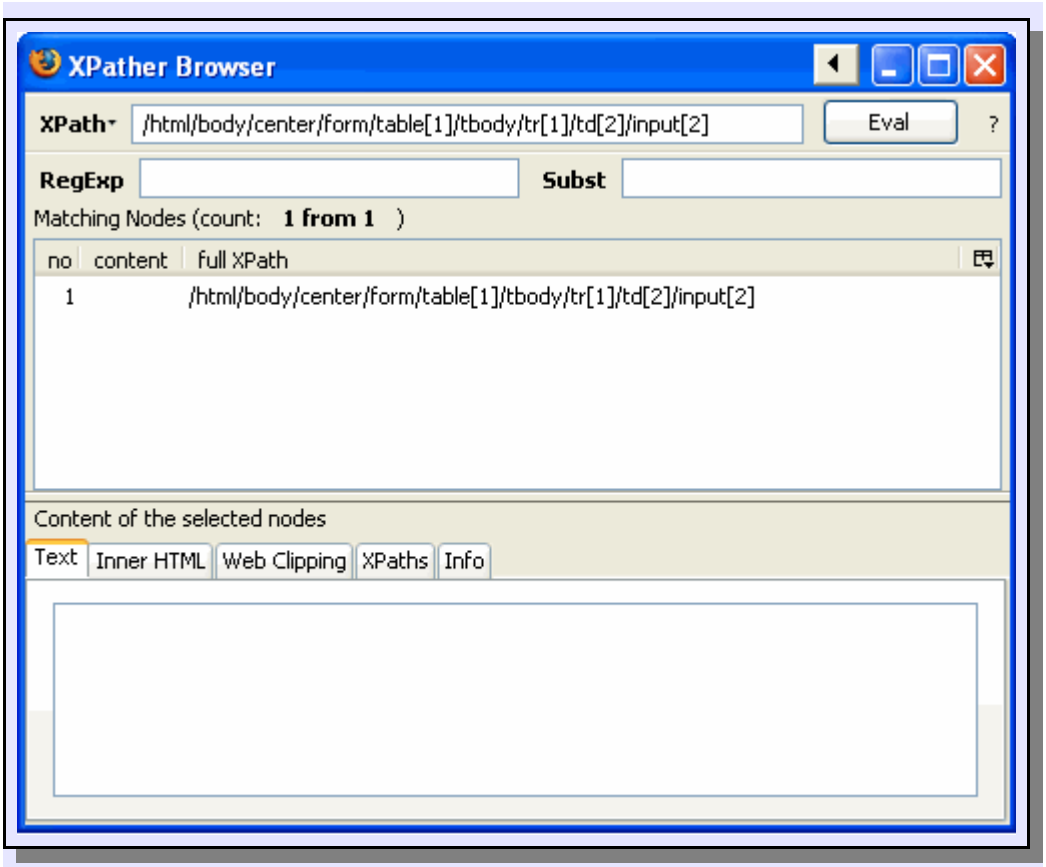


Figure 10.8 : XPather showing a set of matching nodes

Optimise XPath with XPather

Now the XPath supplied by XPather for identifying the input field, while valid, is not very good for our Selenium testing.

```
/html/body/center/form/table[1]/tbody/tr[1]/td[2]/input[2]
```

This says "starting at the top of the document '/', traverse the tree 'html', then 'body' etc."

We really do need to learn some basic XPath to help us get the best from XPather and to make our tests more robust. We will cover this in more detail in later sections, but just to start us off.

One of the things we want to do with our XPath statements is make them as robust to page changes as we can. Here our XPath starts from the top of the document so if the page is restructured in anyway then our test is likely to break. Ideally what I want to do is go straight to the input field. Here the XPath statement is telling me that there is only one form on the page, and I can see this because the 'form' text in the XPath does not have an index on it e.g. "form[1]", or "form[2]"

So I can safely shorten the XPath down and still reach the same part of the page.

```
//form/table[1]/tbody/tr[1]/td[2]/input[2]
```

This shortened form says "start at 'form' wherever it appears in the page".

Using XPather I can amend the XPath and click the [Eval] button to try out the statement. Ideally I

want the XPath statement to return a single response, but Selenium will use the first one in a list if it finds more than one matching entry.

This is a bit better but still dependent on the formatting within the form. Ideally I want to go straight to the input field. Which I could do by saying "//input" but I would need to find some attributes that uniquely identify the input field.

For example if I use the "//input" and evaluate that within XPather I get 8 matching elements returned. And the one I want is not the first one so Selenium would not access the correct element.

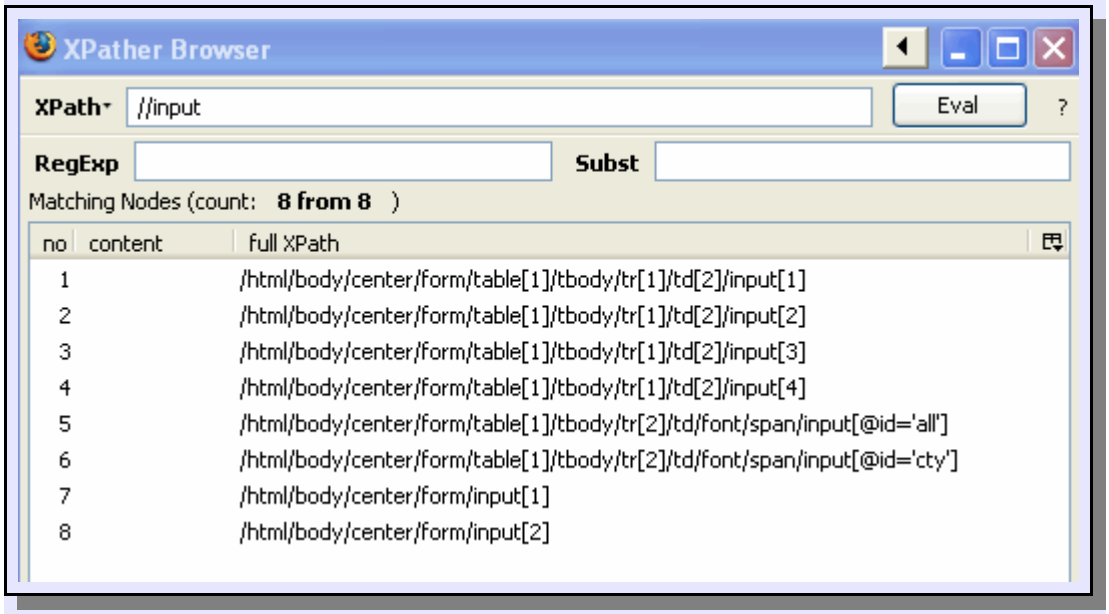


Figure 10.9 : multiple matches for an XPath statement

I could say that I want the 2nd Input, wherever it exists using "//input[2]" since this was in the set of results returned by the original XPath statement. Evaluating this returns 3 results, and the one I want is the first one, so I could use this XPath, but it still seems a little brittle since the page structure might change. I really need a more specific XPath statement.

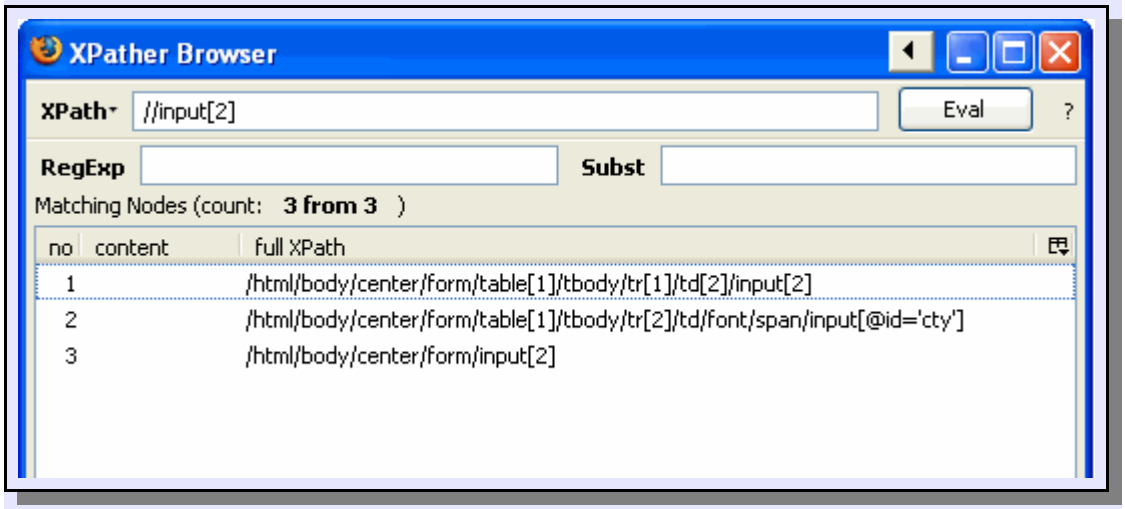


Figure 10.10 : a more specific XPath statement

I do know, from our initial inspection of this field through Firebug that this input field has a

number of attributes associated with it:

```
<input value="" title="Google Search" size="55" name="q" maxlength="2048"
autocomplete="off"/>
```

Sadly it doesn't have an 'id' which would have guaranteed its uniqueness in the page. But the name attribute may be enough to make it unique so I will try an XPath statement that combines the //input with a search for the name attribute.

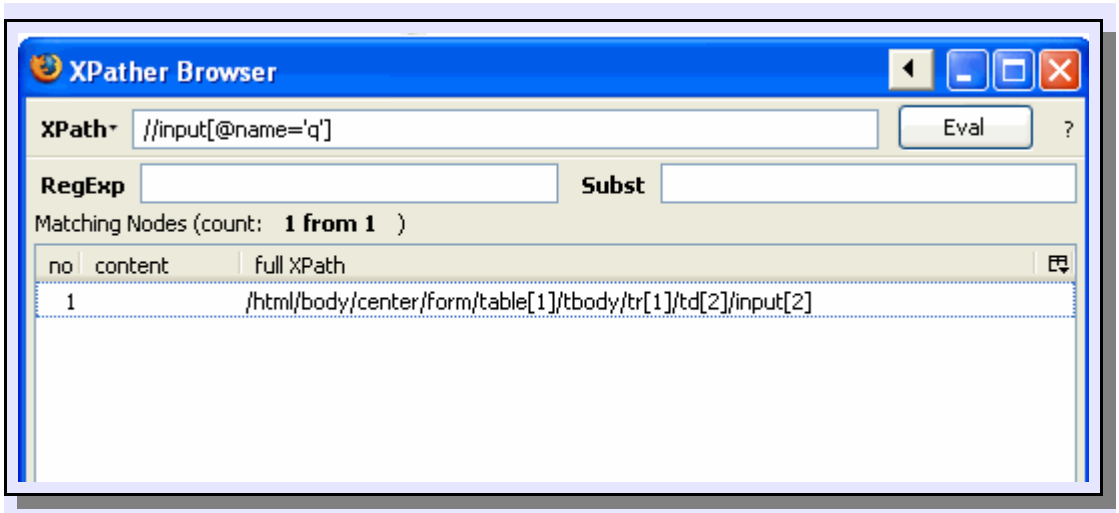


Figure 10.11 : matching on an element's name attribute

This returns a single element so I can most safely use this for the moment. This is the same XPath statement that we constructed earlier in the document. But by walking through this process above you get to see how we can use XPather to help us think about and debug our XPath statements.

Chapter 11: First Steps with Test Automation Thinking

Let's Automate Search

For our first steps in test automation thinking we shall look at a search engine:

http://compendiumdev.co.uk/selenium/search.php

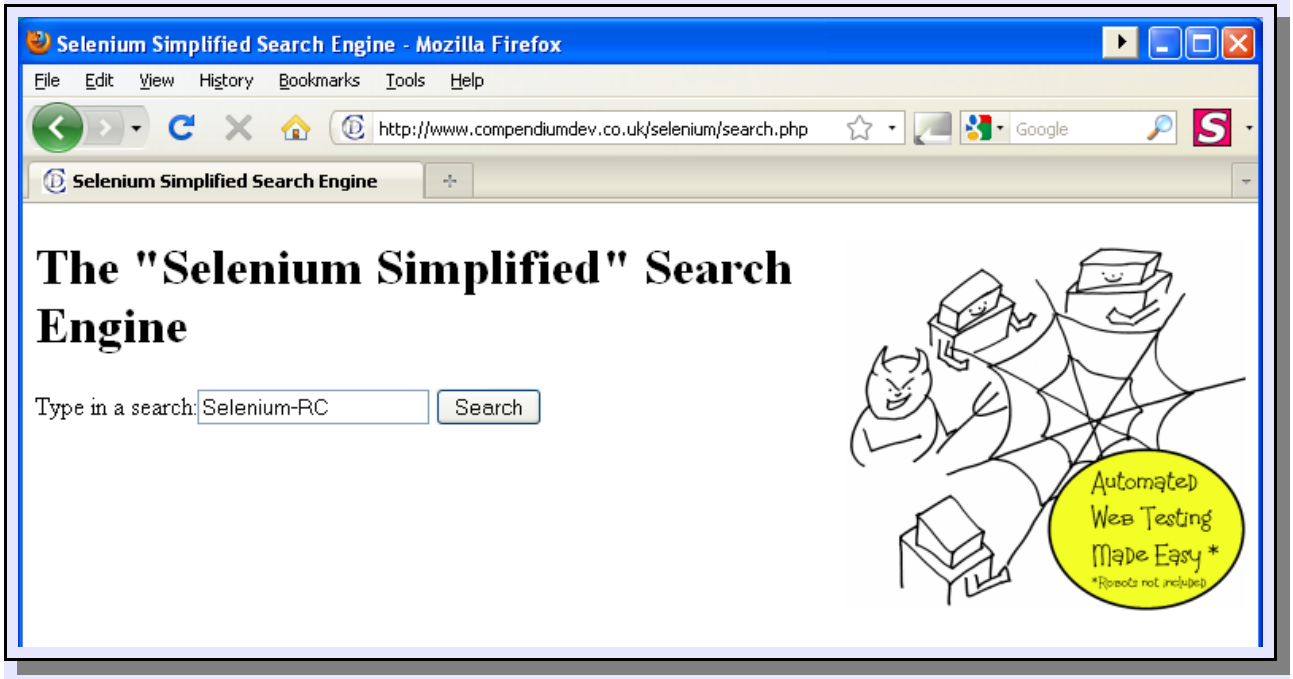


Figure 11.1 : Selenium Simplified Search Engine

OK, so possibly not the search engine you expected, but for the purposes of this book, I need an example application which will not change very much between book releases.

The Selenium Simplified Search Engine interface has:

- A big logo
- A title
- A search input box
- A [Search] button

So some simple tests would be

- typing in a search string
- press the search button
- clicking on the result links

Start writing some tests

First we shall do a basic search. To do that we need to write some code which does the following:

1. type in some text
2. press the Search Button
3. check the results in the page that loads up

OK, so preconditions:

- Start up Eclipse, it should open up your project as we left it
- Start up Selenium-RC in a DOS window as we learned to do earlier

Our very first test, already opened Google, so we can build on that to do some more work:

```
@Test
public void initialiseSeleniumRCServer(){

        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                        "http://www.google.com");
        selenium.start();
        selenium.open("http://www.google.com");
        selenium.close();
        selenium.stop();
}
```

Make Selenium Type In Some Text

Start with a copy and Paste

Copy and paste all that above, into the code window so that we start another test.

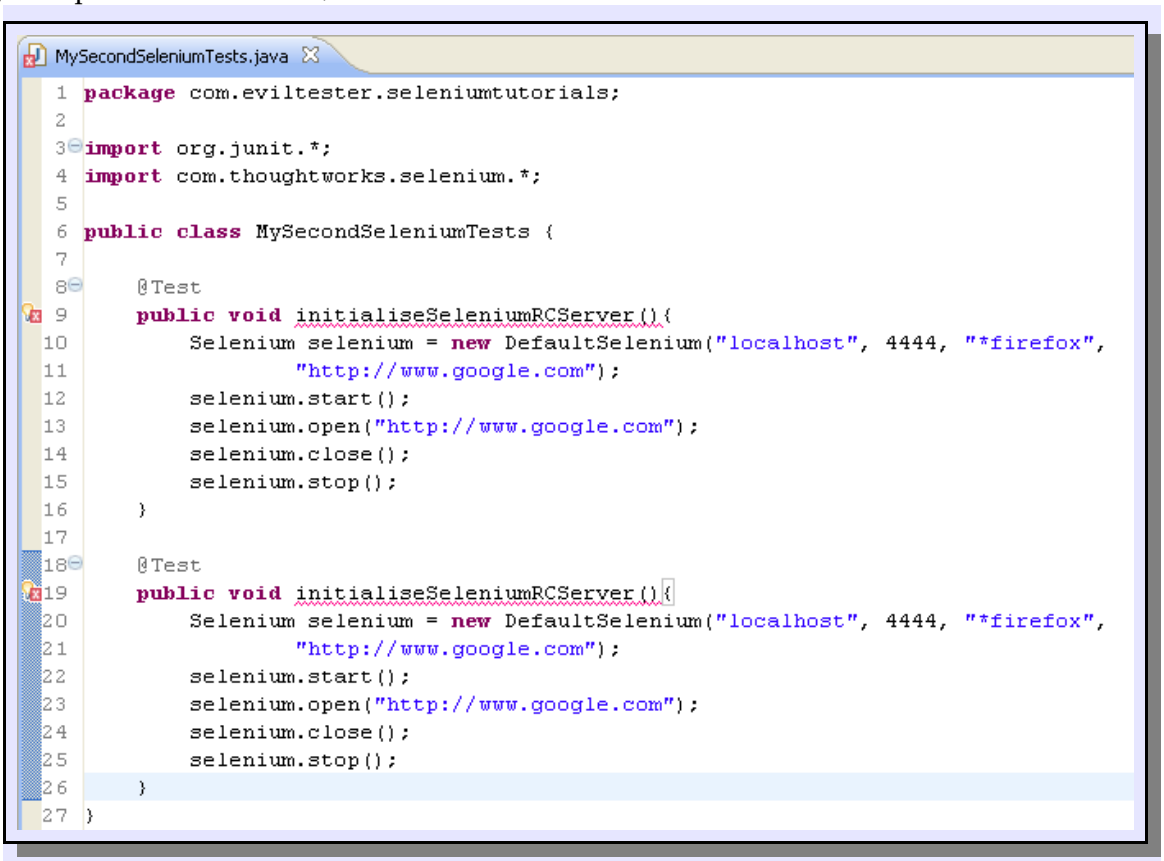


Figure 11.2 : Eclipse shows an error for duplicate test method names

Hopefully you spotted the error symbol when you did this. When we hover over the red [x] we can see the text of the error message.



Figure 11.3: Error message for duplicate method names

Eclipse has given us an error message for a "Duplicate method initialiseSeleniumRCServer() in type MySecondSeleniumTests".

Because each method in a Java class needs to have a unique name, and each of our tests is implemented as a Java method, we need to rename the method name.

Fix the duplicate method name

Amend the Java method name so that it reads "typeInASearchTerm"

```
18    @Test
19    public void typeInASearchTerm(){
20        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
21                "http://www.google.com");
22        selenium.start();
23        selenium.open("http://www.google.com");
24        selenium.close();
25        selenium.stop();
26    }
```

Figure 11.4 : Amended duplicate method name to remove error

Notice that the error indicators in Eclipse have gone away.

Now we have two methods in our Java class:

- initialiseSeleniumRCServer
- typeInSearchTerm

Both methods have the @Test annotation, so JUnit will run them as tests.

Since we are no longer testing Google, change the URL http://www.google.com to http://www.compendiumdev.co.uk

We can do this by pressing Ctrl+F or select the find and replace menu item in the Edit menu.

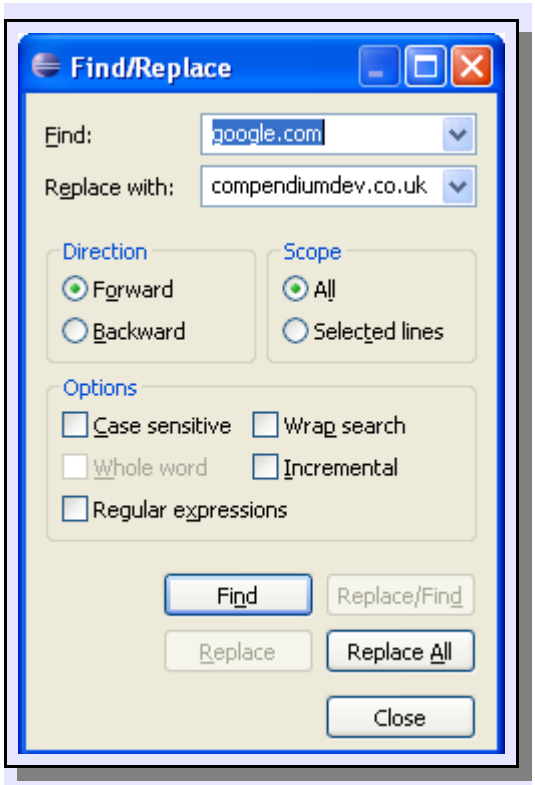


Figure 11.5 : Change URLs

Add a Selenium Command

So now, add a new line of code beneath:

```
selenium.open("http://www.compendiumdev.co.uk");
```

Start the code by typing "selenium." As soon as you typed the "." you should see the code completion dialog pop up. This lists all the Selenium-RC commands available to us as methods of the Selenium Java object.

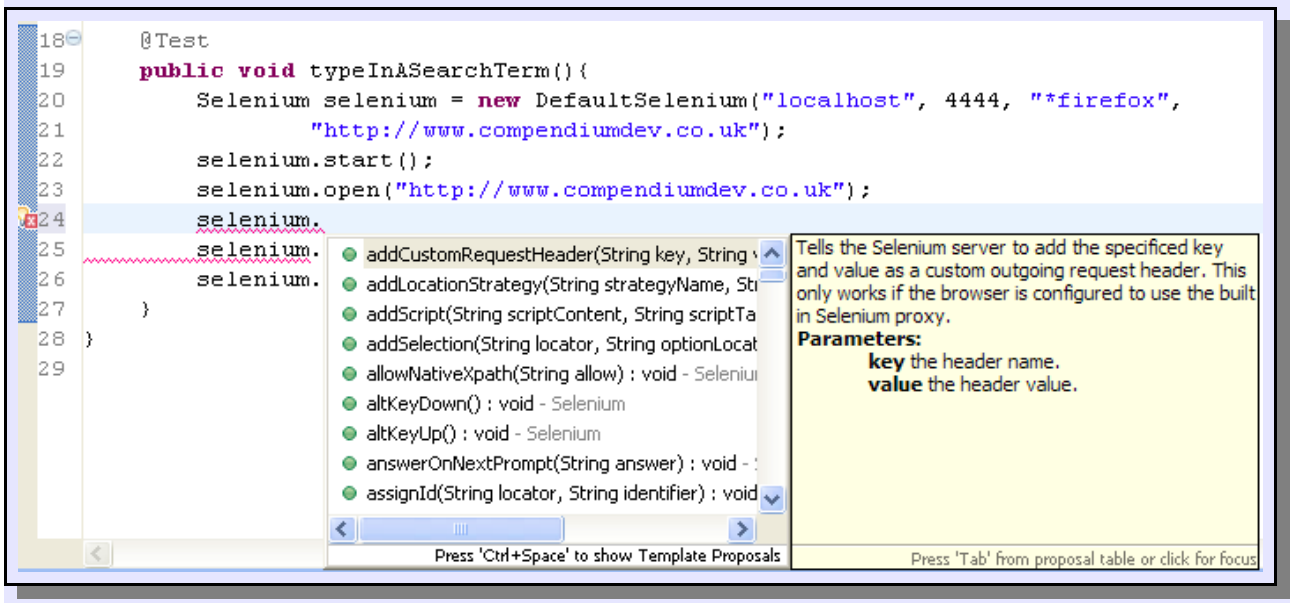


Figure 11.6 : Code completion showing methods of the Selenium interface

The popup dialog goes away after a little while, so if we want it back we put our cursor just after the "." and press Ctrl+Space.

We are going to use the type command so start typing "type" and you should see the number of options in the popup dialog reduce.

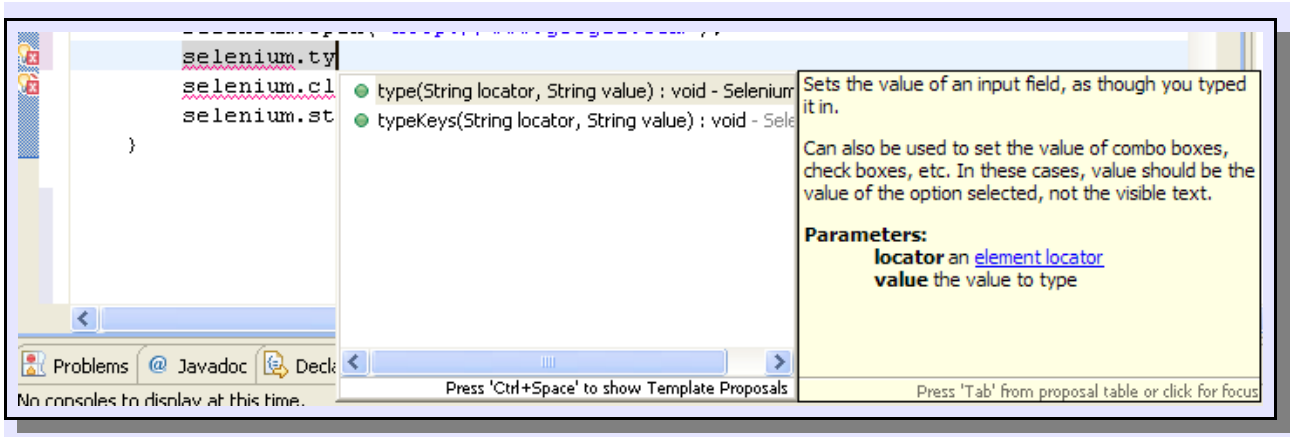


Figure 11.7 : more specific code completion showing commands starting with "ty"

You can choose the "type" method from the popup dialog if you want by pressing "space" or "return" when it is highlighted in the list.

If you do this Eclipse will add the code for you "selenium.type(locator, value)"

Add a ";" to the end of the line since all Java statements terminate with a semi-colon.

Eclipse reports an error on the line because it doesn't know what "locator" or "value" are.

Eclipse added these into the code because the "type" method takes two arguments, both strings.

The "type" method

The Type method takes two arguments:

- the first argument (locator) is a string that tells Selenium where to type the text
- the second argument (value) is the text to type

We will cover locator strings in more depth as we go on. But for now we just need to know that the locator string helps Selenium find the HTML element on the page that we are looking for.

Find the locator with FireBug

I use FireBug when constructing automated Selenium tests.

After installing FireBug do the following:

- open www.compendiumdev.co.uk/selenium/search.php in Firefox

Click on the input field so that the cursor is in the main search box, then right click the mouse and choose "Inspect Element" from the popup menu.

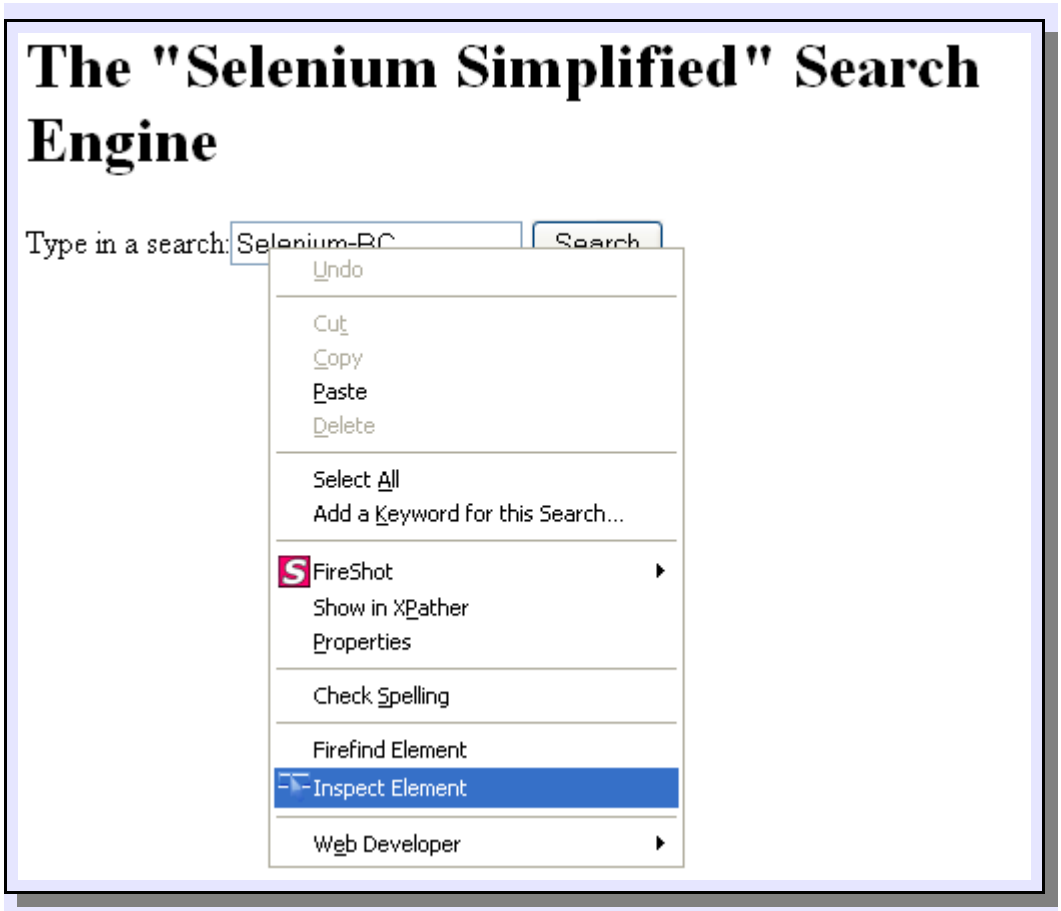


Figure 11.8 : Firebug's "Inspect Element" function from right click context menu

You should see the FireBug window displayed with the input element details visible.

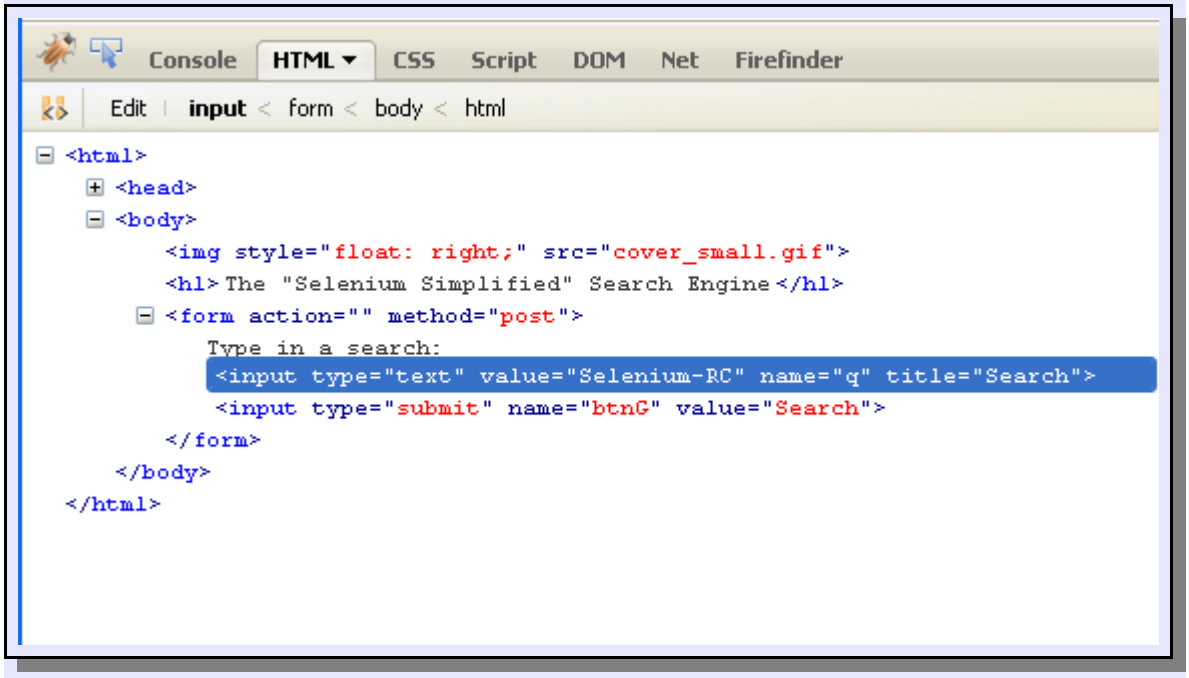


Figure 11.9 : Element is highlighted and displayed in Firebug's HTML Frame

You can also click the 'inspect' icon in the top left of the FireBug menu bar. This will highlight on

the page the elements and display them in the FireBug window.

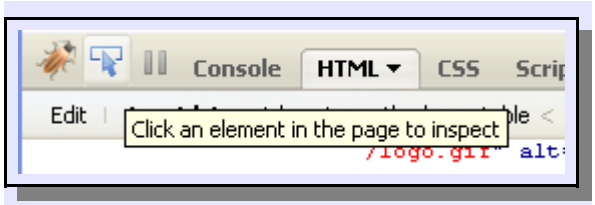


Figure 11.10 : Inspect Icon with tooltip

Having Inspected it, we now have the details we need to automate it.

I could use the name field to identify it by simply typing "q" as the locator, or "name=q".

But I will use a more fully constructed XPath to make the test more robust.

> **Note:**
>
> **I could have used the locator "q" which would try and match something with the id of 'q' or the name of 'q', or been even more specific and used the locator "name=q".**
>
> **But both of these methods are brittle. In HTML a name does not have to be unique, so additional elements with the name 'q' might get added to the page at a later date, making the test fail.**
>
> **Learning how to use XPath to make your tests more robust will serve you well in the future.**

A brief introduction to XPath

XPath is a standard language for querying XML data. The DOM can be queried by XPath as well.

- XPath specification: http://www.w3.org/TR/xpath
- XPath on Wikipedia: http://en.wikipedia.org/wiki/XPath

We will cover XPath in more detail later on in the document but we will use it gradually throughout this tutorial so that you start to learn how to use it through exposure, rather than just reading the definitions about it.

The locator that we will use will look like this:

- //input[@name='q']

This means:

- // - find (anywhere in the document)
- input – an input element
- [- which has
  - @ - an attribute
  - name – called name
  - ='q' – with a value of 'q'
-]

Add the locator into the code

So add this XPath statement into your code:

```
selenium.type("xpath=//input[@name='q']", value);
```

We can tell Selenium that the locator is an XPath statement by writing 'xpath=' before the XPath statement.

> **Note:**
>
> **We can leave out the "xpath=" when we use an XPath starting with "//" because selenium will treat it as an XPath string.**

And all that remains now is to search for something, so change "value" to read "Selenium-RC".

```
selenium.type("xpath=//input[@name='q']", "Selenium-RC");
```

When you do this you should also notice that Eclipse no longer reports an error against this line because we have written a valid Java statement.

Check that it works

If you want, you could run this now by:

- first removing the @Test annotation from the `initialiseSeleniumRCServer` method so we only run the `typeInASearchTerm` test.

> **Note: We need to do this because JUnit will run any of the methods annotated as @Test, and since we are only interested in one at the moment it makes sense to not have the other one annotated at the moment.**
>
> **We could also use the @Ignore annotation and place it before the @Test annotation:**
>
> **e.g. @Ignore @Test**

- putting a debug statement on the "selenium.close();" line
- right click on the test class and choose to "Debug As \ JUnit Test"
- You should see the Selenium test open up the search page and perform a search. Notice that the commands we have used e.g. "selenium.open", get displayed in the top right hand corner of the "Selenium Remote Control" browser window that appears.
- Finish off the debug process by clicking the resume button.
- Then click on the "Java perspective" toolbar button to go back to our source code view



Figure 11.11 : Java Perspective button

Now make it Click on the Search Button

We will follow a similar process for the search button:

- start adding the Selenium code
- find the button on the page
- create an XPath so that selenium can locate it
- add the XPath details to our source code

Selenium "Click" Method

Start adding a new command after the type statement.

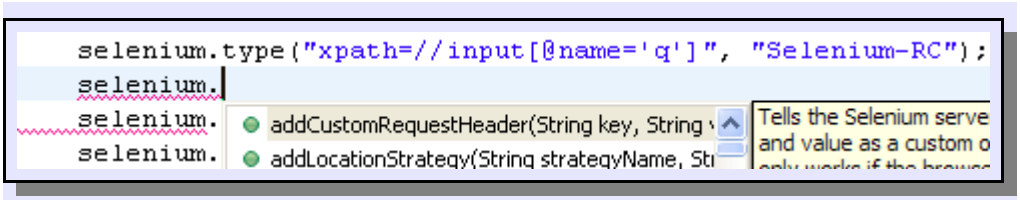


Figure 11.12: Use code completion to identify Selenium commands

By using code completion we can see that the Selenium command we need to use is the "click" command.

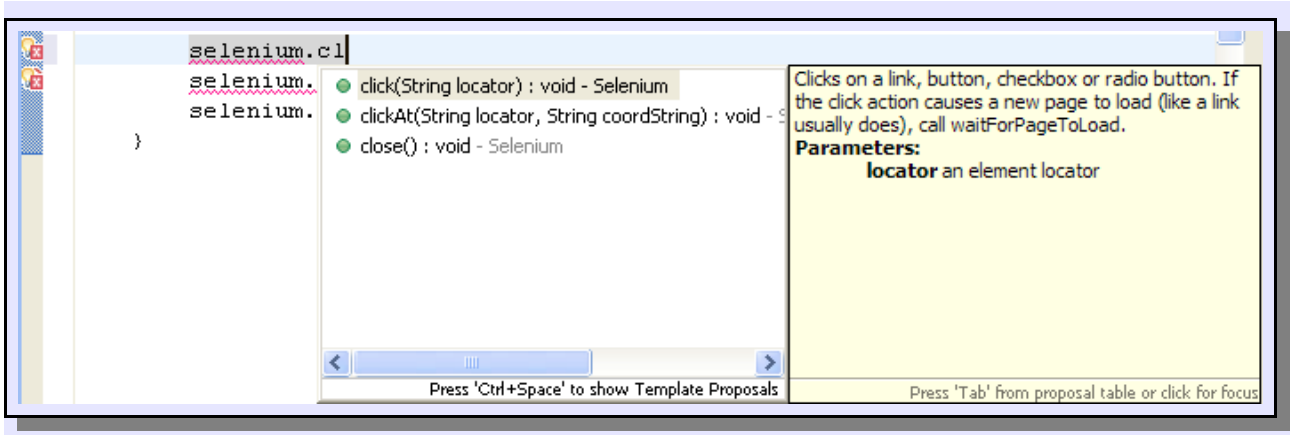


Figure 11.13 : Click command code completion

The click command takes a single argument (locator), the location of the HTML element we want to click on.

So use the code completion context menu to have Eclipse write our basic code for us. Remembering to add the ";" to the end of the line.

```
selenium.click(locator);
```

Locate the button with FireBug

Use the Inspect icon and click on the "[Search]" button. And the details for the Search button are displayed in the source code window.

```
<input type="submit" value="Search" name="btnG" />
```

Create an XPath so that selenium can locate it

A very similar XPath to the last one, but this time we will use two attributes to identify the button:

```
//input[@name='btnG' and @type='submit']
```

Here we are using the 'and' conditional operator to match items with the name 'btnG' and a type of 'submit'.

We could just use the name to find it, but here you are learning that we can use multiple attributes and longer conditional statements to create more specific locator strings.

Add the XPath details to our source code

So overwrite the “locator” in your source code so that the click statement line looks like this:

```
selenium.click("xpath=//input[@name='btnG' and @type='submit']");
```

One last thing to do

When Selenium clicks on the button, the request will be submitted and a new page will load. So I'm going to add a wait for the page to load after the click:

```
selenium.waitForPageToLoad("30000");
```

This will wait 30 seconds for the page to load, and if it hasn't loaded by the time 30 seconds roll around it will timeout with an exception and the test will fail.

Now check that it works

Now run the test in debug mode to make sure that it does what you expected.

- Make sure there is a breakpoint on “selenium.stop();”
- right click on MySecondSeleniumTests.java in the Package Explorer and choose “Debug As \ JUnit test”
- Hopefully you saw the “Selenium Remote Control” window actually do the search.
- Back in Eclipse, press “F8” or click on the “Resume” button in the debug toolbar to finish your test.
- Click on the “Java perspective” toolbar button to go back to your source code

Quick Summary

Great, you made Selenium-RC do something, and you did it by writing code.

So just to remind you of what you learned:

- the Selenium type command
- the Selenium click command
- some XPath
- use of “xpath=” or “//” to create XPath selenium locators
- @Ignore to prevent tests from running
- Selenium uses locator strings to find items on a page

- how to use the FireBug and XPather to find items on a page
- more familiarity with Debugging tests
- using Eclipse to swap perspectives

And just to remind you what the test you wrote looked like if you followed the instructions:

```
@Test
public void typeInASearchTerm(){

        Selenium selenium = new DefaultSelenium("localhost",
                4444, "*firefox", "http://www.compendiumdev.co.uk");

        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");

        selenium.type("xpath=//input[@name='q']", "Selenium-RC");

        selenium.click("xpath=//input[@name='btnG' and @type='submit']");

        selenium.waitForPageToLoad("30000");

        selenium.close();
        selenium.stop();
}
```

But is it a test if we don't check any results?

Back when we started this session the basic test we were going to write was:

1. type in some text
2. press the Search Button
3. check the results in the page that loads up

We have managed to do the first two items.

But is it a 'test' if we don't check the results?

We ultimately want these tests to run without us watching them, and have the tests report accurately the passes and failures.

At the moment these tests might fail for some of the following reasons:

- The names of the input fields on the form changed so our locators stopped working and Selenium could not find the elements
- compendiumdev.co.uk servers got slow and the pages timed out

They would not fail if:

- The search suddenly started performing random searches so instead of searching for “Selenium-RC” like we asked it to, it searched for “Surrealism DC” - our test would not know.

We need to add some code to check for basic acceptance conditions.

But we can't control the data

We have the complication here that, we don't control the data that the search engine searches against so we can't guarantee the results it will return.

Therefore we cannot adopt the strategy of checking each of the links returned and making sure

they match the data we expect to have returned.

What can we do?

Well, we know that the search engine does a pretty good job of returning results that match your searches. We should probably expect to see the Selenium Remote Control homepage in the list of results on the first page.

Certainly when I did the search it was on the first page, and was in fact the first item returned.

So one acceptance criteria we can use is:

- Selenium Remote Control homepage listed in returned results

By looking at the returned page we can see a few things that will help determine if the search engine has done the search we asked for.

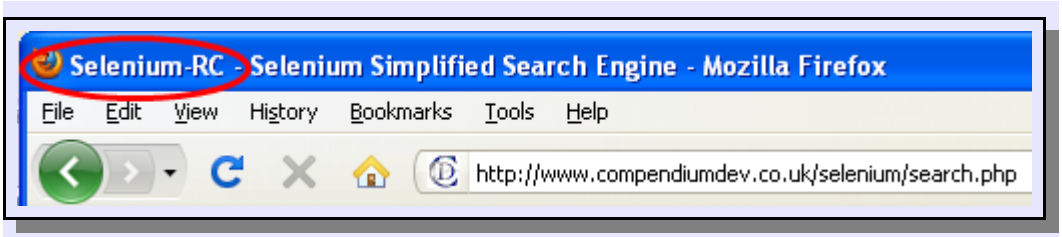


Figure 11.14 : selenium-rc in title

The search box on the page has the search term we typed in so I can check that too.



Figure 11.15 : Selenium-RC was added to the search input field

So the full list of acceptance criteria that I will automate for this 'test' are:

- Selenium Remote Control homepage listed in returned results
- Page title has “Selenium-RC” in it
- Search box on the page has Selenium-RC in it

Automate the Acceptance Criteria

Acceptance Criteria: Selenium Remote Control homepage listed in returned results

We have a number of ways of checking for the homepage as a result in the page.

1. have selenium look for text we know is on the home page
2. get the source code for the page and scan it for the text we want
3. look for a specific URL in the page

The third item will turn out to be the most robust but I think it is important for you to understand alternative strategies as well. Some of these may be suboptimal in this situation, but may help you in the future.

Look for text in the page

Of all the options this one is the weakest and most likely to break or provide a false positive result. A false positive result is when the test passes, but the condition that you wanted tested fails.

But sometimes looking for text may be exactly what you want to do so we'll explore it here.

It only takes one command to check if the text is present on the page.

```
selenium.isTextPresent(pattern);
```

The isTextPresent method takes a single argument, the string you want to search for.

Based on the results of the search we could pick out some of the returned text.

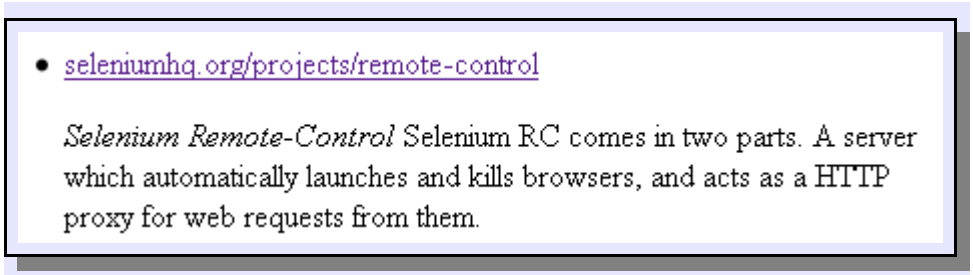


Figure 11.16 : Examine the search results for text to check for

And check for the text using the "isTextPresent" method.

```
selenium.isTextPresent("Selenium Remote-Control Selenium RC comes in two parts. A
server which automatically");
selenium.isTextPresent("launches and kills browsers, and acts as a HTTP proxy for web
requests from them.");
```

This method of checking is pretty brittle since the Selenium Remote Control homepage content is highly likely to change and any time you are testing something with variable content where you embed that content in your test then you better expect a few things to happen:

- increased test maintenance effort
- frequent failures of your automated test pack

So consider carefully if this is a strategy that you really want to pursue. It may be exactly what you want, but you have to be aware of the brittleness that you are building into your test pack.

Initially we might expect the test that uses this method to look like this:

```
@Test
public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
        Selenium selenium = new DefaultSelenium(
                "localhost", 4444, "*firefox", "http://www.compendiumdev.co.uk");

        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        selenium.isTextPresent(
                "Selenium Remote-Control Selenium RC comes in two parts." +
                " A server which automatically"));
        selenium.isTextPresent(
```

```
                "launches and kills browsers, and acts as" +
                " a HTTP proxy for web requests from them."));
        selenium.close();
        selenium.stop();
}
```

Edit your tests so that you create a new test like the above and run it through in debug mode and see if it works.

Your test passed right? So now change the text you were checking the presence of and see if the test fails.

```
selenium.isTextPresent(
        "Selenium Remote-Control Selenium RC blah blah yadayee ya");
```

I suspect that text is not on the page. So running it now, did the test fail?

Mine didn't. But the text is not present on the page. So what went wrong?

Use assertions to make the test fail on error

Some methods in Selenium will throw exceptions if they don't work properly, and that will cause the test to fail – which is what we want since we want to know if the test hasn't run properly. e.g. “open”, “close”, “type”, “click”.

But some methods return values which we can use in our test e.g. isTextPresent returns a boolean value: true, or false.

We need to check the returned value from isTextPresent and fail the test if it is not what we expect.

To do this we are going to use an assert.

An assert is a statement which “makes an assertion that something will be a specific way” (read that with your most authoritative inner voice), and it throws an exception if the world is not the way it expects it to be.

We can use one of the asserts built into JUnit e.g. assertEquals, assertTrue, assertFalse. At the moment all we want to do is check that a value is true so we will use assertTrue.

The assertion wrapped isTextPresent method should look like this:

```
assertTrue(selenium.isTextPresent(
        "Selenium Remote-Control Selenium RC comes in two parts." +
        " A server which automatically"));

assertTrue(selenium.isTextPresent(
        "launches and kills browsers, and acts as" +
        " a HTTP proxy for web requests from them."));
```

What we have just written here means:

- assert that (the value returned from the selenium.isTextPresent method) is true

And if you typed those lines into your source code then you will notice that Eclipse reports an error against these lines, because we haven't told it about the assertTrue method in JUnit.

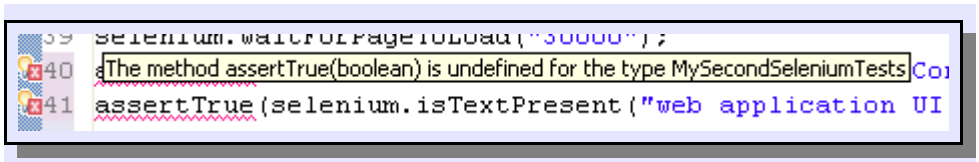


Figure 11.17: Eclipse highlighted assertTrue as erroneous

Eclipse should be able to fix this error because we have told it about JUnit, but the assert methods have to be statically imported.

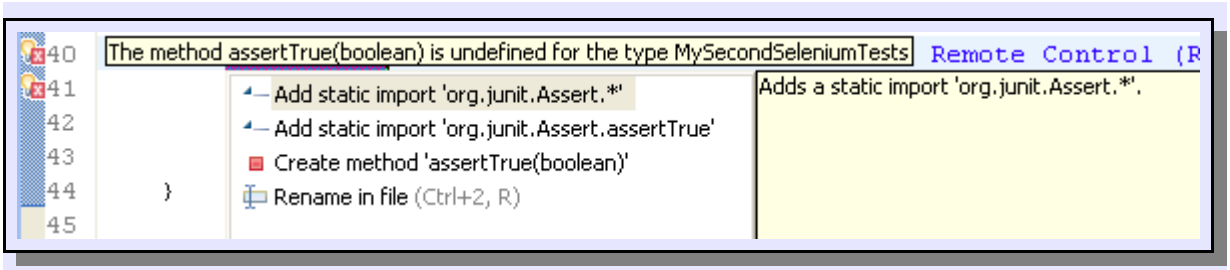


Figure 11.18: Tool tip explaining the error associated with assertTrue

So selecting "Add static import 'org.junit.Assert.*'

Eclipse will add a static import to our code.

```
import static org.junit.Assert.*;
```

This will allow us to use any of the Asserts which JUnit provides.

If you followed the instructions as intended then Eclipse should be showing an error free source code. And if we run the test now with the correct text it should pass, and if we change the text to be text which is not on the page, it should fail.

Now that you have a failing test we should have a quick look at how to debug a failing test.

Debugging a Failing Test

When you ran the test with the incorrect text, hopefully you saw the JUnit tab display a red bar, and identify a failing test.

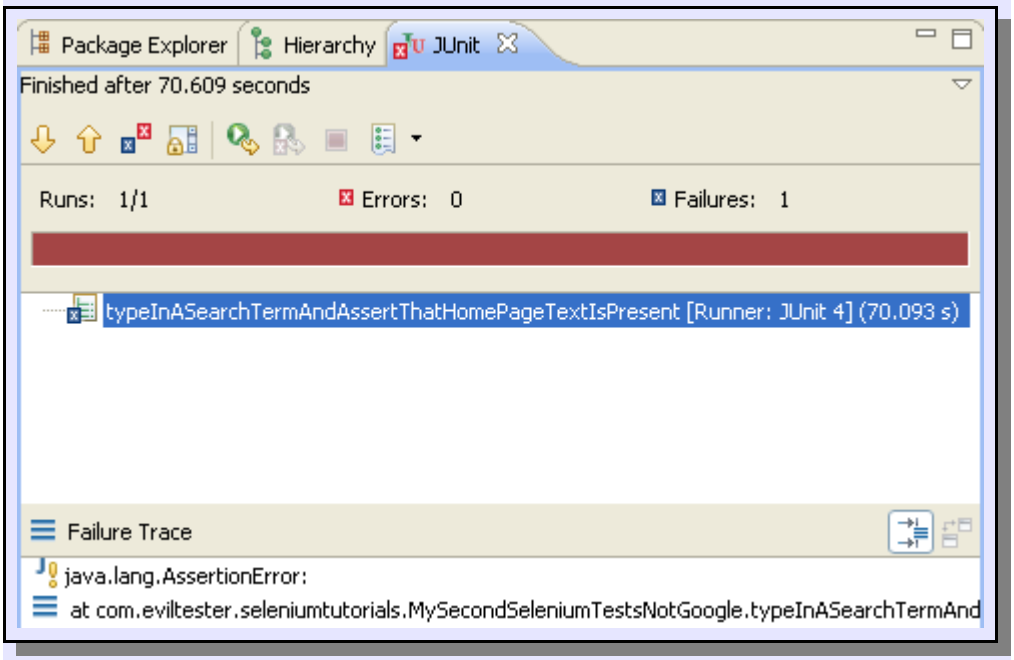


Figure 11.19 : The test runner showing a failed test

The big red bar across the top tells us that a test failed.

The crosses on the tree tell us which tests in the hierarchy failed.

The window on the bottom, when we select one of the failing tests, shows us the Java Stack trace for the test execution. The Java stack trace is the list of error messages that we use to work out what went wrong.

Look at the error reported, and scroll to the right so you can see the error code. It fails with an assertion error.

If we double click on the line in the stack trace then we will jump to the line of code that generated the exception, allowing us to debug and fix it.

Note:

Because the test failed before we reached the selenium.close(); and selenium.stop(); statements, the selenium browser windows will be left open.

We will see how to handle this side-effect later by using the @After and @AfterClass annotations in our refactoring chapters.

Our final test with assertions

Our final test should look like the following:

```
@Test
public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
"http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        assertTrue(selenium.isTextPresent(
                "Selenium Remote-Control Selenium RC comes in two parts." +
                " A server which automatically"));
        assertTrue(selenium.isTextPresent(
                "launches and kills browsers, and acts as" +
                " a HTTP proxy for web requests from them."));
        selenium.close();
        selenium.stop();
}
```

Note:

In JUnit, all assert methods can take a description as the first parameter e.g.

assertTrue("false is not true", false);

Using these textual descriptions can make understanding the stack trace a little easier.

Without the description you have to get used to clicking through to the code to see the line in error.

For JUnit assertTrue in particular the error message can seem confusing:

junit.framework.AssertionFailedError: null

at junit.framework.Assert.fail(Assert.java:47)
at junit.framework.Assert.assertTrue(Assert.java:20)
at junit.framework.Assert.assertTrue(Assert.java:27)

The 'null' is the description of the error, and since we didn't add one it gets reported as 'null'. You can find a useful discussion of this on the JUnit web site http://www.junit.org/node/434

You will see usage of this approach later in this chapter.

Scan the page source code

Selenium's getHtmlSource() method will return the HTML source code of the page so we can parse it and compare it using normal java tools.

In some instances this can speed up our tests. For example, every time we use a Selenium-RC command we are:

- sending a message from Java to the Selenium-RC server
- the Selenium-RC server is communicating with the Selenium Remote Control browser
- the Selenium Remote Control browser is using JavaScript to manipulate the browser DOM

Any results that come back have to come back through that chain as well.

This can make Selenium slow if you do a lot of query commands.

So one strategy we might choose to use is a single command to return the HTML source code and then parse that ourselves.

For our particular purpose here this is overkill, but I'll recreate the isTextPresent test above using this method.

```
@Test

public void typeInASearchTermAndAssertThatHomePageTextIsPresentInSource(){

        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                        "http://www.compendiumdev.co.uk");

        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("10000");
        String pageSource = selenium.getHtmlSource().toLowerCase();
        assertTrue(pageSource.contains("Selenium Remote-Control Selenium RC comes in two
parts. A server which automatically".toLowerCase()));
        assertTrue(pageSource.contains("launches and kills browsers, and acts as a HTTP
proxy for web requests from them.".toLowerCase()));
        selenium.close();
        selenium.stop();
}
```

So here, you can see the changes I had to make.

```
String pageSource = selenium.getHtmlSource().toLowerCase();
assertTrue(pageSource.contains("Selenium Remote-Control Selenium RC comes in two parts.
A server which automatically".toLowerCase()));
assertTrue(pageSource.contains("launches and kills browsers, and acts as a HTTP proxy
for web requests from them.".toLowerCase()));
```

I create a new variable (pageSource) which is a Java String object that will store the source code returned by selenium's getHtmlSource method.

Then I directly replaced the *selenium.isTextPresent* with *pageSource.contains*

contains() is a method of a Java String object which returns true if the object contains the string passed as argument into the method.

Try it and see if this works in the same way as the previous test.

Did yours work?

Mine didn't.

And this illustrates one of the dangers of using the source code directly. The HTML page doesn't display the raw text. Look at the screenshot and we can see that our Search Engine has added formatting in the text to make bold the first words in the search results paragraph.



Figure 11.20 : The search text has HTML formatting

If we look at the source code of the page we can see that there are embedded HTML commands in the page so we might try and search for:

- "Selenium Remote-Control Selenium RC comes in two parts. A server which automatically"
- "launches and kills browsers, and acts as a HTTP proxy for web requests from them."

If you look at the source-code for the test you will see that I have added a .toLowerCase() when retrieving the page source-code and in the assert on our snippet. IE and Firefox store the source code differently internally. Internet Explorer capitalises the HTML commands internally, but not when displaying the source so this would fail in Internet Explorer if we did not convert it to lowercase.

Every time you use the source code directly you run into these potential problems. Also since HTML ignores white space when rendering you may find that you have to add additional new lines into your comparison strings.

So for our purposes getting the source code is not really what we want to do.

Look for a specific URL in the page

Since we know that the homepage for the Selenium-RC project is

http://seleniumhq.org/projects/remote-control

Then we could just check for the presence of an href with this on the page.

But it might equally be:

- http://seleniumhq.org/projects/remote-control/
- http://seleniumhq.org/projects/remote-control/index.html

Also, in a recent re-organisation of the Selenium site. There is now `http://selenium-rc.seleniumhq.org` which redirects to http://seleniumhq.org/projects/remote-control/

So if we consider all these valid alternatives and are happy if any one of them appears on the page then we can use XPath to help us do that.

We will use the selenium method. getXpathCount which returns a Java Number object representing the number of times the XPath matches something on the page.

An XPath which would match the first URL is:

- //a[@href='http://seleniumhq.org/projects/remote-control']
- which means: find any anchor elements with an href attribute of 'http://seleniumhq.org/projects/remote-control'
- e.g. Selenium Home

getXpathCount takes an XPath statement as an argument. Which means that we don't write "xpath=..." when we pass the XPath string in as an argument.

If the URL "`http://seleniumhq.org/projects/remote-control`" was on the page 3 times, then the getXpathCount would return a decimal object of 3.

```
getXpathCount("//a[@href='http://seleniumhq.org/projects/remote-control']")
```

In order to use the returned value, we really want it to be an integer so we will convert it to an integer using the .intValue() method of the Number class e.g.

```
getXpathCount("//a[@href='http://seleniumhq.org/projects/remote-control']").intValue();
```

The test code looks as follows, I have added inline java comments so you can understand the code:

```
@Test
public void typeInASearchTermAndAssertThatHomePageURLExists(){
        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
"http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("10000");

        /* create a new variable called matchingCountTotal to calculate
           the total number of times our xpath matches, initially setting
           it to 0   */
        int matchingCountTotal = 0;

        // call getXpathCount with our first xpath for the url and
        // add the intValue version to the total
        matchingCountTotal += selenium.getXpathCount("//a[@href='http://selenium-
rc.seleniumhq.org']").intValue();
        matchingCountTotal += selenium.getXpathCount("//a[@href='http://selenium-
rc.seleniumhq.org/']").intValue();

        // call the getXpathCount again with our next url and add
        // the returned intValue to the total
        matchingCountTotal +=
selenium.getXpathCount("//a[@href='http://seleniumhq.org/projects/remote-
control']").intValue();

        // call the getXpathCount again with our next url and add
        // the returned intValue to the total
        matchingCountTotal +=
selenium.getXpathCount("//a[@href='http://seleniumhq.org/projects/remote-
control/index.html']").intValue();

        // call getXpathCount with our last url xpath and add
```

```
        // the returned intValue to the total
        matchingCountTotal +=
selenium.getXpathCount("//a[@href='http://seleniumhq.org/projects/remote-
control/']").intValue();

        assertTrue("No homepage URL found",matchingCountTotal>0);

        selenium.close();
        selenium.stop();
}
```

So a few things to note in the above test:

- In Java anything after // is treated as a comment in the line and not as code.
- You can also comment blocks of text by adding /* to start the comment and */ to end it
- We do not have to prefix arguments to getXpathCount with "xpath="
- We convert the results of getXpathCount to an integer to use it easily
- "a += b" in Java means add the value of "b" to the variable "a" (the same as a = a+ b)
- We have used a different form of assertTrue – this time we have added an error message to display if the assert fails, this will make it easier to debug the test.

Try that test out and see how you get on. If it passed then you might want to amend it to make it fail so that you can see the benefit of adding text strings to your assertions. To make it fail, amend the assertTrue statement to read:

```
        assertTrue("No homepage URL found",matchingCountTotal==-1);
```

Running it now, you should notice a few things:

- The Selenium Remote Control window did not close
- The assertion error in the console window and in the JUnit stack trace is more readable

The Selenium Remote Control window did not close because we did not reach the selenium.close(); line. This is one reason for spending some time in the "What if it all goes wrong?" section, as you may find yourself having to tidy up after failing tests. *We will look at some JUnit functionality which can help us tidy up our environment, even in the face of failing tests, in a later section*. The assertion error in the stack trace now displays "No homepage URL found" so it is much easier to see why the test failed when an assertion is not met.

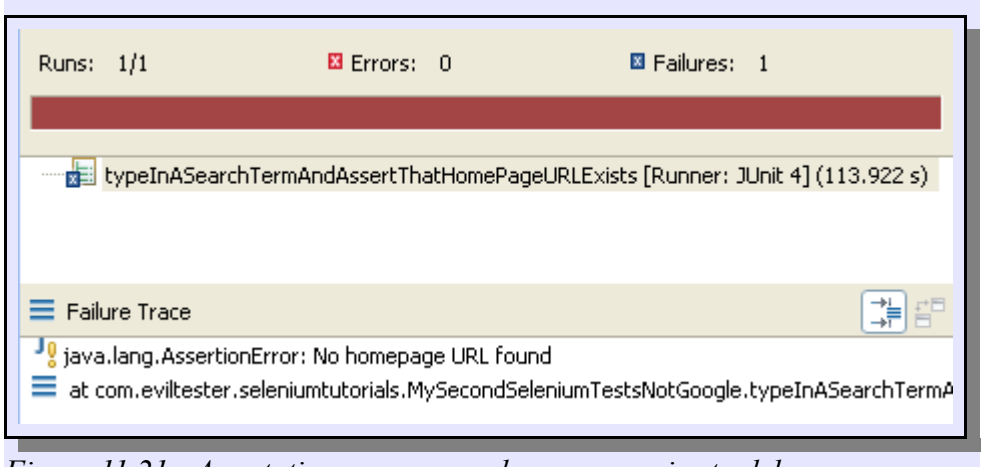


Figure 11.21 : Annotation message makes error easier to debug

Note: A quick note on XPath optimisation.

XPath is a programming language and it is possible to write a single XPath statement which finds all the versions of the homepage URL by using the starts-with function in XPath, since all our URLs start with the same string. e.g.

"//a[starts-with(@href,'http://seleniumhq.org/projects/remote-control')]")

So the test would look like:

```
@Test
public void typeInASearchTermAndAssertThatHomePageURLExistsWithFewerXpath(){
        Selenium selenium = new DefaultSelenium(
                                                "localhost", 4444, "*firefox",
                                        "http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("10000");

        // create a new variable called matchingCountTotal to calculate
        // the total number of times our xpath matches, initially setting
        // it to 0
        int matchingCountTotal = 0;

        // call getXpathCount with our xpath for the url and
        // add the intValue version to the total
        matchingCountTotal += selenium.getXpathCount(
                        "//a[starts-with(@href,'http://selenium-rc.seleniumhq.org')]")
                        .intValue();
        matchingCountTotal += selenium.getXpathCount(
                "//a[starts-with(@href,'http://seleniumhq.org/projects/remote-control')]").intValue();

        assertTrue("No homepage URL found",matchingCountTotal>0);

        selenium.close();
        selenium.stop();
}
```

Most Robust Method

Of the 3 methods we have just looked at for finding the homepage URL in the page:

- text in the page
- scanning the source code
- look for a specific URL

I think that looking for the specific URL is the most robust:

- The URL is less likely to change than text on the home page
- The source code method is horribly brittle and cross browser testing is harder

Acceptance Criteria: Page title has "Selenium-RC" in it

Selenium has a command to return the page title:

- getTitle()

getTitle takes no arguments and returns a String assigned with the title of the page in it.

```
String pageTitle = selenium.getTitle();
assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
       pageTitle.contains("Selenium-RC"));
```

So we end up with a test like the following:

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
       Selenium selenium = new DefaultSelenium(
                                    "localhost", 4444,
                                    "*firefox", "http://www.compendiumdev.co.uk");
       selenium.start();
       selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
       selenium.type("xpath=//input[@name='q']", "Selenium-RC");
       selenium.click("xpath=//input[@name='btnG' and @type='submit']");
       selenium.waitForPageToLoad("30000");

       String pageTitle = selenium.getTitle();
       assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
pageTitle.contains("Selenium-RC"));

       selenium.close();
       selenium.stop();
}
```

Acceptance Criteria: Search box on the page has Selenium-RC in it

The search box on the web page is a form element.

Selenium has a command called getValue to return the value of form elements.

So by going through a process of finding the XPath of the search box form element as we did previously we can construct an XPath locator to use with the getValue statement.

```
String searchTerm = selenium.getValue("xpath=//input[@name='q' and @title='Search']");
```

Our test looks like the following.

```
@Test
public void typeInAGoogleSearchTermAndCheckSearchInputHasSearchTermInIt(){
       Selenium selenium = new DefaultSelenium("localhost", 4444,
                                    "*firefox", "http://www.compendiumdev.co.uk");
       selenium.start();
       selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
       selenium.type("xpath=//input[@name='q']", "Selenium-RC");
       selenium.click("xpath=//input[@name='btnG' and @type='submit']");
       selenium.waitForPageToLoad("30000");
       String searchTerm = selenium.getValue(
              "xpath=//input[@name='q' and @title='Search']");
       assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
              searchTerm.equals("Selenium-RC"));

       selenium.close();
       selenium.stop();
}
```

One test to rule them all

By taking all the above code for the acceptance criteria. We could choose to leave all the individual tests. Or we could roll all the acceptance criteria into a single test.

```
@Test
public void typeInASearchTermAndCheckResultsPage(){
```

```
        Selenium selenium = new DefaultSelenium("localhost", 4444,
                                "*firefox", "http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");

        String searchTerm = selenium.getValue(
                "xpath=//input[@name='q' and @title='Search']");

        assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
                        searchTerm.equals("Selenium-RC"));

        String pageTitle = selenium.getTitle();
        assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
                        pageTitle.contains("Selenium-RC"));

        int matchingCountTotal =
                selenium.getXpathCount(
                "//a[starts-with(@href,'http://seleniumhq.org/projects/remote-
control')]").intValue();

        matchingCountTotal += selenium.getXpathCount(
           "//a[starts-with(@href,'http://selenium-rc.seleniumhq.org')]").intValue();
        assertTrue("No homepage URL found ",matchingCountTotal>0);

        selenium.close();
        selenium.stop();
}
```

In favour of this approach we have:

- A single test to maintain
- All the acceptance criteria have to be met for the functional implementation to pass anyway
- Removed duplication in the set of tests

Against this approach we have:

- Not quite as readable from the test title what is being tested
- If one assert fails, the whole test fails. But there might be a number of problems that we would only find by running the test multiple times i.e. after each fix.

It is a matter of style for your project which you prefer. We will cover techniques for reducing the amount of repeated code, so that will cease to be a problem very soon. Personally I like to have test titles which describe the purpose of the test. I would keep the smaller multiple tests, and if the number of tests started to grow too large I might then merge the tests.

Summary

You have now learned about the following Selenium methods:

- open
- type
- click
- waitForPageToLoad
- isTextPresent
- getHtmlSource

- getXpathCount
- getTitle
- stop

You have also learned a little more Java:

- inline comments
- +=
- variables – String and int
- Java Number object

You have also learned:

- JUnit assertions – specifically assertTrue
- debugging failing tests and using the stack trace
- using local variables for readability and aid debugging
- making assert statements more readable
- adding details into the assert statements to aid debugging

In addition you have learned some pros and cons for a variety of acceptance criteria checking strategies. All of the strategies are valid and may be exactly what you need, but learn to choose appropriately between them and experiment for yourself which ones work best for you.

The next step is to refactor some of this code and take advantage of some of JUnit functionality to help us avoid a lot of code duplication.

Chapter 12: First Steps with Automation Refactored

> "**Code refactoring** is the process of changing a computer program's internal structure without modifying its external *functional* behavior or existing functionality, in order to improve internal *non-functional* properties of the software, for example to improve code readability, to simplify code structure, to change code to adhere to a given programming paradigm, to improve maintainability, or to improve extensibility."
> *Wikipedia @ 24/04/2009, http://en.wikipedia.org/wiki/Refactoring*

Introducing Refactoring

We have written 4 tests:

- typeInASearchTermAndAssertThatHomePageTextIsPresent
- typeInASearchTermAndAssertThatHomePageURLExistsWithFewerXpath
- typeInASearchTermAndCheckPageTitleHasSearchTermInIt
- typeInASearchTermAndCheckSearchInputHasSearchTermInIt

All of which use a common pattern:

- Start the test and goto Search page
- Implement the acceptance criteria
- Tidy up the test

```
@Test
public void typeInASearchTermAndCheckPageURLHasSearchTermInIt(){
    /*****************************************************************
     *          Start the test and goto Search
     *****************************************************************/
    Selenium selenium = new DefaultSelenium("localhost", 4444,
                            "*firefox", "http://www.compendiumdev.co.uk");
    selenium.start();
    selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
    selenium.type("xpath=//input[@name='q']", "Selenium-RC");
    selenium.click("xpath=//input[@name='btnG' and @type='submit']");
    selenium.waitForPageToLoad("30000");

    /*****************************************************************
     *          Implement the Acceptance Criteria
     *****************************************************************/
    String pageTitle = selenium.getTitle();
    assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
            pageTitle.contains("Selenium-RC"));

    /*****************************************************************
     *          Tidy up the test
     *****************************************************************/
    selenium.close();
    selenium.stop();
}
```

Figure 12.1 : A Test without refactoring contains duplicated code

We can refactor the tests to:

- reduce the amount of repeated code
- make the test more readable

I will start by refactoring the block of code before the acceptance criteria into a new method to setup the test. I can do this automatically to help avoiding making any manual errors.

Automatically refactoring the start up code into a new method

Select the code to refactor into a new method.

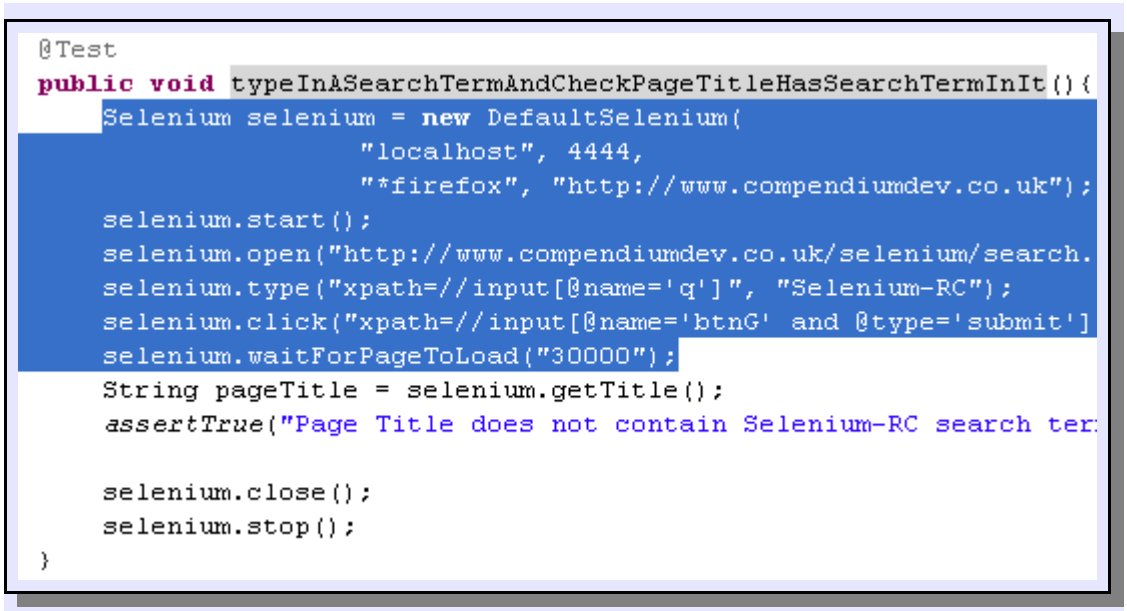


Figure 12.2 : Highlight code ready to refactor it into a new method

Right click and from the popup menu choose “Refactor \ Extract Method...”

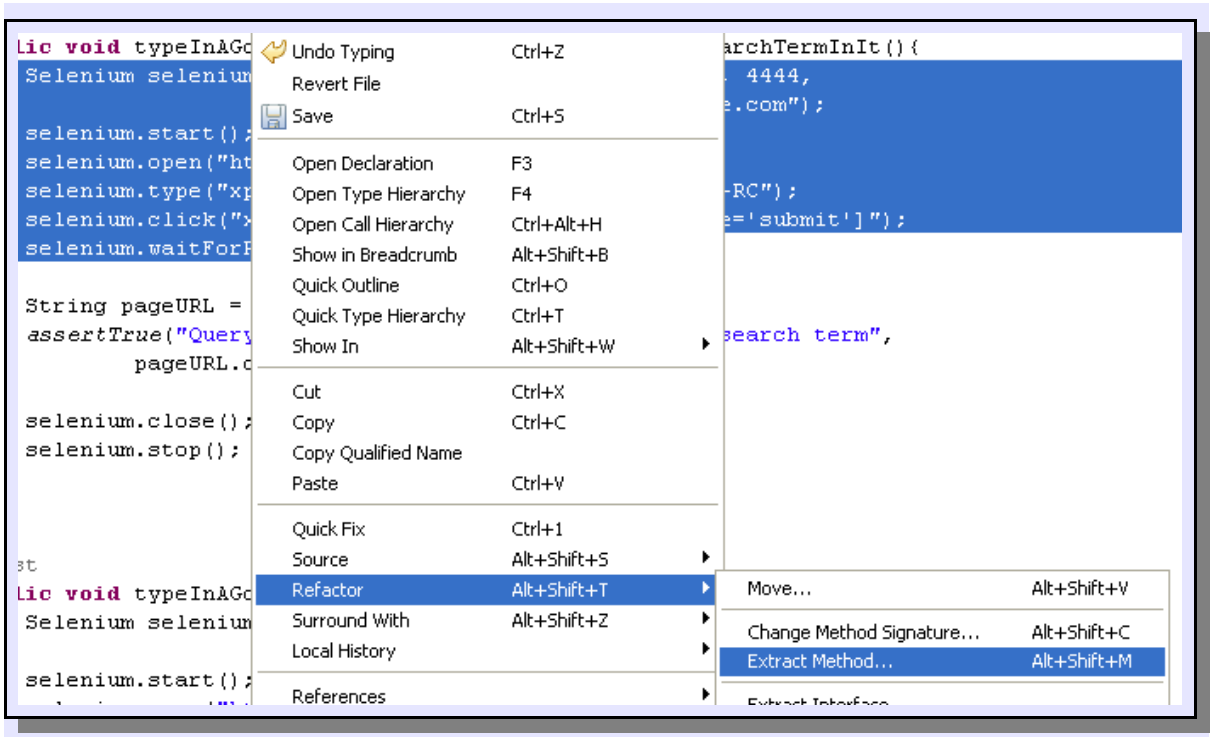


Figure 12.3 : "Extract Method" in the Refactor contextual menu

In the wizard that follows, I enter the "Method name" as "startSeleniumAndSearchForSeleniumRC".

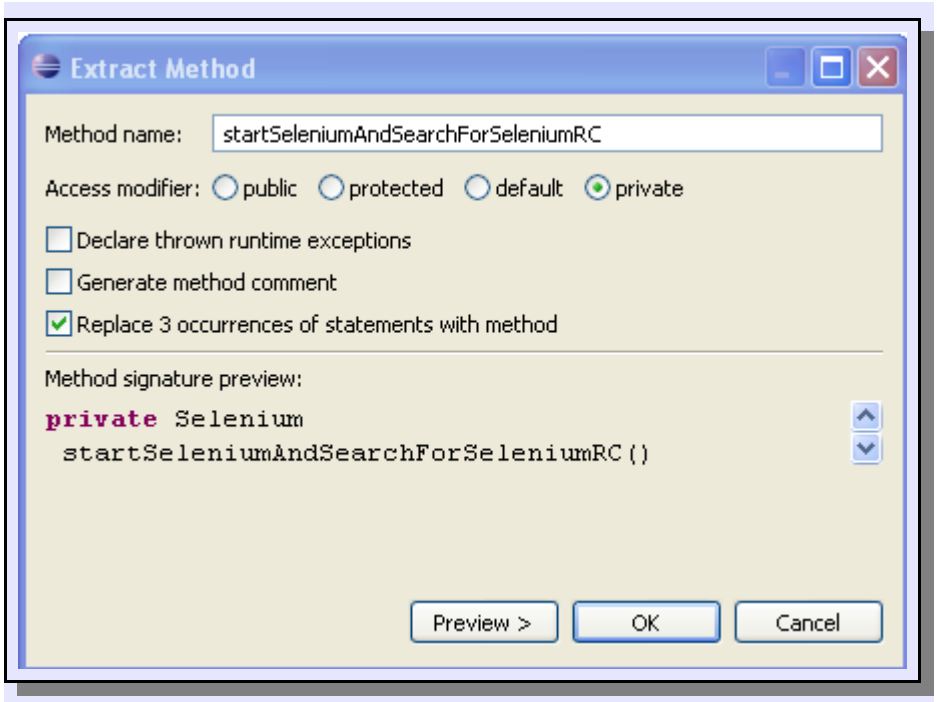


Figure 12.4 : The "Extract Method" Wizard

Because the source code is repeated across all the tests, Eclipse has identified that it can "Replace 3 occurrences of statements with method" which means that it will amend all the tests to use this new start up method.

Note:

Amending all instances only works when the code is repeated exactly throughout the class (as would happen if copy and pasted). So sometimes before an extract method refactor, I do minor code edits to get the code the same throughout the source.

I set the method as private, as I only want to use it with the tests of this class at the moment.

After completing this step my test code has reduced in size and I have a new method containing common code:

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
        Selenium selenium = startSeleniumAndSearchForSeleniumRC();

        String pageTitle = selenium.getTitle();
        assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
                        pageTitle.contains("Selenium-RC"));

        selenium.close();
        selenium.stop();
}

private Selenium startSeleniumAndSearchForSeleniumRC() {
        Selenium selenium = new DefaultSelenium("localhost", 4444,
                                        "*firefox", "http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        return selenium;
}
```

And all the tests now follow this pattern.

Automatically refactor the tear down code into a new method

We can then do the same thing with the tear down code:

```
selenium.close();
selenium.stop();
```

Select this code and refactor it into a new method called “stopSeleniumAndCloseBrowser”.

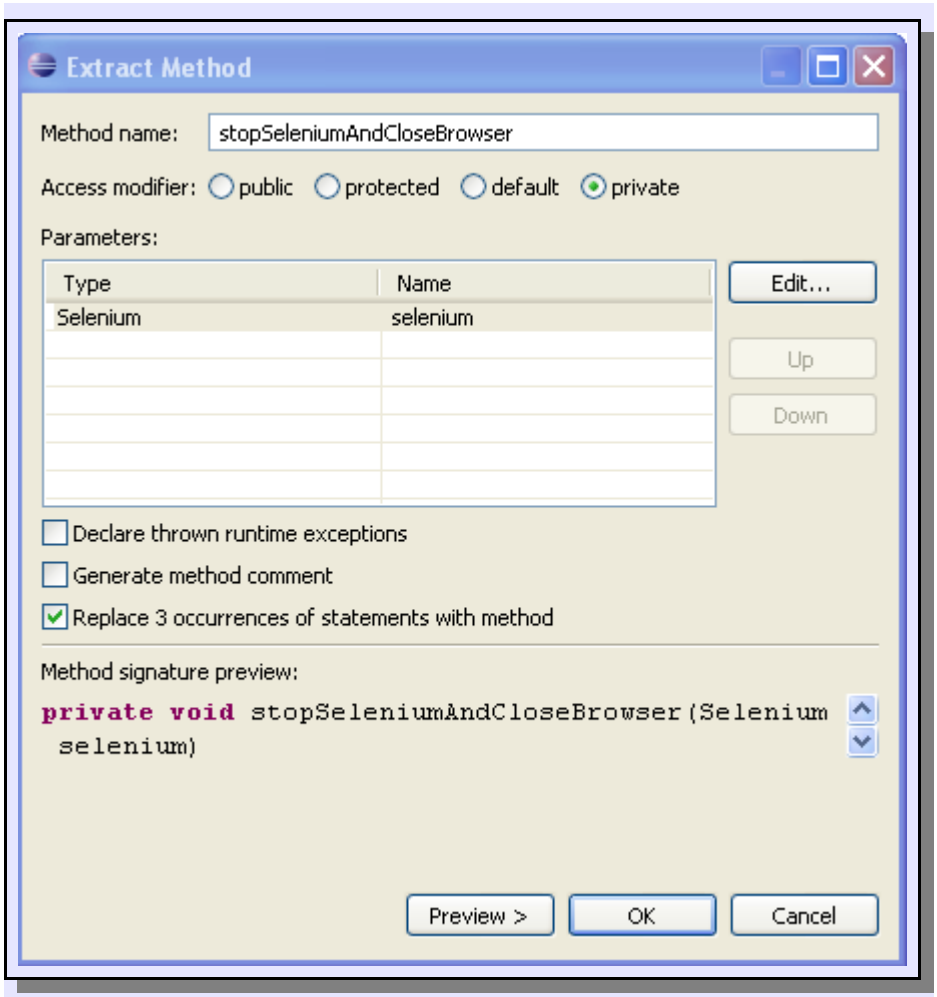


Figure 12.5 : Extract Method Wizard with multiple occurrences

Eclipse identifies that, not only can it do this to the other tests' occurrences, but that it needs to create a parameter so that we can pass in the "selenium" object.

This refactoring creates a new method:

```
private void stopSeleniumAndCloseBrowser(Selenium selenium) {
	selenium.close();
	selenium.stop();
}
```

And our test now looks much cleaner and more readable:

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
	Selenium selenium = startSeleniumAndSearchForSeleniumRC();

	String pageTitle = selenium.getTitle();
	assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
			pageTitle.contains("Selenium-RC"));

	stopSeleniumAndCloseBrowser(selenium);
}
```

The source code is now much more readable and maintainable.

(*See MySecondSeleniumTests_Refactored.java in com.eviltester.seleniumtutorials.chap12 in the downloadable source-code.*)

Using some JUnit functionality we can actually do much better than this and create 'proper' setup and tear down code, to remove the repeated code from each of the test methods.

JUnit @Before and @After

JUnit supports a number of Java annotations that help us do the same thing for the different test automation events that we encounter when running tests.

JUnit has before and after annotations for:

- Method
- Class

A JUnit *Method* is anything in your code annotated with @Test

Any Java method annotated with:

- @Before will be run before every JUnit @Test method
- @After will be run after every JUnit @Test method
- @BeforeClass will be run once before any methods in that class
- @AfterClass will be run once after all methods in the class

Refactoring Plan and Analysis

With our test code looking like the following:

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
        Selenium selenium = startSeleniumAndSearchForSeleniumRC();

        String pageTitle = selenium.getTitle();
        assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
                        pageTitle.contains("Selenium-RC"));

        stopSeleniumAndCloseBrowser(selenium);
}

private void stopSeleniumAndCloseBrowser(Selenium selenium) {
        selenium.close();
        selenium.stop();
}

private Selenium startSeleniumAndSearchForSeleniumRC() {
        Selenium selenium = new DefaultSelenium("localhost", 4444,
                                        "*firefox", "http://www.compendiumdev.co.uk");
        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        return selenium;
}
```

I want to create a method which sets up Selenium and goes to the search engine. This is currently done by the line

```
Selenium selenium = startSeleniumAndSearchForSeleniumRC();
```

I will move this into a method of its own and annotate it @Before. I also want to annotate stopSeleniumAndCloseBrowser with the @After. But first, because the stopSeleniumAndCloseBrowser method takes a parameter I will first have to move selenium into a field instead of a variable local to the method.

So my plan is:

- remove the parameter from stopSeleniumAndCloseBrowser
- move setup code into a new method and annotate it with @BeforeMethod
- remove the setup code from each @Test method
- annotate stopSeleniumAndCloseBrowser with @AfterMethod
- remove the tear down code from each @Test method

Note:

A field is a variable at the class level which is accessible by all methods in the class.

Remove Parameter from stopSeleniumAndCloseBrowser

Eclipse does not have an automated refactoring for the way we have written this code. So I manually delete the parameter, and I can see from the errors in the source code, what additional changes I need to make.

```
private void stopSeleniumAndCloseBrowser() {
        selenium.close();
        selenium.stop();
}
```

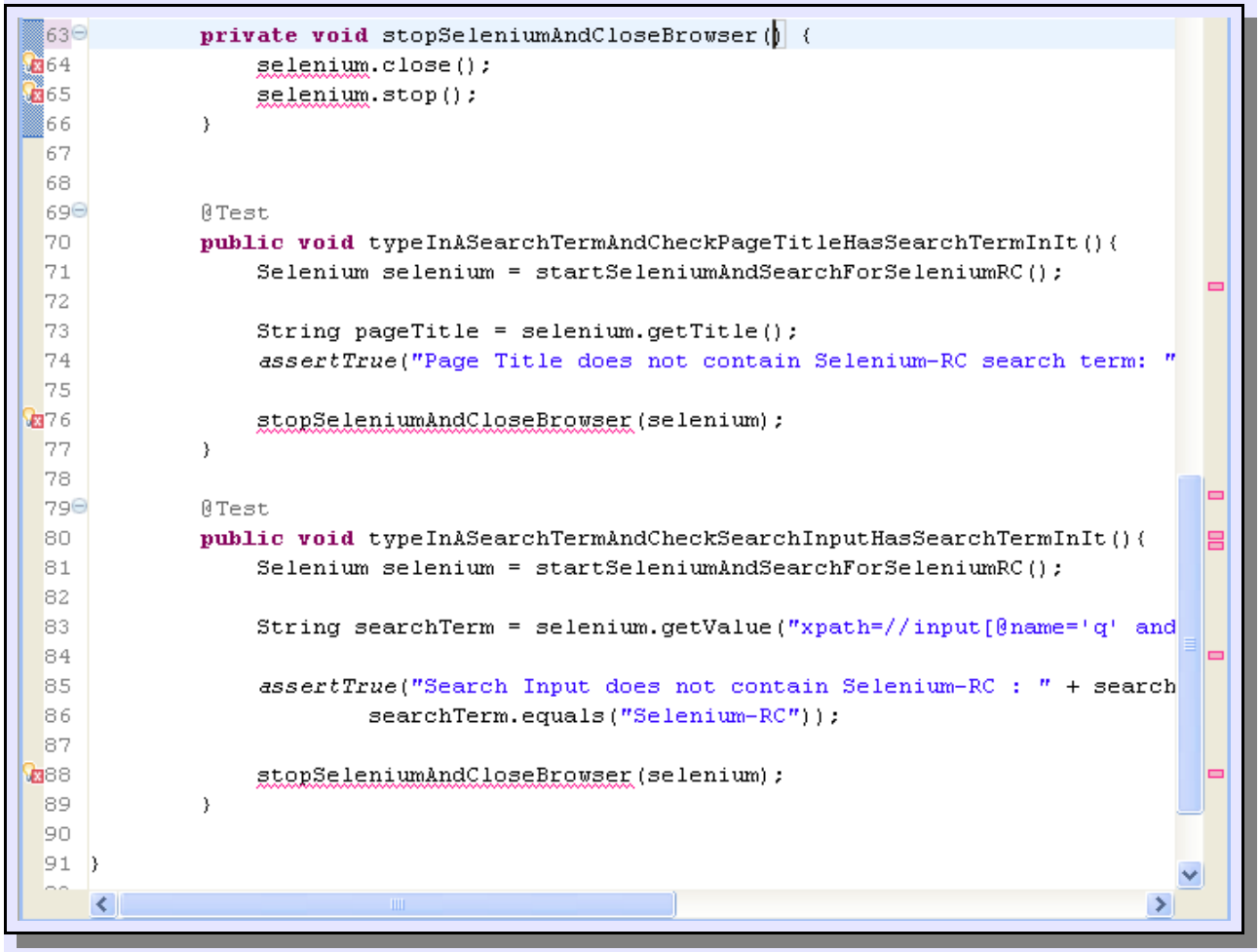


Figure 12.6 : Eclipse marks errors in left info bar and right info bar

Errors are highlighted by red crosses on the left border, and by red rectangles on the right border.

First I will remove the selenium parameter from each of the stopSeleniumAndCloseBrowser calls.

This clears up many of the errors, but the errors against selenium.close(); and selenium.stop(); remind me that I still have to introduce the field.

I can introduce this field by clicking the mouse on the red error cross next to the "selenium.close();" statement and selecting "Create field 'selenium'" from the pop-up menu.

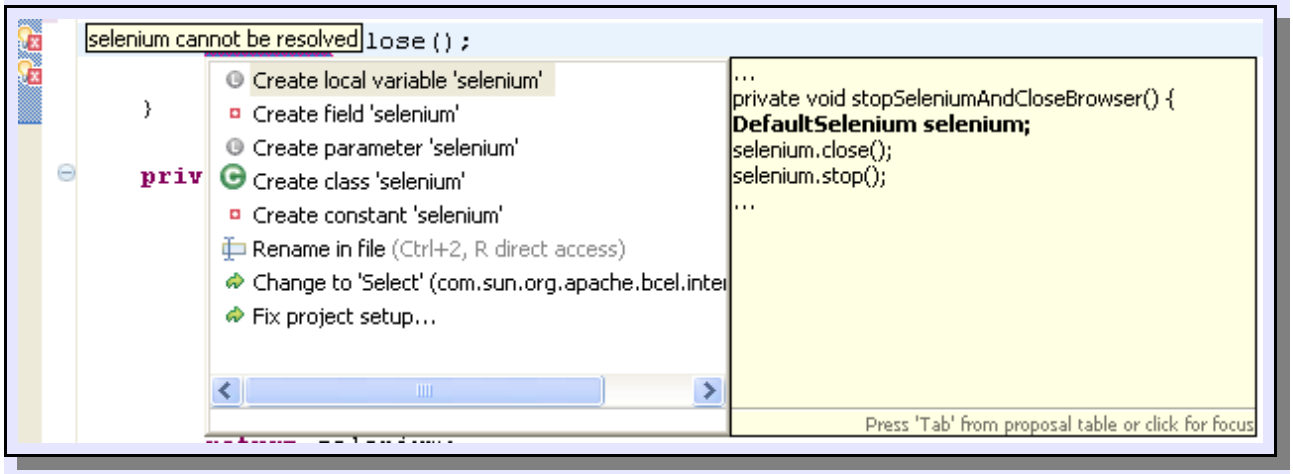


Figure 12.7 : Automatically fix errors through Eclipse context menus

Move setup code into a new method

Once I do this. Eclipse reports no errors against the source code. But at this point we have introduced an error. You can see the error in the following source code snippet:

```
private Selenium selenium;
private Selenium startSeleniumAndSearchForSeleniumRC() {
        Selenium selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                                        "http://www.compendiumdev.co.uk");

        selenium.start();
        selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        return selenium;
}
```

Our last fix introduced the field which you can see at the top of the source code snippet "private Selenium selenium;". This field variable can be used by any of the methods in the class.

But we still have the local declaration of selenium in the test methods e.g.:

```
Selenium selenium = startSeleniumAndSearchForSeleniumRC();
```

If I don't amend this line then each method will create a local instance of the selenium variable and negate the effect of trying to move the test setup into an @Before method.

I will fix this by first creating a method to be annotated with @Before called automateTestSetup.

```
@Before
public void automateTestSetup(){
        selenium = startSeleniumAndSearchForSeleniumRC();
}
```

Remove the setup code from each @Test method

Now our source code has no displayed errors, but we haven't finished fixing the source code yet. We still have to remove the repeated code from each of the @Test methods.

So delete all the “Selenium selenium = startSeleniumAndSearchForSeleniumRC();” lines from each of the @Test methods.

I also need to amend the startSeleniumAndSearchForSeleniumRC method to use the class variable:

I do this by removing the declaration and leaving the assignment.

```
selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                               "http://www.compendiumdev.co.uk");
```

Since I've done so many changes. I'm just going to run the tests and make sure they still run.

All of my tests still run.

I get more nervous about manual refactoring than I do about automated refactoring so I needed to run the tests to convince me that I hadn't broken anything before doing anything else.

Annotate stopSeleniumAndCloseBrowser with @After

I add @After to stopSeleniumAndCloseBrowser.

```
@After
private void stopSeleniumAndCloseBrowser() {
        selenium.close();
        selenium.stop();
}
```

Remove the tear down code from each @Test method

All that remains now is to remove the stopSeleniumAndCloseBrowser(); line from each of the @Test methods.

I will do this manually by deleting the line throughout.

And then run the tests to make sure they still run and tidy up after themselves.

Our tests should have failed when we did that, because JUnit expects all @After and @Before annotated methods to be made public.

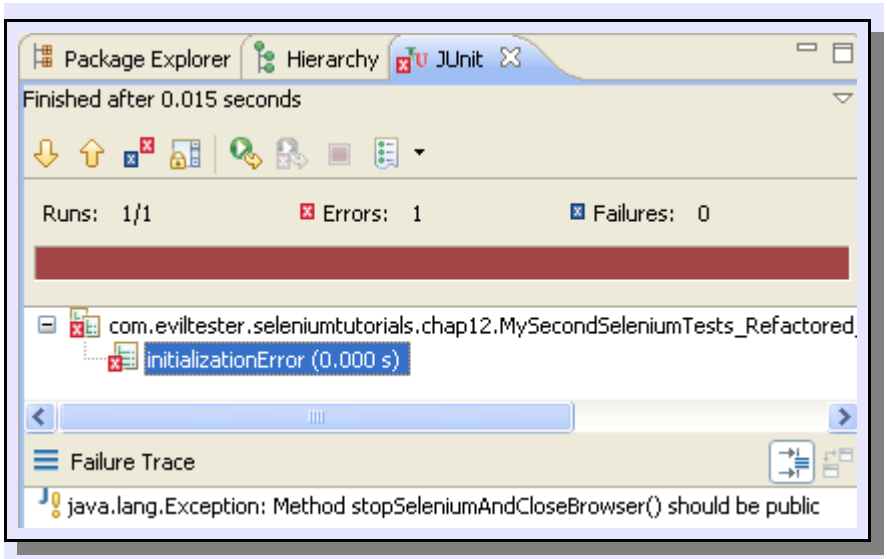


Figure 12.8 : JUnit Test Runner showing our error

To fix this, change the declaration of stopSeleniumAndCloseBrowser to public.

```
@After
public void stopSeleniumAndCloseBrowser() {
        selenium.close();
        selenium.stop();
}
```

Running the tests should now result in them all passing.

The Refactored Code

```
package com.eviltester.seleniumtutorials;
import static org.junit.Assert.*;
import org.junit.*;
import com.thoughtworks.selenium.*;

public class MySecondSeleniumTests{

	private Selenium selenium;

	@Before
	public void automateTestSetup(){
		selenium = startSeleniumAndSearchForSeleniumRC();
	}

	private Selenium startSeleniumAndSearchForSeleniumRC() {
		selenium = new DefaultSelenium("localhost", 4444, "*firefox",
					"http://www.compendiumdev.co.uk");
		selenium.start();
		selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
		selenium.type("xpath=//input[@name='q']", "Selenium-RC");
		selenium.click("xpath=//input[@name='btnG' and @type='submit']");
		selenium.waitForPageToLoad("30000");
		return selenium;
	}

	@Test
	public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
		assertTrue(selenium.isTextPresent(
			"Selenium Remote-Control Selenium RC comes in two parts." +
			" A server which automatically"));
		assertTrue(selenium.isTextPresent(
			"launches and kills browsers, and acts as" +
			" a HTTP proxy for web requests from them."));
	}

	@Test
	public void typeInASearchTermAndAssertThatHomePageURLExistsWithFewerXpath(){
		// create a new variable called matchingCountTotal to calculate
		// the total number of times our XPath matches, initially setting
		// it to 0
		int matchingCountTotal = 0;

		// call getXpathCount with our xpath for the url and
		// add the intValue version to the total
		matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://selenium-rc.seleniumhq.org')]").intValue();
		matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://seleniumhq.org/projects/remote-control')]").intValue();

		assertTrue("No homepage URL found",matchingCountTotal>0);
	}

	@After
	public void stopSeleniumAndCloseBrowser() {
		selenium.close();
		selenium.stop();
	}

	@Test
	public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
		String pageTitle = selenium.getTitle();
		assertTrue("Page Title does not contain Selenium-RC search term: " +
pageTitle, pageTitle.contains("Selenium-RC"));
	}

	@Test
	public void typeInASearchTermAndCheckSearchInputHasSearchTermInIt(){
		String searchTerm = selenium.getValue("xpath=//input[@name='q' and
@title='Search']");
		assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
			searchTerm.equals("Selenium-RC"));
	}
}
```

The refactored code has the following advantages:

- Smaller
- Less code duplication
- @Test methods concentrate on the test and not the environment configuration

We have less code duplication because our setup and tear down code is in a single place. This makes the tests easier to maintain since we only have to make changes in a single place.

Having the @Test methods concentrate only on what needs to be done for the test makes them easier to read, understand and maintain.

A common practice in automated testing is to have a setup and tear down process. JUnit supports us in this by allowing us to do setup and tear down at multiple levels. We are currently only using the method level.

@BeforeClass & @AfterClass

@Before, and @After execute the methods before and after every test method. This means that we start and stop the server and the browser before every test. This makes the test execution slower than we need.

We can use the @BeforeClass and @AfterClass methods so that we start and stop the server once for the entire set of test methods in the class.

Unfortunately using the @BeforeClass and @AfterClass annotations requires a few more amendments to our code. We can't simply replace @Before with @BeforeClass and @After with @AfterClass.

In order to make the methods accessible at the class level the methods have to become static.

So make the following changes:

- The declaration of the selenium variable must become static

```
static Selenium selenium;
```

- The automateTestSetup method must become static

```
@BeforeClass
static public void automateTestSetup(){
	selenium = startSeleniumAndSearchForSeleniumRC();
}
```

- The stopSeleniumAndCloseBrowser must also become static

```
@AfterClass
static public void stopSeleniumAndCloseBrowser() {
	selenium.close();
	selenium.stop();
}
```

- And lastly the private method startSeleniumAndSearchForSeleniumRC must also be declared as static

```
static private Selenium startSeleniumAndSearchForSeleniumRC() {
	selenium = new DefaultSelenium("localhost", 4444, "*firefox",
			"http://www.compendiumdev.co.uk");
	selenium.start();
	selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
	selenium.type("xpath=//input[@name='q']", "Selenium-RC");
```

```
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
        return selenium;
    }
```

Note:

Static means that no matter how many objects of a class are a created, they all share the same 'thing' marked as static. i.e. If I create a method as static then each call to the method on any object, calls the same code. If I update a static variable, then it will be updated on all objects of the class.

Static fields and methods are common to all object instances of a class. For more information on static you might wish to read the official documentation.

http://java.sun.com/docs/books/tutorial/java/javaOO/classvars.html

After making these changes you should see a massive execution speed improvement to the tests.

This speed improvement comes because we only start up the browser once for all the tests, and only connect to Selenium server once for all the tests. The tests themselves have always run quickly, it was only the environmental setup that caused them to run slowly.

Our refactored code

```
package com.eviltester.seleniumtutorials;
import static org.junit.Assert.*;
import org.junit.*;
import com.thoughtworks.selenium.*;

public class MySecondSeleniumTests {

        static Selenium selenium;

        @BeforeClass
        static public void automateTestSetup(){
                selenium = startSeleniumAndSearchForSeleniumRC();
        }

        static private Selenium startSeleniumAndSearchForSeleniumRC() {
                selenium = new DefaultSelenium("localhost", 4444, "*firefox",
                                        "http://www.compendiumdev.co.uk");
                selenium.start();
                selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
                selenium.type("xpath=//input[@name='q']", "Selenium-RC");
                selenium.click("xpath=//input[@name='btnG' and @type='submit']");
                selenium.waitForPageToLoad("30000");
                return selenium;
        }

        @Test
        public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
                assertTrue(selenium.isTextPresent(
                        "Selenium Remote-Control Selenium RC comes in two parts." +
                        " A server which automatically"));
                assertTrue(selenium.isTextPresent(
                        "launches and kills browsers, and acts as" +
                        " a HTTP proxy for web requests from them."));
        }


        @Test
        public void typeInASearchTermAndAssertThatHomePageURLExistsWithFewerXpath(){
                // create a new variable called matchingCountTotal to calculate
                // the total number of times our xpath matches, initially setting
                // it to 0
                int matchingCountTotal = 0;
```

```
                // call getXpathCount with our xpath for the url and
                // add the intValue version to the total
                matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://selenium-rc.seleniumhq.org')]").intValue();
                matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://seleniumhq.org/projects/remote-control')]").intValue();

                assertTrue("No homepage URL found",matchingCountTotal>0);
        }

        @AfterClass
        static public void stopSeleniumAndCloseBrowser() {
                selenium.close();
                selenium.stop();
        }

        @Test
        public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
                String pageTitle = selenium.getTitle();
                assertTrue("Page Title does not contain Selenium-RC search term: " +
pageTitle, pageTitle.contains("Selenium-RC"));
        }

        @Test
        public void typeInASearchTermAndCheckSearchInputHasSearchTermInIt(){
                String searchTerm = selenium.getValue("xpath=//input[@name='q' and
@title='Search']");
                assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
                        searchTerm.equals("Selenium-RC"));
        }
}
```

Note:

You can find this code in the downloadable source-code archive as com.eviltester.seleniumtutorials.chap12.MySecondSeleniumTests_Refactored_BeforeClass_AfterClass

Automatically create JUnit Test Structure

Eclipse has a useful wizard to setup the basic skeleton for tests. “File \ New \ JUnit Test Case”

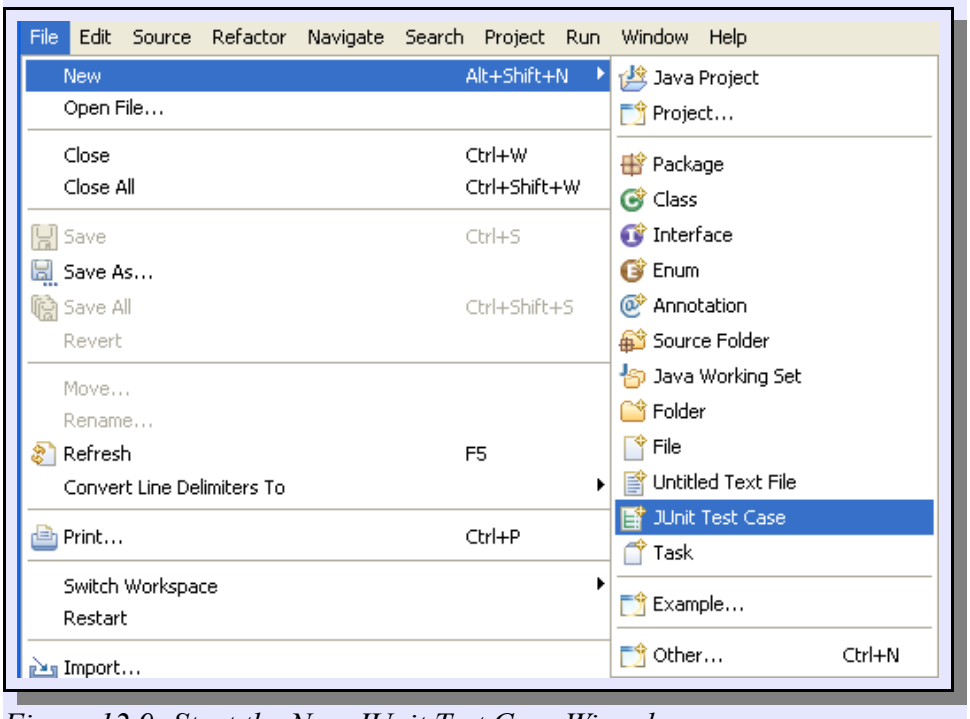


Figure 12.9: Start the New JUnit Test Case Wizard

The “New JUnit Test Case” Wizard will display:

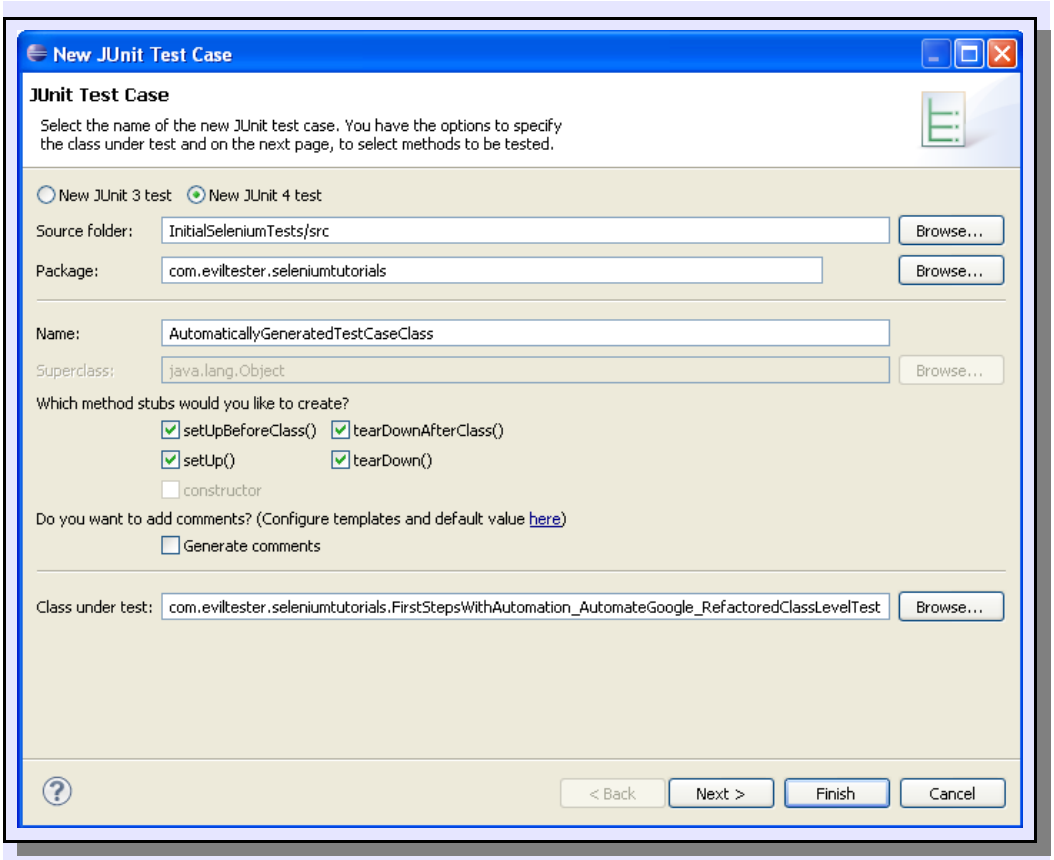


Figure 12.10: The JUnit Test Case Wizard Dialog

We would need to overwrite the name of the test to put in what we want. By Default the wizard uses the last value you put in and appends “Test” to it.

You would then choose which method stubs to create:

- setUpBeforeClass() creates a static method with that name with the @BeforeClass annotation
- tearDownAfterClass() creates a static method with that name annotated with @AfterClass, this can only be selected if you have chosen to create a setUpBeforeClass() method
- setUp() creates a method named setUp() annotated with @Before. It is called setUp because this is the naming convention used when inheriting from the JUnit TestCase class, although it could be named anything you choose.
- tearDown() creates a method named tearDown annotated with @After. Again the name is legacy from the JUnit TestCase class and could be named anything you choose.
- You could choose to generate comments but these don't add much value to the generation and you are probably better off adding your own comments later if you need them.

The generated code:

```
package com.eviltester.seleniumtutorials;

import org.junit.After;
import org.junit.AfterClass;
import org.junit.Before;
```

```
import org.junit.BeforeClass;

public class AutomaticallyGeneratedTestCaseClass {

	@BeforeClass
	public static void setUpBeforeClass() throws Exception {
	}

	@AfterClass
	public static void tearDownAfterClass() throws Exception {
	}

	@Before
	public void setUp() throws Exception {
	}

	@After
	public void tearDown() throws Exception {
	}
}
```

You would then need to add your references to Selenium and fill in the methods with your code. Using this method to create JUnit test classes can prove a time saver. I usually start with the @Before and @After annotated setUp and tearDown methods and manually move to the @BeforeClass and @AfterClass methods if I require the speed improvements, and if it is a suitable approach for the tests.

Summary

You have learned the following.

Java:

- Class Variables vs Local Variables
- Refactoring

Eclipse:

- Automatic Refactoring

Principles:

- Run tests after refactoring
- Separate setup and tear down from tests

JUnit annotations:

- @Before and @After
- @BeforeMethod and @AfterMethod
- New JUnit Test Case wizard

You might want to flip back through the pages and compare the various code snippets to see how much more understandable, and easier to read, the code is now compared to our original version.

Chapter 13: How do I upgrade my Selenium Server when a new version comes out?

Occasionally you will encounter issues with Selenium that have been fixed in the newest version. So how do you upgrade your server?

First Download the New Version

1. Visit http://seleniumhq.org/download/ and download the new version.



Figure 13.1 : The available downloads listed on the Selenium web site

2. Unarchive the downloaded zip file as before

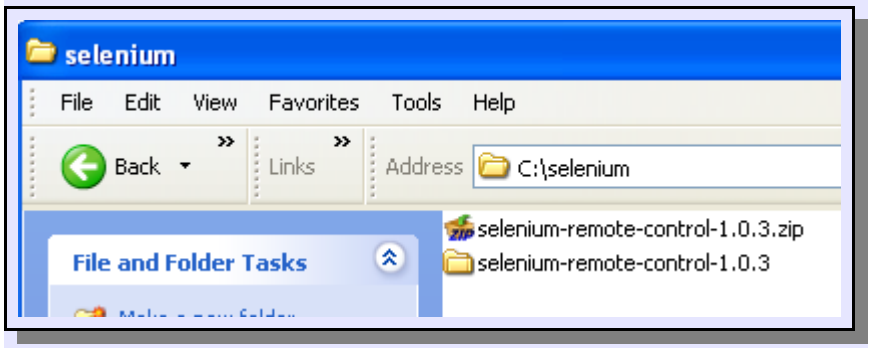


Figure 13.2 : Unarchive Selenium into a folder

Configure Eclipse Project

Then configure your project in Eclipse to use the new .jar files.

1. Select the project in the Package Explorer and then either press Alt+Enter or select "Properties" from the "Project" menu.

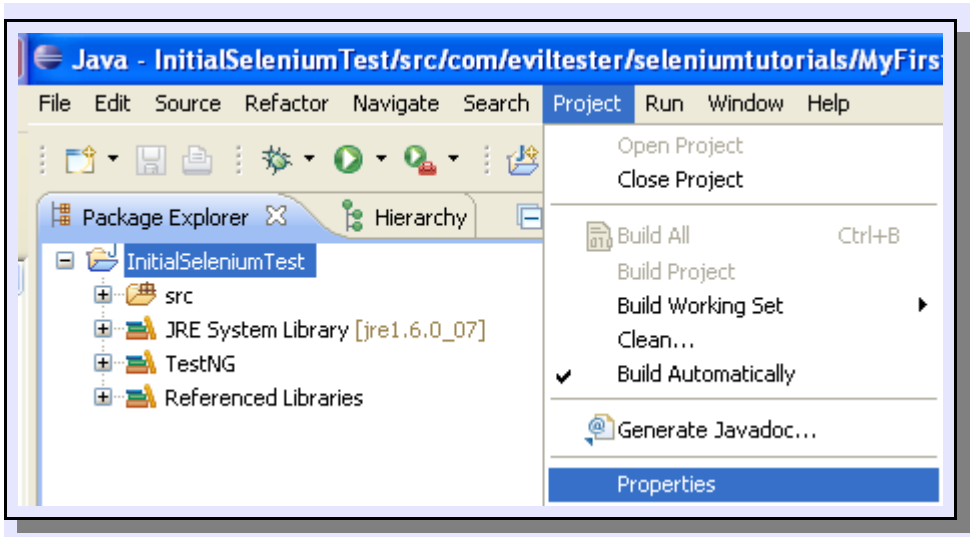


Figure 13.3 : Amend the project properties

2. Change the referenced libraries on the Java Build Path.
   i. Select Java Build Path on the properties tree
   ii. Select Libraries from the tab
   iii. Select the selenium-java-client-driver.jar then click [edit] and choose the new path where you unarchived the new version

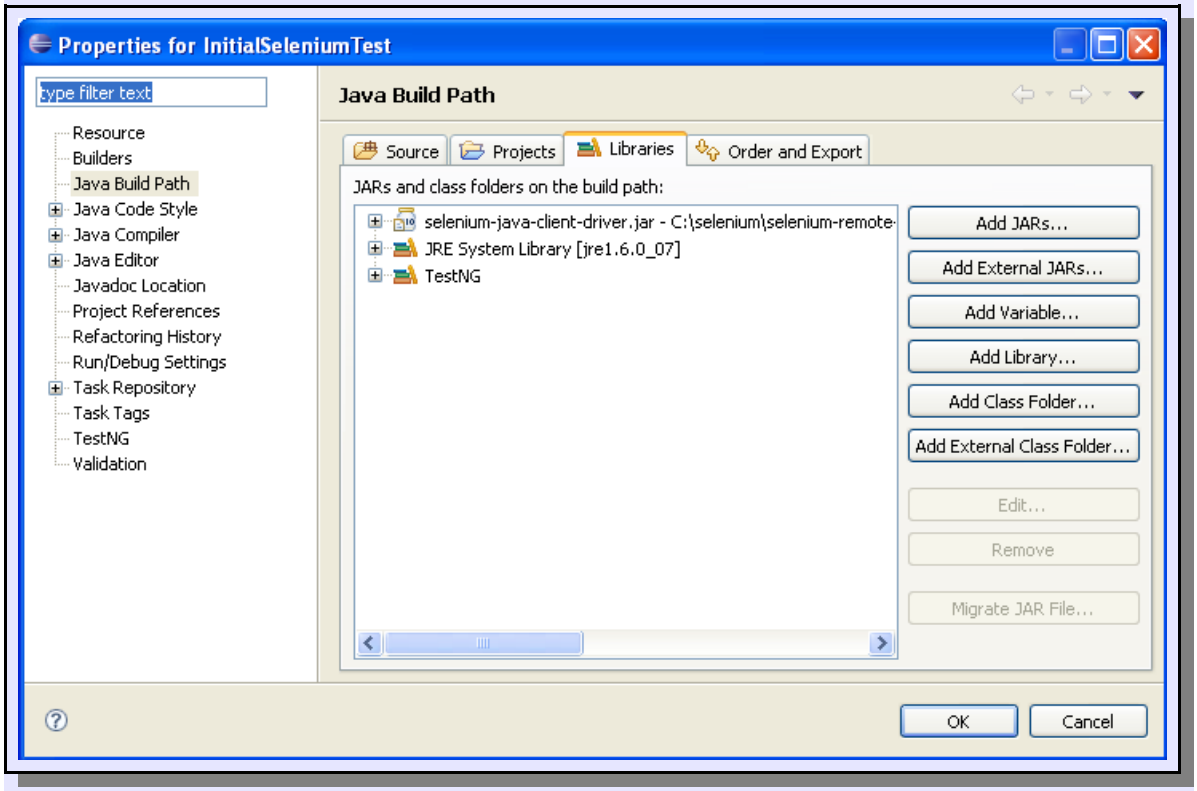


Figure 13.4 : Edit the properties of the library to point to the new location

Remember to re-attach the source

In one of your tests, select one of the selenium commands and press F3, you should be prompted to re-attach the source. Navigate to the selenium-java-client-driver-sources.jar and re-attach it as you did in Chapter 7.

Chapter 14: Chapter - Basic HTML Theory

This chapter will provide you with a basic understanding of HTML. Just enough to cover the basic requirements for writing Selenium tests.

If you visit a web page in your browser, and then view source on that page. You will see that an HTML page has a very simple structure.

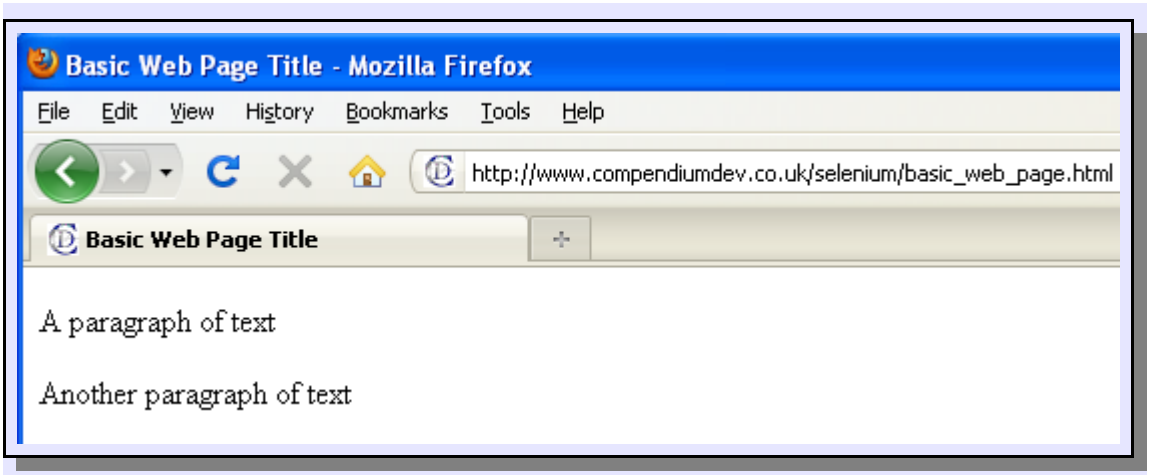


Figure 14.1 : A very simple HTML page

Open http://www.compendiumdev.co.uk/selenium/basic_web_page.html in a browser and view the source code and you will see a very simple set of markup.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
                "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html>
        <head>
                <title>Basic Web Page Title</title>
        </head>
        <body>
                <p id="para1">A paragraph of text</p>
                <p id="para2">Another paragraph of text</p>
        </body>
</html>
```

Page Structure

An HTML page has a very simple structure.

- A doctype to help the browser identify what to do with the file
- an <html> block which contains the page code
    - a <head> section
    - a <body> section

Elements & Tags

HTML consists of nested elements. Elements are represented by an opening tag: e.g. , followed by the corresponding closing tag .

In a well formed XHTML document each opening tag has a corresponding closing tag. Some older

doctypes allow you to leave off the closing tag and let the browser try and work out where an element finishes.

Some elements do not have any content, so don't have an open and close tag, they are self contained e.g.
.

Because elements are nested, an HTML document can be viewed as a tree. This allows XPath to be used to traverse it.

Attributes

Elements can have attributes e.g. The id in .

Attributes help to specify display styles, code, and in the case of id – a unique identifier for the element in the document.

Expanded

Just to be clear, in the line: A paragraph of text

- The Paragraph **element** is: A paragraph of text
- The Paragraph element **content** (or **text**) is: “A paragraph of text”
- The **tags** are: ,
- The **attribute** is: 'id'
- The attribute **value** is: “para1”

Chapter 15: Basic XPath Theory

This chapter will provide you with a basic understanding of XPath. Just enough to cover the basic requirements for writing Selenium tests.

XPath is the XML Path Language. Since all HTML, once loaded into a browser, becomes well structured and can be viewed as an XML tree, we can use XPath to traverse it.

> **Note:**
>
> **To help follow this section you might want to visit the web page http://compendiumdev.co.uk/selenium/basic_web_page.html and use the Firefox plugin XPather to try out the XPath statements listed.**

I'll include the listing of the XHTML for the basic_web_page.html here so you can follow along:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
                 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html>
        <head>
                <title>Basic Web Page Title</title>
        </head>
        <body>
                <p id="para1" class="main">A paragraph of text</p>
                <p id="para2" class="sub">Another paragraph of text</p>
        </body>
</html>
```

XPath Expressions

XPath expressions select 'nodes' or 'node-sets' from an XML document.

e.g. The XPath expression "//p" would select the following node-set from the example in the "Basic HTML Theory" section:

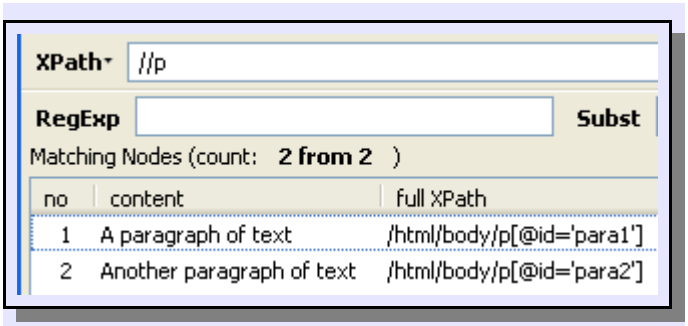


Figure 15.1 : XPather matching //p

Node Types

XPath has different types of nodes. In the example XHTML these are:

- Document node (the root of the XML tree): <html>
- Element node e.g.:
  - <head><title>Basic Web Page Title</title></head>

    - <title>Basic Web Page Title</title>
    - A paragraph of text
- Attribute node e.g.
    - id="para1"

Selections

- /
    - start selection from the document node
    - allows you to create 'absolute' path expressions
    - e.g. “/html/body/p” matches all the paragraph element nodes
        - `assertEquals(2,selenium.getXpathCount("/html/body/p"));`
- //
    - start selection matching anywhere in the document
    - allows you to create 'relative' path expressions
    - e.g.
        - //p matches all paragraph element nodes
            - `assertEquals(2,selenium.getXpathCount("//p"));`
- @
    - selects attribute elements
    - e.g. //@id would select all the attribute nodes
        - `assertEquals(2,selenium.getXpathCount("//@id"));`

Predicates

Predicates help make selections more specific and are put in square brackets.

Predicates can be indexes. e.g.

- //p[2] selects the second p element node in the node-set,
    - `assertEquals("Another paragraph of text",selenium.getText("//p[2]"));`
- and //p[1] would select the first.
    - `assertEquals("A paragraph of text",selenium.getText("//p[1]"));`

Predicates can be attribute selections e.g.

- //p[@id='para1'] selects the p element node where the value of the attribute id is 'para1'
    - `assertEquals("A paragraph of text", selenium.getText("//p[@id='para1']"));`
- //p[@class='main'] selects the p element node where the value of the attribute class is 'main'
    - `assertEquals("A paragraph of text", selenium.getText("//p[@class='main']"));`

Predicates can be XPath functions e.g.

- //p[last()] would select the last paragraph

- `assertEquals("Another paragraph of text",selenium.getText("//p[last()]"));`

Predicates can be comparative statements e.g.

- //p[position()>1] would return all but the first p element
  - ```
    assertEquals("Another paragraph of text",
           selenium.getText("//p[position()>1]"));
    ```
  - ```
    assertEquals("A paragraph of text",
           selenium.getText("//p[position()>0]"));
    ```

Advanced

XPath is a full programming language so you can perform calculations (e.g. "last()-1"), and use boolean operations (e.g. or, and)

Combining Match Queries

You can combine several selections by using "|" e.g.

- //p | //head
  - get any paragraph element node and also get the head element node
  - `assertEquals(3,selenium.getXpathCount("//p | //head"));`

Wild Card Matches

You can also use wild cards:

- "node()" matches any type of node (document, element, attribute) e.g.
  - //node()[@id='para1']
    - get any node with an id of para1
    - `assertEquals(1,selenium.getXpathCount("//node()[@id='para1']"));`
  - //node()
    - get all the nodes (try it, you may not get the results you expect)
  - //body/node()
    - get all the nodes in the body (again, try it to see if you get the value you expect)
- * matches anything depending on its position
  - "@*" matches any attribute
    - e.g. //p[@*='para1'] would match the first paragraph
    - `assertEquals(1,selenium.getXpathCount("//p[@*='para1']"));`
  - "*" can match nodes e.g.
    - //*[@*] anything with any attribute
      - `assertEquals(2,selenium.getXpathCount("//*[@*]"));`
    - //*[@id] anything with an id attribute
      - `assertEquals(2,selenium.getXpathCount("//*[@id]"));`
    - /html/* all children of the document node

- `assertEquals(2,selenium.getXpathCount("/html/*"));`

Boolean Operators

You can setup matches with multiple conditions e.g.

- //p[starts-with(@id,'para') **and** contains(.,'Another')]
  - find all paragraphs where the id starts with 'para' and the text contains 'Another' i.e. the second paragraph/

```
assertEquals("Another paragraph of text",
 selenium.getText(
       "//p[starts-with(@id,'para') and contains(.,'Another')]"));
```

- //*[@id='para1' or @id='para2']
  - find any node where the id is 'para1' or the id is 'para2' i.e. our two paragraphs

```
assertEquals(2,
       selenium.getXpathCount("//*[@id='para1' or @id='para2']"));
```

XPath Functions

Since XPath is actually a programming language it has built in functions which we can use in our XPath statements. Some common XPath functions are listed below

- contains()
  - allows you to match the value of attributes and elements based on text anywhere in the comparison item
  - e.g.
    - //p[contains(.,'text')]
      - would match any paragraph with text in the main paragraph e.g. Both our paragraphs
      - `assertEquals(2, selenium.getXpathCount("//p[contains(.,'text')]"));`
    - //p[contains(.,'Another')]
      - would match any paragraph with “Another” in the paragraph text, in our example this would match the second paragraph.
      - `assertEquals("Another paragraph of text", selenium.getText("//p[contains(.,'Another')]"));`
    - //p[contains(@id,'1')]
      - would match any paragraph where the id had '1' in it, in our example this is the first paragraph
      - `assertEquals("A paragraph of text", selenium.getText("//p[contains(@id,'1')]"));`
- starts-with()
  - allows you to match the value of attributes and elements based on text at the start of the comparison item
  - e.g.

- //*[starts-with(.,'Basic')]
  - would match any node where the contents of that node start with 'Basic', in our example this would match the title

```
assertEquals("Basic Web Page Title",
selenium.getText("//*[starts-with(.,'Basic')]"));
```

- //*[starts-with(@id,'p')]
  - would match any node where the id name started with 'p', in our example this would match the paragraphs

```
assertEquals(2,
selenium.getXpathCount("//*[starts-with(@id,'p')]"));
```

There are many XPath functions available to you, I have just picked a few of the most common ones that I use. I recommend that you visit some of the web sites below to learn more about XPath functions, and experiment with them.

Recommended web sites for function references:

- http://www.w3schools.com/XPath/xpath_functions.asp
- http://msdn.microsoft.com/en-us/library/ms256115.aspx
- http://www.xmlpitstop.com/ListTutorials/DispContentType/XPath/PageNumber/1.aspx

XPath optimisation

For our testing we typically want to get the shortest and least brittle XPath statement to identify elements on a page.

Some XPath optimisation strategies that I have used are:

- use the id,
- use a combination of attributes to make the XPath more specific,
- start at the first unique element

We have to make a trade off between handling change and false positives. So we want the XPath to return the correct item, but don't want the test to break when simple changes are made to the application under test.

Use the ID

If the element has a known id then use that e.g.

- //*[@id='para2']

Or you probably want to be even more specific and state the type e.g.

- //p[@id='para2']

Use the attributes

If it doesn't have an id, but you can identify it with a combination of attributes then do that. Our example XHTML doesn't have enough nodes to make this clear, but we did this with our Google testing.

e.g. //input[@name='q' and @title='Google Search']

Start at the first unique element

If there is really nothing to distinguish the element then look up the Ancestor chain and find the first unique element.

e.g. //form/table[1]/tbody/tr[1]/td[2]/input[2]

This approach starts to introduce the chance of false positives since a new input might be added before the one we want, and the test would start using that instead.

Selenium XPath Usage

Selenium uses XPath in locators to identify elements e.g.

```
selenium.isElementPresent("xpath=//p[@id='para1']")
```

Because only XPath locators start with // it is possible to write XPath locators without adding "xpath=" on the front. e.g.

```
selenium.isElementPresent("//p[@id='para1']")
```

The specific XPath command getXpathCount expects an XPath statement as its argument so you should not use "xpath=" in front of the XPath locator. Possibly a good reason for not using "xpath=" in any of your locators, but each of us has personal coding styles so you get to make a choice as to which you prefer. e.g.

```
selenium.getXpathCount("//p"); //return a count of the <p> elements
```

You can combine the XPath statement in the getAttribute statement to get specific attributes from elements e.g.

```
assertEquals("para2", selenium.getAttribute("xpath=//p[2]@id"));
assertEquals("para2", selenium.getAttribute("//p[2]@id"));
```

The @id (or more specifically @<attribute-name> means that the statement is not valid XPath but Selenium parses the locator and knows to split the @id on the end off before using it.

Note:

The source code for tests illustrating these XPath commands can be found in the source code archive described in Chapter 31 and in the package com.eviltester.seleniumtutorials.chap15

Chapter 16: Basic CSS Selector Theory

XPath statements can slow down your tests as they evaluate slowly in Internet Explorer. CSS selectors run faster than the equivalent XPath. As you become more experienced with Selenium, you will probably gravitate towards using CSS selectors as your locators.

Firefinder for Firebug

I use "Firefinder for Firebug" to help me with my CSS Selectors. You can install it by following the link below in Firefox.

- https://addons.mozilla.org/en-US/firefox/addon/11905

This Firebug plugin adds a few additional abilities to Firefox. By right clicking on an element in a page in Firefox, you now have the ability to "Firefind Element".

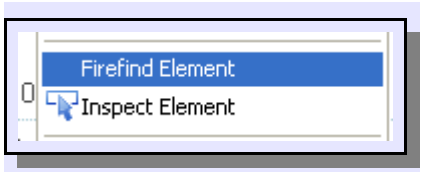


Figure 16.1 : Firefind Element

This opens Firefind, with the HTML of the element displayed.

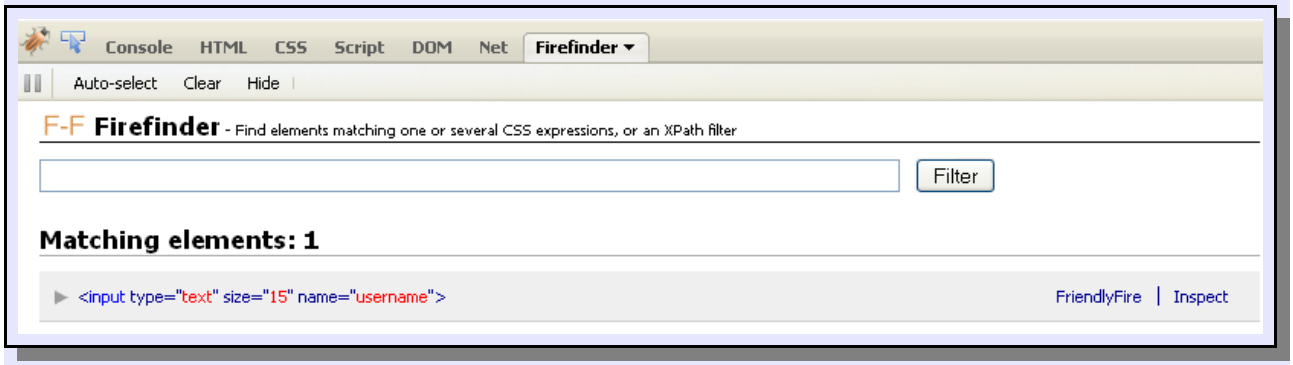


Figure 16.2 : Display after having found an element with Firefind

By typing a CSS selector as the 'filter' input, and pressing the [Filter] button. You can see the elements in the page which match the CSS selector – this allows you to check if your CSS selector constrains the search results enough.

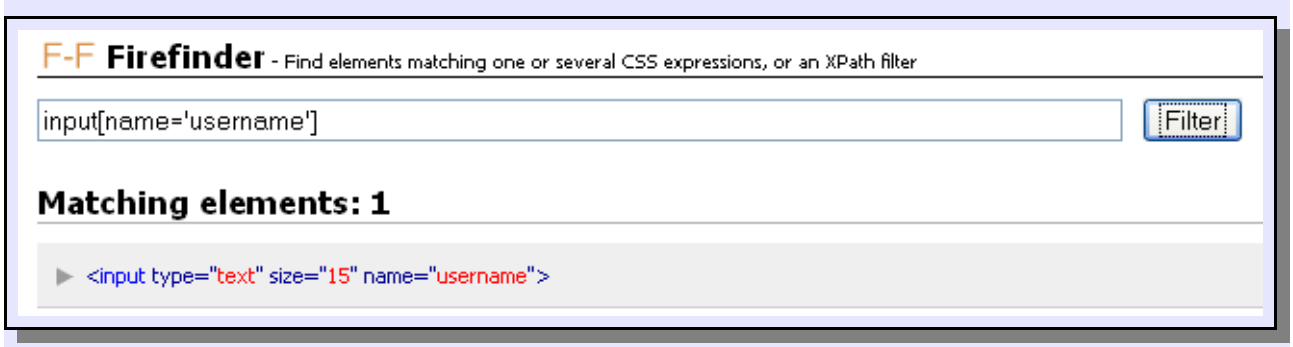


Figure 16.3 : Filter on a CSS selector

Also the item on the page itself will be shown as selected.

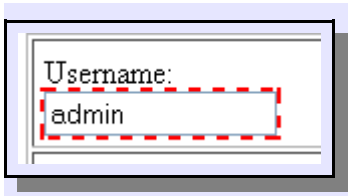


Figure 16.4 : Element matching the CSS selector

You can also use Firefind to match XPath e.g. //input[@name='username'] and the same result will be achieved.

CSS Selector Expressions

In Selenium, you prefix a CSS locator with "css=" e.g. The CSS selector "css=p" would match all the paragraphs in a web page.

> **Note:**
>
> **When using Firefinder, you do not add the "css=" prefix. Only use this in Selenium.**

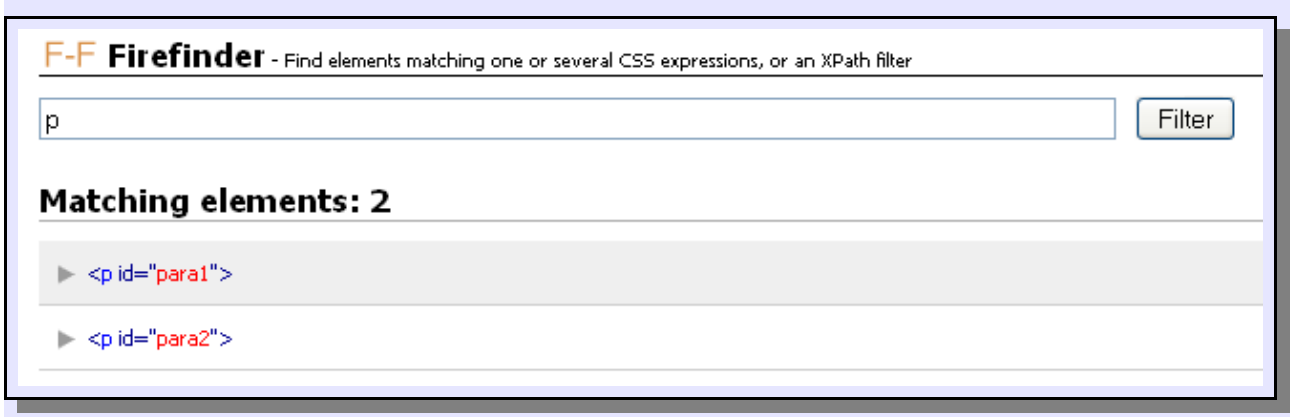


Figure 16.5 : Match all paragraph elements

> **NOTE:**
>
> **As with the XPath chapter. Most of the examples in this section will use the simple page at http://www.compendiumdev.co.uk/selenium/basic_web_page.html**

Selenium & CSS

Selenium uses CSS in locators to identify elements e.g.

```
selenium.isElementPresent("css=p[id='para1']")
```

You can also use the CSS statement in the getAttribute statement to get specific attributes from elements e.g.

```
assertEquals("para2", selenium.getAttribute("css=p.main@id"));
```

Selenium does not provide a getCSSCount function, like the getXPathCount function, but we can create a simple getCSSCount function using the getEval command that we will explain later.

```
// based on http://www.ivaturi.org/home/addgetcsscountcommandtoselenium
private int getCSSCount(String aCSSLocator){
        String jsScript =
        "var cssMatches = eval_css(\"%s\", window.document);cssMatches.length;";

        return Integer.parseInt(
                selenium.getEval(String.format(jsScript, aCSSLocator)));
}
```

Note:

String.format is a particularly useful Java command for avoiding concatenating strings together. String.format has the following form:

String.format(<a format string>,<list of arguments>);

For the format string, you create a string e.g. "hello there %s, I have %d for sale" and then add the replacement items for the % markers as arguments:

String name = "alan";

int amount=10;

String.format("hello there %s, I have %d for sale",name,amount);

This allows you to have constants which you can add values into without concatenating lots of variables together.

For more information on String.format visit:
- http://java.sun.com/j2se/1.5.0/docs/api/java/util/Formatter.html#summary

If you want to use *getCSSCount* in the short term then add it as a private method in the test class, and you can use it in your tests as follows.

```
@Test
public void someCounts(){
        assertEquals(2,getCSSCount("p"));
        assertEquals(6, getCSSCount("*"));
        assertEquals(2, getCSSCount("body > *"));
        assertEquals(1,getCSSCount("p[id='para1']"));
}
```

Selections

CSS selectors, use a hierarchical definition much like XPath to match elements on the page.

Direct Descendents and Absolute Paths

In XPath we use '/' to build a path to an element on the page e.g. //head/title

With CSS selectors we use the '>' notation e.g.

- head > title
  - ```
    assertEquals("Basic Web Page Title",
            selenium.getText("css=head > title"));
    ```
    ```

Sub Elements and Relative Paths

In XPath we can chain // constructs to match any depth in the tree. e.g. //html//title to find all 'title' elements anywhere below an 'html' element

In CSS, add white space between the elements e.g.

- html title
  - ```
    assertEquals("Basic Web Page Title",
            selenium.getText("css=html title"));
    ```

Attribute and Index matching

Attribute Matching

CSS selectors support a variety of notation for making selections more specific. There is a very close equivalent to the XPath [] notation with CSS.

We can use [] to match attribute selections in both CSS and XPath e.g.

- p[id='para1']
  - selects the p element where the value of the id attribute is 'para1'
  - ```
    assertEquals("A paragraph of text",
            selenium.getText("css=p[id='para1']"));
    ```
- p[class='main']
  - selects the p element where the value of the class attribute is 'main'
  - ```
    assertEquals("A paragraph of text",
            selenium.getText("css=p[class='main']"));
    ```
- p[id]
  - select all p elements with an id attribute
  - although this matches more than one element, Selenium will always use the first
  - ```
    assertEquals("A paragraph of text",
            selenium.getText("css=p[id]"));
    ```

Special attribute selectors

Some of the attributes are so commonly used that they have special notation to help make the CSS selectors shorter:

- ID - for Id's we can use the shortcut notation #
  - p#para1
    - we can use # to select an element with a specific id
  - #para1
    - we can also find something using just the id, equivalent to the XPath statement //*[@id='para1']
- Class – for class we can use the shortcut notation '.'
  - p.main

        - will match the p element with the class main

Indexed Matching

CSS Selectors also supports Indexed matching. The w3c specification lists all the indexing predicates, these are called pseudo classes in the w3c specification e.g.

- **first-child**, matches the first child of an element e.g.
    - body *:first-child
        - matches the first child of the body element
        - ```
          assertEquals("A paragraph of text",
                  selenium.getText("css=body *:first-child"));
          ```
- **last-child**, matches the last child of an element e.g.
    - Body *:last-child, matches the last child of the body element
    - ```
      assertEquals("Another paragraph of text",
              selenium.getText("css=body *:last-child"));
      ```
- **nth-child()**, matches the nth child of an element
    - ```
      assertEquals("A paragraph of text",
       selenium.getText("css=body *:nth-child(1)"));
      ```
        - match the first child of any type
    - ```
      assertEquals("A paragraph of text",
       selenium.getText("css=body p:nth-child(1)"));
      ```
        - match the first child of type p
    - ```
      assertEquals("Another paragraph of text",
       selenium.getText("css=body *:nth-child(2)"));
      ```
        - match the 2nd child of any type
- **nth-last-child()**, matches the nth child of an element counting backwards from the last child
    - body p:nth-last-child(1), returns the last child for the body element
        - ```
          assertEquals("Another paragraph of text",
                  selenium.getText("css=body p:nth-last-child(1)"));
          ```
    - body:nth-last-child(2), returns the 2nd last child for the body element
        - ```
          assertEquals("A paragraph of text",
                  selenium.getText("css=body p:nth-last-child(2)"));
          ```

The Selenium documentation describes support for all css1, css2 and css3 selectors. With the following exceptions, so no support yet for:

- CSS3 Name spaces,
- the following pseudo classes(:nth-of-type, :nth-last-of-type, :first-of-type, :last-of-type, :only-of-type, :visited, :hover, :active, :focus, :indeterminate)
- Also no support for the pseudo elements(::first-line, ::first-letter, ::selection, ::before, ::after).
          ```

Advanced

Combining Matches

You can combine several selections by chaining them in a comma separated list, the items will be returned the order listed e.g.

- p , title
  - get any paragraph element and the title element
  - `assertEquals("A paragraph of text",selenium.getText("css=p, title"));`
  - `assertEquals("Basic Web Page Title",selenium.getText("css=title, p"));`

Wild Card Matches

CSS has a universal selector * which matches any element type

- * matches all elements in the page
  - `assertEquals("Basic Web Page Title A paragraph of text\n Another paragraph of text", selenium.getText("css=*"));`
- body > *, matches all children under body
  - Since 2 elements get returned, Selenium will use the first one
  - `assertEquals("A paragraph of text",selenium.getText("css=body > *"));`

Attribute Substring matching

There are three attribute selectors which match substrings in the value of the attribute:

- ^=, matches the prefix
  - p[class^='ma']
    - would match any paragraph which has a class name starting with “ma”
    - `assertEquals("A paragraph of text", selenium.getText("css=p[class^='ma']"));`
- $=, matches a suffix
  - p[class$='n']
    - would match any paragraph with a class name ending in “n”
    - `assertEquals("A paragraph of text", selenium.getText("css=p[class$='n']"));`
- *=, matches a substring anywhere in the attribute value
  - p[class*='u']
    - would match any paragraph with “u” in the class name
    - `assertEquals("Another paragraph of text", selenium.getText("css=p[class*='u']"));`

Boolean Operators

You can setup matches with multiple conditions e.g.

- p[class='main'][id='para1']

- match any paragraph with a class of “main” and an id of “para1”
- ```
  assertEquals("A paragraph of text",  selenium.getText("css=p[class='main'][id='para1']"));
  ```

You can negate conditions e.g.

- p:not([class='main'])[id^='para']
  - would match any paragraph which does not have the class “main” and the id starts with “para”
  - ```
    assertEquals("Another paragraph of text",   selenium.getText("css=p:not([class='main'])[id^='para'] "));
    ```
- p:not([class='main'])[id^='para']:not([class='sub'])
  - you can have multiple negations in the selector
  - would match any paragraph which does not have the class “main” and the id starts with “para” and does not have the class “sub” - in our example this would match no elements
  - ```
    assertEquals(0,
      getCSSCount("p:not([class='main'])[id^='para']:not([class='sub'])"));
    ```

Sibling Combinators

As well as traversing a hierarchy with “ “ and “>” we can also check for siblings before and after a particular element e.g.

- +, match an element immediately following another element e.g.
  - p + p
    - match a paragraph that immediately follows another paragraph
    - ```
      assertEquals("Another paragraph of text",
             selenium.getText("css=p + p"));
      ```
- ~, match an element, following, but not immediately following another item

Useful Links

- http://kimblim.dk/css-tests/selectors/
  - A page that lists browser compatibility with various CSS selectors.
- http://robertnyman.com/firefinder/
  - Homepage for Firefinder with links to the support google group and instructions on its use.
- http://www.w3.org/TR/css3-selectors/
  - The official W3C selectors specification
  - The W3c specification, has an excellent summary of the CSS selection patterns

Note:

The source code for tests illustrating these CSS examples can be found in the source code archive described in Chapter 31 and in the package com.eviltester.seleniumtutorials.chap16

Chapter 17: Learning The Selenium API

Now that you have learned the basics of the unit test framework, Eclipse and simple Java. It is time to start learning the Selenium API.

Selenium has a very simple and unstructured API. Basically a long list of methods on the Selenium object. This makes the API large and unwieldy, but it also makes it very simple for beginners which is one reason why you don't need to know much Java to get started.

The most up to date documentation for the API is found in the Selenium source code, and since it is added as JavaDoc comments it will appear in Eclipse as the comments in the code completion documentation.

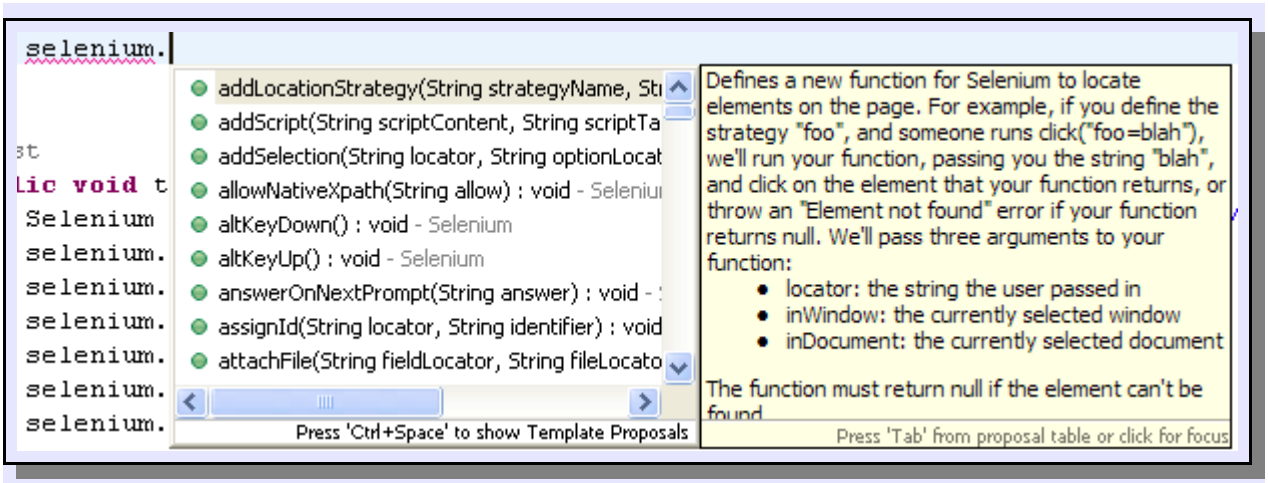


Figure 17.1 : JavaDoc displayed as code completion contextual help

In the following few chapters we will build some familiarity with the API methods by using them to complete common tasks that you will encounter when testing web sites.

Chapter 18: Testing HTML Forms

When testing websites we often have to input data which we send to the server.

This is typically done by using POST commands initiated from Forms.

To help I have created a simple form on the web at
http://www.compendiumdev.co.uk/selenium/basic_html_form.html

This contains all the common HTML Form Elements so we can manipulate them in a safe and simple environment.

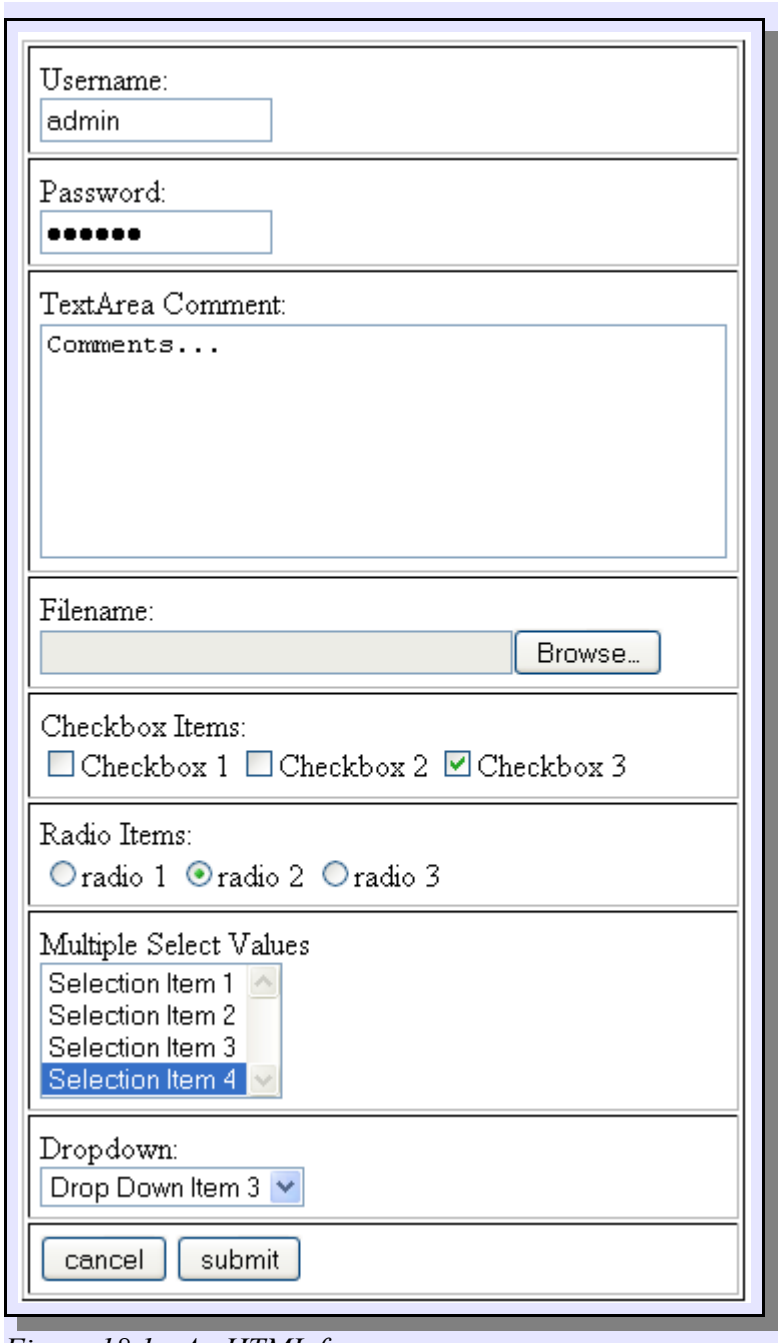


Figure 18.1 : An HTML form

An HTML Form

You can recognise an HTML form in the source code for a web page by looking for a *form* element.

```
<form name="HTMLFormElements" action=
       "http://www.compendiumdev.co.uk/selenium/form_processor.php"
    method="post" id="HTMLFormElements">
```

Forms have the following child elements which capture the input for the form:

- **Single Line Text**

```
<input type="text" name="username" size="15" />
```

- **Passwords**

```
<input type="password" name="password" size="15" />
```

- **Multi-line text**

```
<textarea cols="40" name="comments" rows="6">
```

- **Files**

```
<input type="file" name="filename" size="35" />
```

- **Hidden Fields**

```
<input type="hidden" name="hiddenField" value="Hidden Field Value" />
```

- **Check Box Items**

```
<input type="checkbox" name="checkboxes[]" value="cb1" />
```

- **Radio Buttons**

```
<input type="radio" name="radioval" value="rd1" />
```

- **Multi-Select Items**

```
<select multiple="multiple" name="multipleselect" size="4">
       <option value="ms1">
              Selection Item 1
       </option>
       <option value="ms2">
              Selection Item 2
       </option>
</select>
```

- **Dropdown selection items**

```
<select name="dropdown">
       <option value="dd1">
               Drop Down Item 1
      </option>
      <option value="dd2">
               Drop Down Item 2
      </option>
</select>
```

- **Submit Button**

```
<input type="submit" name="submitbutton" value="submit" />
```

We need to learn how to handle all of these form elements with Selenium to test forms effectively.

Submit a Form with Default Values

The easiest way of learning how to do something simple in Selenium is to record it through the IDE. So we will use the IDE to visit the web form and submit it.

Use Firefox to visit http://www.compendiumdev.co.uk/selenium/basic_html_form.html

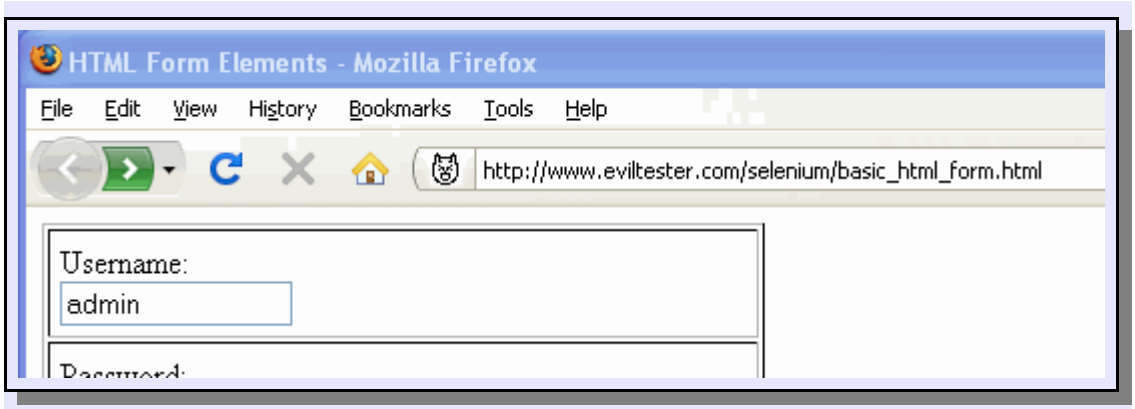


Figure 18.2: The URL to access an HTML form you can use for learning

Start up the Selenium IDE.

Then, type the page URL (http://www.compendiumdev.co.uk/selenium/basic_html_form.html) into the "Base URL" field in the IDE.

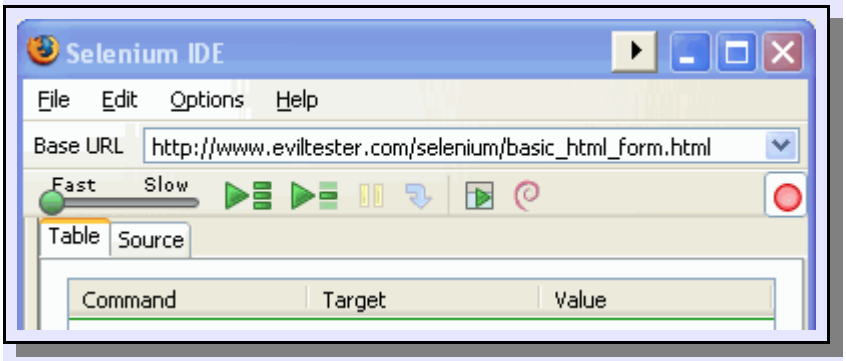


Figure 18.3: The URL in the IDE

After making sure that the IDE is recording, click the Submit button on the Form.



Figure 18.4 : Submit Button

You should now see the IDE showing a simple Selenese Script.

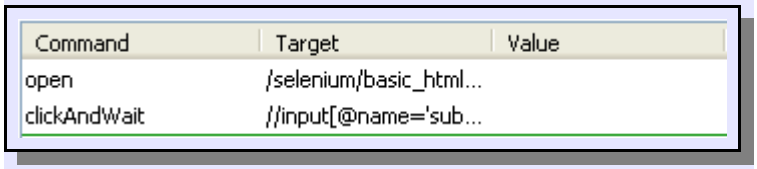


Figure 18.5: A simple Selenese Script

Use the Format Option in the IDE "Options \ Format\ Java (JUnit) – Selenium RC" to convert this

into a simple Java test.

Now, in Eclipse create a new JUnit Test Case called "HTML_form_tests"to copy the automatically generated code into. "File \ New \ JUnit Test Case". Automatically generating the setUpBeforeClass(), tearDownAfterClass() and setUp methods.

> **Note:**
>
> **I changed the package so that these tests were stored in com.eviltester.seleniumtutorials.chap18 as this will help you locate the tests in the downloadable source code (as covered in Chapter "Playing along at home")**

Now copy in only the testUntitled() method with the test details in it, to the Eclipse IDE. Rename the test method to "test_submit_form_with_default_values"

In order to run, we need to:

- add @Test to the test method
- add "import org.junit.Test;"
- create the code in our @BeforeClass and @AfterClass methods,
- and make references to Selenium
- all our tests will be done on the basic_html_form.html so I will move the selenium.open statement into the @Before method
- and I know we will be adding asserts so I also add an import for those
    - `import static org.junit.Assert.*;`

To make the test look like this:

```
package com.eviltester.seleniumtutorials.chap18;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Test;
import static org.junit.Assert.*;
import com.thoughtworks.selenium.DefaultSelenium;
import com.thoughtworks.selenium.Selenium;

public class HTML_form_tests {
	private static Selenium selenium;
	@BeforeClass
	public static void setUpBeforeClass() throws Exception {
		selenium = new DefaultSelenium("localhost", 4444,
				"*firefox", "http://www.compendiumdev.co.uk");
		selenium.start();
	}

	@AfterClass
	public static void tearDownAfterClass() throws Exception {
		selenium.close();
		selenium.stop();
	}

	@Before
	public void setUp() throws Exception {
		selenium.open("/selenium/basic_html_form.html");
	}

	@Test
	public void test_submit_form_with_default_values(){
		selenium.click("//input[@name='submitbutton' and @value='submit']");
		selenium.waitForPageToLoad("30000");
	}
}
```

The steps I took to get the code to this point were:

- First add the "`private Selenium selenium;`" command. When I added this code, Eclipse automatically added in the necessary import statements.
- Manually added the code to start selenium in setUp
- Manually added the close and stop calls in tearDownAfterClass

Commands Used

First we had to *open* the page which we wanted.

Submitting a form requires the use of the *click* command on the submit button element.

The IDE did a good job of constructing the XPath statement that uniquely identifies the button on our form so there is no need to change that.

Also note that since we expect the click action to open a new page we also use the *waitForPageToLoad* command to pause the test until a new page is loaded.

Submit Form without clicking button

Some forms allow you to submit them without clicking a button, e.g. A search field.

Selenium has the submit command for these particular situations. e.g.

```
selenium.submit("//form[@id='HTMLFormElements']");
```

This doesn't work exactly the same way as clicking the button since the name and value of the button will not be part of the request sent to the server.

If you try this on the example application we are using for testing then you will notice that in the results page, text is displayed saying that you did not click the button. So this may prove useful for some applications but it probably should not be your default submit process.

```
@Test
public void test_submit_form_without_clicking_submit(){
        selenium.submit("//form[@id='HTMLFormElements']");
        selenium.waitForPageToLoad("30000");
}
```

Amend a Text, Password, TextArea or File Field

Useful Commands: Type, TypeKeys

Text Fields

We can enter values into a text field using the *type* command. e.g.

```
selenium.type("username", "eviltester");
```

type takes a locator ("username") and a value to type ("eviltester");

For this form we can use "username" as the locator because the source of the input field is:

```
<input type="text" size="15" name="username"/>
```

Selenium is smart enough to use the *name* attribute to find the element to use. We could also have used a more complicated XPath to achieve the same results e.g.

```
//input[@name="username"]
```

So a test which types in a username and submits the form would look like:

```
@Test
public void test_submit_form_with_new_username() throws Exception {
        selenium.type("username","eviltester");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Type vs TypeKeys

Selenium provides us with two commands for typing in data.

Type, simulates the typing of values into the field.

TypeKeys, triggers the keyDown, keyPress, and keyUp events for each character that it inputs. This can be useful when entering data into javaScript enabled fields.

For most forms you will get by with the *type* command.

Password Fields

Password fields are basically just text fields but the value typed in is not visible to the user.

Use the same type or typekeys command covered above.

So to type "myPassword" in to the password field we would use:

```
selenium.type("password","myPassword");
```

e.g.

```
@Test
public void test_submit_form_with_new_password(){
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

TextArea Fields

TextArea fields are essentially big text fields so we would use the type command again.

But we will probably want to take advantage of the multi-line nature of the textarea.

To do this we insert a \n into the string we pass to the type command. e.g.

```
selenium.type("comments", "a multiline\ncomment\nin the TextArea");
```

This would enter:

```
a multiline
comment
in the TextArea
```

Here we are simply using the Java escape characters.

Note:

In java putting \ into a string does not add \ it sets java up to expect an escape character.

Java escape characters:

\b – backspace

\t – tab

\n – newline

\f – form feed

\r – carriage return

\" – double quote

\' – single quote

\\ – backslash

\uHHHH – unicode escape where HHHH is a hex value encoding (for more see http://www.utf8-chartable.de/)

Example:

```
selenium.type("comments", "\\\n\t\"I said, give me your pound sign\"\u00A3");
```

Would type:

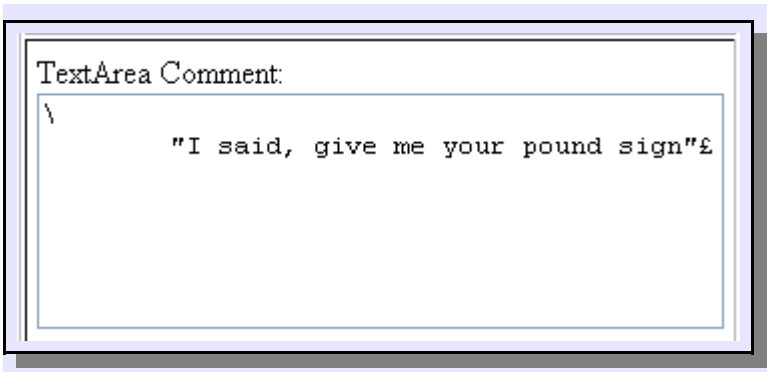


Figure 18.6: Escaped Chars added to a Text Area

You can still use escape characters in a normal text field and they will be passed through in the form, but you won't see the effect of new lines in the text box. e.g.

```
@Test
public void test_submit_form_with_html_escaped_text(){
        selenium.type("comments", "\\\n\t\"I said, give me your pound sign\"\u00A3");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

File Field

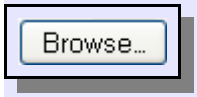


Figure 18.7: Browse Button

If you have ever used the browse button on IE or Firefox to upload a file then you might wonder how Selenium handles the popup dialog box that appears.

On older browsers e.g. IE6 and Firefox below 3.5, the common way of avoiding this was to use the *type* command to add the full path name of a file into the File field.

```
selenium.type("filename", "C:\\Selenium\\readme.txt");
```

The above statement fills the filename box with: c:\selenium\readme.txt

Note the use of the escape characters to add \ into the string.

```
@Test
public void test_submit_form_with_filename(){
       selenium.type("filename", "C:\\Selenium\\readme.txt");
       selenium.click("//input[@name='submitbutton' and @value='submit']");
       selenium.waitForPageToLoad("30000");
}
```

This no longer seems to work because browsers are now more secure against JavaScript hacks.

This is still a tricky area of web applications to work with.

Attachfile command

The attach file command is typically used when your server is not running on the same box as your client, or in a grid scenario. The Attach file command differs because it takes the URL of a file as its argument. i.e.

```
attachFile(locator, url)
```

So if I want to use attachFile for a local file then I would have to use

```
selenium.attachFile("filename", "file:\\C:\\Selenium\\readme.txt");
```

Note also that in the file box, the file path is not "c:\selenium\readme.txt" it is a temporary file location e.g.

```
"C:\DOCUME~1\Alan\LOCALS~1\Temp\se-567909924.txt"
```

Because the file has had to be transferred from the client, to the server.

Selenium Documentation lists this as only supporting the Firefox *chrome browser mode for Selenium.

Amending Checkboxes and Radio Items

Checkboxes and radio boxes use exactly the same commands either:

- click,
- check,
- uncheck

Click

Differences in behavior with click are only apparent on the browser.

So for a checkbox:

- If selected then click on it will unselect it.
- If unselected then click on it will select it.

For a radio button:

- click will always select it

e.g.

```
selenium.click("//input[@name='checkboxes[]' and @value='cb3']");
selenium.click("//input[@name='radioval' and @value='rd2']");
```

Click will only raise an exception if it cannot click on the element. i.e. If the element is not found or not visible or not enabled.

In a checkbox group, multiple items can be selected.

In a radio button group, only one radio button can be selected at a time. As soon as you click on one of the radio buttons, whichever of the others that was previously selected, will be unselected.

```
@Test
public void test_submit_form_with_click_check_and_radio(){
        selenium.click("//input[@name='checkboxes[]' and @value='cb2']");
        selenium.click("//input[@name='checkboxes[]' and @value='cb3']");
        selenium.click("//input[@name='radioval' and @value='rd3']");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Check

Check will make sure that the item is checked after the action has run.

So in the following scenario where checkbox 3 is selected, and checkbox 2 is not selected.

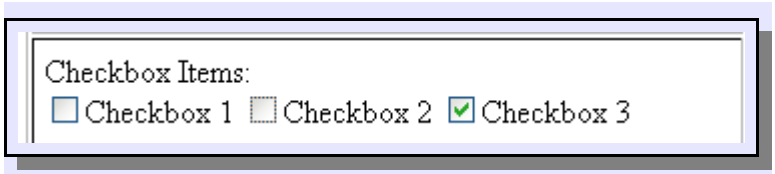


Figure 18.8: Checkbox 3 Selected

If I run:

```
selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
selenium.check("//input[@name='checkboxes[]' and @value='cb3']");
```

Then the end result would be checkbox 3 selected and checkbox 2 selected.

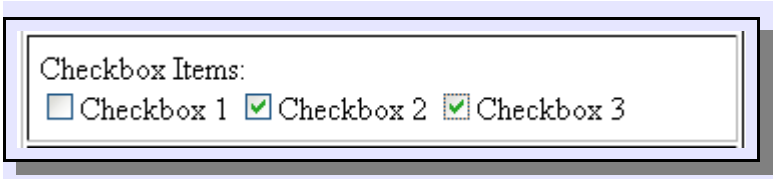


Figure 18.9: Two Checkboxes selected

To contrast this with the click method, starting from the same point.

```
selenium.click("//input[@name='checkboxes[]' and @value='cb2']");
selenium.click("//input[@name='checkboxes[]' and @value='cb3']");
```

Had I used click then I would have ended up with checkbox 2 selected, but because checkbox 3 was selected prior to the click, checkbox 3 would have been unselected.

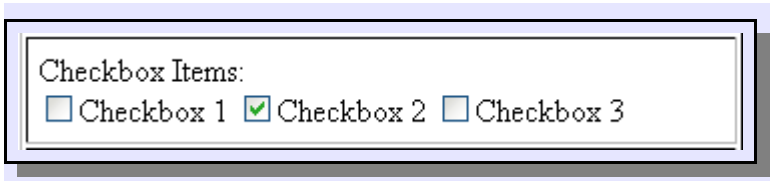


Figure 18.10: No checkboxes selected

For radio buttons *check* works much the same as *click* since you can't uncheck a radio button by clicking on it, only by selecting another radio button in the group.

Uncheck

Uncheck guarantees that whatever the status of the element before we issue the uncheck command it will be unchecked afterward.

So in the following scenario for checkboxes where checkbox 3 is selected:

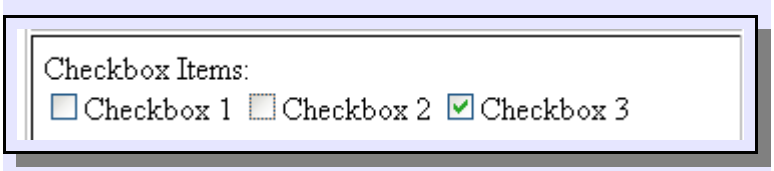


Figure 18.11: A Single Checkbox selected

Running

```
selenium.uncheck("//input[@name='checkboxes[]' and @value='cb3']");
selenium.uncheck("//input[@name='checkboxes[]' and @value='cb1']");
```

Would result in no checkboxes being selected. Since cb3 would be unchecked and cb1 would remain unchecked.

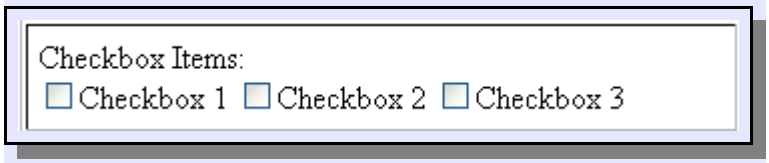


Figure 18.12: After 'unchecking' the checkbox

Uncheck has a different effect on radio buttons and allows you to bypass normal browser functionality.

Given the situation where Radio button 2 was selected.



Figure 18.13: A single radio button selected

If I were to *click* on any of the items, then that item and only that item would be selected.

If I were to *check* any of the items then that item and only that item would be selected.

If I were to *uncheck* any unselected item then it will remain unselected.

But if I *uncheck* a selected item I can *uncheck* that item and have the situation where none of the radio buttons are selected. Something you can't do normally in the browser. e.g.

```
selenium.uncheck("//input[@name='radioval' and @value='rd2']");
```

The above code would result in the view below:

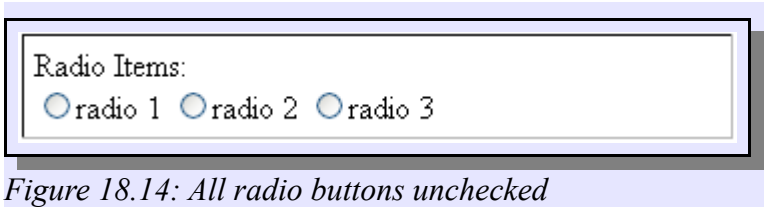


Figure 18.14: All radio buttons unchecked

Amending Multiple Select Values

Selenium provides two commands for working with multiple selection items.

- removeSelection – unselects a selected item
- removeAllSelections – unselect every selected item
- addSelection – adds an unselected item to the selection

Both commands take 2 arguments (locator, optionLocator):

- locator – a locator for the selection element itself
- optionLocator – the item in the selection element

So in on our example page we have

```
<select multiple="multiple" name="multipleselect[]" size="4">
       <option value="ms1">
              Selection Item 1
       </option>
       <option value="ms2">
              Selection Item 2
       </option>
       ...
</select>
```

I can select an item using the following code:

```
selenium.addSelection("multipleselect[]", "label=Selection Item 1");
```

If I addSelection an item that is already selected then it remains selected and no exception is thrown.

And I can remove a selection with:

```
selenium.removeSelection("multipleselect[]", "label=Selection Item 4");
```

If I removeSelection on an item that is not selected then the item remains unselected and no exception is thrown.

Alternatively I could remove all the selected items:

```
selenium.removeAllSelections("multipleselect[]");
```

I am not limited to using 'label' as my locator for the option element, I could just as easily have used the 'value'

```
selenium.addSelection("multipleselect[]", "value=ms2");
```

So If I start with the following situation:

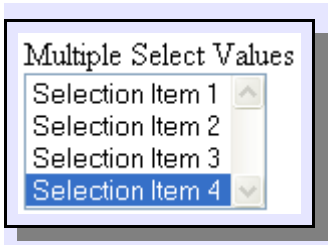


Figure 18.15: A Multiple Selection Item

And run the following test:

```
@Test
public void test_submit_form_with_multiple_select_values(){
        selenium.removeSelection("multipleselect[]", "label=Selection Item 3");
        selenium.addSelection("multipleselect[]", "label=Selection Item 4");
        selenium.addSelection("multipleselect[]", "label=Selection Item 1");
        selenium.removeSelection("multipleselect[]", "label=Selection Item 4");
        selenium.addSelection("multipleselect[]", "value=ms2");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

We will end up with the following form submitted through to the application:

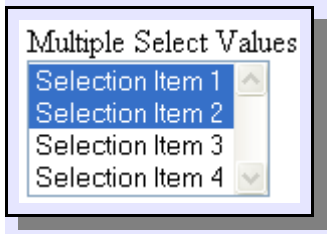


Figure 18.16: Two Items Selected

Amending Dropdown Elements

Drop down elements have a very similar html syntax to the multi-select box above, but because of the absence of the attribute *multiple="multiple"*. They are displayed as a drop down.

```
<select name="dropdown">
        <option value="dd1">
                Drop Down Item 1
        </option>
        <option value="dd2">
                Drop Down Item 2
        </option>
</select>
```

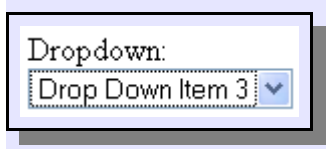


Figure 18.17 : A dropdown item

Because only one item can be selected in a dropdown, Selenium provides us with a single command for manipulating dropdown elements:

- select

Select takes two attributes (selectLocator, optionLocator):

- selectLocator – a locator for the select element
- optionLocator – a locator for the option element

So running a test with the code:

```
selenium.select("dropdown", "label=Drop Down Item 1");
```

Would select item 1.

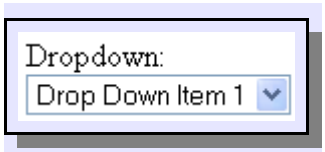


Figure 18.18: Item1 selected in dropdown

As this test code demonstrates:

```
@Test
public void test_submit_form_with_dropdown_values(){
	// by default 3 is selected, so select item 1
	selenium.select("dropdown", "label=Drop Down Item 1");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
}
```

Amending Hidden Field Values

Amending a hidden field value is unlikely to be something that you will often want to do in your automated tests. Although it is something that we frequently want to do in our manual tests.

The Selenium API does not have a method designed for this specific task. But it does have a very generic method for executing arbitrary JavaScript. And so, by supplementing our learning of Java with a little JavaScript we can do more things to the web application than we can through Selenium alone.

The command we will use here, since we just want to execute a small snippet of JavaScript is:

getEval

getEval allows us to pass in some JavaScript which Selenium will execute in the browser on our behalf and return the value of the script. e.g.

```
selenium.getEval("this.browserbot.findElement(\"name=hiddenField\").value=\"amended value\"");
```

To amend a hidden field we will take advantage of the fact that Selenium creates an object in the Browser context called browserbot. So I can use browserbot to find the element I want to amend and then use normal JavaScript DOM manipulation to amend the value of the element. e.g.

```
@Test
public void test_submit_form_with_hidden_field(){
	selenium.getEval(
	"this.browserbot.findElement(\"name=hiddenField\").value=\"amended value\"");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
```

```
}
```

Checking the Values of the Input Fields

Simply issuing the commands to change form values is not good enough when writing automated tests so we need the ability to query the values of the form items. Either:

- before we change them to take action based on the default state of the form, or
- after we change them to make sure that our actions did what we expected.

Selenium provides a number of methods for this. Some very general like getValue which we can apply to many of the elements. And others more specific for working with <select> items.

Text, Password, TextArea, File

For all these items we use getValue to give us the value of the elements. e.g.

```
selenium.getValue("username");   // will return the text entered as the username
```

As illustrated by the following test. The test types in some values for the text items then uses assertions to check if the value entered is the same as the value that we typed in.

```
@Test
public void test_check_text_entered_values(){
        selenium.type("username","eviltester");
        selenium.type("filename", "C:\\Selenium\\readme.txt");
        selenium.type("password","myPassword");
        selenium.type("comments", "simple text");

        assertEquals("username not as entered",
                        "eviltester",selenium.getValue("username"));
        assertEquals("password not as entered",
                        "myPassword",selenium.getValue("password"));
        assertEquals("filename not as entered",
                        "C:\\Selenium\\readme.txt",selenium.getValue("filename"));
        assertEquals("comments not as entered",
                        "simple text",selenium.getValue("comments"));
}
```

CheckBox & Radio Items

For Checkboxes and Radio Items we have two commands that we can use:

- getValue, which returns the string “off” or “on” depending on the state of the checkbox/radio button.
- isChecked, which returns a boolean *true* or *false* depending on the state of the checkbox/radio button. isChecked takes a single parameter *locator* to identify the item you want to check.

e.g. The following tests use both getValue and isChecked to query the default settings of the checkboxes and radio buttons and then, after selecting them, checks the values again.

```
@Test
public void test_check_checkbox_values(){

        assertEquals("by default checkbox 1 is not selected", "off",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb1']"));
        assertEquals("by default checkbox 2 is not selected", "off",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb2']"));
        assertEquals("by default checkbox 3 is selected", "on",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb3']"));

        selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
```

```
        selenium.uncheck("//input[@name='checkboxes[]' and @value='cb3']");

        assertFalse("checkbox 1 is still not selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb1']"));
        assertTrue("checkbox 2 is now selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb2']"));
        assertFalse("default checkbox 3 is no longer selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb3']"));
    }

    @Test
    public void test_check_radio_values(){

        assertEquals("by default radio 1 is not selected", "off",
                selenium.getValue("//input[@name='radioval' and @value='rd1']"));
        assertEquals("by default radio 2 is selected", "on",
                selenium.getValue("//input[@name='radioval' and @value='rd2']"));
        assertEquals("by default radio 3 is not selected", "off",
                selenium.getValue("//input[@name='radioval' and @value='rd3']"));

        selenium.check("//input[@name='radioval' and @value='rd1']");

        assertTrue("radio 1 is now selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd1']"));
        assertFalse("radio 2 is no longer selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd2']"));
        assertFalse("radio 3 is not selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd3']"));
    }
```

Multiple Select & DropDown

MultiSelect boxes and DropDowns use the same commands (since they are essentially the same html markup that should not come as too much of a surprise).

- `getValue`
- `isSomethingSelected`
- `getSelectedId`
- `getSelectedIds`
- `getSelectedIndex`
- `getSelectedIndexes`
- `getSelectedLabel`
- `getSelectedLabels`
- `getSelectedValue`
- `getSelectedValues`
- `getSelectOptions`

A quick reminder of the syntax of a Select element

```
<select multiple="multiple"
name="multipleselect" size="4">
    <option value="ms1">
         Selection Item 1
    </option>
    <option value="ms2">
         Selection Item 2
    </option>
    ...

</select>
```

```
<select name="dropdown">
    <option value="dd1">
         Drop Down Item 1
    </option>
    <option value="dd2">
         Drop Down Item 2
    </option>
    ...
</select>
```

getValue

getValue is useful for drop downs where a single item can be selected, but less so for multi selection items. For single dropdown items getValue returns the value attribute of the item selected.

For the default on our test page it is "dd3" : e.g.

```
selenium.getValue("dropdown")
```

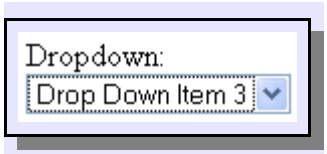


Figure 18.19 : Item 3 selected by default

isSomethingSelected

isSomethingSelected returns a boolean value which tells us if at least one item has been selected in either the dropdown or the multi-select element. e.g.

```
selenium.isSomethingSelected("dropdown")
```

getSelectedId & getSelectedIds

getSelectedId returns the *id* attribute of the selected option in a dropdown.

In the markup of our test page we have no id values so this will return an empty string.

In order to return a value we would need markup like the following:

```
<option id="selectedbydefault" value="dd3" selected="selected">
        Drop Down Item 3
</option>
```

With this markup, if dd3 were selected then getSelectedId would return the string "selectedbydefault".

getSelectedIds returns a string array of the *id* attributes of any selected options. If, as is the case in our example page, there are no ids, then you will receive an array with 1 item which is an empty string.

getSelectedIndex & getSelectedIndexes

getSelectedIndex returns a string containing the index value of the selected item. e.g.

```
selenium.getSelectedIndex("dropdown");
```

On our example page the third item is selected by default and *getSelectedIndex* returns "2" because the Indexes of the option elements start at 0. So "dd1" is at index 0.

getSelectedIndexes returns a string array containing the index values of the selected items. This can be used on both drop down and multi-selects e.g.

```
System.out.println("dropdown getSelectedIndexes:");
for (String item : selenium.getSelectedIndexes("dropdown")) {
```

```
        System.out.println(item);
}

System.out.println("multiselect getSelectedIndexes:");
for (String item : selenium.getSelectedIndexes("multipleselect")) {
        System.out.println(item);
}
```

This would print out:

```
dropdown getSelectedIndexes:
2
multiselect getSelectedIndexes:
3
```

If more items were selected in the multiselect then they would all be printed out.

getSelectedLabel & getSelectedLabels

getSelectedLabel returns a string containing the label of selected item (the text displayed in the dropdown). e.g.

```
selenium.getSelectedLabel("dropdown");
```

For our example page this would return "Drop Down Item 3" which is the text for the selected option. e.g.

```
<option value="dd3" selected="selected">
        Drop Down Item 3
</option>
```

getSelectedLabels returns a string array containing the labels of all the selected items. This can be used on both dropdown and multi-selects e.g.

```
System.out.println("dropdown getSelectedLabels:");
for (String item : selenium.getSelectedLabels("dropdown")) {
        System.out.println(item);
}

System.out.println("multiselect getSelectedLabels:");
for (String item : selenium.getSelectedLabels("multipleselect")) {
        System.out.println(item);
}
```

This would print out:

```
dropdown getSelectedLabels:
Drop Down Item 3
multiselect getSelectedLabels:
Selection Item 4
```

getSelectedValue & getSelectedValues

getSelectedValue returns a string containing the value attribute of selected item e.g.

```
selenium.getSelectedValue("dropdown");
```

For our example page this would return "dd3", the value attribute in the defining HTML e.g.:

```
<option value="dd3" selected="selected">
        Drop Down Item 3
</option>
```

getSelectedValues returns a string array containing the values of all the selected items. This can be used on both drop down and multi-selects e.g.

```
System.out.println("dropdown getSelectedValues:");
for (String item : selenium.getSelectedValues("dropdown")) {
```

```
        System.out.println(item); // dd3
}

System.out.println("multiselect getSelectedValues:");
for (String item : selenium.getSelectedValues("multipleselect")) {
        System.out.println(item); // ms4
}
```

This would printout:

```
dropdown getSelectedValues:
dd3
multiselect getSelectedValues:
ms4
```

getSelectOptions

GetSelectOptions applies to both drop downs and multi select elements, and it returns a string array of all the option labels. e.g.

```
System.out.println("dropdown getSelectOptions:");
for (String item : selenium.getSelectOptions("dropdown")) {
        System.out.println(item); // all the labels of items in the drop down
}

System.out.println("multiselect getSelectOptions:");
for (String item : selenium.getSelectOptions("multipleselect")) {
        System.out.println(item); // all the labels of items in the multi select
}
```

displays:

```
dropdown getSelectOptions:
Drop Down Item 1
Drop Down Item 2
Drop Down Item 3
Drop Down Item 4
Drop Down Item 5
Drop Down Item 6

multiselect getSelectOptions:
Selection Item 1
Selection Item 2
Selection Item 3
Selection Item 4
```

Put It All Together

To consolidate our learning we can put this all together into an example test.

This test will go to the page and randomly select an item from the drop down, while making sure that it does not reselect the value that was there by default.

Many people don't use random values in tests because they want the tests to be “repeatable”. For some tests that is important. But sometimes we can increase the data coverage of our tests by using random selections and expose bugs after repeated runs of the test, thereby increasing the bug finding potential of the tests.

This example covers the dropdown, but the multiselect code is much the same.

```
@Test
public void test_randomly_select_value_from_dropdown(){

        // get all options in the dropdown
        String dropDownOptions[] = selenium.getSelectOptions("dropdown");
```

```
        // get the index of the currently selected item in the dropdown
        String currentDropDownItem = selenium.getSelectedIndex("dropdown");
        int previouslySelectedIndex =Integer.valueOf(currentDropDownItem);

        // randomly choose an item in the dropdown
        Random generator = new Random(); // requires import java.util.Random to be added
        int dropDownIndex = generator.nextInt(dropDownOptions.length);

        // make sure it isn't the same as the current one
        if(dropDownIndex==previouslySelectedIndex){
                // it is the same, so add one
                dropDownIndex++;
                // but this might push it out of bounds so modulus it
                dropDownIndex = dropDownIndex % dropDownOptions.length;
        }

        // select this new one
        selenium.select("dropdown", "label=" + dropDownOptions[dropDownIndex]);

        // check that something is selected
        assertTrue(selenium.isSomethingSelected("dropdown"));

        // check that what we wanted is selected
        assertEquals(dropDownIndex,
                Integer.valueOf(selenium.getSelectedIndex("dropdown")).intValue());

        //double check we didn't select the same thing
        assertTrue(dropDownIndex!=previouslySelectedIndex);
    }
```

A few explanations for readers with less Java experience:

- Since getSelectedIndex returns a String, I have to convert it to an Integer to allow us to do the arithmetical comparisons which I do with the Integer.ValueOf method which converts the String *currentDropDownItem* into an integer
    - `Integer.valueOf(currentDropDownItem);`
- I use the standard Java Random class to generate a random number between 0 and the length of the dropDownOptions string array. So if the array is 6 in length, then I get a random number from 0 to 5 inclusive. I also have to add "`import java.util.Random;`" to the head of my .java file
- Since we are making the test random and expect the test to last the lifetime of the application. I have taken the precaution that the default value might change. Currently the default value is 4 out of 6 possible items. But a later change may make "Drop Down Item 6" the default, and if that happens then adding 1 in the event of a conflict would create an array index of 6 and push it out of bounds. After adding 1 to it, I use the modulus operator "%", which acts as a remainder operation, and in the event of choosing 6 we do 6%6. The result of 6%6 is 0 which is the first item in the array. If the random value conflicts but does not push it out of bounds e.g. 4, then the result of 4%6 is 4 , so we get the value we wanted.

Chapter 19: Testing Static HTML

By static HTML I mean the standard HTML markup commands e.g <h1>, <p>, <li>, <ul> etc. Ignoring Forms (which we covered) and ignoring JavaScript, which we will cover later.

Despite the large amount of HTML markup elements (see http://en.wikipedia.org/wiki/HTML_element). Selenium handles these the same way.

Much of the time when we test applications we check if the page has displayed values that we want to see displayed, and traverse through links on the site. Most of this type of processing is done via static HTML commands and we can get by with a minimal set of Selenium commands and an understanding of XPath or CSS selectors.

In this section we will cover the following commands (the commands that I think you will use most, I have formatted with bold text):

- assignid
- **click**
- getAttribute
- getBodyText
- getHTMLSource
- **getText**
- **getTitle**
- **getXPathCount**
- highlight
- goBack
- **isElementPresent**
- **isTextPresent**

A Basic Static HTML Test

In this example we will submit a form and check some basic results on the processor page, then navigate back to the form, change it, and check that our changes were passed through.

First I create a new class called Static_HTML_tests.java, based on the previous HTML_form_tests.java, I added this to the package com.eviltester.seleniumtutorials.chap19 with the following code:

```
package com.eviltester.seleniumtutorials.chap19;

import static org.junit.Assert.*;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;
import com.thoughtworks.selenium.DefaultSelenium;
import com.thoughtworks.selenium.Selenium;


public class Static_HTML_tests {
```

```
	private static Selenium selenium;

	@BeforeClass
	public static void setUpBeforeClass() throws Exception {
		selenium = new DefaultSelenium("localhost", 4444,
				"*firefox", "http://www.compendiumdev.co.uk");
		selenium.start();
	}

	@AfterClass
	public static void tearDownAfterClass() throws Exception {
		selenium.close();
		selenium.stop();
	}


	@Before
	public void setUp() throws Exception {
		selenium.open("/selenium/basic_html_form.html");
	}

	@Test
	public void submit_form_with_values_and_check_static_html(){
		// add test details in here
	}
}
```

The first thing I do in the test is fill details into the form and then submit it. We have covered this in the previous section so no real explanation is required:

```
@Test
public void submit_form_with_values_and_check_static_html(){
	selenium.type("username","eviltester");
	selenium.type("password","myPassword");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
}
```

I want to investigate the resulting page to help me build my test results.

So I will manually goto http://www.compendiumdev.co.uk/selenium/basic_html_form.html and type in the username and password above, and then press submit.

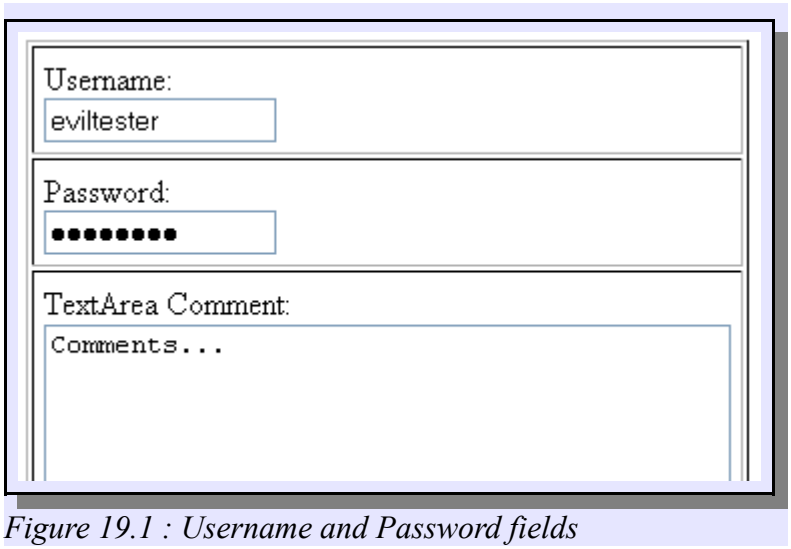


Figure 19.1 : Username and Password fields

When I submit the form I see the form processed page which is static HTML generated by the PHP application.



Figure 19.2 : Results of form submission

Based on what I can see here I will check:

- that the title is as I expect
- the username I typed in
- the password I typed in
- a comment was added by default
- no filename was entered
- the number of checkboxes was as I expected

Then I will click the "Go back to the main form" link.

And in the form, amend the checkboxes, and add a filename.

Then resubmit the form and check those details.

getTitle – Check the page title

Selenium comes with a command to return the title of the page as a String:

- getTitle

getTitle takes no arguments and I will add it to the test directly in an Assert.

```
assertEquals("Processed Form Details",selenium.getTitle());
```

So how did I know what text to add as my 'expected result' text?

Well, I can see it displayed at the top of the browser, so I could type it into the test from there.

Or I can more accurately view the page source and copy it from there.

In Firefox, to view the page source I right click on the page and select “View Page Source”

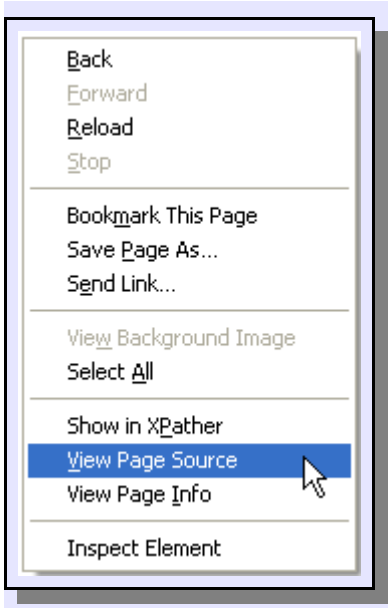


Figure 19.3 : View Source

Then in the resulting source display, select the text in the <title> element



Figure 19.4 : Source Displayed

By putting the check inside the AssertEquals, if the page title is not as I expect then the test will throw an assertion exception and fail.

getText – Check the username

Selenium's getText command allows us to retrieve the text for any element.

- getText(String locator)

getText takes a String as an argument the locator for the element we want to get the text for.

So if we look at the page in Firefox I can right click on the username text and (using the Firebug plugin which we installed) choose *Inspect Element* to see its details.

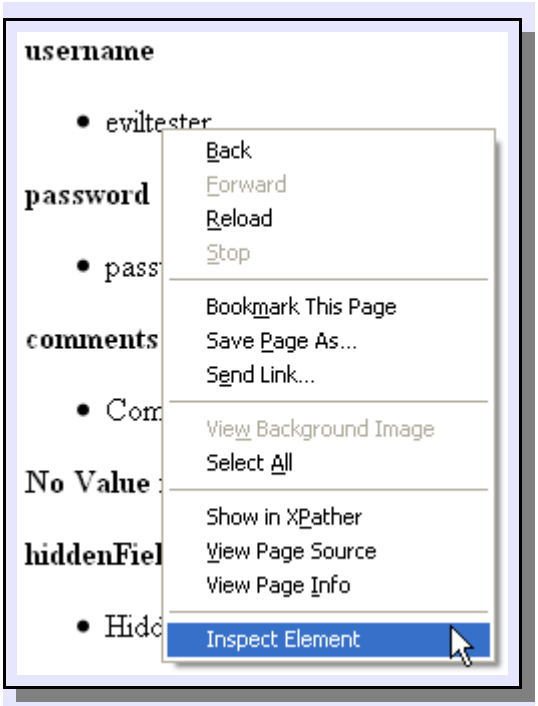


Figure 19.5 : Inspect Element Option

I can see that the <li> element for the username has an id of "_valueusername".

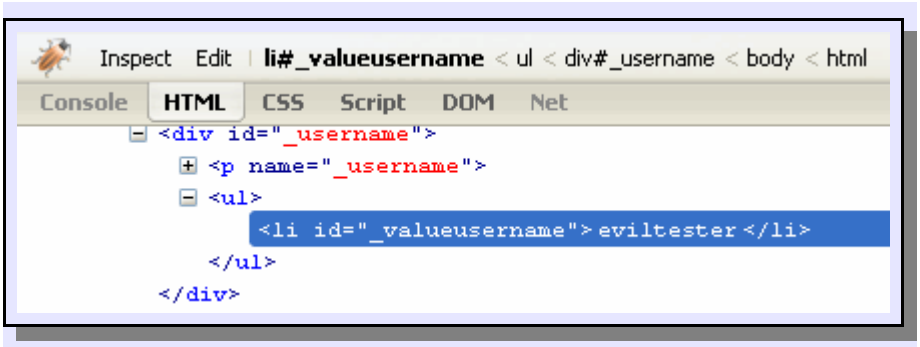


Figure 19.6 : Username field inspected

So I can use that id directly as my locator string. And again I will use this directly within an Assert statement.

```
assertEquals( "eviltester",selenium.getText("_valueusername"));
```

NOTE:

I can use the getText to access the text of any element. So I could have used it in place of getTitle (had Selenium not provided that command) by using:

selenium.getText("//head/title")

If you want to confirm this for yourself, try the following test:

```
@Test
public void getText_equals_getTitle() {
        selenium.open("/selenium/basic_html_form.html");
        assertEquals("HTML Form Elements",selenium.getTitle());
        assertEquals(selenium.getTitle(),selenium.getText("//head/title"));
}
```

isTextPresent – check the 'heading'

Our example page has the text "Submitted Values" at the top, so I will do a quick check to make sure that the text is present on the page.

- isTextPresent(String pattern)

isTextPresent takes a single String as an argument and this can be written in 5 different ways:

- "Submitted Values"
  - note: with no prefix (as above) this defaults to glob pattern matching so you can use * and ? as wild cards.
- "glob:Sub*d V?lu?s"
  - prefix the string with "glob:" to use this format deliberately
  - "*" stands for any sequence of characters
  - "?" stands for any single character
  - also known as "wildmat" pattern matching (http://en.wikipedia.org/wiki/Wildmat)
- "regexp:S..m.tted V(.)*s"
  - prefix with "regexp:" to use JavaScript regular expression patterns
  - for more details see http://www.javascriptkit.com/javatutors/redev.shtml and http://www.regular-expressions.info/javascript.html
- "regexpi:s..m.tted v(.)*s"
  - prefix with "regexpi:" to use a case insensitive JavaScript regular expression patterns
- "exact:Submitted Values"
  - find the string exactly on the page with no wild card processing

In our test I will use a glob search but with no wild cards.

```
assertTrue(selenium.isTextPresent("Submitted Values"));
```

isElementPresent – check comments and filename

Sometimes we want to check if an HTML Element is present on the page or not, and we can use Selenium's isElementPresent command to do this:

- isElementPresent(String locator)

isElementPresent takes a locator as the argument and if found returns a boolean value of True.

The example static HTML for the form processing has a very simple structure.

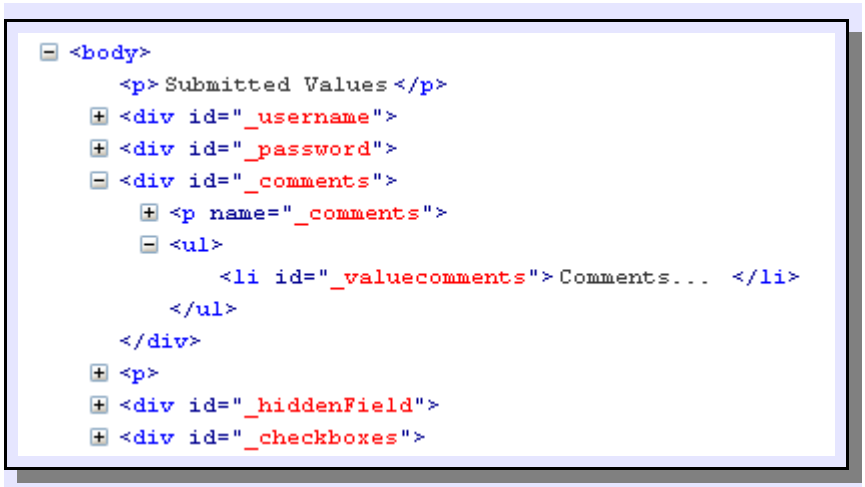


Figure 19.7 : HTML for a simple page structure

Each field that I submitted in the form is contained in a div with an id of "_fieldname" and within the div is a descriptive paragraph with the "name" of the field and an unordered list, where each list item has the value entered in the field and an id of "_value<insertfieldname".

When a form field was not entered there is no div; simply an unidentified paragraph with text describing the missing field.

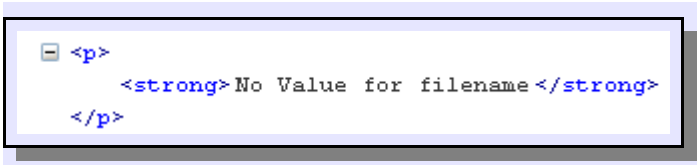


Figure 19.8 : No Value for filename Paragraph

I will check for the presence of the div for "_comments" and for the absence of the div for "_filename" with the following code:

```
assertTrue(selenium.isElementPresent("_comments"));
assertFalse(selenium.isElementPresent("_filename"));
```

getXPathCount – count the checkboxes entered

XPath is an incredibly powerful mechanism for querying the HTML page returned. A basic understanding of XPath will go a long way in helping you construct robust automated tests.

XPath can be used in locators by prefixing a locator with "xpath:" but we can use XPath statements directly in the getXPathCount method.

```
getXPathCount(string xpath)
```

No need to prefix the string passed into getXPathCount with "xpath:", if you do then getXPathCount will fail.

e.g. Because I know that all the checkbox values are rendered as list items within an unordered list within the "_checkboxes" div I can use the following XPath statement to select them all from the HTML document: `"//div[@id='_checkboxes']/ul/li"`

If you are not yet confident enough with XPath to write XPath statements directly into your tests. Then the Firefox plugin XPather (which we installed earlier) is a tremendous help.

Use the XPather plugin to help:

- Right Click on the item in the page you want to derive the XPath for, and select "Show in XPather" from the context menu.

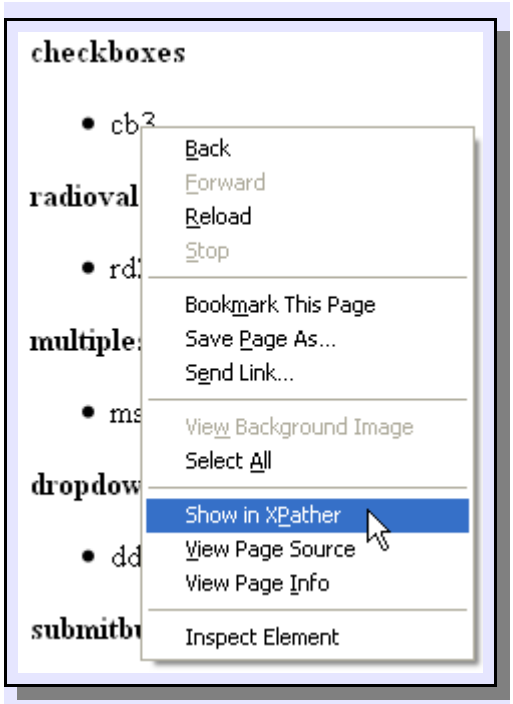


Figure 19.9 : Show in XPather option

- Then use the XPather dialog to refine the XPath Statement.

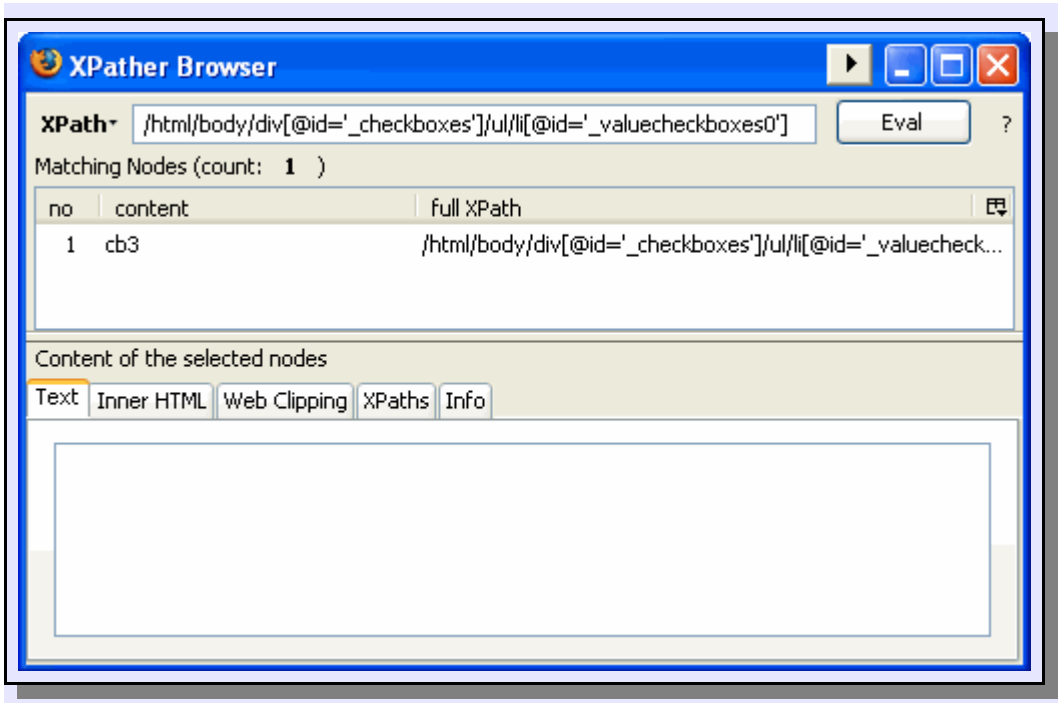


Figure 19.10 : Default XPath statement from XPather

- We want to cut the XPath down to minimal size so, since id's are unique in an HTML page we can make the first item in our XPath the first id we encounter so our XPath becomes:

    - //div[@id='_checkboxes']/ul/li[@id='_valuecheckboxes0']
- Next we want to make it generic so it matches all *li* items by removing the *id* from the *li*
    - //div[@id='_checkboxes']/ul/li
- And now we have an XPath which will match all *li* items in the unordered list within the *div* with an id of "_checkboxes"

Our final code for counting the number of checkboxes selected reads:

```
assertEquals(1,selenium.getXpathCount("//div[@id='_checkboxes']/ul/li").intValue());
```

getXPathCount returns a Number so I convert that into an integer for comparing in the Assert because I know that for our first test, we expect to have 1 checkbox item selected.

click – navigate to the form

HTML pages have hyperlinks to move between pages using the anchor tags in the document.

As users we click on these to navigate to another page. As automators we use the selenium command click.

- click(String locator)

Click takes, as the argument, the locator for the anchor we wish to click on.

By examining the source code using Firebug in Firefox we can see that the link has an id of "back_to_form" so we can use this directly as our locator string.

```
selenium.click("back_to_form");
selenium.waitForPageToLoad("30000");
```

The rest of the test

- For the rest of the test I want to check that the click takes me back to the form, by checking the title of the form page.

```
assertEquals("HTML Form Elements",selenium.getTitle());
```

- Then I will add a filename and check all the checkboxes (remembering that typing into filename will not work on all browsers)

```
selenium.type("filename", "C:\\Selenium\\readme.txt");
selenium.check("//input[@name='checkboxes[]' and @value='cb1']");
selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
selenium.check("//input[@name='checkboxes[]' and @value='cb3']");
```

- Then resubmit the form

```
selenium.click("//input[@name='submitbutton' and @value='submit']");
selenium.waitForPageToLoad("30000");
```

- And check that the filename is present with a value

```
assertTrue(selenium.isElementPresent("_filename"));
assertEquals("readme.txt",selenium.getText("_valuefilename"));
```

- And then check that the count for the checkboxes has increased to 3

```
assertEquals(3,
       selenium.getXpathCount("//div[@id='_checkboxes']/ul/li").intValue());
```

The full test and summary

In the preceding test we used the following commands:

- click
- getTitle
- isTextPresent
- isElementPresent
- getXpathCount
- getText

These represent the core of the commands for working with static HTML so here is the full test

```
@Test
public void submit_form_with_values_and_check_static_html(){
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        // check the title on the page
        assertEquals("Processed Form Details",selenium.getTitle());

        // check the username entered
        assertEquals( "eviltester",selenium.getText("_valueusername"));

        // check the submitted values text is on the page
        assertTrue(selenium.isTextPresent("Submitted Values"));

        //isElementPresent to check comments and filename
        assertTrue(selenium.isElementPresent("_comments"));
        assertFalse(selenium.isElementPresent("_filename"));

        // use getXPathCount to check the number of checkboxes
        assertEquals(1
                ,selenium.getXpathCount("//div[@id='_checkboxes']/ul/li").intValue());

        // click to go back to the form
        selenium.click("back_to_form");
        selenium.waitForPageToLoad("30000");

        // check the page title with getTitle
        assertEquals("HTML Form Elements",selenium.getTitle());

        //add a filename
        selenium.type("filename", "C:\\Selenium\\readme.txt");

        //check all the checkboxes
        selenium.check("//input[@name='checkboxes[]' and @value='cb1']");
        selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
        selenium.check("//input[@name='checkboxes[]' and @value='cb3']");

        // submit the form
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        //check filename is present
        assertTrue(selenium.isElementPresent("_filename"));
        assertEquals("readme.txt",selenium.getText("_valuefilename"));

        //check the checkbox count
        assertEquals(3,
        selenium.getXpathCount("//div[@id='_checkboxes']/ul/li").intValue());
}
```

getAttribute

Just as we can access the text of an element, we can also access the values of any of its attributes.

- getAttribute(String attributeLocator)

getAttribute takes a single argument, the locator to an attribute.

Selenium distinguishes between a 'normal' locator and an 'attribute' locator by having an attribute locator end with @<attribute_name> e.g. @id to get the value of the id attribute.

So were I to get the name of one of the paragraphs under an identified div.

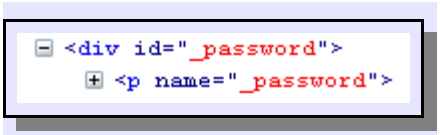


Figure 19.11 : div with an ID

I could use the following attributeLocator to get the value of the name attribute:

```
"xpath=//div[@id='_password']/p@name"
```

And use it in a test like this:

```
@Test
public void check_static_html_password_name(){
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        // check the name of the password display paragraph is _password
        assertEquals("_password",
                selenium.getAttribute("xpath=//div[@id='_password']/p@name"));
}
```

assignId

Because Selenium runs as JavaScript within the browser itself, we can actually amend the HTML in the browser. One command that Selenium provides for this is the assignId command.

- assignId(String locator, String identifier)
    - locator is a String to allow us to locate the element we want to add an id to
    - identifier is the String we want to add as the id on the element

We can use this to help us limit the number of complicated XPath statements we use and to enable us to go directly to specific page elements instead of long XPath statements.

As an example in the static HTML page we have been looking at the filename error message is in an element without an id or a name so we have to use an XPath to access the text. e.g.

```
assertEquals("No Value for filename",
                selenium.getText("xpath=//body/p[2]/strong"));
```

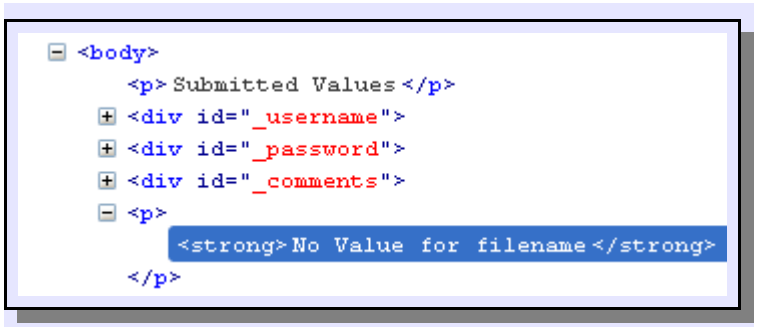


Figure 19.12 : No ID on the filename message

We could instead assign an id to the element and then use that id for any subsequent methods. e.g.

```
@Test
public void useAssignIDToCheckFilenameBlank(){
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        selenium.assignId("xpath=//body/p[2]/strong", "_filename");
        assertEquals("No Value for filename",selenium.getText("_filename"));
}
```

getBodyText & getHtmlSource

We may decide that we want to do a comprehensive set of actions on the static HTML and that doing it through lots of Selenium commands would slow our testing down, therefore we would rather process the HTML directly.

We can access the HTML using two commands:

- **getBodyText** – returns all the text on the page
- **getHtmlSource** – returns the HTML source of the page

Neither takes an argument and just return a string with all the details we require.

e.g. For the following code

```
@Test
public void getBodyText_getHtmlSource(){
        selenium.open("/selenium/basic_html_form.html");
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        System.out.println(selenium.getBodyText());
        System.out.println("-----");
        System.out.println(selenium.getHtmlSource());
}
```

getBodyText() returns all the text for all the elements:

```
Submitted Values
username
eviltester
password
myPassword
comments
Comments...
No Value for filename
hiddenField
Hidden Field Value
```

```
checkboxes
cb3
radioval
rd2
multipleselect
ms4
dropdown
dd3
submitbutton
submit
Go back to the main form
```

getHtmlSource() returns the full HTML of the page

```
<head>

<title>Processed Form Details</title>

</head><body>

<p>Submitted Values</p><div id="_username"><p
name="_username"><strong>username</strong></p><ul><li
id="_valueusername">eviltester</li></ul></div><div id="_password"><p
name="_password"><strong>password</strong></p><ul><li
id="_valuepassword">myPassword</li></ul></div><div id="_comments"><p
name="_comments"><strong>comments</strong></p><ul><li id="_valuecomments">Comments...

...

</body>
```

goBack

In one of the previous examples we used click to return to the form entry page. We could also have used the back button.

Selenium provides a mechanism for simulating this with the goBack command.

- goBack()

goBack takes no arguments

```
@Test
public void submit_form_with_values_and_go_back(){
        // check the title on the form page
        assertEquals("HTML Form Elements",selenium.getTitle());
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        // check the title on the detail page
        assertEquals("Processed Form Details",selenium.getTitle());

        selenium.goBack();

        //check title for the form page again
        assertEquals("HTML Form Elements",selenium.getTitle());
}
```

highlight

Selenium's highlight command briefly highlights an element on the screen to help you debug your tests.

- highlight(String locator)

The single argument to highlight is the locator of the element you want to draw attention to. e.g.

```
selenium.highlight("username");
selenium.type("username","eviltester");
```

The above code would briefly highlight the username input field:

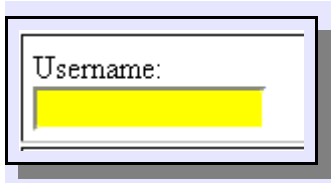


Figure 19.13 : highlighted username

Figure 19.13 shows the field highlighted before typing. And Figure 19.14 after the typing.



Figure 19.14 : not highlighted after typing

The brief flashes on screen may help you follow what Selenium is doing.

Chapter 20: Using JavaScript with Selenium

Selenium has commands for using JavaScript within Selenium.

These commands are a little more advanced than the commands we have used before but they allow us to expand Selenium functionality within our tests without relying on Selenium IDE User Extensions:

- isAlertPresent – returns true if an alert has been generated in JavaScript
- getAlert – returns the text of the last alert generated in JavaScript
- fireEvent – trigger an event on an element
- getEval – run some JavaScript and return the result
- runScript – inject some JavaScript into the page and run it without returning a result

A Basic JavaScript Enabled Page

To provide some context to understand these commands I have created a small page with some JavaScript elements at http://www.compendiumdev.co.uk/selenium/basic_ajax.html.

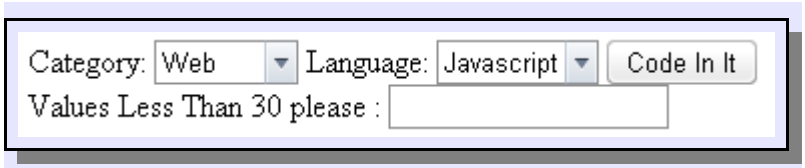


Figure 20.1: JavaScript enabled web page

This has two combo boxes wired together by javascript and a text input box with some javascript validation.

```
<form method="post" action="form_processor.php?ajax=1">
  <label for="combo1">Category:</label>
  <select name="id" id="combo1">
    <option value="1">Web</option>
    <option value="2">Desktop</option>
    <option value="3">Server</option>
  </select>
  <noscript>
    <input type="submit" name="submitbutton" value="Load Individuals" />
  </noscript>
  <label for="combo2">Language:</label>
  <select name="language_id" id="combo2">
    <option value="1">Javascript</option>
    <option value="2">VBScript</option>
    <option value="3">Flash</option>
  </select>

<input type="submit" name="submitbutton" value="Code In It" />
```

```
        </br>
        Values Less Than 30 please : <input type="text" id="lteq30"
        onBlur="checkGT30(this.value)">
    </form>
```

The code on the text input field lteq30 is easy to spot in the code since you can see the call definition in the onBlur attribute.

```
onBlur="checkGT30(this.value)
```

The code on the combo1 select is far less visible as it is added dynamically when the page has loaded.

If a value greater than 30 is entered into the lteq30 field then an error alert is displayed to the user.



Figure 20.2 : Alert Validation

The Class

As an example, I have created a new class "JavaScript_With_Selenium_Tests.java", in the package com.eviltester.seleniumtutorials.chap20, with the following basic setup and tear down code:

```
package com.eviltester.seleniumtutorials.chap20;

import static org.junit.Assert.*;
import org.junit.AfterClass;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;
import com.thoughtworks.selenium.DefaultSelenium;
import com.thoughtworks.selenium.Selenium;

public class JavaScript_With_Selenium_Tests {

        private static Selenium selenium;

        @BeforeClass
        public static void setUpBeforeClass() throws Exception {
                selenium = new DefaultSelenium("localhost", 4444,
                                "*firefox", "http://www.compendiumdev.co.uk");
                selenium.start();
        }

        @AfterClass
        public static void tearDownAfterClass() throws Exception {
                selenium.close();
                selenium.stop();
        }

        @Before
        public void setUp() throws Exception {
                selenium.open("/selenium/basic_ajax.html");
        }
}
```

FireEvent, isAlertPresent, and getAlert

I once had to automate a particularly tricky HTML form. I was using type and typekeys to get data into field values, but the application wasn't functioning as I expected.

After a bit of analysis I investigated the controls on the form and saw that they had events associated with them in javascript. Events on fields take the form onX where X is the name of the event. e.g.

```
Values Less Than 30 please : <input type="text" id="lteq30"
onBlur="checkGT30(this.value)">
```

The above HTML snippet shows an input field with an onBlur event which, when triggered, calls a JavaScript function called *checkGT30* to validate this field.

If I create a test to automate this which types into this field. The onBlur event will not fire. If I try and select one of the other fields on the form, e.g. An item from a combo box, the event will not fire.

You can try it for yourself by visiting
http://www.compendiumdev.co.uk/selenium/basic_ajax.html

Manually type "45" into the input box, then try selecting a value from the first combo box, a dialog pops up with a validation warning.

If you use a Selenium test to do this. Then the dialog box does not appear.

```
@Test
public void enterAnInvalidValueInBlurInput(){
        selenium.type("xpath=//input[@id='lteq30']", "45");
        selenium.select("combo1", "Desktop");
        assertFalse(selenium.isAlertPresent());
}
```

isAlertPresent returns a boolean letting you know if you need to perform a getAlert function.

In the above example, isAlertPresent returns false because no JavaScript alert was created because the validation routine was not called.

In order to trigger the validation routine, we have to trigger the blur event on lteq30 using the **fireEvent** command. As in the following test:

```
@Test
public void enterAnInvalidValueInBlurInputAndTriggerValidation(){
        selenium.type("xpath=//input[@id='lteq30']", "45");
        selenium.fireEvent("xpath=//input[@id='lteq30']", "blur");
        assertTrue(selenium.isAlertPresent());
        assertEquals("Enter a value less than 30", selenium.getAlert());
}
```

Here we know that the alert is generated because the isAlertPresent function returns true and so the **getAlert** function returns the text of the alert.

If you attempt to call getAlert without an alert having been triggered in JavaScript then an exception will be thrown.

Sometimes code of the following form might be used if the alert is irrelevant to our test and all we want to do is clear the alert:

```
if(selenium.isAlertPresent()){
        selenium.getAlert();
}
```

getEval

getEval takes a single String argument which it evaluates as JavaScript in the browser and returns the result of executing that JavaScript.

Complicated uses of this require JavaScript knowledge.

I have used this to get the actual HTML of elements on a page, and often use it in conjunction with assignId e.g.

```
@Test
public void showHTMLOfCombo1WithGetEval(){
        selenium.assignId("xpath=//form", "form1");
        String theInnerHTML =
                selenium.getEval("window.document.getElementById('form1').innerHTML;");
        System.out.println(theInnerHTML);
}
```

The above 'test' would display on the console the HTML contained between the 'form' element tags in the HTML. i.e.

```
      <label for="combo1">Category:</label>
      <select name="id" id="combo1">
        <option value="1">Web</option>
        <option value="2">Desktop</option>
        <option value="3">Server</option>
      </select>
      <noscript>
        <input type="submit" name="submitbutton" value="Load Individuals" />
      </noscript>
      <label for="combo2">Language:</label>
      <select name="language_id" id="combo2">
        <option value="1">Javascript</option>
        <option value="2">VBScript</option>
        <option value="3">Flash</option>
      </select>
    <input name="submitbutton" value="Code In It" type="submit">
       <br>
       Values Less Than 30 please : <input id="lteq30" onblur="checkGT30(this.value)"
type="text">
```

You can use getEval to evaluate any JavaScript that you can run in the browser.

RunScript

Runs a script in the context of the application but does not keep the script.

In the following example I use runScript to create a named JavaScript 'object' (actually a function) called EvilTester with a method called returnBob (which returns the text “bob”). It also creates an Alert.

To demonstrate that the function created in the RunScript can not be used later in the test I try to call it. Since I know that trying to call an undefined function generates an exception, I trap the exception and assert that we got the expected assertion.

```
@Test
public void showValueOfInjectedJavaScriptFromRunScript(){
        // using runScript the function does not persist
        selenium.runScript("function EvilTester(){}; EvilTester.returnBob = function()
{return 'bob';}; alert(EvilTester.returnBob());");
```

```
        // if dialog present then function ran
        assertTrue(selenium.isAlertPresent());

        // should not be able to call again so expect an exception
        assertEquals("bob",selenium.getAlert());
        try{
                selenium.getEval("EvilTester.returnBob();");
        }catch(Exception e){
                assertEquals(e.getMessage(),
                        "ERROR: Threw an exception: EvilTester is not defined");
        }
}
```

I could also have used the JUnit *expected* functionality. e.g.

```
@Test(expected=com.thoughtworks.selenium.SeleniumException.class)
public void showValueOfInjectedJavaScriptFromRunScriptExpectedException(){

        // using runScript the function does not persist
        selenium.runScript("function EvilTester(){}; EvilTester.returnBob = function()
{return 'bob';}; alert(EvilTester.returnBob());");

        assertTrue(selenium.isAlertPresent());
        assertEquals("bob",selenium.getAlert());
        selenium.getEval("EvilTester.returnBob();");
}
```

This tells JUnit to expect the test to throw an exception of type "com.thoughtworks.selenium.SeleniumException.class" and fail the test, if the exception does not get thrown.

AddScript

Runs a script in the context of Selenium and keep it around for later use.

In the following script I use the addScript function to create a function called returnBob which we can use elsewhere in the test.

I use the JavaScript function created by the addScript command in the following getEval command..

```
@Test
public void showValueOfInjectedJavaScriptFromAddScript(){
        // using addScript the function persists for future use

        selenium.addScript(
        "function EvilTester(){}; EvilTester.returnBob = function(){return 'bob';};",
        "evilTester");

        // use the function we created
        assertEquals("bob",selenium.getEval("EvilTester.returnBob();"));

}
```

Chapter 21: Start Selenium Programmatically

The Selenium distribution's selenium-server.jar can also be called programmatically. So that instead of having to type "java -jar selenium-server.jar" at the command line, we can have our tests start up and close down a server when they run.

To do this we use the `startSeleniumServer()` command from the SeleniumServer Class.

Add Selenium Server to the project

You need to add selenium-server.jar to your class path. As before, amend the properties of your project to amend the Java Build Path, and choose to add selenium-server.jar as an external jar.

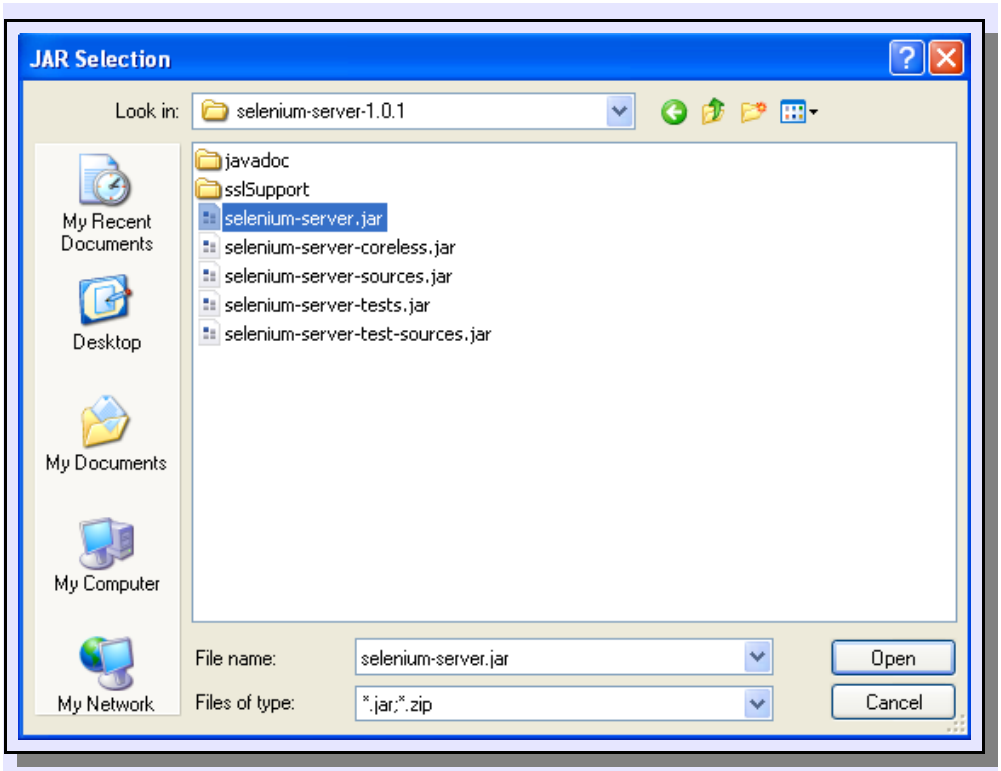


Figure 21.1: Add selenium-server.jar to the project

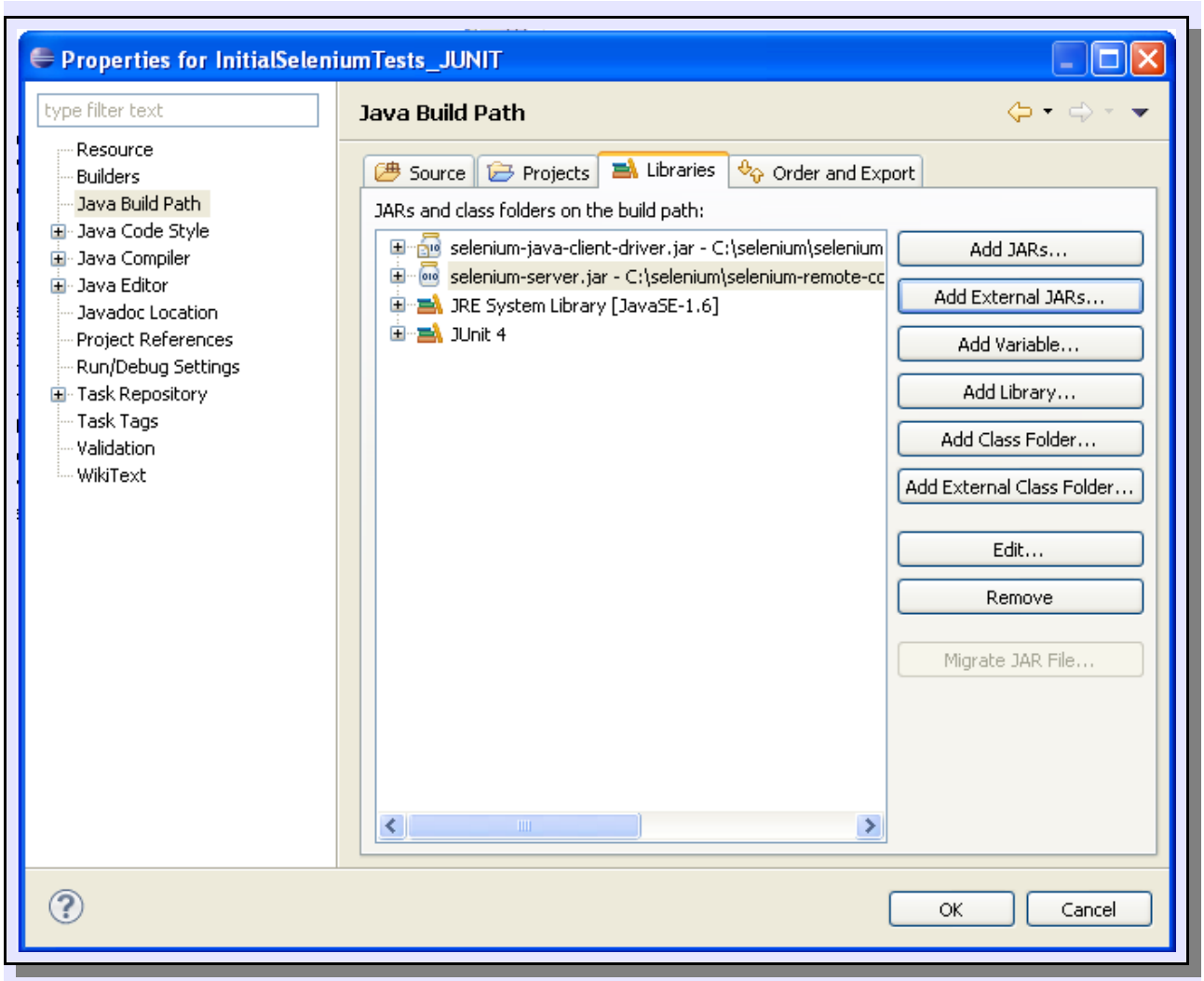


Figure 21.2: Properties Dialog with selenium-server.jar added to build path

Start it in the code

You start Selenium Server by declaring a SeleniumServer, creating a new instance and then starting it. We wrap it with a try catch block because SeleniumServer will throw an exception if it can't start.

```
SeleniumServer seleniumServer=null;
try {
        seleniumServer = new SeleniumServer();
        seleniumServer.start();
} catch (Exception e) {
        e.printStackTrace();
}
```

Then connect to the server in the normal way. Your server has still started on your local machine using the same port.

```
Selenium selenium=null;
selenium = new DefaultSelenium("localhost", 4444,
                "*firefox", "http://www.compendiumdev.co.uk");
selenium.start();
```

Then do your normal test commands.

```
selenium.open("selenium/search.php");
selenium.type("xpath=//input[@name='q']", "Selenium-RC");
```

```
selenium.click("xpath=//input[@name='btnG' and @type='submit']");
selenium.waitForPageToLoad("10000");
```

And close selenium in the normal way.

Then close down the server.

```
if(seleniumServer!=null){
	seleniumServer.stop();
}
```

So a full individual test to do this would look like the following:

```
package com.eviltester.seleniumtutorials;


import org.junit.Test;
import org.openqa.selenium.server.SeleniumServer;
import com.thoughtworks.selenium.DefaultSelenium;
import com.thoughtworks.selenium.Selenium;

public class SeleniumServerInATest {

	@Test
	public void doASearch(){

		SeleniumServer seleniumServer=null;

		try {
			seleniumServer = new SeleniumServer();
			seleniumServer.start();
		} catch (Exception e) {
			e.printStackTrace();
		}

		Selenium selenium=null;
		selenium = new DefaultSelenium("localhost", 4444,
				"*firefox", "http://www.compendiumdev.co.uk");
		selenium.start();

		selenium.open("selenium/search.php");
		selenium.type("xpath=//input[@name='q']", "Selenium-RC");
		selenium.click("xpath=//input[@name='btnG' and @type='submit']");
		selenium.waitForPageToLoad("10000");

		selenium.close();
		selenium.stop();

		if(seleniumServer!=null){
			seleniumServer.stop();
		}
	}
}
```

Obviously we wouldn't start and stop the server in each test so we would refactor this to use the @BeforeClass, @AfterClass annotations to start the server.

The Start Up routine explored

Typically the Selenium Server will not start because something else is already using the Port: e.g. Another instance of Selenium.

BindException Handling

We could amend the code to check if the exception raised was a BindException. If this was the case then we could assume that a selenium server was already running and use the existing Selenium Server. e.g.

```
Try {
```

```
        seleniumServer = new SeleniumServer();
        seleniumServer.start();
} catch (java.net.BindException bE){
        // could not bind, assume that server is currently running
        // and carry on
        System.out.println("could not bind - carrying on");
}
catch (Exception e) {
        // any other exception stop
        throw new IllegalStateException("Can't start selenium server", e);
}
```

Stop Existing Server

We might choose to stop the running server and start a new one.

You would have to decide if this was a valid thing to do in your environment or not. If during the course of your testing you could not start a server then it might mean that your test setup was not closing down the server properly. And rather than 'working around' this, you might find it more beneficial to work out how to stop the server automatically during your previous test run.

But since I think this is a useful technique to know, I include it here.

This will get a little more complicated so if you are new to Java I don't necessarily expect you to understand all of this.

Our main block doesn't change very much:

```
String stopSeleniumCommand =
        "http://localhost:4444/selenium-server/driver/?cmd=shutDownSeleniumServer";
try {
        seleniumServer = new SeleniumServer();
        seleniumServer.start();
} catch (java.net.BindException bE){
        // could not bind, assume that server is currently running
        // and carry on
        System.out.println("could not bind - carrying on");
        // try and stop it
        if( runHTTPCommand(stopSeleniumCommand)){
                try {
                        seleniumServer = new SeleniumServer();
                        seleniumServer.start();
                } catch (Exception e) {
                        throw new IllegalStateException(
                                "Could not stop existing server on blocked port 4444", e);
                }
        }
}
catch (Exception e) {
        // any other exception stop and start
        throw new IllegalStateException("Can't start selenium server", e);
}
```

This time, if we could not bind, we send an HTTP request to the server, and if it is a selenium server on that port, it will stop the server.

If that seemed to work then we try and start the server again.

The runHTTPCommand that does the magic to stop Selenium uses a little more complicated Java.

```
private boolean runHTTPCommand(String theCommand) throws IOException{
        URL url = new URL(theCommand);

        URLConnection seleniumConnection = url.openConnection();
        seleniumConnection.connect();

        InputStream inputStream = seleniumConnection.getInputStream();
        ByteArrayOutputStream outputSteam = new ByteArrayOutputStream();
        byte[] buffer = new byte[2048];
        int streamLength;
```

```
        while ((streamLength = inputStream.read(buffer)) != -1) {
                outputSteam.write(buffer, 0, streamLength);
        }
        inputStream.close();

        // give command some time to finish
        try {
                Thread.sleep(1000);
        } catch (InterruptedException e) {
                e.printStackTrace();
        }

        String stringifiedOutput = outputSteam.toString();
        if (stringifiedOutput.startsWith("OK"))
                return true;

        return false;
}
```

This code, requires the addition of the following imports:

```
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.URL;
import java.net.URLConnection;
```

Basically it:

- creates a connection to run the command.
- issues the HTTP command
- retrieves the output result
- waits for the Selenium server to shutdown fully
- If the output result starts with “OK” then it returns true
- otherwise returns false

Try this code by adding it in a test and running it when a Selenium Server is already running from the command line. You should see the server shut down, and then the test run.

Custom Remote Control Configurations

Because we are now starting Selenium-RC from the command line, we have the option to create custom configurations. At its most simple this might just mean choosing a different port to run the server against, but we can control any element of Selenium-RC that is also configurable from the command line.

To do this we use the RemoteControlConfiguration class which is also contained in Selenium-server.jar:

```
import org.openqa.selenium.server.RemoteControlConfiguration;
```

In the following test I create the server on a different port, using the RemoteControlConfiguration object:

```
@Test
public void remoteControlConfigExample(){

        final int SELENIUM_PORT = 8888;
        final int SELENIUM_TIMEOUT = 45;

        RemoteControlConfiguration rcConfig = new RemoteControlConfiguration();
        rcConfig.setTimeoutInSeconds(SELENIUM_TIMEOUT);
        rcConfig.setPort(SELENIUM_PORT);
```

```
        SeleniumServer seleniumServer=null;
        try {
                seleniumServer = new SeleniumServer(rcConfig);
                seleniumServer.start();
        } catch (Exception e) {
                e.printStackTrace();
        }
        Selenium selenium=null;

        selenium = new DefaultSelenium("localhost", SELENIUM_PORT,
                        "*firefox", "http://www.compendiumdev.co.uk");
        selenium.start();

        selenium.open("selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("10000");

        selenium.close();
        selenium.stop();

        if(seleniumServer!=null){
                seleniumServer.stop();
        }
}
```

Note:

I also have to use the same port when I define the DefaultSelenium server.

I have declared SELENIUM_PORT and SELENIUM_TIMEOUT as final because they are constants that will not change in the test.

Chapter 22: Running Tests Outside Eclipse

Thus far we have run all our tests from within Eclipse.

This has allowed us to get used to writing tests and debugging them, but has not prepared us for a production environment.

In a production environment, a robust test suite will run as part of a continuous integration build and on multiple browser configurations.

To support this we must have the ability to run our tests outside of the Eclipse environment.

Many tools exist to support this way of working and for simplicity I will demonstrate these principles using Ant , Hudson and Subversion.

What is Ant?

Ant is a build tool written in Java. This means that given a configuration file, written in XML, it will carry out a series of actions to compile the java application, run the tests, report the results, and package up the files we need for deployment.

http://ant.apache.org/

Other build systems exist, but Ant is simple enough to get to grips with that it makes a suitable tool for beginners.

What is Hudson?

Hudson is a continuous integration server written in Java. This means that you can setup repeating jobs which will build your software and run your tests regularly.

http://wiki.hudson-ci.org/

Again, other continuous integration systems exist, but Hudson has a very simple install routine and easy to use front end GUI, again making it suitable for beginners.

What is Subversion?

Subversion is a version control system. We will store our code in the version control system and allow Hudson to check out the code as required to run the most recent version of the tests automatically.

http://subversion.tigris.org/

Chapter 23: Using Ant to Run Selenium Tests

Install Ant

To Install Ant we are basically going to follow the instructions listed in the Ant manual

http://ant.apache.org/manual/index.html

1. Download the current version of Ant from the binary downloads page http://ant.apache.org/bindownload.cgi (I downloaded the zipped version, at the time of writing this was "apache-ant-1.8.0-bin.zip")
2. Extract the contents of the archive to a folder on your drive. I unarchived the contents to c:\apache-ant-1.8.0

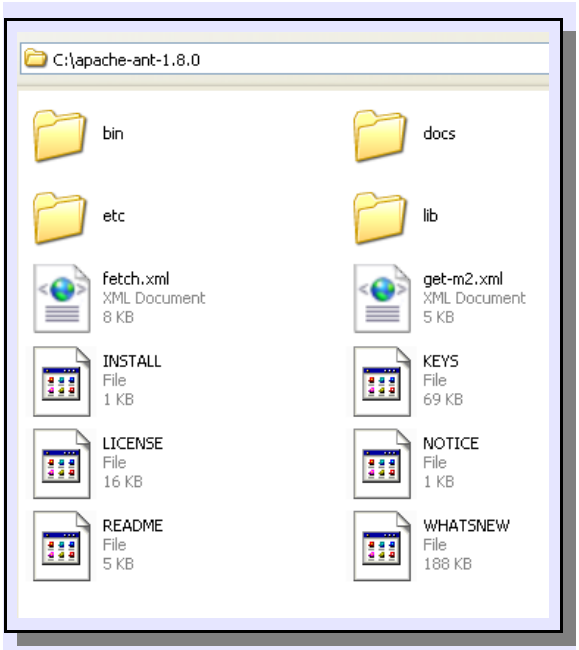


Figure 23.1 : Ant installed to c:\apache-ant-1.8.0

We still have additional configuration to do but first we need to install a JDK version of Java.

Install Java JDK

Because we are now moving into the territory of compiling java code to run outside of Eclipse, we need to install the Java Development Kit version of Java. This is the version that has the java compiler.

1. Visit the Java JDK download page http://java.sun.com/javase/downloads/index.jsp
2. Choose to download the most recent JDK

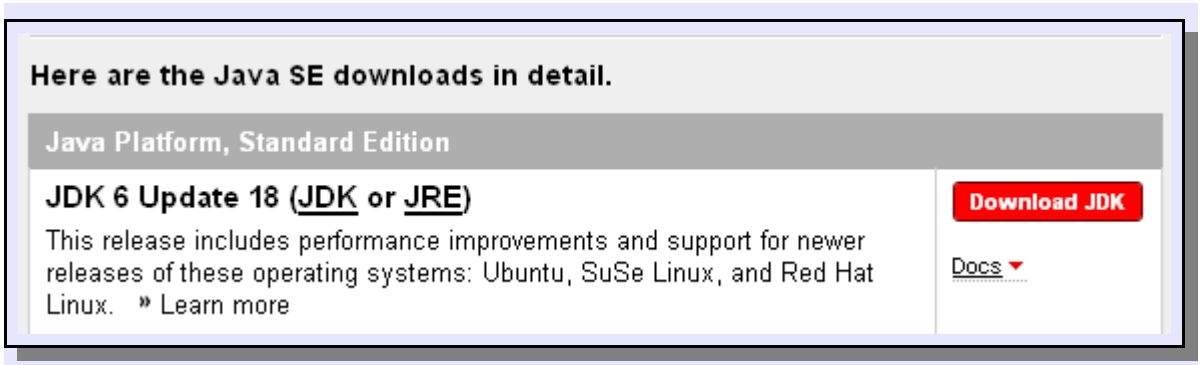


Figure 23.2 : Download the Standard Edition JDK

3. Run the executable that you download to install the JDK. By default it will install to "C:\Program Files\Java\jdk1.6.0_18\". This path should work fine as this "C:\Program Files\Java\" is the default directory for all versions of Java.

Update the Environment Variables and Path

We need to add a few environment variables to our windows configuration. On your "My Computer" icon, right click and select "properties" to access the "System Properties" dialog.

The environment variables are accessible by clicking the [Environment Variables] button on the Advanced tab.

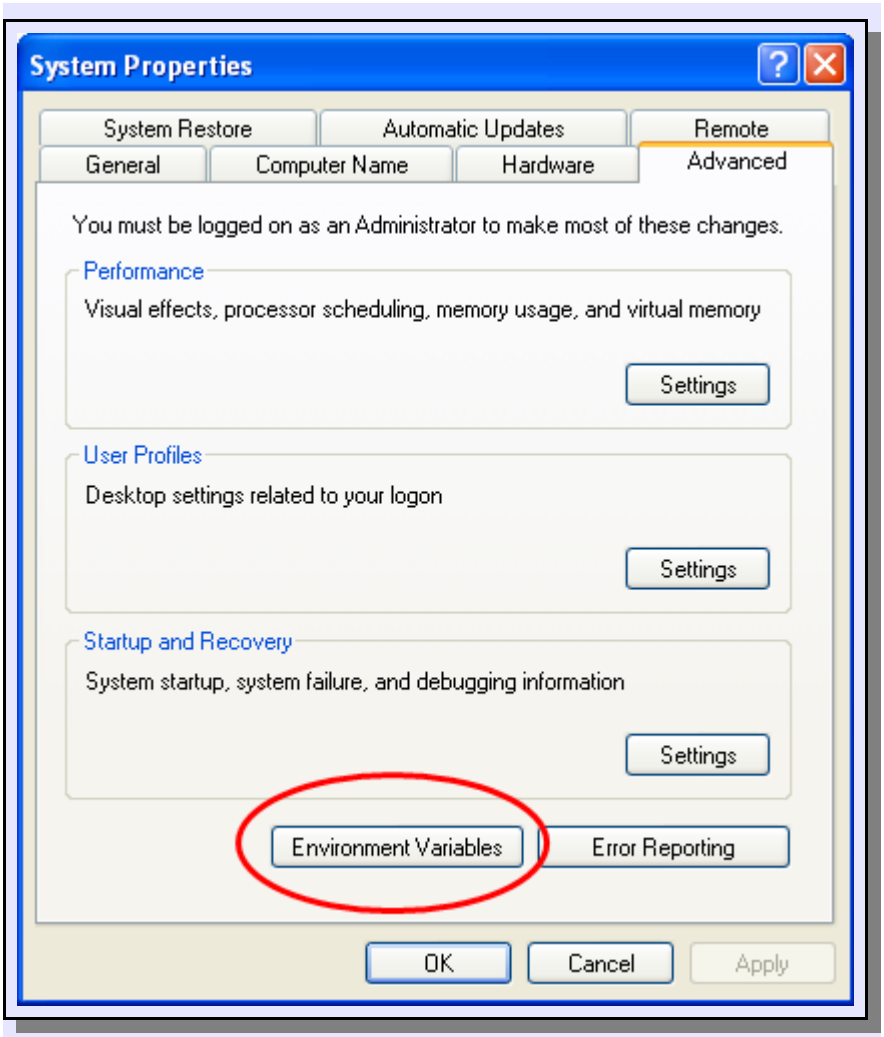


Figure 23.3 : Environment Variables button on System Properties

Click the [Environment Variables] button and use the Environment Variables dialog to add the following new System Variables.

- ANT_HOME
- JAVA_HOME

Click the [New] button on the "Environment Variables" dialog, in the "System variables" pane and enter ANT_HOME with the path that you installed Ant to, for me this was c:\apache-ant-1.8.0

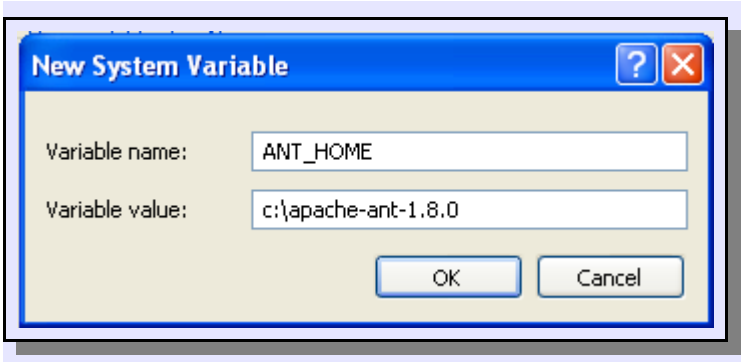


Figure 23.4 : Set ANT_HOME system variable

Follow the same procedure to add a system variable for JAVA_HOME with the install directory of the Java JDK. For me this was "C:\Program Files\Java\jdk1.6.0_18"

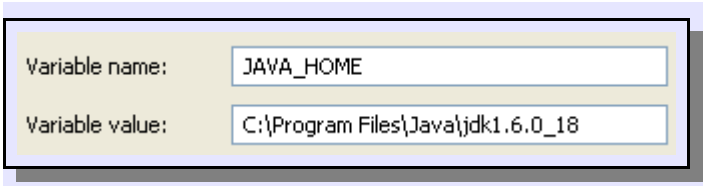


Figure 23.5 : Set JAVA_HOME system variable

We also have to update the Path variable, so edit the existing Path variable (or create one if it does not exist) and append the paths of Apache and java. e.g. In my case, I had to add "c:\apache-ant-1.8.0\bin;C:\Program Files\Java\jdk1.6.0_18\bin;" Path entries are separated by ";".

Check that Ant works

If you create a command line now and type the command "ant".You should see the message

```
Buildfile: build.xml does not exist!
Build failed
```

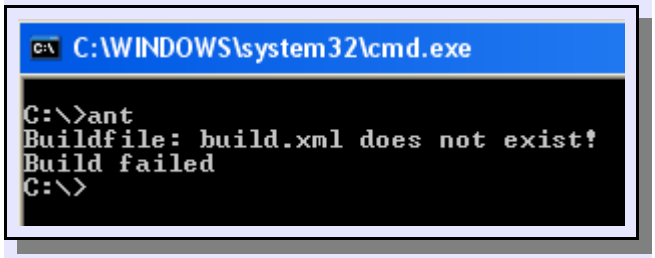


Figure 23.6 : Ant installed correctly

While this may look as though Ant has failed to install correctly. It means that Ant has been found by the command line, has run, and is reporting the message that it could not find a build.xml file to run. That is what we expect, and we will create a build file for it in the next section.

Check that Java works

If you do the same thing for java, and type the command "java -version". You should see a version string matching the JDK that you installed.

```
java version "1.6.0_18"
Java(TM) SE Runtime Environment (build 1.6.0_18-b07)
Java HotSpot(TM) Client VM (build 16.0-b13, mixed mode, sharing)
```

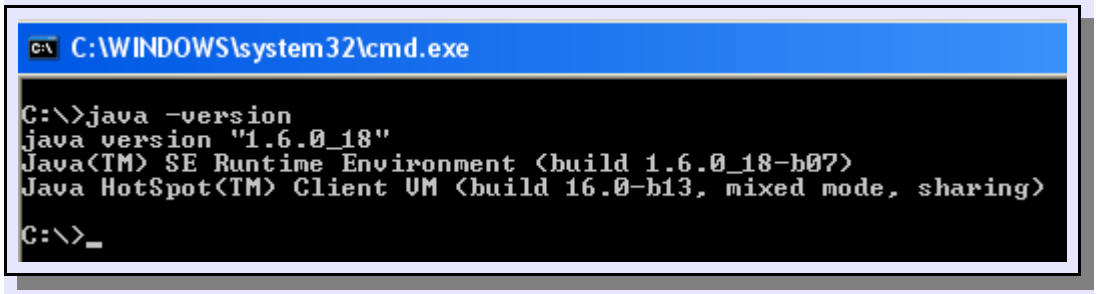


Figure 23.7 : JDK version installed

Create a Lib Folder

Since we are going to be running under the command line. We are no longer going to rely on the Eclipse properties configuration to locate the library files that we need.

We will create a library folder in our Java project, and this will store the Selenium libraries and JUnit libraries. Then we will reference these libraries from Ant.

By moving the libraries into a folder associated with the project source code, it will also make it easier to integrate Ant with Hudson, and Hudson with Subversion.

In your source project folder, create a directory called lib.

For me this required creating "c:\selenium_tests\InitialSeleniumTests\lib"

Into this lib folder we will copy, from the Selenium installation:

- selenium-java-client-driver.jar
- selenium-server.jar

We also have to download and install JUnit because we want to use a .jar version from the lib directory rather than the version bundled with Eclipse.

To do this:

1. visit http://junit.org
2. follow the link to "Download JUnit"
3. download the most recent JUnit jar file to your lib directory.

Amend Eclipse Project Properties

We will now amend our Eclipse project properties to use these library files, so that when we develop within Eclipse, we use the same library files as Ant.

Amend the Java Build Path to use the libraries in the lib folder by removing the existing JUnit, and Selenium libraries and instead adding the libraries from the lib folder.

You will need to re-associate these with the source code, so you can either include the source jars in the lib folder, or point to the source jars in the folder where you installed Selenium.

Create an initial build.xml file

We have two different sets of tests in the book. Tests which rely on the server being started, in order to run. And tests which start the server themselves. This has made running all the tests from within Eclipse a little difficult and really we need to refactor all the tests to use the same mechanism. Which we will do in a later section.

Initially we can have both tests co-existing in the same build.xml file, with Ant starting and stopping the Selenium server as required.

We will start by creating a build.xml file with the tests which require the Selenium server, and then add the remaining tests in the next section.

Note:

This is stored as build_server_tests.xml in the supporting source code for the book.

You would run this from the command line as "ant -f build_server_tests.xml"

If you create the file in stages by following along with the book, you will be able to run it, just by typing "ant"

A build.xml file is an XML file which consists of:

- configuration elements, e.g. property, path etc.
- target elements – these 'do' something

The target elements allow you to define a set of tasks, with dependencies between them. e.g.

  - <target name="target1"/>
  - <target name="target2" depends="target1"/>
  - <target name="target3" depends="target2"/>
  - <target name="target4"/>
  - <target name="target5" depends="target4"/>

In the above example I could start Ant with:

- "ant target1" - only target1 would execute
- "ant target2" - Ant would execute target1 (because target2 depends on it) and if target1 completed successfully, then Ant would execute target2
- "ant target3" - Ant would execute, target1, target2 and target3
- "ant target4 target2" - Ant would execute target4, target1 and target2
- "ant target5 target3" - Ant would execute target4, target5, target1, target2, target3

So for each target which 'depends' on other targets, Ant has to execute the 'depends on' targets first to ensure they complete correctly before executing the targets which depend on them.

You can also see from the example command lines above that we can execute multiple targets from the command line, and subsets of the targets in the file.

The build.xml file explained:

```
<project name="Run Selenium Tests" default="all_selenium_tests" basedir=".">
```

- We start the Ant build file with a definition of the project. Naming it "Run Selenium Tests". The default parameter is the name of the 'target' that we are going to run.
- When we start Ant from the command line we can just run "ant", which will look for a build.xml file, and if found will use that and will start execution with the target defined as default.

```
<property name="src" location="src" />
<property name="build" location="build" />
<property name="libs" location="lib" />
<property name="testresults" location="junit-results" />
<property name="testreport" location="junit-report" />
```

- Properties are 'variables' that we can refer to later in the Ant file. This helps make the Ant file more maintainable because we can amend locations in a single place. e.g. If I want to move the location of where the JUnit result files get written then I simply change the 'location' of the “testresults” property rather than amending all the directory references in the build file.

```
<path id="junit.class.path">
	<pathelement location="${libs}/junit-4.8.1.jar" />
	<pathelement location="${libs}/selenium-java-client-driver.jar" />
	<pathelement location="${libs}/selenium-server.jar" />
	<pathelement location="${build}" />
</path>
```

- Using the 'path' element I create a 'classpath' which I will use to run junit. So here I define the set of folders and files that I want junit to search through when it runs the tests.
- You can see that we refer to the properties that we set up at the start of the file using the notation ${*propertyname*}
- I am defining the class path to point to the location of the junit and selenium jars stored in the /lib folder. And the compiled jar files which get placed into the /build directory.

```
<target name="clean">
	<delete dir="${build}" />
	<delete dir="${testresults}" />
	<delete dir="${testreport}" />
</target>
```

- The 'clean' target will delete all the folders that are created during an Ant build. This uses the properties we setup earlier.
- We could use this build file to delete the /build, /junit-results and /junit-report folders by writing “ant clean” at the command line.

```
<target name="makedir">
	<mkdir dir="${build}" />
	<mkdir dir="${testresults}" />
	<mkdir dir="${testreport}" />
</target>
```

- The 'makedir' target will create all the folders that are required for our Ant build.
- We could use this build file to make the /build, /junit-results and /junit-report folders by issuing “ant make” from the command line.

```
<target name="compile" depends="clean, makedir">
	<javac
		srcdir="${src}" destdir="${build}"
		debug="true" includeAntRuntime="false">
		<classpath refid="junit.class.path" />
	</javac>
</target>
```

- The 'compile' target will create all the folders that are required for our Ant build.
- This target 'depends' on clean and makedir so we know that before Ant compiles our code, it will delete the ${build}, ${testresults} and ${testreport} folder, then create them.
- This target uses an Ant command called javac, this compiles the java code that we have written (all our tests) and writes the jar file into the ${build} folder.
- You can see what this command does by issuing an “ant compile” command and looking in the ${build} directory “/build”. You should see a folder structure that mirrors our package structure, and within the folders, .class files instead of .java files. The .class files are compiled versions of our source code.

```
<target name="run_junit_tests_requiring_server_started">
	<antcall target="start-selenium-server"/>
```

```
        <junit printsummary="yes" fork="yes" errorproperty="junit.error"
failureproperty="junit.error">
                <classpath refid="junit.class.path" />
                <formatter type="xml" />
                <test todir="${testresults}"
name="com.eviltester.seleniumtutorials.MyFirstSeleniumTests" />
                <test todir="${testresults}"
name="com.eviltester.seleniumtutorials.MySecondSeleniumTests" />
                <test todir="${testresults}"
name="com.eviltester.seleniumtutorials.chap18.HTML_form_tests" />
                <test todir="${testresults}"
name="com.eviltester.seleniumtutorials.chap19.Static_HTML_tests" />
                <test todir="${testresults}"
name="com.eviltester.seleniumtutorials.chap20.JavaScript_With_Selenium_Tests" />
        </junit>
        <antcall target="report-junit"/>
        <antcall target="stop-selenium-server"/>
        <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
</target>
```

- The 'run_junit_tests_requiring_server_started' is the target that does the most work for us.
- You can see that it does not have a 'depends' statement, so it could be called from the command line, but we actually call it from within one of the other targets.
- Working through the target you can see an <antcall target="start-selenium-server"/>. The antcall element allows us to treat Ant targets as though they were functions or subroutines in a program, effectively saying 'run the target start-selenium-server'.
- We then use the JUnit Ant command. This runs the JUnit tests which we just compiled and writes the test results to XML files.
    - This sets up a property "junit.error" which traps the JUnit values "errorproperty" and "failureproperty", this causes JUnit to run all the tests, and if there is a failure it will set one of these properties and we can then use that to report a failure. If we didn't do this we would have to use the haltonerror attribute, but this would stop the build on the first test failure – this might be desirable with unit tests, but with the functional tests we run, we want as many of those to run as we can.
    - The *printsummary* attribute is currently set to "yes", which means that we get a one line summary for each test case. Another useful value is "withOutAndErr" which shows all the output that we might have written to System.out and System.err
    - We set the classpath for JUnit to use so that it knows where to find our tests and the libraries that they use.
    - Then the sequence of <test> elements, defines the classes we want to run as tests and the directory to write the XML results to
    - We call the "report-junit" target – this converts the output XML files to an HTML report
    - We call the "stop-selenium-server" target which issues the http command to stop the selenium server.
    - We then have a <fail> element, which causes the build to fail, if the "junit.error" property was set when running the JUnit tests.

```
    <target name="all_selenium_tests" depends="compile" description="The Main Target
for running all tests">
        <antcall target="run_junit_tests_requiring_server_started"/>
    </target>
```

- The 'all_selenium_tests' target will ensure that the "compile" step is done.
- Then it runs the JUnit tests.
- This is the default target triggered when calling "ant".

```
    <target name="report-junit">
        <junitreport todir="${testreport}">
                <fileset dir="${testresults}">
                        <include name="TEST-*.xml"/>
```

```
                <fileset>
                <report format="frames" todir="${testreport}"/>
            </junitreport>
        </target>
```

- The 'report-junit' target uses the junitreport task element which takes all the files in the ${testresults} directory which match the naming convention “TEST-*.xml” and converts them into an HTML report which is written to the ${testreport} directory.

```
    <target name="start-selenium-server">
            <java jar="${libs}/selenium-server.jar" fork="true" spawn="true" />
        <waitfor maxwait="2" maxwaitunit="minute" checkevery="100">
            <http url="http://localhost:4444/selenium-server/driver/?
cmd=testComplete"/>
        </waitfor>
    </target>
```

- The 'start-selenium-server' target could be initiated from the command line with “ant start-selenium-server” command.
- In this build file we call it using an *antcall* during the run_junit_tests_requiring_server_started target.
- This makes a java call to run selenium-server.jar located in the ${libs} directory
- Then uses the Waitfor task (http://ant.apache.org/manual/CoreTasks/waitfor.html) to wait for 2 minutes, checking every 100 milliseconds if the http task (http://ant.apache.org/manual/CoreTasks/conditions.html) has returned a 200 status
    - If at the end of 2 minutes Selenium had not started, then this task would fail and the build would fail

```
    <target name="stop-selenium-server">
        <get taskname="selenium-shutdown"
            src="http://localhost:4444/selenium-server/driver/?
cmd=shutDownSeleniumServer"
            dest="${testresults}\selenium-shutdown-output.txt" ignoreerrors="true" />
    </target>
```

- The 'stop-selenium-server' target could be initiated from the command line with “ant stop-selenium-server”.
- This uses the 'get' task to call the URL which stops selenium, and write the results of the call to an output file in the ${testresults} folder

```
</project>
```

- Finally we make the XML well formed by closing the <project> element

Running the Ant file

This gives us an Ant file which, when run, will start up the selenium server, run the tests which require the server to be running, report the results and stop the server.

I run this by starting a command line in the source directory and issuing “ant” as a command.

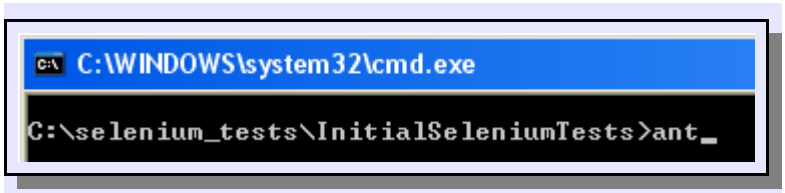


Figure 23.8 : Issue ant from command line

The output from running this will show you the Ant targets run, and the error and success messages generated.
e.g.

```
Buildfile: C:\selenium_tests\InitialSeleniumTests\build.xml

clean:
   [delete] Deleting directory C:\selenium_tests\InitialSeleniumTests\build
   [delete] Deleting directory C:\selenium_tests\InitialSeleniumTests\junit-
results
   [delete] Deleting directory C:\selenium_tests\InitialSeleniumTests\junit-
report

makedir:
    [mkdir] Created dir: C:\selenium_tests\InitialSeleniumTests\build
    [mkdir] Created dir: C:\selenium_tests\InitialSeleniumTests\junit-results
    [mkdir] Created dir: C:\selenium_tests\InitialSeleniumTests\junit-report

compile:
    [javac] Compiling 18 source files to
C:\selenium_tests\InitialSeleniumTests\build

all_selenium_tests:

run_junit_tests_requiring_server_started:

start-selenium-server:
    [junit] WARNING: multiple versions of ant detected in path for junit
    [junit]          jar:file:/C:/apache-ant-
1.8.0/lib/ant.jar!/org/apache/tools/ant/Project.class
    [junit]      and
jar:file:/C:/selenium_tests/InitialSeleniumTests/lib/selenium-
server.jar!/org/apache/tools/ant/Project.class
    [junit] Running com.eviltester.seleniumtutorials.MyFirstSeleniumTests
    [junit] Tests run: 2, Failures: 0, Errors: 0, Time elapsed: 23.625 sec
    [junit] Running com.eviltester.seleniumtutorials.MySecondSeleniumTests
    [junit] Tests run: 10, Failures: 0, Errors: 0, Time elapsed: 161.297 sec
    [junit] Running com.eviltester.seleniumtutorials.chap18.HTML_form_tests
    [junit] Tests run: 14, Failures: 0, Errors: 0, Time elapsed: 43.203 sec
    [junit] Running com.eviltester.seleniumtutorials.chap19.Static_HTML_tests
    [junit] Tests run: 6, Failures: 0, Errors: 0, Time elapsed: 31.125 sec
    [junit] Running
com.eviltester.seleniumtutorials.chap20.JavaScript_With_Selenium_Tests
    [junit] Tests run: 6, Failures: 0, Errors: 0, Time elapsed: 86.219 sec

report-junit:
[junitreport] Processing C:\selenium_tests\InitialSeleniumTests\junit-
report\TESTS-TestSuites.xml to C:\DOCUME~1\Alan\LOCALS~1\Temp\null562923061
[junitreport] Loading stylesheet jar:file:/C:/apache-ant-1.8.0/lib/ant-
junit.jar!/org/apache/tools/ant/taskdefs/optional/junit/xsl/junit-frames.xsl
[junitreport] Transform time: 10172ms
[junitreport] Deleting: C:\DOCUME~1\Alan\LOCALS~1\Temp\null562923061
```

```
stop-selenium-server:
[selenium-shutdown] Getting: http://localhost:4444/selenium-server/driver/?
cmd=shutDownSeleniumServer
[selenium-shutdown] To: C:\selenium_tests\InitialSeleniumTests\junit-
results\selenium-shutdown-output.txt

BUILD SUCCESSFUL
Total time: 6 minutes 13 seconds
```

From the above output you can see the following sequence:

- clean task ran and deleted the directories
- makedir task ran and created the directories
- the compile step compiled 18 source code files
- the all_selenium_tests step executed which started selenium and ran the tests
    - Note also the warning about duplicate Ant. It seems as though Selenium-server has the ant.jar bundled within it. This should not give us any issues.
- the JUnit report was created
- the selenium server was stopped
- the build was reported as successful

After running this build file, the junit-report directory contains a report of the tests run:

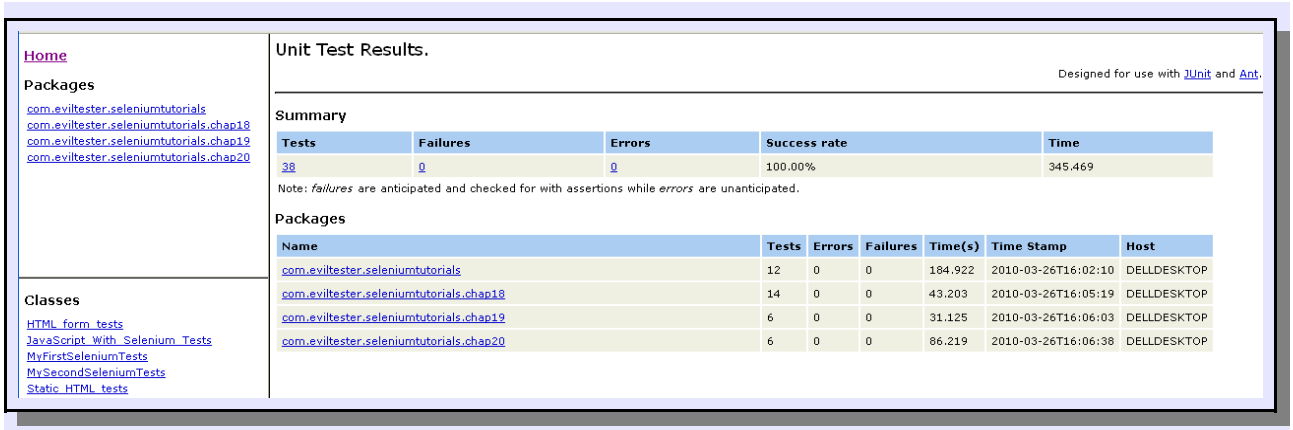


Figure 23.9 : The JUnit HTML report

When we move the tests over to Hudson, the JUnit reports will be collated and stored with their build so that you have a historic record of the test runs for each build.

Add the Tests which require no server

Having created a build file which runs the tests requiring a selenium server to start. We now add the additional complexity of tests which require no server.

Given the way we have structured the build file, it is not hard to add this functionality.

In fact we only have to make two changes.

The addition of a new target to run those tests, and a call to run that target.

```
<target name="run_junit_tests_requiring_no_server">
       <junit printsummary="yes" fork="yes" errorproperty="junit.error"
failureproperty="junit.error">
              <classpath refid="junit.class.path" />
              <formatter type="xml" />

              <test todir="${testresults}"
 name="com.eviltester.seleniumtutorials.chap21.Start_Selenium_Server_In_A_Test" />
              <test todir="${testresults}"
 name="com.eviltester.seleniumtutorials.chap15.XpathExamplesAsTests" />
              <test todir="${testresults}"
 name="com.eviltester.seleniumtutorials.chap16.CSSSelectorExamplesAsTests" />
       </junit>

       <antcall target="report-junit"/>

       <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
</target>
```

- This JUnit test target does basically the same as the previous test runner, the only differences being the omission of the start-selenium-server and stop-selenium-server antcalls
- This has led to a repetition of the antcall to report-junit, which means that the first time it is run it collates the results from running the tests with the server, and the second time it collates all the test runs.

```
<target name="all_selenium_tests" depends="compile" description="The Main Target for
running all tests">

       <antcall target="run_junit_tests_requiring_server_started"/>

       <antcall target="run_junit_tests_requiring_no_server"/>

</target>
```

- The all_selenium_tests target now calls both JUnit test runs

Refactor The Build File

I will also refactor the build file. We have created the basics for running our tests in the above build file. I will want to have multiple build files to illustrate different setups and configurations – I could do this in a single build file but it will become complicated to read.

Instead I will take advantage of the fact that the Ant file is XML and use the standard XML Entity Inclusion mechanism to split my build file into components.

I will move the properties, build, clean, makedir, compile and report-junit targets into one file, and the start and stop selenium targets into another and include them as entities in my build file. This will result in a clean build file, and make it easy to see the core of what the build file does.

I will create a file called properties_build_clean_compile_report.xml which has the Ant code we have discussed above.

I will create a second file called start_stop_selenium.xml into which I will put the start-selenium-server and stop-selenium-server targets.

- properties_build_clean_compile_report.xml

```
<!-- Sets variables which can later be used. -->
<!-- The value of a property is accessed via ${} -->
<property name="src" location="src" />
<property name="build" location="build" />
```

```
<property name="libs" location="lib" />
<property name="testresults" location="junit-results" />
<property name="testreport" location="junit-report" />

<!-- Define the classpath which includes the junit.jar and the classes after
compiling-->
<path id="junit.class.path">
  <pathelement location="${libs}/junit-4.8.1.jar" />
  <pathelement location="${libs}/selenium-java-client-driver.jar" />
  <pathelement location="${libs}/selenium-server.jar" />
  <pathelement location="${build}" />
</path>

<!-- Deletes the existing build and result directories-->
<target name="clean">
       <delete dir="${build}" />
       <delete dir="${testresults}" />
       <delete dir="${testreport}" />
</target>

<!-- Creates the  build, and test results directories-->
<target name="makedir">
       <mkdir dir="${build}" />
       <mkdir dir="${testresults}" />
       <mkdir dir="${testreport}" />
</target>

<!-- Compiles the java code -->
<target name="compile" depends="clean, makedir">
  <javac srcdir="${src}" destdir="${build}" debug="true"
    includeAntRuntime="false">
    <classpath refid="junit.class.path" />
  </javac>
</target>

<target name="report-junit">
  <junitreport todir="${testreport}">
       <fileset dir="${testresults}">
            <include name="TEST-*.xml"/>
       </fileset>
       <report format="frames" todir="${testreport}"/>
  </junitreport>
</target>
```

- start_stop_selenium.xml

```
<target name="start-selenium-server">

       <java jar="${libs}/selenium-server.jar"
            fork="true" spawn="true" />
         <waitfor maxwait="2" maxwaitunit="minute"
              checkevery="100">

           <http url="http://localhost:4444/selenium-server/driver/?
cmd=testComplete"/>
         </waitfor>
</target>


<target name="stop-selenium-server">

        <get taskname="selenium-shutdown"

           src="http://localhost:4444/selenium-server/driver/?
cmd=shutDownSeleniumServer"

           dest="${testresults}\selenium-shutdown-output.txt" ignoreerrors="true" />

</target>
```

I then need to turn the build file into a 'proper' XML file with the doctype to allow me to use entity notation. I do this by including the following at the top of the build file:

```
<?xml version="1.0"?>

<!DOCTYPE project [
       <!ENTITY start_stop_selenium SYSTEM "start_stop_selenium.xml">
       <!ENTITY properties_build_clean_compile_report SYSTEM
```

```
                "properties_build_clean_compile_report.xml">
]>
```

I include the XML files by referencing them in the build file by adding

```
&properties_build_clean_compile_report;
```

and

```
&start_stop_selenium;
```

at the point where I want to include the files.

So my build file would become:

```
<?xml version="1.0"?>

<!DOCTYPE project [
       <!ENTITY start_stop_selenium SYSTEM "start_stop_selenium.xml">
       <!ENTITY properties_build_clean_compile_report SYSTEM
                "properties_build_clean_compile_report.xml">
]>

<project name="Run Selenium Tests" default="all_selenium_tests">

   &properties_build_clean_compile_report;

   <target name="run_junit_tests_requiring_server_started">

      <antcall target="start-selenium-server"/>
      <junit printsummary="yes" fork="yes"
             errorproperty="junit.error" failureproperty="junit.error">
         <classpath refid="junit.class.path" />
         <formatter type="xml" />
         <test todir="${testresults}"
               name="com.eviltester.seleniumtutorials.MyFirstSeleniumTests" />
         <test todir="${testresults}"
               name="com.eviltester.seleniumtutorials.MySecondSeleniumTests" />

         <test todir="${testresults}"
               name="com.eviltester.seleniumtutorials.chap18.HTML_form_tests" />
         <test todir="${testresults}"
               name="com.eviltester.seleniumtutorials.chap19.Static_HTML_tests" />
         <test todir="${testresults}"
          name="com.eviltester.seleniumtutorials.chap20.JavaScript_With_Selenium_Tests" />
      </junit>

      <antcall target="report-junit"/>
      <antcall target="stop-selenium-server"/>
      <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
   </target>

   <target name="run_junit_tests_requiring_no_server">
      <junit printsummary="yes" fork="yes" errorproperty="junit.error"
             failureproperty="junit.error">
         <classpath refid="junit.class.path" />
         <formatter type="xml" />

         <test todir="${testresults}"
      name="com.eviltester.seleniumtutorials.chap21.Start_Selenium_Server_In_A_Test" />
         <test todir="${testresults}"
               name="com.eviltester.seleniumtutorials.chap15.XpathExamplesAsTests" />
         <test todir="${testresults}"
           name="com.eviltester.seleniumtutorials.chap16.CSSSelectorExamplesAsTests" />
      </junit>

      <antcall target="report-junit"/>
      <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
   </target>

   <target name="all_selenium_tests" depends="compile"
           description="The Main Target for running all tests">
```

```
        <antcall target="run_junit_tests_requiring_server_started"/>
        <antcall target="run_junit_tests_requiring_no_server"/>
    </target>

    &start_stop_selenium;

</project>
```

XML Entity Includes allow you to reuse parts of your Ant build files or cut back on the complexity of the build file to make it more readable.

> **Note:**
>
> **In the source code download, this build file is build_server_and_without.xml which can be run with "ant -f build_server_and_without.xml"**
>
> **Use "ant" if you have copy and pasted the code into build.xml**

Summary

Now that we have a build.xml file, all we have to do is open a command line and change directory to where the build file resides, then issue the command "ant" and all the tests that we have configured to run in the build.xml file will run.

A human readable report of the test run will be placed in the /test-reports folder.

Additional Reading

> **Note: I found the following references particularly useful while creating this chapter on Ant. You may like to read these as follow on reading to this chapter:**
>
> **– http://ant.apache.org/faq.html#xml-entity-include**
>
> **– http://ant.apache.org/manual/tutorial-HelloWorldWithAnt.html**
>
> **– http://java.sun.com/developer/Books/javaprogramming/ant/**
>
> **– http://en.wikibooks.org/wiki/Apache_Ant**
>
> **– http://www.vogella.de/articles/ApacheAnt/ar01s04.html**
>
> **– http://pub.admc.com/howtos/junit4x/ant-chapt.html**
>
> **– http://wiki.openqa.org/display/SRC/Selenium-RC+and+Continuous+Integration**

You can find details of all the task used in the Ant documentation (http://ant.apache.org/manual/):

- property http://ant.apache.org/manual/CoreTasks/property.html
- antcall http://ant.apache.org/manual/CoreTasks/antcall.html
- delete http://ant.apache.org/manual/CoreTasks/delete.html
- javac http://ant.apache.org/manual/CoreTasks/javac.html
- get http://ant.apache.org/manual/CoreTasks/get.html

- mkdir http://ant.apache.org/manual/CoreTasks/mkdir.html
- JUnit http://ant.apache.org/manual/OptionalTasks/junit.html
- junitreport http://ant.apache.org/manual/OptionalTasks/junitreport.html
- fail http://ant.apache.org/manual/CoreTasks/fail.html
- waitfor http://ant.apache.org/manual/CoreTasks/waitfor.html
- echo http://ant.apache.org/manual/CoreTasks/echo.html

Chapter 24: Using Hudson to Run Ant

Introduction

Hudson is a continuous integration server. This means that it can poll a source-code repository and rebuild an application, running all its tests, whenever the source code changes.

Or, we can configure a test run to start at regular periods throughout the day. This is the approach that we will cover in this chapter.

Install Hudson

Hudson has a particularly simple install process.

1. Visit http://hudson-ci.org/



Figure 24.1 : Visit hudson-ci.org to download hudson.war

2. Download the hudson.war to a directory of your choosing. I used c:\hudson
3. A .war file is simply another java archive so we can run Hudson by typing "java -jar hudson.war" at the command line

```
C:\hudson>java -jar hudson.war
Running from: C:\hudson\hudson.war
[Winstone 2010/03/27 11:39:05] - Beginning extraction from war file
hudson home directory: C:\Documents and Settings\Alan\.hudson
[Winstone 2010/03/27 11:39:12] - Winstone Servlet Engine v0.9.10 running: contro
lPort=disabled
[Winstone 2010/03/27 11:39:12] - HTTP Listener started: port=8080
[Winstone 2010/03/27 11:39:12] - AJP13 Listener started: port=8009
27-Mar-2010 11:39:13 hudson.model.Hudson$4 onAttained
INFO: Started initialization
27-Mar-2010 11:39:14 hudson.model.Hudson$4 onAttained
INFO: Listed all plugins
27-Mar-2010 11:39:14 hudson.model.Hudson$4 onAttained
INFO: Prepared all plugins
27-Mar-2010 11:39:14 hudson.model.Hudson$4 onAttained
INFO: Started all plugins
27-Mar-2010 11:39:28 hudson.model.Hudson$4 onAttained
INFO: Loaded all jobs
27-Mar-2010 11:39:28 hudson.model.Hudson$4 onAttained
INFO: Completed initialization
27-Mar-2010 11:39:28 hudson.TcpSlaveAgentListener <init>
INFO: JNLP slave agent listener started on TCP port 1171
```

Figure 24.2 : Start hudson.war from the command line

4. From the messages displayed in the console window you can see the port that the http listener has started on e.g. Port=8080. So visit "http://localhost:8080" to see the Hudson GUI.



Figure 24.3 : The Hudson GUI

5. Then we want to configure Hudson to use our Ant setup. So click the "Manage Hudson" link from the left hand menu. Then on the "Manage Hudson" screen, click on "Configure System".

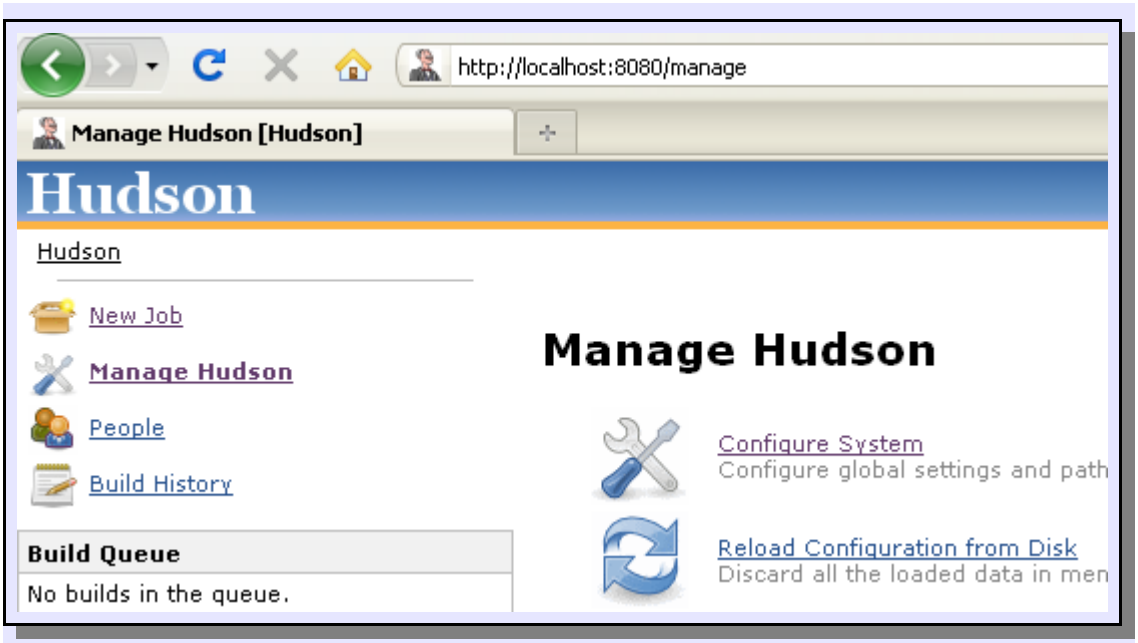


Figure 24.4 : Manage Hudson

6. In the Hudson configuration we need to add an Ant configuration. So click the "Add Ant" button and type in a name e.g. "Ant 1.8" and set the ANT_HOME field to the directory that you have Ant installed in. You also need to check that the Shell executable is set to cmd.exe for Windows.

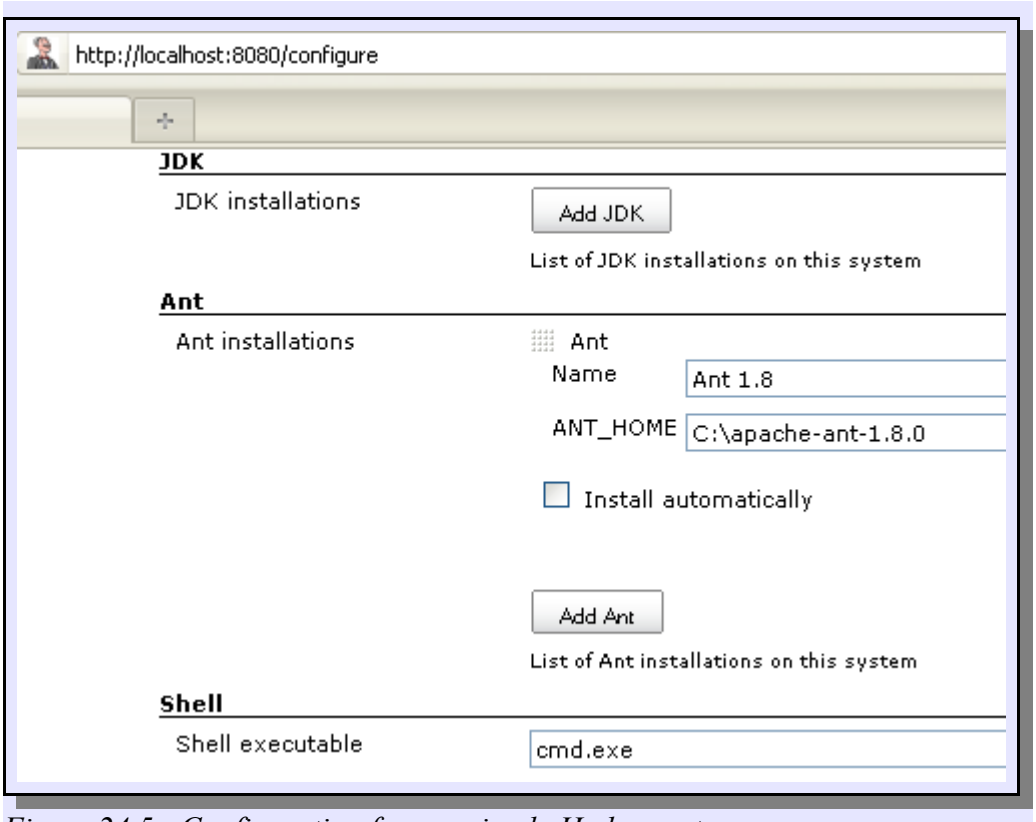


Figure 24.5 : Configuration for our simple Hudson setup

7. Now we want to create a Hudson Job to run our tests automatically. Click on "New Job"

from the Menu, type in a job name and choose the “free-style” software project. And click the “OK” button.

> **Note**
>
> **Hudson uses the term Job and Project interchangeably, and inconsistently, between forms. So when you create a Hudson Job, it is also a Hudson Project.**

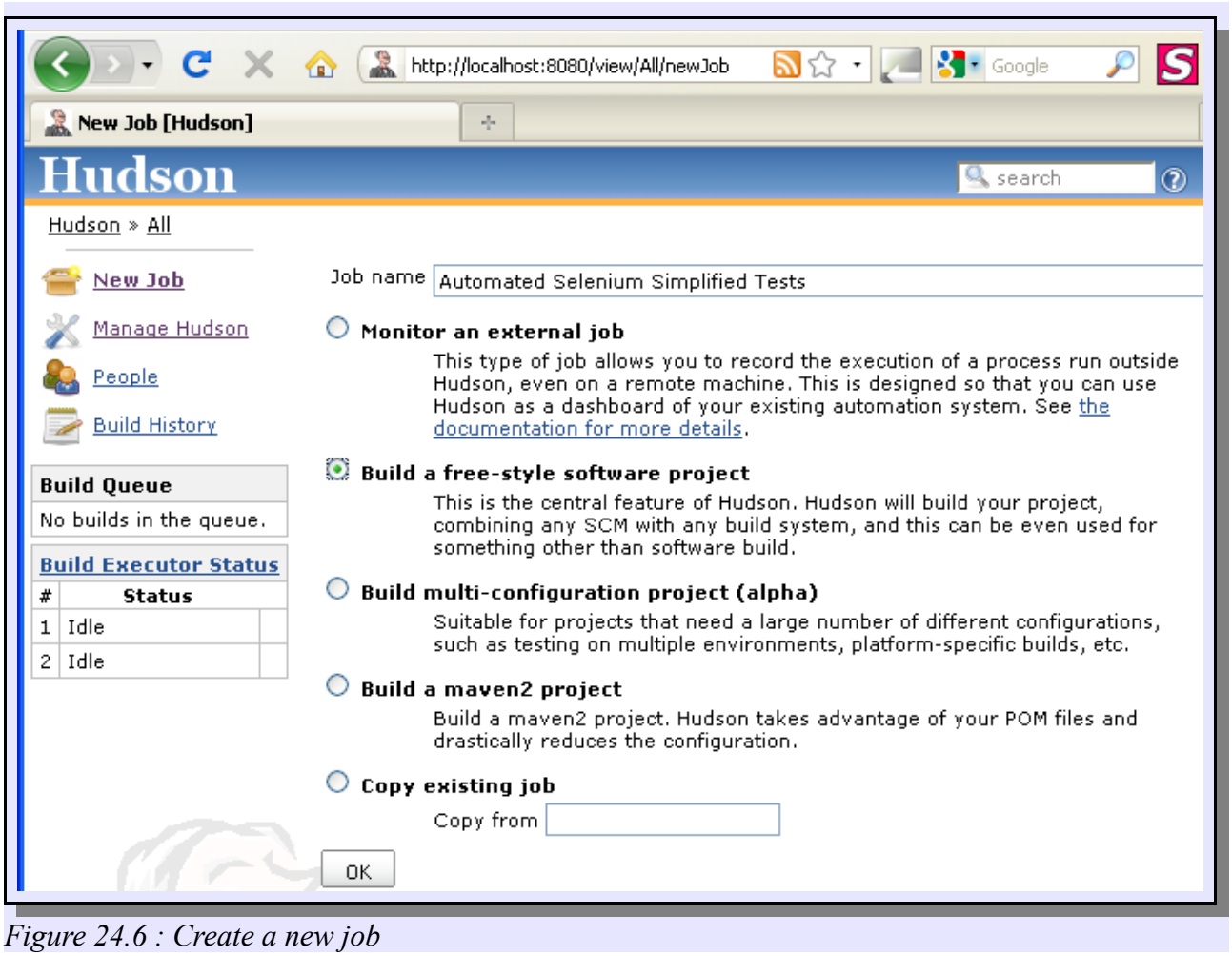


Figure 24.6 : Create a new job

Using Hudson without Version Control

In order to demonstrate the use of Hudson without imposing too many setup issues on you when you are learning the principles. I have chosen not to add the complexity of a version control system in to the examples.

In the next section I will provide an overview of using a version control system with Hudson, but I am not going to cover in this book how to setup a version control system. As that will increase the size of the book and increase the complexity of what we have to cover.

So we need to understand a little bit about how Hudson works and where it stores its files.

Open a file explorer and look in your user id's “Documents and Settings” folder for a “.hudson” folder e.g. I log in to my computer as “Alan”, so I look in “C:\Documents and

Settings\Alan\.hudson"

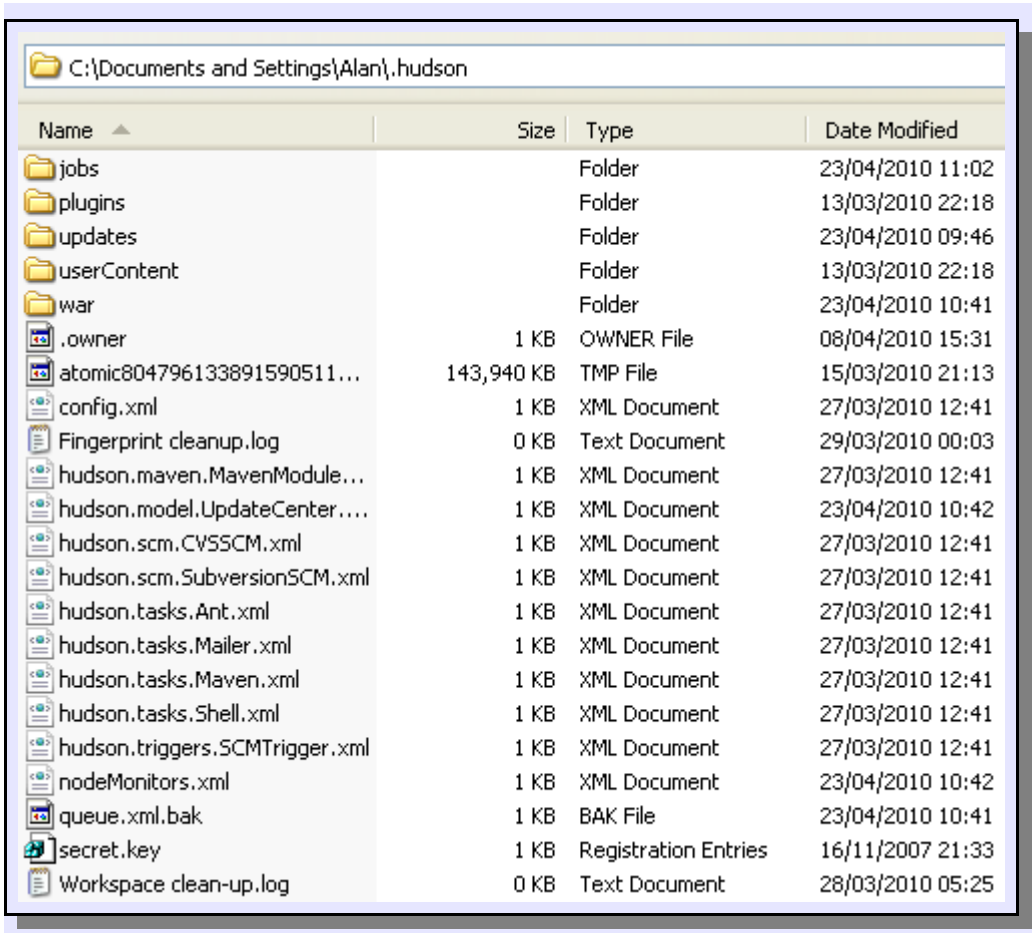


Figure 24.7 : Hudson folder structure

In the jobs folder, Hudson creates a folder for each job that you configure. So we should now have a folder for "Automated Selenium Simplified" Tests.

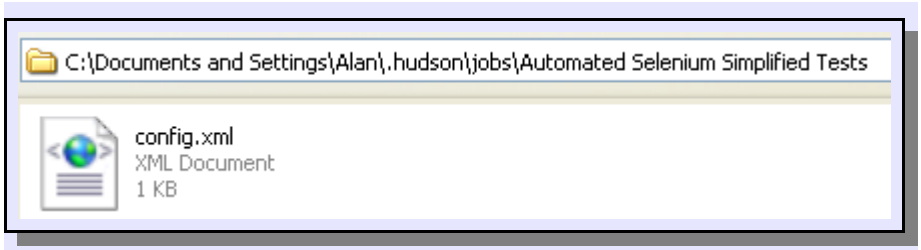


Figure 24.8 : The Job Folder

Normally, we would have Hudson "checkout" our source-code from the version control system into a 'workspace' folder of the Hudson job. Because we have not setup a version control system. We will simulate the 'checkout' process with a .bat file which copies the files from our source-code folder to the Hudson workspace.

Once the files have been 'checked out' Hudson will run the Ant build file to build the project and run the tests.

Then Hudson will take the JUnit test reports from the workspace and copy them into a "builds" folder that stores the details of all or builds to keep a historic record of the test runs.

So I create a nosvn.bat file

```
	rmdir /s /q "C:\Documents and Settings\Alan\.hudson\jobs\automated selenium simplified tests\workspace\"

	mkdir "C:\Documents and Settings\Alan\.hudson\jobs\automated selenium simplified tests\workspace\"

	xcopy "C:\selenium_tests\InitialSeleniumTests" "C:\Documents and Settings\Alan\.hudson\jobs\automated selenium simplified tests\workspace\" /e
```

All this file does is:

1. remove the workspace directory for the "automated selenium simplified tests" project
2. create the workspace directory for the "automated selenium simplified tests" project so we have a blank workspace
3. copy in the files from where I keep my source-code to the workspace we just created

This is exactly what the subversion job would do, but the subversion job, instead of copying files from a directory, would take the most recent versions from the version control repository and copy them to the workspace folder.

Having created your own nosvn.bat file, we can complete the configuration of the Hudson project.

Configure the Hudson Project

We need to add some "Build Steps" to the project.

So, click the "Add Build Step" button, and select an "Execute shell" build step.

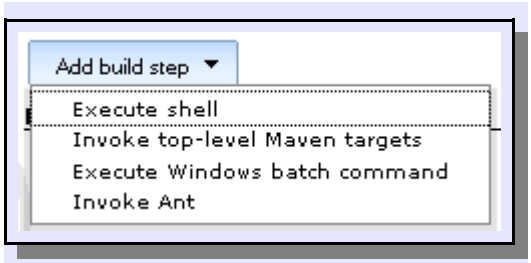


Figure 24.9 : Add Execute Shell Step

We will use this build step to run our "nosvn.bat" file to get all the files in the correct place for Hudson to execute and track the results. The command we want to run is our nosvn.bat file, so I just put the full path for that file in the command field.

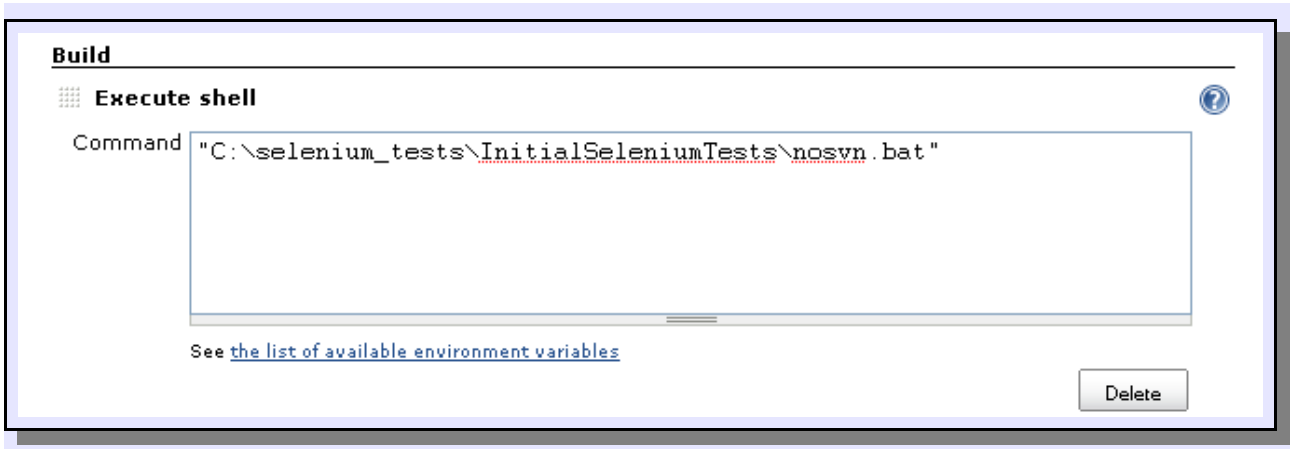


Figure 24.10 : Execute the nosvn.bat file

Next we want to run the Ant command so click "Add build step" and select "Invoke Ant" as the next build step to add.

Here you can leave the invoke Ant command as the default Ant Version as we only have one setup. If we had configured multiple Ant installs on the Hudson configuration we would have to choose the name of the Ant configuration from the drop down.



Figure 24.11 : Use Default "Invoke Ant" configuration

The last thing we have to do before being able to run the test, is to gather the JUnit reports when the tests have finished.

We collate the JUnit reports in the "Post-build Actions" section.

Enable the "Publish JUnit test results report" and for the "Test report XMLs" we use "**/junit-results/*.xml"

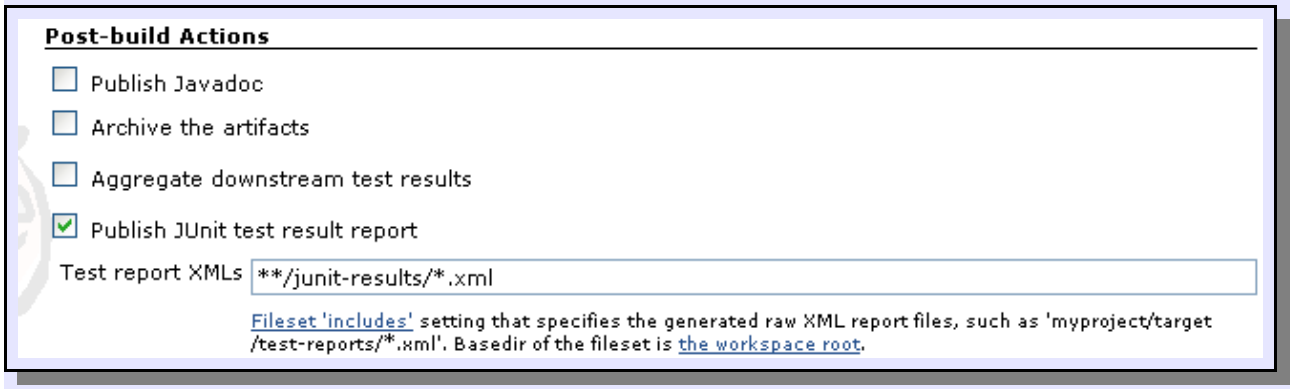


Figure 24.12 : JUnit test result collation configuration

The "**/junit-results/*.xml" means, all the .xml files in the junit-results folder.

This will collate all the XML files and generate the HTML reports and store them in the Hudson builds for future use.

Save the configuration by pressing the [Save] button.

Now you should see the project GUI.

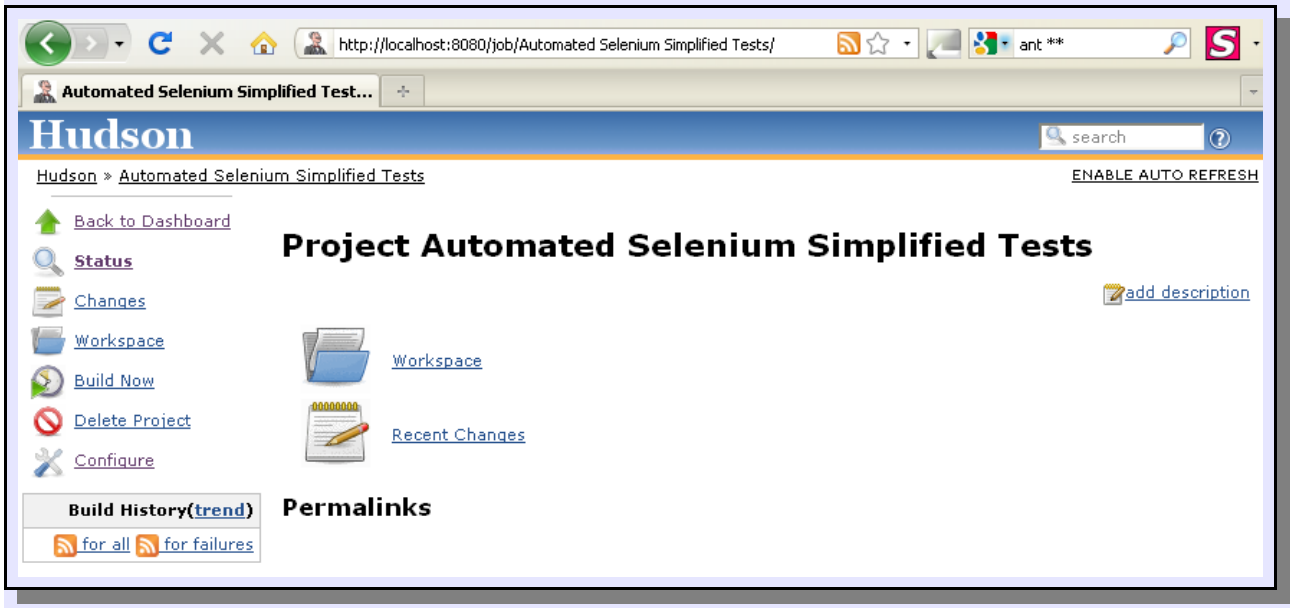


Figure 24.13 : Initial view of the project GUI

Manually start a build by clicking "Build Now" and if you have everything setup correctly. Your tests should start running.

An item will show in the "Build History" with the date and time, follow the link for this build.

You monitor the progress of the build by clicking on the "Console Output", and you see all the console messages written as the tests run.

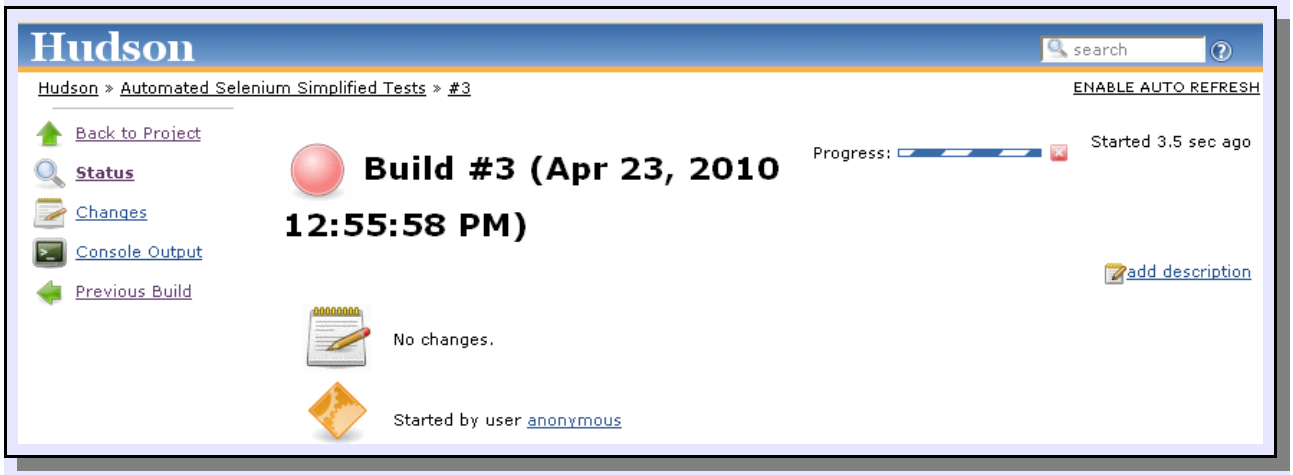


Figure 24.14 : Progress Screen for a build in progress

You should then be able to view the test results by clicking on the "Latest Test Result" link from the project GUI.

Scheduling the build automatically

We can configure it to run automatically. Click on the "Configure" link from the Project GUI and amend the "Build Triggers" section to use the "Build Periodically". Click on the (?) button to the right of the field to see the options that you can add to the schedule.

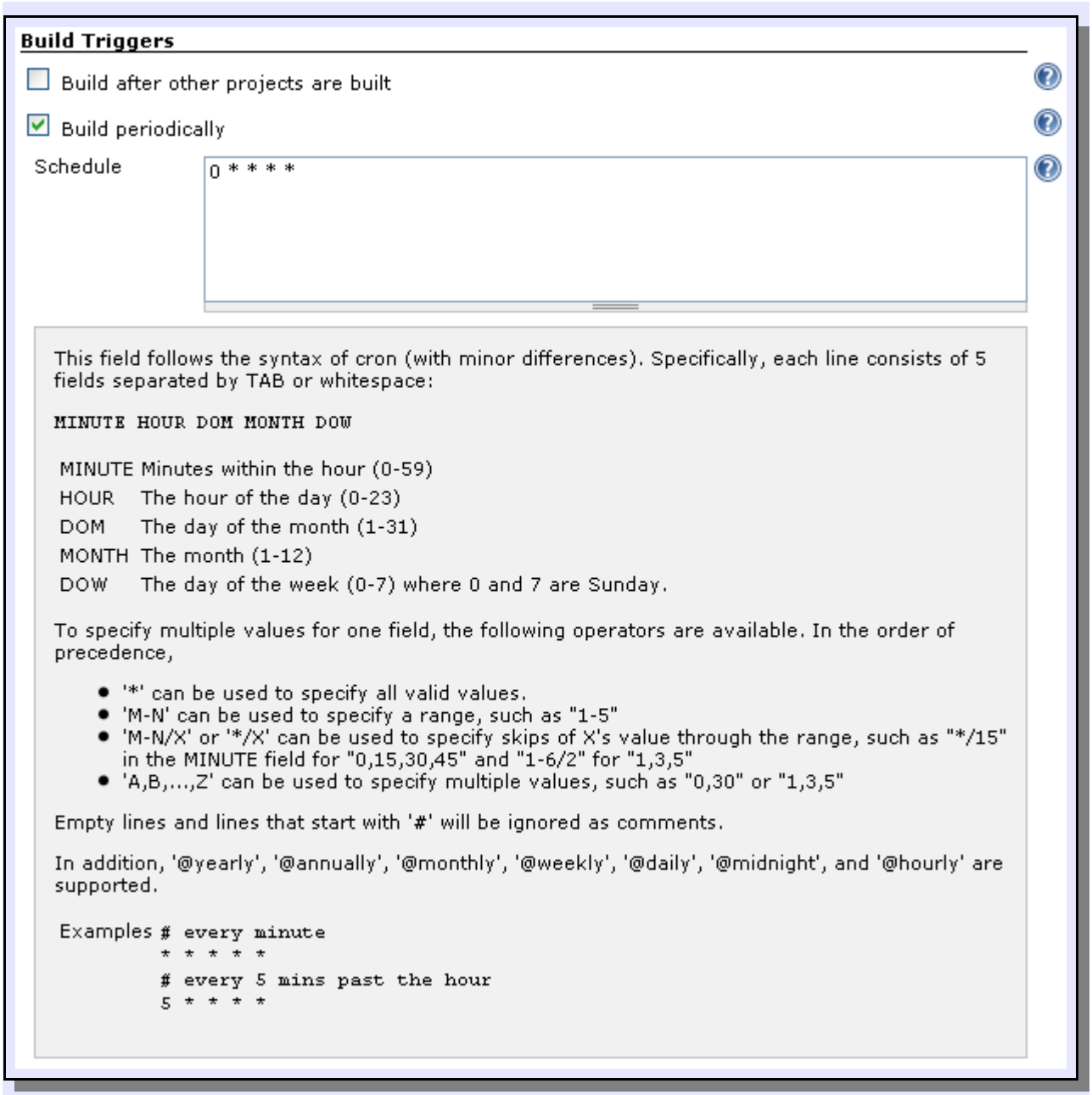


Figure 24.15 : Build Triggers with Help expanded

By entering "0 * * * *" I have scheduled it for the 0th minute of every hour, of every day of every month no matter what day of the week it is.

The full options for the field are explained above, here are a few examples:

- "5 * * * 1-5" - 5 minutes after the hour on every working weekday (Monday – Friday)
- "0 */3 * * *" - 0 minutes after the hour, every 3 hours of every day

Chapter 25: Adding Subversion to the mix

I use Subversion as my version control system.

If you want to install subversion locally then Visual SVN server seems like the easiest option:

- http://visualsvn.com/server/
- http://visualsvn.com/server/getting-started/

You would also want to install Tortoise SVN

- http://tortoisesvn.tigris.org/

TortoiseSVN provides right context menu integration in explorer to access your repositories.

After you have installed subversion, and imported your code into the repository, then check the code out so you can work on it. You can integrate subversion into Hudson by selecting the "subversion" option in the "Source Code Management" section. Adding in the URL to access your subversion repository and the folder you wish to checkout into the workspace.

You will be prompted to enter your authentication credentials.

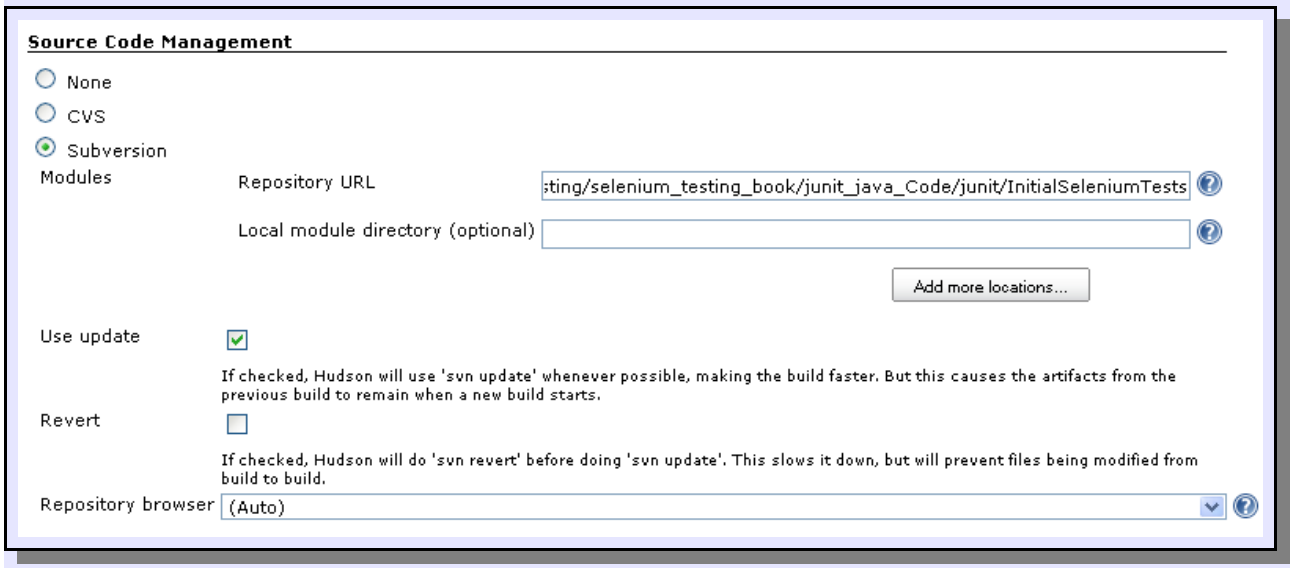


Figure 25.1 : Subversion configuration in Hudson

You would also remove the "Execute Shell" step from the "Build" section.

This should be all you have to do to amend your Hudson setup from the non-version controlled approach to a robust version control setup.

Chapter 26: Take Stock of Where we are

Before we move on, we should take a quick look at what we have managed to achieve, look at the weaknesses we have, and consider what to do next.

We have managed to:

- Write some tests using the IDE and run them from the command line
- Write some tests by converting from IDE tests into Eclipse
- Write tests which require the Selenium Server to be active:
  - com.eviltester.seleniumtutorials.MyFirstSeleniumTests
  - com.eviltester.seleniumtutorials.MySecondSeleniumTests
  - com.eviltester.seleniumtutorials.chap18.HTML_form_tests
  - com.eviltester.seleniumtutorials.chap19.Static_HTML_tests
  - com.eviltester.seleniumtutorials.chap20.JavaScript_With_Selenium_Tests
- Write tests which start the server themselves:
  - com.eviltester.seleniumtutorials.chap21.Start_Selenium_Server_In_A_Test
  - com.eviltester.seleniumtutorials.chap15.XpathExamplesAsTests
  - com.eviltester.seleniumtutorials.chap16.CSSSelectorExamplesAsTests
- Use Ant to run the tests
- Automatically run the tests from Hudson
- Integrating a version control system into Hudson

All of which means that you can start automating your tests.

Now we have to start making your automation "Industrial Strength":

1. Our tests should all use a common way of interacting with Selenium, so they should all start Selenium automatically.
2. We should run our tests across different browsers e.g. IE and Firefox and Safari, and we should do this automatically.
3. We should start using Page Object Models and abstraction layers to make our tests more readable and more robust.

Plan Of Action

We will create a class to manage our connection to a Selenium Server. This will use an existing server if one exists, or start one if it doesn't. This way our tests don't have to keep containing code to manage the server.

We only really care about the tests in a few of the packages since many of our tests were "examples" of Selenium functionality:

- MySecondSeleniumTests

- HTML_form_tests
- Static_HTML_tests
- JavaScript_With_Selenium_Tests

We will refactor these tests after copying them into a new package. That way we can safely refactor and organise without disturbing the originals.

The first thing we will do is have all the tests use the SeleniumManager class.

Then we will start working through each test class and moving all references to selenium into page objects.

Ideally we don't want our tests to use selenium, we want our tests to be written in the language of the application, not the tool. So we want to create 'Page Object Models' instead of writing tests using a 'Tool Object Model'.

Currently we have:

test -> tool

What we want is:

test -> application page object model -> tool

So instead of:

```
assertEquals("this page title",selenium.getTitle());
```

we would want to see:

```
assertEquals("this page title",resultsPage.getTitle());
```

This will make the tests more readable.

And instead of:

```
assertEquals(1,selenium.getXpathCount(
                                "//div[@id='_checkboxes']/ul/li").intValue());
```

we would want to see

```
assertEquals(1,resultsPage.getNumberOfCheckBoxes());
```

This makes the tests more readable and more maintainable. More maintainable because the XPath statements are contained in the page object model classes, rather than in all the tests. So they are localised into a single area. When the application changes, hopefully all we have to do is change the application page object model, and not the tests.

The tests should only have to change when the structure of the application fundamentally changes.

So we will build page object models for each of the pages that we have tested:

- http://compendiumdev.co.uk//selenium/basic_html_form.html
- http://compendiumdev.co.uk/selenium/search.php
- http://compendiumdev.co.uk/selenium/form_processor.php

We don't have to plan what the page object models will look like in advance. They will be supporting the tests, so we will take the approach of writing the test so that it looks the way we want it to. Then use Eclipse to create the objects and methods, and move the existing code from our tests into the objects.

We will be incrementally updating the tests so that at any point in time during the 'refactoring' the tests remain functional and we will not be breaking the tests. If anything, the tests will start to run faster and become more maintainable with minimal effort.

Once we have done this, we will see the inadequacies of the current test set and will be able to easily expand the tests so that they add more value.

We will also add some syntactic sugar to the page object models to increase the readability of the tests:

e.g.

- .and()
- .then()
- by passing back 'this' from the above methods

We will also refactor the page object models themselves to create constants.

Then, we will set the tests to run on multiple browsers.

We will also see which tests can add value by making them data driven to increase the data coverage that can be achieved with a single test path.

Chapter 27: Evolving A Selenium Manager Class

We currently have two ways of starting and connecting to selenium:

- from the command line
- programmatically

Both of these are valid options, and we are using very similar code for all the connection methods.

We will use a few guiding principles from this point forward:

- remove repeating code
- delegate responsibility to specific classes

Following these principles, we will create a single manager class which hides these implementation details for us so that we can have a common set of code to start and connect to the selenium server.

I want the basic code in the tests to look like the following:

```
@BeforeClass
static public void startServer(){
         sm = new SeleniumManager();
         sm.start("http://www.compendiumdev.co.uk");
         selenium = sm.getSelenium();
}

@AfterClass
static public void stopServer(){
        sm.stop();
}
```

This means that before a class of tests I will create a SeleniumManager object, start the server at the domain of my choice, and then 'get' a Selenium session from the manager to use in my tests.

After the tests have finished I just want to issue a single call to the Selenium Manager Object to close the browser, connection and the Selenium Server.

Also I want this to work regardless of whether the Selenium server is already running locally or not.

We will start by moving the source-code that we created in Chapter 21 “Start Selenium Programmatically” into a new class called SeleniumManager.

To do this, let's create a new test class called Selenium_Manager_Tests, in a new package called com.eviltester.seleniumtutorials.refactored so that the code for this looks like the following:

```
package com.eviltester.seleniumtutorials.refactored;

import com.thoughtworks.selenium.Selenium;

public class Selenium_Manager_Tests {

        static SeleniumManager sm;
        static Selenium selenium;

        @BeforeClass
        static public void startServer(){
                sm = new SeleniumManager();
```

```
                sm.start("http://www.compendiumdev.co.uk");
                selenium = sm.getSelenium();
        }


        @AfterClass
        static public void stopServer(){
                sm.stop();
        }
}
```

This will look like it has a lot of errors in Eclipse.

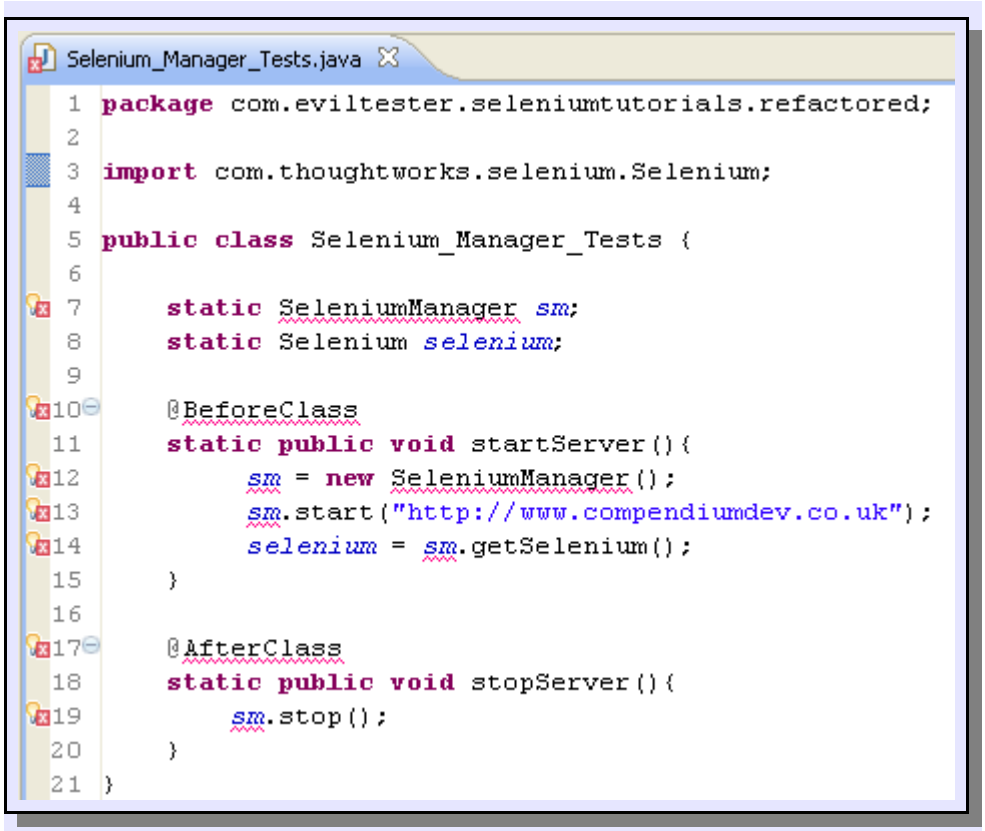


Figure 27.1 : A Test Class using Selenium Manager with initial errors

Let me explain my reasoning for developing the class in this way.

We are about to start refactoring our existing test code into a page object model. This is a set of page object classes which will model our application and act as an abstraction layer which removes the implementation code of how to access a page. The page object model provides an interface which makes tests easier to read and easier to understand. I build the abstraction layer by writing tests which are readable.

I am doing the same here. The SeleniumManager abstracts away the details of how to start selenium and connect to the server. So create the interface by writing tests which are concise and readable.

Also by doing this, we can use the Eclipse automatic error-fixing code to help us write the class.

As an example, our first line with an error:

```
static SeleniumManager sm;
```

By clicking on the error marker on the left border, Eclipse will tell us that “SeleniumManager

cannot be resolved to a type", because we have not created the SeleniumManager class yet.

Selecting the option "Create class 'SeleniumManager'" in the drop down, will cause Eclipse to create the basic class file for us and we can add the additional code we need.

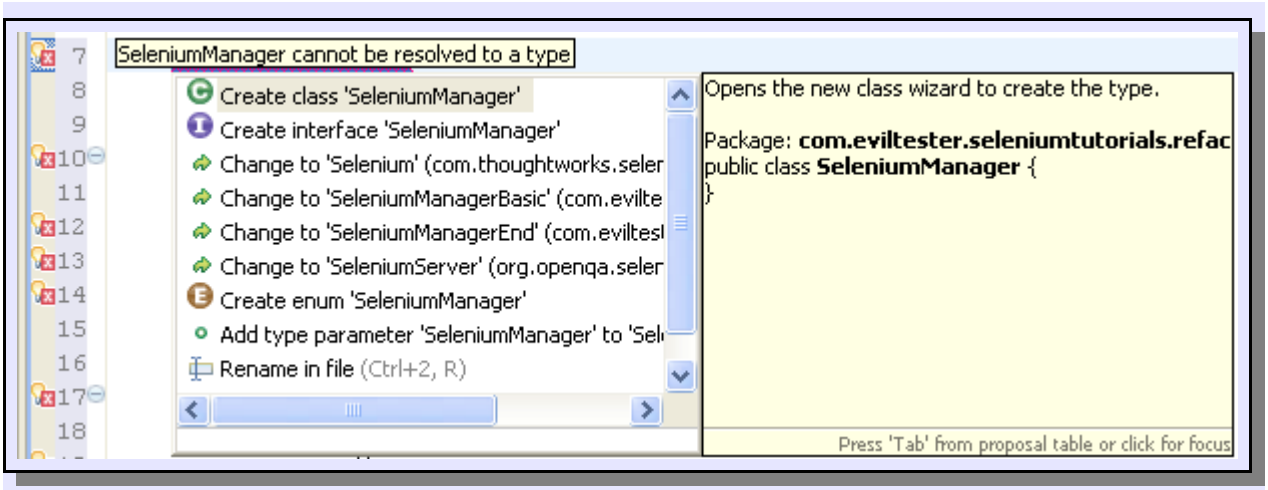


Figure 27.2 : Click on error marker on left to see error and fixing options

This is all functionality you have seen before, but we can use it to very quickly build up the scaffolding code that we need to build our abstraction layers.

Select the option to "Create class 'SeleniumManager'" I added it to a package com.eviltester.seleniumutils to keep the 'helper' classes separate from my page abstractions and tests.

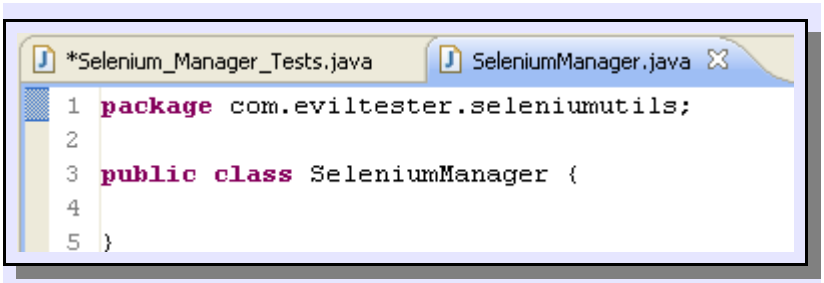


Figure 27.3 : Created Selenium Manager Class

Back to our Selenium_Manager_Tests class:

1. Fix the error on @BeforeClass by selecting the option to "Import 'BeforeClass' (org.junit)" from the drop down menu.
2. Fix the error on @AfterClass by selecting the option to "Import 'AfterClass' (org.junit)" from the drop down menu.

We use the remaining errors to create the public methods that we want the SeleniumManager class to expose to our tests.

Click on the error marker for line "sm.start("http://www.compendiumdev.co.uk");" and choose the option to "Create Method". This will create a basic 'start' method in SeleniumManager which we can amend to implement starting the server from the manager.

Follow this process for all the lines with error in the Selenium_Manager_Tests class.

You will end up with a SeleniumManager class like the following:

```
package com.eviltester.seleniumutils;

import com.thoughtworks.selenium.Selenium;

public class SeleniumManager {

	public void start(String string) {
		// TODO Auto-generated method stub
	}

	public Selenium getSelenium() {
		// TODO Auto-generated method stub
		return null;
	}

	public void stop() {
		// TODO Auto-generated method stub
	}
}
```

Clearly this class doesn't 'do' anything. But it is good enough to remove all the errors in our test class.

So we will add a test into our Selenium_Manager_Tests class and this will force us to write the code to implement the SeleniumManager.

Add the following test into Selenium_Manager_Tests.java

```
@Test
public void openCompendiumDevHomePage(){
	selenium.open("/");
}
```

Remove the error on "@Test" by selecting the option to "Import 'Test' (org.junit)" from the drop down menu.

If you now run this test by right clicking on "Selenium_Manager_Tests.java" in Eclipse and selecting "Run as \ JUnit Test" then the test will run and fail because the selenium variable is null, because we haven't implemented any code in the Selenium Manager.

Fortunately we already have this code in one of our tests, so it will be easy to fill in the template provided by Eclipse.

Looking at the code we wrote when following Chapter 21 in "Start_Selenium_Server_In_A_Test.java" we can copy the code from here into our SeleniumManager class.

We will copy in code from the *checkForBindExceptionAndStopExistingServer* test.

Copy the following code into the SeleniumManager's start() method.

```
SeleniumServer seleniumServer=null;
String stopSeleniumCommand =
	"http://localhost:4444/selenium-server/driver/?cmd=shutDownSeleniumServer";
try {
	seleniumServer = new SeleniumServer();
	seleniumServer.start();
} catch (java.net.BindException bE){
	// could not bind, assume that server is currently running
	// and carry on
	System.out.println("could not bind - carrying on");

	// try and stop it
	if( runHTTPCommand(stopSeleniumCommand)){
		try {
			seleniumServer = new SeleniumServer();
			seleniumServer.start();
```

```
                } catch (Exception e) {
                        throw new IllegalStateException(
                        "Could not stop existing server on blocked port 4444", e);
                }
        }
} catch (Exception e) {
        // any other exception stop and start
        throw new IllegalStateException("Can't start selenium server", e);
}

Selenium selenium=null;
selenium = new DefaultSelenium("localhost", 4444,
                        "*firefox", "http://www.compendiumdev.co.uk");
selenium.start();
```

Eclipse will show an error on the line "`if( runHTTPCommand(stopSeleniumCommand)){`" because we need to also copy in the private method runHTTPCommand. So copy that across too, exactly as it is, so that runHTTPCommand is a private method of SeleniumManager.

There will still be an error on this line because of an unhandled IOException so select the option to "Add throws declaration".

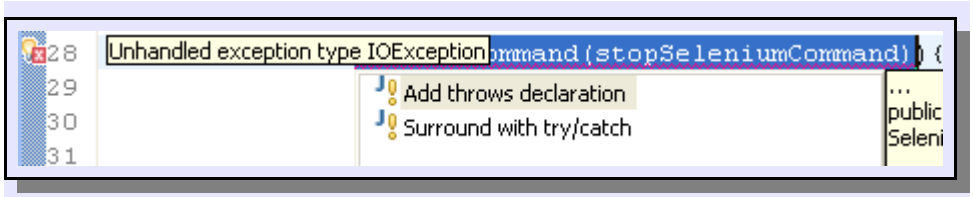


Figure 27.4 : Add throws declaration to start method

There should now be no errors in the SeleniumManager class. But we haven't finished implementing everything we need to have our test pass.

We have passed in a URL to our start method, so we should amend the method definition signature to make it more descriptive. Change:

```
public void start(String string) throws IOException {
```

to

```
public void start(String baseURL) throws IOException {
```

Now, use the baseURL in the start method.

So amend:

```
selenium = new DefaultSelenium("localhost", 4444,
                        "*firefox", "http://www.compendiumdev.co.uk");
```

to

```
selenium = new DefaultSelenium("localhost", 4444,
                                "*firefox", baseURL);
```

Now fix the test we wrote in Selenium_Manager_Tests. To do that we need to return the selenium object which we instantiate with the "new DefaultSelenium..." command in start(), when we call the getSelenium() method.

To do that we have to change the selenium variable from being in scope of the start() method and instead be in scope to the class.

We can do this by moving the line "Selenium selenium=null;" out of the method and up to the top

of the class declaration. When we move this we should also move the SeleniumServer declaration as this will be required in the stop method.

```
public class SeleniumManager {

	SeleniumServer seleniumServer=null;
	Selenium selenium=null;
	...
```

So our getSelenium method can now be amended to read:

```
	public Selenium getSelenium() {
		return selenium;
	}
```

We do have a reminder in our SeleniumManager code in the form of a TODO, even though our Start_Selenium_Server test would pass. So we can write the code for the stop() method now too.

```
	public void stop() {

		if(selenium!=null){
			selenium.close();
			selenium.stop();
		}

		if(seleniumServer!=null){
			seleniumServer.stop();
		}
	}
```

In the stop method we check to see if the selenium object has been set, and if it has, we issue a close, followed by a stop.

Then if the seleniumServer object has been set (meaning that we created it programmatically) then we issue a stop command for the server.

We have one thing to do in our Selenium_Manager_Tests class because the sm.start(...) line is showing an error.

```
15	@BeforeClass
16	static public void startServer(){
17		sm = new SeleniumManager();
18		sm.start("http://www.compendiumdev.co.uk");
19		selenium = sm.getSelenium();
20	}
```

Figure 27.5 : One last error to fix in the test

Use the right-click menu in Eclipse to "Add throws declaration" so that the @BeforeClass looks like the following:

```
@BeforeClass
static public void startServer() throws IOException{
	sm = new SeleniumManager();
	sm.start("http://www.compendiumdev.co.uk");
	selenium = sm.getSelenium();

}
```

And our Start_Selenium_Server test, should now pass and cleanly tidy up Selenium for us.

Perform the following actions and see:

1. Run the test and see that it creates the expected browser instances, the test passes, and the browser instances close

2. Check that selenium server is not running
    - opening http://localhost:4444/selenium-server/driver/?cmd should show an error message. e.g. IE will say “Internet Explorer cannot display the webpage”, Firefox will report “Problem loading page – Unable to connect”
3. Start up a selenium server, “java -jar selenium-server.jar” in the selenium server folder. And check that the server is running
    - opening http://localhost:4444/selenium-server/driver/?cmd in a browser should show the error message “ERROR Server Exception: sessionId should not be null; has this session been started yet?”
4. run the test
5. check that the selenium server has stopped by following step 2.1

Note:

You can find the finished Selenium Manager in the source code distribution as SeleniumManager02FirstEvolution.java in com.eviltester.seleniumutils

We now have an abstraction layer to help remove the complexity of working with the Selenium Server. This will make the tests more consistent and easier to write.

But also:

- If we choose to amend our Selenium Server control behaviour and have it not stop a server if one is already running, we just have to amend SeleniumManager and not our test code.
- When we come to add multiple browsers in our Ant and Hudson builds we can do that by amending the SeleniumManager and not any of our tests.
- If we choose to move to Selenium 2.0 we just have to change SeleniumManager and not any of our tests.

We want our tests to remain stable, unless the conditions that they are testing change in the app. We don't want to have to amend our tests if the app structure or tool changes (where possible).

We achieve these aims through abstractions like the SeleniumManager and Page Object Models.

Chapter 28: Create Our First Page Objects

This chapter will provide the detail of how to convert a test using direct selenium commands into one using Page Objects. This will reinforce the basic techniques demonstrated in our previous chapter.

I will start by converting the Search page tests that we wrote in the MySecondSeleniumTests.java file.

> **Note:**
>
> **This was heavily refactored in Chapter 12, so I am starting with the refactored version with the @BeforeClass and @AfterClass methods in it.**
>
> **In the downloadable source-code package this can be found in com.eviltester.seleniumtutorials.chap12 as MySecondSeleniumTests_Refactored_BeforeClass_AfterClass.java**

I will initially go through the steps and refactoring in detail, but as we go through the chapter you will see that the basic steps are the same, so in later test refactorings I will only dwell on those steps which are different. As I step through I will explain the thought processes behind each step.

Create a SearchPageTests class

Since we are going to change these heavily. Rather than amending them in the MySecondSeleniumTests.java class we will take the opportunity to rename the class to make it more representative of the area being tested and move it to a new package to keep it separate from the previous tests.

So:

1. Create a copy of the class by right-clicking on the class in the package explorer and choosing 'copy'
2. Right click on the com.eviltester.seleniumtutorials.refactored package and choose paste

This will copy the class into the package. So the first thing we will do is rename it.

Right click on the MySecondSeleniumTests.java in the com.eviltester.seleniumtutorials.refactored package and select “refactor \ rename”, rename it as SearchPageTests

> **Note:**
>
> **The only reason for doing a copy and paste here is to keep your old tests so that you can see the difference. Normally we would rely on a version control system to maintain versions and we would refactor or migrate to page object models directly. Learning to rely on copy and paste may mean that you do not learn to refactor as much as you need to, and risk tolerating repeated code across your tests.**

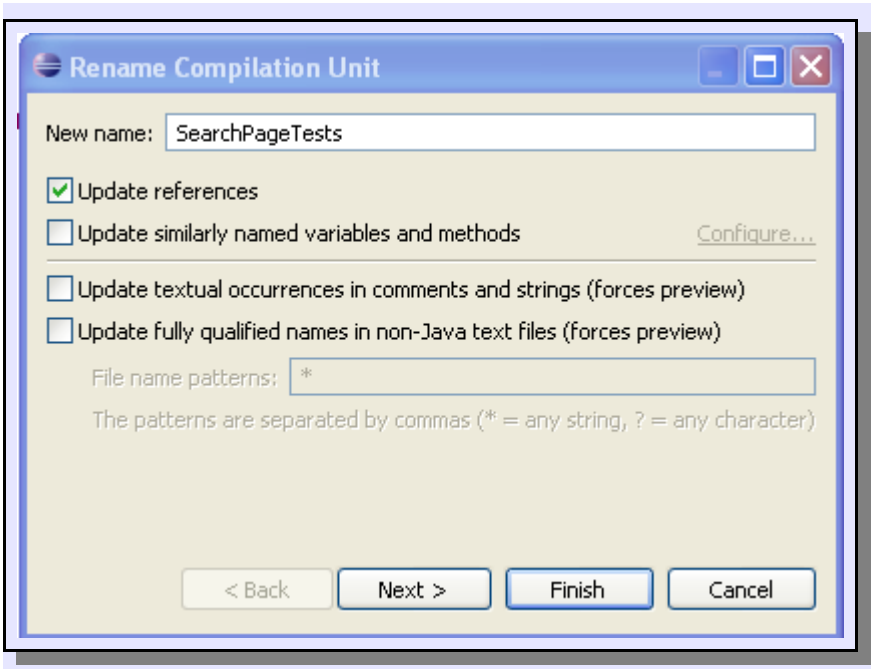


Figure 28.1 : Rename to SearchPageTests

Add the SeleniumManager

The first thing we need to do, is bring in the SeleniumManager that we created.

Add the lines of code to use the Selenium Manager to the SearchPageTests class as illustrated below:

```
public class SearchPageTests {
		static SeleniumManager sm;
		static Selenium selenium;

		@BeforeClass
		static public void automateTestSetup() throws IOException{
			sm = new SeleniumManager();
			sm.start("http://www.compendiumdev.co.uk");
			selenium = sm.getSelenium();
			startSeleniumAndSearchForSeleniumRC();
		}

		@AfterClass
		static public void stopSeleniumAndCloseBrowser() {
			sm.stop();
		}

		static void startSeleniumAndSearchForSeleniumRC() {
			selenium.open("/selenium/search.php");
			selenium.type("xpath=//input[@name='q']", "Selenium-RC");
			selenium.click("xpath=//input[@name='btnG' and @type='submit']");
			selenium.waitForPageToLoad("30000");
		}
```

To get to the above point I had to:

1. Remove errors about the SeleniumManager not being recognised by importing the SeleniumManager class – you should be able to do this by right clicking on the error indicator and choosing the automated fix to "Import 'SeleniumManager'"

```
import com.eviltester.seleniumutils.SeleniumManager;
```

2. Remove the error on the sm.start(...) line about an unhandled exception by right clicking on

the error indicator and choosing the automated fix to “Add throws declaration”. This adds the “throws IOException” text to the @BeforeClass annotated method.

3. Change the startSeleniumAndSearchForSeleniumRC method by deleting the instantiation of the selenium object, remove the return statement and change the method declaration to void. Remove the selenium.start(...) statement.
4. I also took the opportunity to change the open statement from a fully qualified URL, to a URL relative to the base URL we entered in the sm.start(...) method.

Run the tests to make sure that they still pass.

Now we will illustrate the steps to move these tests over to page object models on a method by method basis.

Convert startSeleniumAndSearchForSeleniumRC

The first method we will look at is startSeleniumAndSearchForSeleniumRC:

```
static void startSeleniumAndSearchForSeleniumRC() {
        selenium.open("/selenium/search.php");
        selenium.type("xpath=//input[@name='q']", "Selenium-RC");
        selenium.click("xpath=//input[@name='btnG' and @type='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Our basic approach will be to remove any of the statements using “selenium.” and move this functionality into a page object in a way that makes the test readable and maintainable.

We will use the following pattern initially in these tests to open pages with the page object model.

```
SearchPage searchPage = new SearchPage(selenium);
searchPage.open();
```

Add this code to your test and then we will look at it in more depth.

```
37      static void startSeleniumAndSearchForSeleniumRC() {
38          SearchPage searchPage = new SearchPage(selenium);
39          searchPage.open();
40          selenium.open("/selenium/search.php");
41          selenium.type("xpath=//input[@name='q']", "Selenium-RC");
42          selenium.click("xpath=//input[@name='btnG' and @type='submit']");
43          selenium.waitForPageToLoad("30000");
44      }
```

Figure 28.2 : Adding initial SearchPage references to the test

I add the code to create and use the SearchPage object into the test before creating the SearchPage object and methods. I do this because I want to model what the interface will look like and how it will be used, and I want to take advantage of Eclipse's code fixing technology to make it easier for me to write the code, as we saw in Chapter 27 Evolving A Selenium Manager Class,

I know that my SearchPage object will need to call selenium to implement the test, so I will pass the current selenium session to the SearchPage object when I create it. And I know that once I have the searchPage object, I want to call an open() method to open the page.

Now we can start implementing the page object using the Eclipse error fixing technology.

Left-click on the error in the left border and choose the option to “Create class 'SearchPage'”

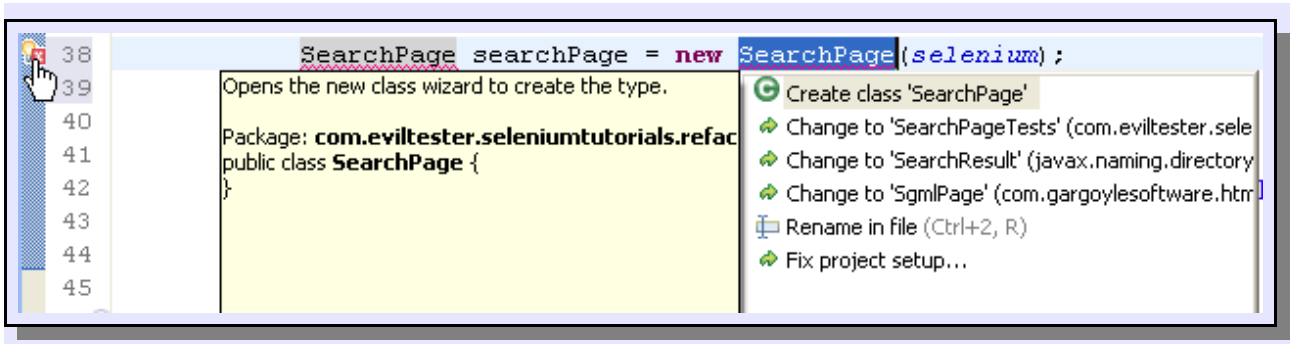


Figure 28.3 : Create class 'SearchPage' automatically

In the New Java Class wizard, I will change the package location so that all the page objects are kept in the same package

```
com.eviltester.seleniumtutorials.refactored.pageObjectModel
```

By finishing the wizard, Eclipse will create a new class for us:

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;

public class SearchPage {
}
```

We now have more errors to deal with in our test. First “The constructor SearchPage(Selenium) is undefined”, which we can deal with by choosing “Create constructor 'SearchPage(Selenium)'” from the drop down menu.

Save “SearchPage.java” to remove the error display in “SearchPageTests.java”. Do this after every move of code or creation of methods into SearchPage.java. This will force Eclipse to recompile the class and use the new version, which will fix the errors in our “SearchPageTests.java”.

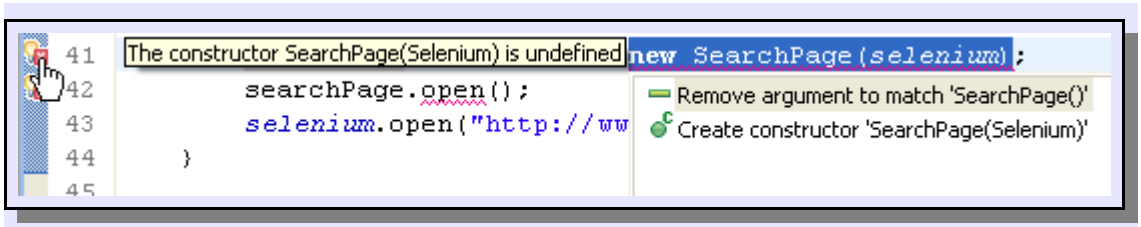


Figure 28.4 : Left-click on error to use drop down to create constructor

Next we have to deal with the error on the “searchPage.open();” line. So choose “Create method 'open()' in type 'SearchPage'”.

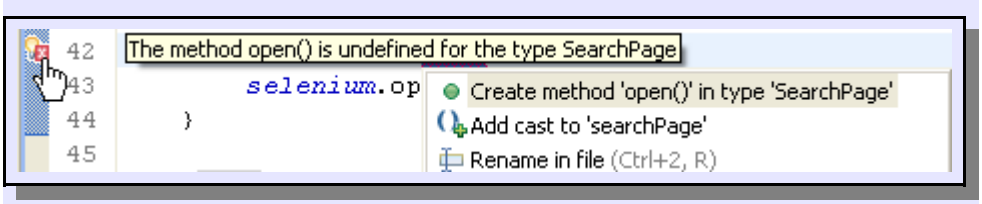


Figure 28.5 : Left-click on error to user drop down to create open method

This will remove all the errors in our test, although we haven't implemented anything in our page object.

So now cut and paste the “selenium.open("/selenium/search.php")” line into the SearchPage object

```
selenium.open("selenium/search.php");
```

We end up with a page object like this:

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;

import com.thoughtworks.selenium.Selenium;

public class SearchPage {

	public SearchPage(Selenium selenium) {
		// TODO Auto-generated constructor stub
	}

	public void open() {
		selenium.open("http://www.compendiumdev.co.uk/selenium/search.php");
	}

}
```

Now that our test has no errors we can turn our attention to the page object and fix the errors here.

Because we have references to a selenium object in the open method which we haven't declared or set, we are seeing the error “selenium cannot be resolved”.

We can fix this by amending the constructor as follows:

```
	private Selenium selenium;

	public SearchPage(Selenium selenium) {
		this.selenium = selenium;
	}
```

Here we have declared a variable local to the class called selenium, or type Selenium, which will act as our reference to the current Selenium session.

In the constructor, we take the selenium value passed in as a parameter and assign it to our local class variable.

At this point we have the basics of our page object model for the search page. All it does is open the page, but it abstracts where that page is located from our tests, and all our test has to do is create the correct object and call the method to open the page.

If you run the SearchPageTests now you should find that they run without errors and do exactly the same thing.

Where possible I prefer to take the approach of refactoring to page object models incrementally from existing tests, and doing it in such a way that the existing tests do not break.

We still have lines in the test which use the Selenium object, so we need to move them into the SearchPage object.

Again, I start by writing what I want the test code to look like and I will initially do that with in line comments:

```
static void startSeleniumAndSearchForSeleniumRC() {
	SearchPage searchPage = new SearchPage(selenium);
	searchPage.open();
	//searchPage.typeSearchTerm("Selenium-RC");
	//searchPage.clickSearchButton();
	selenium.type("xpath=//input[@name='q']", "Selenium-RC");
	selenium.click("xpath=//input[@name='btnG' and @type='submit']");
	selenium.waitForPageToLoad("30000");
}
```

So remove comments from the line:

```
searchPage.typeSearchTerm("Selenium-RC");
```

This now has an error against it because the typeSearchTerm method does not exist.

By writing a call for a method that does not exist, I can quickly create it with a left-click on the error marker and choosing the option “Create method ...” from the pop-up menu.

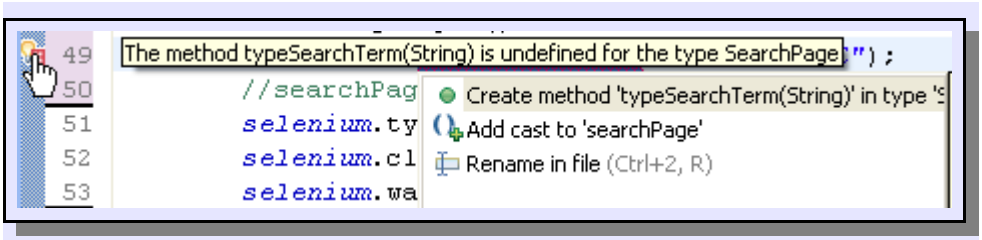


Figure 28.6 : Create method typeSearchTerm

The method will be created in the page object, and all we have to do, is cut the code we are moving from the test.

```
selenium.type("xpath=//input[@name='q']", "Selenium-RC");
```

And paste it into the page object method.

```
public void typeSearchTerm(String string) {
	selenium.type("xpath=//input[@name='q']", "Selenium-RC");
}
```

Then adjust the page object method to use the passed in parameters, and name the parameters appropriately.

```
public void typeSearchTerm(String searchTerm) {
	selenium.type("xpath=//input[@name='q']", searchTerm);
}
```

Note:

This is the basic pattern I use for refactoring from tests using Selenium directly into page object models:

- write the test as you want it to read

- make those statements comments until you are ready to implement them

- remove the comment marks and use Eclipse to create the method

- cut the code out of the test and paste it into the page object method

- amend the method to have sensible parameter names

- use the parameters in the body of the method

Now we will remove the comment marks from the code for:

```
searchPage.clickSearchButton();
```

Use the Eclipse pop-up menu to “Create Method 'clickSearchButton'”.

Cut and paste the code into the 'clickSearchButton' method in SearchPage:

```
public void clickSearchButton() {
	selenium.click("xpath=//input[@name='btnG' and @type='submit']");
	selenium.waitForPageToLoad("30000");
```

```
}
```

We don't even have to amend any parameters this time.

Then we end up with the test that we wanted:

```
static void startSeleniumAndSearchForSeleniumRC() {
	SearchPage searchPage = new SearchPage(selenium);
	searchPage.open();
	searchPage.typeSearchTerm("Selenium-RC");
	searchPage.clickSearchButton();
}
```

typeInASearchTermAndAssertThatHomePageTextIsPresent

The next test to refactor is typeInASearchTermAndAssertThatHomePageTextIsPresent:

```
@Test
public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){

	assertTrue(selenium.isTextPresent(
			"Selenium Remote-Control Selenium RC comes in two parts." +
			" A server which automatically"));
	assertTrue(selenium.isTextPresent(
			"launches and kills browsers, and acts as" +
			" a HTTP proxy for web requests from them."));
}
```

This time, I can see that I'm going to create an isTextPresent method on searchPage, so I will do that by amending the text directly, by changing *selenium* to *SearchPage*

```
assertTrue(searchPage.isTextPresent(
		"Selenium Remote-Control Selenium RC comes in two parts." +
		" A server which automatically"));
```

Then use the same process as before to fix the error using Eclipse, amend the parameter and copy in the code to create the following method in SearchPage:

```
public boolean isTextPresent(String textToSearchFor) {
	return selenium.isTextPresent(textToSearchFor);
}
```

All I do in the isTextPresent method is delegate the processing to the selenium method of the same name and return the result of doing this. The selenium method is named effectively enough that I don't see the need to change it in my test.

I make the same change on the other assertTrue.

One extra thing I will do here is refactor some of the code in the test.

I need the searchPage object created in startSeleniumAndSearchForSeleniumRC to be accessible in the tests. So I need to move the declaration for this object to the top of the code. e.g.

```
public class SearchPageTests {
	static SeleniumManager sm;
	static Selenium selenium;
	static SearchPage searchPage;
```

And in the startSeleniumAndSearchForSeleniumRC method:

```
static void startSeleniumAndSearchForSeleniumRC() {
			searchPage = new SearchPage(selenium);
```

Having done this, I run the tests again and make sure they still pass.

Note:

After a few refactoring steps, and amending the tests, run the tests and make sure that they still pass. If you run them from within Eclipse, hopefully you will find that they all run and continue to pass.

They should, as you haven't done anything to materially change the tests, all you have done is move the code which implements the tests into abstraction layers.

typeInASearchTermAndAssertThatHomePageURLExists

I rename typeInASearchTermAndAssertThatHomePageURLExistsWithFewerXpath to the shorter typeInASearchTermAndAssertThatHomePageURLExists.

```
@Test
public void typeInASearchTermAndAssertThatHomePageURLExists(){

        // create a new variable called matchingCountTotal to calculate
        // the total number of times our XPath matches, initially setting
        // it to 0
        int matchingCountTotal = 0;

        // call getXpathCount with our XPath for the url and
        // add the intValue version to the total
        matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://selenium-rc.seleniumhq.org')]").intValue();
        matchingCountTotal += selenium.getXpathCount("//a[starts-
with(@href,'http://seleniumhq.org/projects/remote-control')]").intValue();
        assertTrue("No homepage URL found",matchingCountTotal>0);
}
```

Note:

There is a "rename" refactoring which renames all instances and references in the code.

The next step involves moving the XPath statements into the page object. It seems to me that a general statement of intent that I have for those lines is:

```
searchPage.numberOfURLsThatStartWith
```

So really I want my test to look like this:

```
matchingCountTotal += searchPage.numberOfURLsThatStartWith(
                                "http://selenium-rc.seleniumhq.org");
matchingCountTotal += searchPage.numberOfURLsThatStartWith(
                                "http://seleniumhq.org/projects/remote-control");
```

So if I add that code into the test, I can use Eclipse to build the "numberOfURLsThatStartWith" method.

```
public int numberOfURLsThatStartWith(String aHREF) {
        return selenium.getXpathCount(
                "//a[starts-with(@href,'" + aHREF + "')]").intValue();
}
```

All I have done here is rename the parameter to be readable. Delegated the functionality to selenium's getXPathCount method. And replaced the URL which was hard coded into the XPath statement with the parameter we pass in.

One interesting side-effect is that I no longer need the comments that I added in to the test to explain what was going on, as the test is now more readable.

```
@Test
```

```
public void typeInASearchTermAndAssertThatHomePageURLExists(){

        int matchingCountTotal = 0;

        matchingCountTotal += searchPage.numberOfURLsThatStartWith(
                                        "http://selenium-rc.seleniumhq.org");
        matchingCountTotal += searchPage.numberOfURLsThatStartWith(
                                        "http://seleniumhq.org/projects/remote-control");

        assertTrue("No homepage URL found",matchingCountTotal>0);
}
```

typeInASearchTermAndCheckPageTitleHasSearchTermInIt

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){

        String pageTitle = selenium.getTitle();
        assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
                                pageTitle.contains("Selenium-RC"));
}
```

All I have to do here is create a searchPage.getTitle method:

```
String pageTitle = searchPage.getTitle();
```

Which looks like the following when added to the SearchPage object and delegating to the selenium.getTitle() method:

```
public String getTitle() {
        return selenium.getTitle();
}
```

So our test becomes:

```
@Test
public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
        String pageTitle = searchPage.getTitle();
        assertTrue("Page Title does not contain Selenium-RC search term: " + pageTitle,
                        pageTitle.contains("Selenium-RC"));
}
```

typeInASearchTermAndCheckSearchInputHasSearchTermInIt

```
@Test
public void typeInASearchTermAndCheckSearchInputHasSearchTermInIt(){

        String searchTerm = selenium.getValue(
                        "xpath=//input[@name='q' and @title='Search']");

        assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
                        searchTerm.equals("Selenium-RC"));
}
```

I think that this test should read:

```
String searchTerm = searchPage.getSearchTerm();
```

So I add that code to the test, then use Eclipse to create the getSearchTerm method.

Then move the code from my test into the getSearchTerm method in SearchPage:

```
public String getSearchTerm() {
        return selenium.getValue("xpath=//input[@name='q' and @title='Search']");
}
```

So the final test reads:

```
@Test
public void typeInASearchTermAndCheckSearchInputHasSearchTermInIt(){
        String searchTerm = searchPage.getSearchTerm();
        assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
                        searchTerm.equals("Selenium-RC"));
}
```

After all that ...

SearchPageTests.java

```
package com.eviltester.seleniumtutorials.refactored;
import static org.junit.Assert.*;
import java.io.IOException;
import org.junit.*;
import com.eviltester.seleniumtutorials.refactored.pageObjectModel.SearchPage;
import com.eviltester.seleniumutils.SeleniumManager;
import com.thoughtworks.selenium.*;

public class SearchPageTests {

	static SeleniumManager sm;
	static Selenium selenium;
	private SearchPage searchPage;

	@BeforeClass
	static public void startServer() throws IOException{
		sm = new SeleniumManager();
		sm.start("http://www.compendiumdev.co.uk");
		selenium = sm.getSelenium();
		startSeleniumAndSearchForSeleniumRC();
	}

	@AfterClass
	static public void stopServer(){
		sm.stop();
	}

	static void startSeleniumAndSearchForSeleniumRC() {
		searchPage = new SearchPage(selenium);
		searchPage.open();
		searchPage.typeSearchTerm("Selenium-RC");
		searchPage.clickSearchButton();
	}

	@Test
	public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
		assertTrue(searchPage.isTextPresent(
			"Selenium Remote-Control Selenium RC comes in two parts." +
			" A server which automatically"));
		assertTrue(searchPage.isTextPresent(
			"launches and kills browsers, and acts as" +
			" a HTTP proxy for web requests from them."));
	}

	@Test
	public void typeInASearchTermAndAssertThatHomePageURLExists(){
		int matchingCountTotal = 0;
		matchingCountTotal += searchPage.numberOfURLsThatStartWith(
					"http://selenium-rc.seleniumhq.org");
		matchingCountTotal += searchPage.numberOfURLsThatStartWith(
					"http://seleniumhq.org/projects/remote-control");
		assertTrue("No homepage URL found",matchingCountTotal>0);
	}

	@Test
	public void typeInASearchTermAndCheckPageTitleHasSearchTermInIt(){
		String pageTitle = searchPage.getTitle();
		assertTrue("Page Title does not contain Selenium-RC search term: "
				+ pageTitle, pageTitle.contains("Selenium-RC"));
	}

	@Test
	public void typeInASearchTermAndCheckSearchInputHasSearchTermInIt(){
		String searchTerm = searchPage.getSearchTerm();
		assertTrue("Search Input does not contain Selenium-RC : " + searchTerm,
				searchTerm.equals("Selenium-RC"));
	}
}
```

SearchPage.java

And the page object abstraction looks as follows:

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;
import com.thoughtworks.selenium.Selenium;

public class SearchPage {

	private Selenium selenium;

	public SearchPage(Selenium selenium) {
		this.selenium = selenium;
	}

	public void open() {
		selenium.open("/selenium/search.php");
	}

	public void typeSearchTerm(String searchTerm) {
		selenium.type("xpath=//input[@name='q']", searchTerm);
	}

	public void clickSearchButton() {
		selenium.click("xpath=//input[@name='btnG' and @type='submit']");
		selenium.waitForPageToLoad("30000");
	}

	public boolean isTextPresent(String textToSearchFor) {
		return selenium.isTextPresent(textToSearchFor);
	}

	public int numberOfURLsThatStartWith(String aHREF) {
		return selenium.getXpathCount(
				"//a[starts-with(@href,'" + aHREF + "')]").intValue();
	}

	public String getTitle() {
		return selenium.getTitle();
	}

	public String getSearchTerm() {
		return selenium.getValue("xpath=//input[@name='q' and @title='Search']");
	}
}
```

Final Notes

The tests and page object will evolve over time, as more tests get added, more methods will move into the SearchPage page object. If the application changes, hopefully all amendments can be made to the SearchPage object and not the test.

By removing the selenium command execution from the tests we can see the intent of those tests clearly and make them more readable.

Chapter 29: More Page Object Creation

All that remains now, is to take the other tests that we are interested in retaining, and refactor them to use page objects:

- HTML_form_tests.java
- JavaScript_With_Selenium_Tests.java
- Static_HTML_tests.java

The same basic principles apply for all of these.

The Tests

This chapter will go through the refactoring of these test files very quickly. Only covering the decisions that were made, or the points where we had to refactor differently.

You can find all the final code in the code distribution in the package com.eviltester.seleniumtutorials.refactored

A useful exercise for you is to go through and do the refactorings yourself and create the page object model as outlined below. You have all the source-code to refer to if you make a mistake. Follow the same procedures as the previous chapter and the act of doing the refactoring will eventually make the thought processes and actions second nature for you.

HTML_form_tests.java

First I copy the HTML_form_tests.java from com.eviltester.seleniumtutorials.chap18 and paste it into com.eviltester.seleniumtutorials.refactored

And then add the SeleniumManager code.

I change the setUp method to instantiate a BasicHTMLForm object, and call the open method on this object, instead of opening the page directly.

Before	After

`private static Selenium selenium;`
 	
`@BeforeClass`
`public static void setUpBeforeClass()`
`throws Exception {`
`selenium = new DefaultSelenium(`
`"localhost", 4444,`
`"*firefox","http://www.eviltester.com");`
`selenium.start();`
`}`
 	
`@AfterClass`
`public static void tearDownAfterClass() throws Exception {`
`selenium.close();`
`Selenium.stop();`
`}`
 	
`@Before`
`public void setUp() throws Exception {`
`selenium.open("/selenium/basic_html_form.html");`
`}`
	`static SeleniumManagerEnd sm;`
	`static Selenium selenium;`
	`BasicHTMLForm htmlForm;`
	
	`@BeforeClass`
	`static public void startServer() throws IOException{`
	`sm = new SeleniumManagerEnd();`
	`sm.start("http://www.compendiumdev.co.uk");`
	`selenium = sm.getSelenium();`
	`}`
	
	`@AfterClass`
	`static public void stopServer(){`
	`sm.stop();`
	`}`
	
	`@Before`
	`public void setUp() throws Exception {`
	`htmlForm = new BasicHTMLForm(selenium);`
	`htmlForm.open();`
	`}`

test_submit_form_with_default_values

```
@Test
public void test_submit_form_with_default_values(){
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_default_values(){
        htmlForm.clickSubmitButton();
}
```

Where all we did was move the method code into the page object clickSubmitButton method.

test_submit_form_without_clicking_submit

```
@Test
public void test_submit_form_without_clicking_submit(){
        selenium.submit("//form[@id='HTMLFormElements']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_without_clicking_submit(){
        htmlForm.submitForm();
}
```

Again, all we did was move the method code into the page object submitForm method.

test_submit_form_with_new_username

```
@Test
public void test_submit_form_with_new_username(){
	selenium.type("username","eviltester");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_new_username(){
	htmlForm.typeUserName("eviltester");
	htmlForm.clickSubmitButton();
}
```

Here we reused the clickSubmitButton, which we refactored earlier, and we will re-use this throughout the tests. And added a more domain level "typeUserName", this takes a parameter so that we can add any username we want. But hides away the information about how you type this, or where on the form it actually goes.

test_submit_form_with_new_password

```
@Test
public void test_submit_form_with_new_password(){
	selenium.type("password","myPassword");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_new_password(){
	htmlForm.typePassword("myPassword");
	htmlForm.clickSubmitButton();
}
```

Here we keep the approach for typePassword consistent with typeUserName. It doesn't really matter what I call these methods because we can easily refactor them later. I am keeping this simple and using very similar language to the Selenium methods.

At the moment keeping it consistent will make the tests readable.

test_submit_form_with_new_password

```
@Test
public void test_submit_form_with_html_escaped_text(){
	selenium.type("comments", "\\\n\t\"I said, give me your pound sign\"\u00A3");
	selenium.click("//input[@name='submitbutton' and @value='submit']");
	selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_html_escaped_text(){
	htmlForm.typeComments("\\\n\t\"I said, give me your pound sign\"\u00A3");
	htmlForm.clickSubmitButton();
}
```

Another very simple move of code into the page object, keeping the naming convention consistent.

test_submit_form_with_filename

```
@Test
public void test_submit_form_with_filename(){
	selenium.type("filename", "C:\\Selenium\\readme.txt");
```

```
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_filename(){
        htmlForm.typeFilename("C:\\Selenium\\readme.txt");
        htmlForm.clickSubmitButton();
}
```

Again, a simple move of code into the page object, keeping the naming convention consistent.

test_submit_form_with_click_check_and_radio

```
@Test
public void test_submit_form_with_click_check_and_radio(){
        selenium.click("//input[@name='checkboxes[]' and @value='cb2']");
        selenium.click("//input[@name='checkboxes[]' and @value='cb3']");
        selenium.click("//input[@name='radioval' and @value='rd3']");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_click_check_and_radio(){
        htmlForm.clickCheckBox("2");
        htmlForm.clickCheckBox("3");
        htmlForm.clickRadioButton("3");
        htmlForm.clickSubmitButton();
}
```

Here I made a decision to create a single clickCheckBox method, which would take as a parameter the 'digit' of the checkbox value, since all checkbox values are very similar (cb1, cb2, cb3). This will work for the moment since the form is very simple. There is scope for the user to get the test wrong and choose a checkbox that doesn't exist e.g. htmlForm.clickCheckBox("27"); But at the moment, refactoring quickly and keeping the abstraction layers simple seems to make sense.

I made the same decision for clickRadioButton.

test_submit_form_with_multiple_select_values

```
@Test
public void test_submit_form_with_multiple_select_values(){
        selenium.removeSelection("multipleselect[]", "label=Selection Item 3");
        selenium.addSelection("multipleselect[]", "label=Selection Item 4");
        selenium.addSelection("multipleselect[]", "label=Selection Item 1");
        selenium.removeSelection("multipleselect[]", "label=Selection Item 4");
        selenium.addSelection("multipleselect[]", "value=ms2");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_multiple_select_values(){
        htmlForm.removeMultiSelection("3");
        htmlForm.selectMultiSelection("4");
        htmlForm.selectMultiSelection("1");
        htmlForm.removeMultiSelection("4");
        htmlForm.selectMultiSelection("2");
        htmlForm.clickSubmitButton();
}
```

One useful side-effect of using page object models is that it makes it very clear in the test that we are not actually 'testing' anything. We are certainly 'doing stuff' with the system. But we are not

checking any results or conditions on the form. This is slightly less easy to see with the raw selenium access code because the test is not quite as readable.

> **Note:**
>
> **A useful exercise for the reader would be to add asserts into these tests.**
>
> **e.g. After each of the remove or select MultiSelection we should really check that the values we expect to be selected are in fact selected.**

test_submit_form_with_dropdown_values

```
@Test
public void test_submit_form_with_dropdown_values(){
        // by default 3 is selected, so select item 1
        selenium.select("dropdown", "label=Drop Down Item 1");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_dropdown_values(){
        htmlForm.selectDropDownItem("1");
        htmlForm.clickSubmitButton();
}
```

Create the abstraction layer (htmlForm) methods, and copy in the implementation code from the test. I also removed the code comments because the method names are explanatory enough that the comments were no longer adding value.

test_submit_form_with_hidden_field

```
@Test
public void test_submit_form_with_hidden_field(){
        selenium.getEval(
        "this.browserbot.findElement(\"name=hiddenField\").value=\"amended value\"");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
@Test
public void test_submit_form_with_hidden_field(){
        htmlForm.setHiddenField("amended value");
        htmlForm.clickSubmitButton();
}
```

Move the code into htmlForm methods and reuse the form submit method. I took the opportunity to make the hidden field's new value a parameter to the htmlform method. This instantly makes the HtmlForm.setHiddenField method more reusable.

test_check_text_entered_values

```
@Test
public void test_check_text_entered_values(){
        selenium.type("username","eviltester");
        selenium.type("filename", "C:\\Selenium\\readme.txt");
        selenium.type("password","myPassword");
        selenium.type("comments", "simple text");
```

```
        assertEquals("username not as entered",
                      "eviltester",selenium.getValue("username"));
        assertEquals("password not as entered",
                      "myPassword",selenium.getValue("password"));
        assertEquals("filename not as entered",
                      "C:\\Selenium\\readme.txt",selenium.getValue("filename"));
        assertEquals("comments not as entered",
                      "simple text",selenium.getValue("comments"));
}
```

Becomes:

```
@Test
public void test_check_text_entered_values(){
        htmlForm.typeUserName("eviltester");
        htmlForm.typeFilename("C:\\Selenium\\readme.txt");
        htmlForm.typePassword("myPassword");
        htmlForm.typeComments("simple text");

        assertEquals("username not as entered", "eviltester",htmlForm.getUserName());
        assertEquals("password not as entered", "myPassword",htmlForm.getPassword());
        assertEquals("filename not as entered",
                       "C:\\Selenium\\readme.txt",htmlForm.getFileName());
        assertEquals("comments not as entered", "simple text",htmlForm.getComments());
}
```

This test reuses some of the 'type' methods created earlier, but introduces the 'accessor' or 'get' methods required in our test to have the ability to make assertions. I could have added the 'get' methods at the same time as the 'type' (or 'set') methods. But I prefer to refactor the abstraction layer and therefore only add code into the abstraction layer when tests actually call for it. In this way I don't build methods which require maintenance but are never used.

test_check_checkbox_values

```
@Test
public void test_check_checkbox_values(){
        assertEquals("by default checkbox 1 is not selected", "off",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb1']"));
        assertEquals("by default checkbox 2 is not selected", "off",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb2']"));
        assertEquals("by default checkbox 3 is selected", "on",
                selenium.getValue("//input[@name='checkboxes[]' and @value='cb3']"));

        selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
        selenium.uncheck("//input[@name='checkboxes[]' and @value='cb3']");

        assertFalse("checkbox 1 is still not selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb1']"));
        assertTrue("checkbox 2 is now selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb2']"));
        assertFalse("default checkbox 3 is no longer selected",
                selenium.isChecked("//input[@name='checkboxes[]' and @value='cb3']"));
}
```

Becomes:

```
@Test
public void test_check_checkbox_values(){

        assertFalse("by default checkbox 1 is not selected",
                       htmlForm.isCheckBoxSelected("1"));
        assertFalse("by default checkbox 2 is not selected",
                       htmlForm.isCheckBoxSelected("2"));
        assertTrue("by default checkbox 3 is selected",
                       htmlForm.isCheckBoxSelected("3"));

        htmlForm.setChecked_CheckBox("2");
        htmlForm.setUnChecked_CheckBox("3");
```

```
        assertFalse("checkbox 1 is still not selected",
                        htmlForm.isCheckBoxSelected("1"));
        assertTrue("checkbox 2 is now selected",
                        htmlForm.isCheckBoxSelected("2"));
        assertFalse("default checkbox 3 is no longer selected",
                        htmlForm.isCheckBoxSelected("3"));
}
```

I had a number of decision to make when refactoring the test. I already have some checkbox methods 'clickCheckBox' but here I made the decision to add new methods to setChecked and setUnchecked. This is in keeping with the intent of the tests, and if it proves confusing later on then I can amend the methods. I also added accessors to check the state of the checkboxes, I did this by parameterising the isCheckBoxSelected method with a string representation of the numeric postfix value on the element id. This keeps the method consistent with previous usage.

test_check_radio_values

```
@Test
public void test_check_radio_values(){
        assertEquals("by default radio 1 is not selected", "off",
                selenium.getValue("//input[@name='radioval' and @value='rd1']"));
        assertEquals("by default radio 2 is selected", "on",
                selenium.getValue("//input[@name='radioval' and @value='rd2']"));
        assertEquals("by default radio 3 is not selected", "off",
                selenium.getValue("//input[@name='radioval' and @value='rd3']"));

        selenium.check("//input[@name='radioval' and @value='rd1']");


        assertTrue("radio 1 is now selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd1']"));
        assertFalse("radio 2 is no longer selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd2']"));
        assertFalse("radio 3 is not selected",
                selenium.isChecked("//input[@name='radioval' and @value='rd3']"));
}
```

Becomes:

```
@Test
public void test_check_radio_values(){

        assertFalse("by default radio 1 is not selected",
                htmlForm.isRadioButtonSelected("1"));
        assertTrue("by default radio 2 is selected",
                htmlForm.isRadioButtonSelected("2"));
        assertFalse("by default radio 3 is not selected",
                htmlForm.isRadioButtonSelected("3"));

        htmlForm.setChecked_RadioButton("1");

        assertTrue("radio 1 is now selected",
                htmlForm.isRadioButtonSelected("1"));
        assertFalse("radio 2 is no longer selected",
                htmlForm.isRadioButtonSelected("2"));
        assertFalse("radio 3 is not selected",
                htmlForm.isRadioButtonSelected("3"));
}
```

Here I took the same approach as used for the checkbox test. With the exception that I did not add a 'setUnChecked' method. I did not add this because it was not called for in the test. At this point, I'm not amending the tests to make them better, I'm working with the test as written. After I convert the tests to use an abstraction layer I will review the tests to make them more robust and this may increase the methods in the abstraction layer.

test_randomly_select_value_from_dropdown

```
@Test
public void test_randomly_select_value_from_dropdown(){

	// get all options in the dropdown
	String dropDownOptions[] = selenium.getSelectOptions("dropdown");

	// get the index of the currently selected item in the dropdown
	String currentDropDownItem = selenium.getSelectedIndex("dropdown");
	int previouslySelectedIndex =Integer.valueOf(currentDropDownItem);

	// randomly choose an item in the dropdown
	Random generator = new Random(); // requires import java.util.Random to be added
	int dropDownIndex = generator.nextInt(dropDownOptions.length);

	// make sure it isn't the same as the current one
	if(dropDownIndex==previouslySelectedIndex){
		// it is the same, so add one
		dropDownIndex++;

		// but this might push it out of bounds so modulus it
		dropDownIndex = dropDownIndex % dropDownOptions.length;
	}

	// select this new one
	selenium.select("dropdown", "label=" + dropDownOptions[dropDownIndex]);

	// check that something is selected
	assertTrue(selenium.isSomethingSelected("dropdown"));

	// check that what we wanted is selected
	assertEquals(dropDownIndex,
		Integer.valueOf(selenium.getSelectedIndex("dropdown")).intValue());

	//double check we didn't select the same thing
	assertTrue(dropDownIndex!=previouslySelectedIndex);
}
```

Becomes:

```
@Test
public void test_randomly_select_value_from_dropdown(){

	String dropDownOptions[] = htmlForm.getAllDropDownValues();
	int previouslySelectedIndex = htmlForm.getCurrentDropDownSelectionIndex();

	// randomly choose an item in the dropdown
	Random generator = new Random(); // requires import java.util.Random to be added

	int dropDownIndex = generator.nextInt(dropDownOptions.length);

	// make sure it isn't the same as the current one
	if(dropDownIndex==previouslySelectedIndex){
		// it is the same, so add one
		dropDownIndex++;

		// but this might push it out of bounds so modulus it
		dropDownIndex = dropDownIndex % dropDownOptions.length;
	}

	htmlForm.selectDropDown(dropDownOptions[dropDownIndex]);

	assertTrue(htmlForm.isADropDownItemSelected());
	assertEquals(dropDownIndex,htmlForm.getCurrentDropDownSelectionIndex());

	//double check we didn't select the same thing
	assertTrue(dropDownIndex!=previouslySelectedIndex);
}
```

Most of this test is test code, so very little had to be moved into the abstraction layer. For those

abstraction layer items that I did add, the refactoring was the simple approach: design method usage, create method, move implementation code from test.

Static_HTML_tests.java

Before refactoring static_html_tests.java into page objects, I first of all have a look at the code to see if there are any beneficial local refactorings first.

> **Note:**
>
> **You can find the code for the local refactorings in the package com.eviltester.seleniumtutorials.chap19 and the java source-code file, Static_HTML_tests_refactored_prior_to_page_objects.java**

I can see that there is considerable reuse of the following chunk of code:

```
selenium.type("username","eviltester");
selenium.type("password","myPassword");
selenium.click("//input[@name='submitbutton' and @value='submit']");
selenium.waitForPageToLoad("30000");
```

So I first extract this into a method called submitFormWithUserNameAndPassword using the automated “refactor \ Extract Method”

```
private void submitFormWithUserNameAndPassword() {
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

This then reduces the amount of work I have to do during the refactoring to page objects.

First I copy the Static_HTML_tests.java from com.eviltester.seleniumtutorials.chap19 and paste it into com.eviltester.seleniumtutorials.refactored

And then add the SeleniumManager code.

I change the setUp method to instantiate a BasicHTMLForm object, and call the open method on this object, instead of opening the page directly.

Before	After

`private static Selenium selenium;`
 	
`@BeforeClass`
`public static void setUpBeforeClass()`
`throws Exception {`
`selenium = new DefaultSelenium(`
`"localhost", 4444, "*firefox",`
`"http://www.compendiumdev.co.uk");`
`selenium.start();`
`}`
 	
`@AfterClass`
`public static void tearDownAfterClass()`
`throws Exception {`
`selenium.close();`
`selenium.stop();`
`}`
`@Before`
`public void setUp() throws Exception {`
`selenium.open("/selenium/basic_html_form.html");`
`}`
	`static SeleniumManagerEnd sm;`
	`static Selenium selenium;`
	`BasicHTMLForm htmlForm;`
	
	`@BeforeClass`
	`static public void startServer()`
	`throws IOException{`
	`sm = new SeleniumManagerEnd();`
	`sm.start("http://www.compendiumdev.co.uk");`
	`selenium = sm.getSelenium();`
	`}`
	
	`@AfterClass`
	`static public void stopServer(){`
	`sm.stop();`
	`}`
	`@Before`
	`public void setUp() throws Exception {`
	`htmlForm = new BasicHTMLForm(selenium);`
	`htmlForm.open();`
	`}`

SubmitFormWithUserNameAndPassword

```
private void submitFormWithUserNameAndPassword() {
        selenium.type("username","eviltester");
        selenium.type("password","myPassword");
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
}
```

Becomes:

```
HTMLFormResultsPage resultsPage;

private void submitFormWithUserNameAndPassword() {
        htmlForm.typeUserName("eviltester");
        htmlForm.typePassword("myPassword");
        resultsPage = htmlForm.clickSubmitButton();
}
```

One interesting point of this refactoring is that it re-uses methods we created in previous refactorings and because I now want to use the page that follows the submitting of the form , I amended the clickSubmitButton method to return a new page object for the ResultsPage.

So the 'clickSubmitButton' method now looks like:

```
public HTMLFormResultsPage clickSubmitButton() {
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
        return new HTMLFormResultsPage(selenium);
}
```

Here you can see that my method does not check to see if, after the waitforPageToLoad, the page loaded is a result page or not. It assumes that it has and returns an HTMLFormResultPage instantiated with the current Selenium session.

I do not add a check here because I assume that if the application fails to return a search results page that there will be assertions in at least one of the tests to check if this is the case.

If I did want to add checks then I would probably rename the method. At the moment the method

doesn't mention a side-effect about checking the 'type' of the new page. So I would perhaps say clickSubmitButtonAndWaitForResultsPage.

But I would rather let the tests fail on follow on actions than put checks and asserts in the abstraction layer – this is based on hard won experience of constructing and maintaining suboptimal abstraction layers.

I did decide not to update the submitForm method to return an HTMLFormResultsPage object. Since the test did not require this.

> **Note:**
>
> **This was probably a mistake and I should have returned an HTMLFormResultsPage object. I can easily fix this with a future refactoring step. Part of the reason for engaging in refactoring, is to review the code that we have produced and determine how to make it better.**
>
> **The important lesson? It doesn't matter that I made the mistake:**
>
> **– I can easily change the method to return an HTMLFormResultsPage object.**
>
> **– At the point that I actually need the method to return an HTMLFormResultsPage object I can change the method.**
>
> **– It is just code, if it works and is clumsy, it still works. Code is one of the most fungible objects known to mankind.**
>
> **I offer you this advice to try and loosen you up. You will take decisions in your refactoring and abstraction layer construction that are suboptimal. None of these decisions are irreparable, and they offer you the chance to learn not to do it again. I have made my share of mistakes. I will try and share some of them in a later chapter, but there is no shame in this. We call this learning. When we approach mistake making with an attitude of flexibility, and the the thought that we did the best that we knew how to do at the time. Then we learn, and we change. Don't beat yourself up about it – that's your boss's job.**

submit_form_with_values_and_check_static_html

```
@Test
public void submit_form_with_values_and_check_static_html(){
        submitFormWithUserNameAndPassword();

        // check the title on the page
        assertEquals("Processed Form Details",selenium.getTitle());

        // check the username entered
        assertEquals( "eviltester",selenium.getText("_valueusername"));

        // check the submitted values text is on the page
        assertTrue(selenium.isTextPresent("Submitted Values"));

        //isElementPresent to check comments and filename
        assertTrue(selenium.isElementPresent("_comments"));
        assertFalse(selenium.isElementPresent("_filename"));

        // use getXPathCount to check the number of checkboxes
        assertEquals(1,selenium.getXpathCount(
                "//div[@id='_checkboxes']/ul/li").intValue());

        // click to go back to the form
```

```
        selenium.click("back_to_form");
        selenium.waitForPageToLoad("30000");

        // check the page title with getTitle
        assertEquals("HTML Form Elements",selenium.getTitle());

        //add a filename
        selenium.type("filename", "C:\\Selenium\\readme.txt");

        //check all the checkboxes
        selenium.check("//input[@name='checkboxes[]' and @value='cb1']");
        selenium.check("//input[@name='checkboxes[]' and @value='cb2']");
        selenium.check("//input[@name='checkboxes[]' and @value='cb3']");

        // submit the form
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");

        //check filename is present
        assertTrue(selenium.isElementPresent("_filename"));
        assertEquals("readme.txt",selenium.getText("_valuefilename"));

        //check the checkbox count
        assertEquals(3, selenium.getXpathCount(
                              "//div[@id='_checkboxes']/ul/li").intValue());
}
```

Becomes:

```
@Test
public void submit_form_with_values_and_check_static_html(){
        submitFormWithUserNameAndPassword();

        assertEquals("Processed Form Details",resultsPage.getTitle());
        assertEquals( "eviltester",resultsPage.getUserName());

        assertTrue(resultsPage.hasHeading());
        assertTrue(resultsPage.hasComments());
        assertFalse(resultsPage.hasFilename());
        assertEquals(1,resultsPage.getCheckboxCount());

        htmlForm = resultsPage.goBackToForm();

        assertEquals("HTML Form Elements",htmlForm.getTitle());

        htmlForm.typeFilename("C:\\Selenium\\readme.txt");
        htmlForm.setChecked_CheckBox("1");
        htmlForm.setChecked_CheckBox("2");
        htmlForm.setChecked_CheckBox("3");
        resultsPage = htmlForm.clickSubmitButton();

        assertTrue(resultsPage.hasFilename());
        assertEquals("readme.txt",resultsPage.getFilename());
        assertEquals(3,resultsPage.getCheckboxCount());
}
```

We had already created many of the methods we needed and those that we didn't have, were easily created by writing the appropriate method and using Eclipse to create the skeleton, then moving across the code from the test.

check_static_html_password_name

```
@Test
public void check_static_html_password_name(){
        submitFormWithUserNameAndPassword();

        // check the name of the password display paragraph is _password
        assertEquals("_password",
                selenium.getAttribute("xpath=//div[@id='_password']/p@name"));
```

```
}
```

Becomes:

```
@Test
public void check_password_field_exists(){
	submitFormWithUserNameAndPassword();
	assertTrue("Does not have a password field",resultsPage.hasPassword());
}
```

One thing this test conversion showed me was that the test added no value. I decided to keep it because I thought it might be useful to have a test that checks that the form has a password element. I could easily have deleted this test and felt no ill effects.

useAssignIDToCheckFilenameBlank

```
@Test
public void useAssignIDToCheckFilenameBlank(){
	submitFormWithUserNameAndPassword();

	selenium.assignId("xpath=//body/p[2]/strong", "_filename");
	assertEquals("No Value for filename",selenium.getText("_filename"));
}
```

Becomes:

```
@Test
public void useAssignIDToCheckFilenameBlank(){
	submitFormWithUserNameAndPassword();
	assertEquals("No Value for filename",resultsPage.getFilename());
}
```

Because this initial test was intended to illustrate the assignment of an id, it doesn't add much value in its current form as there are other ways of checking if the filename is present. And indeed, in the implementation of getFilename I choose a more generic way of implementing the check. The check itself is valuable, and the approach is useful to know, but I will always attempt to use the simplest mechanism to implement an abstraction layer function. I find “simple” easier to maintain and understand.

getText_equals_getTitle

```
@Test
public void getText_equals_getTitle(){
	assertEquals("HTML Form Elements",selenium.getTitle());
	assertEquals(selenium.getTitle(),selenium.getText("//head/title"));
}
```

Becomes:

```
@Test
public void checkPageTitleAsExpected(){
	assertEquals("HTML Form Elements",htmlForm.getTitle());
}
```

Since this test was again a demonstration of alternative approaches. The final test adopts an implementation using the simplest and cleanest of the approaches. In this case this means delegating through to the selenium.getTitle method.

While some of the tests above had little value, I was still able to refactor them into something that provided some coverage value. Some of them had no value as they were purely illustrative. I list

these below by name, but have removed them from the final source-code.

- getBodyText_getHtmlSource
- submit_form_with_values_and_go_back

JavaScript_With_Selenium_Tests.java

First I copy the JavaScript_With_Selenium_Tests.java from com.eviltester.seleniumtutorials.chap20 and paste it into com.eviltester.seleniumtutorials.refactored

And then add the SeleniumManager code.

I change the setUp method to instantiate a BasicHTMLForm object, and call the open method on this object, instead of opening the page directly.

Before	After

`private static Selenium selenium;`
 	
`@BeforeClass`
`public static void setUpBeforeClass()`
`  throws Exception {`
`  selenium = new DefaultSelenium(`
`  "localhost", 4444, "*firefox",`
`  "http://www.compendiumdev.co.uk");`
 	
`  selenium.start();`
`}`
 	
`@AfterClass`
`public static void tearDownAfterClass()`
`  throws Exception {`
`  selenium.close();`
`  selenium.stop();`
`}`
 	
`@Before`
`public void setUp() throws Exception {`
`  selenium.open("/selenium/basic_ajax.html");`
`}`
	`static SeleniumManagerEnd sm;`
	`static Selenium selenium;`
	`BasicAjaxPage basicAjax;`
	
	`@BeforeClass`
	`static public void startServer()`
	`  throws IOException{`
	`  sm = new SeleniumManagerEnd();`
	`  sm.start("http://www.compendiumdev.co.uk");`
	
	`  selenium = sm.getSelenium();`
	`}`
	
	`@AfterClass`
	`static public void stopServer(){`
	`  sm.stop();`
	`}`
	
	`@Before`
	`public void setUp() throws Exception {`
	`  basicAjax = new BasicAjaxPage(selenium);`
	`  basicAjax.open();`
	`}`

enterAnInvalidValueInBlurInput

```
@Test
public void enterAnInvalidValueInBlurInput(){
	selenium.type("xpath=//input[@id='lteq30']", "45");
	selenium.select("combo1", "Desktop");
	assertFalse(selenium.isAlertPresent());
}
```

Becomes:

```
@Test
public void enterAnInvalidValueInBlurInput(){
	basicAjax.typeInputValue("45");
	basicAjax.selectCategory("Desktop");

	assertFalse(basicAjax.hasShownValidationAlert());
}
```

This test is a simple matter of creating some 'setter' methods in basicAjax and an accessor method to allow the test to assert values.

enterAnInvalidValueInBlurInputAndTriggerValidation

```
@Test
```

```
public void enterAnInvalidValueInBlurInputAndTriggerValidation(){
        selenium.type("xpath=//input[@id='lteq30']", "45");
        selenium.fireEvent("xpath=//input[@id='lteq30']", "blur");
        assertTrue(selenium.isAlertPresent());
        assertEquals("Enter a value less than 30", selenium.getAlert());
}
```

Becomes:

```
@Test
public void enterAnInvalidValueInBlurInputAndTriggerValidation(){
        basicAjax.typeInputValue("45");
        basicAjax.triggerInputValidation();

        assertTrue(basicAjax.hasShownValidationAlert());
        assertEquals("Enter a value less than 30",
                        basicAjax.getValidationAlertMessage());
}
```

Here I had to take a decision about where to trigger the validation. I could have triggered it in the typeInputValue method, but I decided, for this test, to keep the two methods separate.

This does not mean that I always have to do this and it will be a simple matter, later, if required, to create a method which does both: type the value, trigger the validation. Again, I will wait until a test itself needs the combined method before constructing it. And I won't spend any time wondering if this refactoring was the 'best' I could do or not. It will become clear in later usage scenarios if this was a useful refactoring or not.

The following tests were deleted from the refactored test class as they added no value to the final set of tests, the were simply methods which illustrated how to incorporate JavaScript in the test cases:

- showHTMLOfCombo1WithGetEval
- showValueOfInjectedJavaScriptFromRunScriptExpectedException
- showValueOfInjectedJavaScriptFromRunScript
- showValueOfInjectedJavaScriptFromAddScript

The Page Objects

BasicHTMLForm.java

The page object created during the HTML_form_tests.java and Static_HTML_tests.java refactoring can be found below.

All of the methods were built initially by using Eclipse's "create method X" option when fixing errors on the test. Then the code was copied in from the test. I then changed the name of the parameters to make them more representative of their function. In some cases I had to change the return type from 'Object' to the appropriate type required.

Have a read through of the code and see if you identify any possible refactorings in this code.

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;

import com.thoughtworks.selenium.Selenium;

public class BasicHTMLForm {

        Selenium selenium;
```

```
public BasicHTMLForm(Selenium selenium) {
        this.selenium = selenium;
}

public HTMLFormResultsPage clickSubmitButton() {
        selenium.click("//input[@name='submitbutton' and @value='submit']");
        selenium.waitForPageToLoad("30000");
        return new HTMLFormResultsPage(selenium);
}

public void submitForm() {
        selenium.submit("//form[@id='HTMLFormElements']");
        selenium.waitForPageToLoad("30000");
}

public void typeUserName(String userName) {
        selenium.type("username",userName);
}

public void typePassword(String password) {
        selenium.type("password",password);
}

public void typeComments(String comments) {
        selenium.type("comments", comments);
}

public void typeFilename(String fileName) {
        selenium.type("filename", fileName);
}

public void clickCheckBox(String checkBoxNumber){
        selenium.click("//input[@name='checkboxes[]' and @value='cb" +
                        checkBoxNumber +"']");
}

public void clickRadioButton(String radioButtonNumber) {
        selenium.click("//input[@name='radioval' and @value='rd" +
                        radioButtonNumber + "']");
}

public void removeMultiSelection(String selectionID) {
        selenium.removeSelection("multipleselect[]", "label=Selection Item " +
                selectionID);
}

public void selectMultiSelection(String selectionID) {
        selenium.addSelection("multipleselect[]", "label=Selection Item " +
                selectionID);
}

public void open() {
        selenium.open("/selenium/basic_html_form.html");
}

public void selectDropDownItem(String itemID) {
        selectDropDown("Drop Down Item " + itemID);
}

public void selectDropDown(String itemLabel) {
        selenium.select("dropdown", "label=" + itemLabel);
}

public void setHiddenField(String newHiddenFieldValue) {
        selenium.getEval(
                "this.browserbot.findElement(\"name=hiddenField\").value=\"" +
                 newHiddenFieldValue + "\"");
}

public String getUserName() {
        return selenium.getValue("username");
}

public String getPassword() {
        return selenium.getValue("password");
}
```

```
	public String getFileName() {
		return selenium.getValue("filename");
	}

	public String getComments() {
		return selenium.getValue("comments");
	}

	public boolean isCheckBoxSelected(String checkBoxIDNumber ) {
		return selenium.isChecked("//input[@name='checkboxes[]' and @value='cb" +
			checkBoxIDNumber +"']");
	}

	public void setChecked_CheckBox(String checkBoxIDNumber) {
		selenium.check("//input[@name='checkboxes[]' and @value='cb" +
			checkBoxIDNumber + "']");
	}

	public void setUnChecked_CheckBox(String checkBoxIDNumber) {
		selenium.uncheck("//input[@name='checkboxes[]' and @value='cb" +
			checkBoxIDNumber + "']");
	}

	public boolean isRadioButtonSelected(String radioButtonIDNumber) {
		return selenium.isChecked("//input[@name='radioval' and @value='rd" +
			radioButtonIDNumber + "']");
	}

	public void setChecked_RadioButton(String radioButtonIDNumber) {
		selenium.check("//input[@name='radioval' and @value='rd" +
			radioButtonIDNumber + "']");
	}

	public String[] getAllDropDownValues() {
		return selenium.getSelectOptions("dropdown");
	}

	public int getCurrentDropDownSelectionIndex() {
		// get the index of the currently selected item in the dropdown
		String currentDropDownItem = selenium.getSelectedIndex("dropdown");
		return Integer.valueOf(currentDropDownItem);
	}

	public boolean isADropDownItemSelected() {
		return selenium.isSomethingSelected("dropdown");
	}

	public String getTitle() {
		return selenium.getTitle();
	}
}
```

Possible refactorings

Most of the code is simple enough that it will work fine if left without refactoring, I note however that:

- My locators for checkbox, multi-select, radio buttons, username, password, etc. have all been repeated in the code. I try to avoid repeated code, so would refactor this.
- I would create a constant for each of the locator strings.

For the more complicated locators, such as checkbox and radio buttons, I would use a String.format, using a %s representing the location of the parameter to add to the locator.

e.g.

from the function

```
public void setUnChecked_CheckBox(String checkBoxIDNumber) {
	selenium.uncheck("//input[@name='checkboxes[]' and @value='cb" +
		checkBoxIDNumber + "']");
```

```
}
```

I will start by making that take the form I want:

```
public void setUnChecked_CheckBox(String checkBoxIDNumber) {
       selenium.uncheck(String.format("//input[@name='checkboxes[]' and @value='cb
%s']",checkBoxIDNumber));
}
```

Then select the string to refactor into a constant. And use the Eclipse refactoring "Extract Into Constant". This will create a constant with the string that I can use in the other methods that access checkboxes.

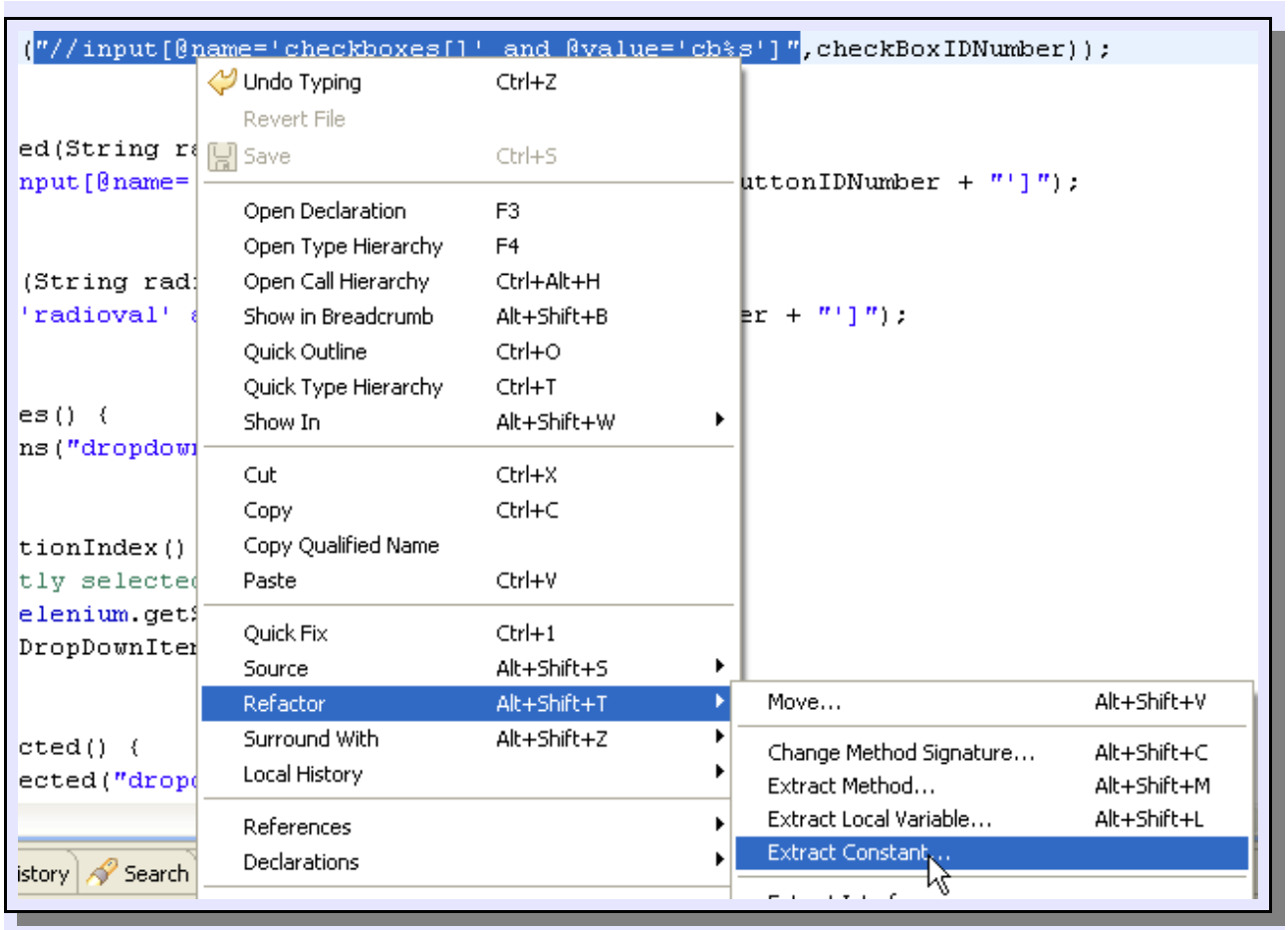


Figure 29.1 : Refactor String into a constant

And in the wizard that follows I type in the name of the constant to create.

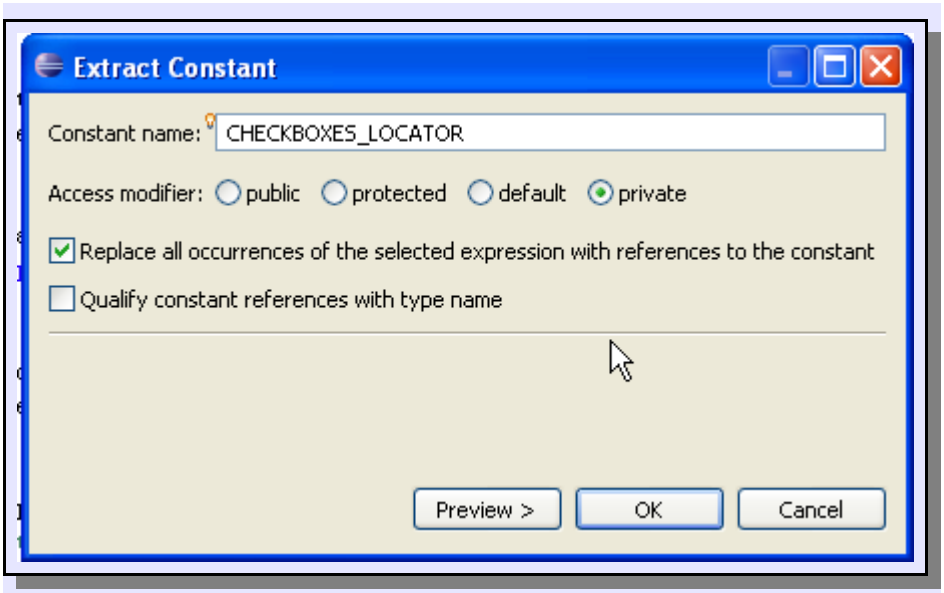


Figure 29.2 : Enter a name for the constant

This refactoring gives us:

```
private static final String CHECKBOXES_LOCATOR = "//input[@name='checkboxes[]' and
@value='cb%s']";

public void setUnChecked_CheckBox(String checkBoxIDNumber) {
        selenium.uncheck(String.format(CHECKBOXES_LOCATOR,checkBoxIDNumber));
}
```

I then amend all the places which previously had the locator to instead use:

```
String.format(CHECKBOXES_LOCATOR,checkBoxIDNumber)
```

This has the advantage that should the locator to access the checkbox change, I only have a single place to remember to change the locator, which will have the knock on effect of fixing all methods that access the checkbox.

Based on the current needs of the tests, and given what we have currently learned, other than these changes I would probably keep this file as it is for just now.

Note:

You can see this refactoring fully completed in the BasicHTMLForm.java class in package com.eviltester.seleniumtutorials.refactored.pageObjectModel

The initial page object model, prior to this additional refactoring is in BasicHTMLForm.java in com.eviltester.seleniumtutorials.refactored.pageObjectModel.notFullyRefactored

HTMLFormResultsPage.java

This Java class was built using the exact method described above in BasicHTMLForm.java

One thing I did have to change was the getFilename method.

The getFilename method illustrates an interesting choice for me as an automator building an

abstraction layer. I will discuss this method, after the source-code listing.

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;

import com.thoughtworks.selenium.Selenium;

public class HTMLFormResultsPage {

	Selenium selenium;

	public HTMLFormResultsPage(Selenium selenium) {
		this.selenium = selenium;
	}

	public String getTitle() {
		return selenium.getTitle();
	}

	public String getUserName() {
		return selenium.getText("_valueusername");
	}

	public boolean hasHeading() {
		return selenium.isTextPresent("Submitted Values");
	}

	public boolean hasComments() {
		return selenium.isElementPresent("_comments");
	}

	public boolean hasFilename() {
		return selenium.isElementPresent("_filename");
	}

	public int getCheckboxCount() {
		return selenium.getXpathCount(
			"//div[@id='_checkboxes']/ul/li").intValue();
	}

	public BasicHTMLForm goBackToForm() {
		// click to go back to the form
		selenium.click("back_to_form");
		selenium.waitForPageToLoad("30000");
		return new BasicHTMLForm(selenium);
	}

	public String getFilename() {
		if(hasFilename()){
			return selenium.getText("_valuefilename");
		}else{
			return selenium.getText("xpath=//body/p[2]/strong");
		}
	}

	public boolean hasPassword() {
		return  selenium.isElementPresent("_password");
	}
}
```

Discussion of getFilename

```
public String getFilename() {
	if(hasFilename()){
		return selenium.getText("_valuefilename");
	}else{
		return selenium.getText("xpath=//body/p[2]/strong");
	}
}
```

My abstraction layer methods are 'logical' rather than 'physical' in that they describe what will happen, but not how. So getFilename is to return the filename text, regardless of the page

structure. The test page to automate is structured differently depending on whether a filename was entered or not.

I could take the view that the application needs to be more testable by being made consistent. But in the short term I need to implement the method to support the test so I add a conditional into the method implementation so, regardless of the state of the page, the test can still get the information it needs.

I suspect that were I to expand the tests to have more tests with optional entries that I would have to take the above conditional method approach for many of the get*X* methods on the results page.

BasicAjaxPage.java

There was nothing new in the BasicAjaxPage.java refactoring. This was a simple cut and paste building exercise.

```
package com.eviltester.seleniumtutorials.refactored.pageObjectModel;

import com.thoughtworks.selenium.Selenium;

public class BasicAjaxPage {

	Selenium selenium;

	public BasicAjaxPage(Selenium selenium) {
		this.selenium = selenium;
	}

	public void open() {
		selenium.open("/selenium/basic_ajax.html");
	}

	public void typeInputValue(String inputValue) {
		selenium.type("xpath=//input[@id='lteq30']", inputValue);
	}

	public void selectCategory(String string) {
		selenium.select("combo1", "Desktop");
	}

	public boolean hasShownValidationAlert() {
		return selenium.isAlertPresent();
	}

	public void triggerInputValidation() {
		selenium.fireEvent("xpath=//input[@id='lteq30']", "blur");
	}

	public String getValidationAlertMessage() {
		return selenium.getAlert();
	}
}
```

Chapter 30: Page Object Models Summary

Now that you have seen Page Object Models in action, we can cover a little theory and thoughts on the topic.

What is a Page Object Model?

I use the term Page Object Model because what we build is a collection of Objects, each of which models a Page or a Component on a page. The methods exposed by the Page Object are the interface that the user has with the page:

- accessors – get the values
  - getTitle()
  - getDisplayedUsername()
- setters – type and select values
  - typeUserName("alan");
- actions – e.g. clicking things
  - clickLoginButton();
  - submitLoginForm();
- answers – answers to questions we might ask about the page – like accessors but a slightly higher level of abstraction, In this way our tests can assert the truth or falsehood of the answer.
  - isTitle("a title"); // this would use the getTitle method and return a boolean rather than asserting the value.

Grow an abstraction layer

I have had more success 'growing' an abstraction layer, by creating a simple layer and refactoring it over time to meet the evolving needs of the test automation.

- Sometimes I create the selenium statements in the test, to get the test working and then refactor them out.
- Sometimes I start with what I want the test to look like, and grow the abstraction layer as I write the test. I prefer this method as it 'feel's more like test driven development of tests.

Refactor the Page Objects internally

Treat all your test code as production quality code. So refactor all your code and have it as clean and waste free as possible. You do not initially need complicated mechanisms to move out your locators into text files, property files etc. Refactor your code into different levels of abstraction and move to higher levels of abstraction as you need to over time.

e.g.

- move literal strings into constants
- use String.Format to parametrize constants
- refactor into higher level methods
    - e.g. Example 1 below might lead to the creation of a loginAs method, which opens the page, types in the username and password and clicks the login button. You might choose to make the typeUsername and other methods private. Or keep the public for use in more detailed tests. The construction of abstraction layers is a set of choices which are up to you.
        - ManageAccountPage loginAs(String username, String password)

Create Domain Level Abstractions

Once you have a collection of Page Objects, consider the use of a User class and refactor into a higher level abstraction. e.g.

Instead of:

```
// example 1
LoginPage loginPage = new LoginPage(selenium);
loginPage.open();
loginPage.typeUsername("alan");
loginPage.typePassword("mypass");
ManageAccountPage manageAccount = loginPage.clickLoginButton();
assertTrue(manageAccount.isTitle("Manage Account");
```

consider refactoring the above into a method on a User class:

```
// example 2
User me = new User(selenium);
user.setUsername("alan");
user.setPassword("mypass");
user.logsIn();
```

Behind the scenes, the user maintains its state and provides high level methods to access the system.

There are dangers with this approach as it can lead to a very large user class with lots of methods.

But I find that judicious use of this approach works well and helps makes the tests more readable.

So in addition to modelling the structure of your application. You create usage models which allow you to create more literate tests.

Build the abstraction layers appropriate to you which:

- help make the tests readable, and maintainable,
- model the application under tests
- model the use of the application
- model the style of testing you are doing

Lessons learned

Below are a few hints and tips relating to abstraction models:

- return other page objects from methods

- model state as different methods
  - loginForm.clickLoginButton()
  - loginForm.clickLoginButtonExpectingFailedLogin()
- return “this” for syntactic sugar methods
  - e.g. and(), then()
    - both of the methods would contain the code {return this;}
- grow the model organically through refactoring
- do not use asserts in the page object model
- collate page objects into a domain model (which can use asserts)
- refactor heavily within the page objects
  - split out components into new objects

Recommended Don'ts

- Don't try and create a whole abstraction layer at the start. Let it grow.

Related Study

- http://code.google.com/p/selenium/wiki/PageObjects A short discussion on page objects with examples using WebDriver.
- http://www.youtube.com/watch?v=tGu1ud7hk5I watch this video from GTAC 2007 by Simon Stewart on Web Driver. It provides a good overview of abstraction layers for testing and discusses web driver in detail, which is the important base technology for Selenium 2.x
- Growing Object Oriented Software Guided by Tests by Steve Friedman and Nat Pryce
  - An excellent book about evolving software with many important lessons for maintaining tests and abstractions.
  - http://www.growing-object-oriented-software.com/

Chapter 31: Data Driven Tests in JUnit

Data Driven testing is an important method of increasing data coverage without writing a lot of tests.

To support Data Driven testing we will look at JUnit's Parameterized class runner.

Basic Data Driven Testing

I have created a small application to make the examples for the use of Data Driven testing simpler at http://www.compendiumdev.co.uk/selenium/calculate.php

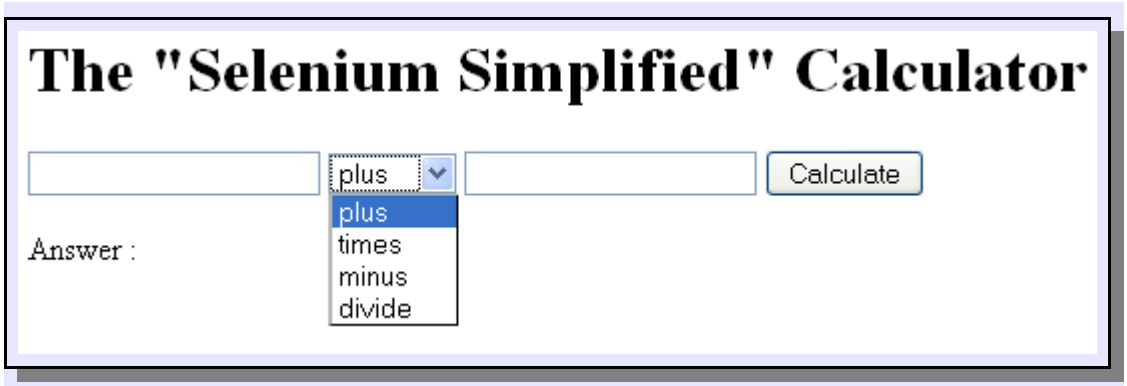


Figure 31.1 : A simple calculator example for data driven testing

In the code examples, you will see that I have already created a page object for the calculator and am using it in the test.

> **Note:**
>
> **The code for this example is in com.eviltester.seleniumtutorials.chap31.datadriven as CalculateTwoNumbersTests.java**
>
> **The page object CalculateForm.java, can be found in the package: com.eviltester.seleniumtutorials.chap31.datadriven.pageObjectModel**

This calculate.php form has two input fields for entering numbers, a central drop down for the calculation function to perform. A Calculate button which submits the form and displays the answer.

This has a very simple test path:

- type first number
- select function
- type second number
- press calculate button
- check answer is correct

But we want to vary the data across this test path to choose different numbers, and different functions.

So I will show the important new code in snippets, explaining as I go, and then we can look at the whole code.

JUnit Parameterized Class Runner And Constructor

At a high level, we will create a new test class which uses a parameterized runner. We will define a data provider for the test class which returns a collection of objects. For each object in the collection, the test runner will instantiate our test class, passing in the objects as parameters to the test class constructor, and then execute all the tests in the test class.

First, when I declare my class I annotate it with @RunWith(Parameterized.class)

```
@RunWith(Parameterized.class)
public class CalculateTwoNumbersTests {
```

This is a different runner, which does a lot of magic under the covers to create our test class as many times as our data provider returns data, and runs all the tests in our test class with the parameters.

```
private String number1;
private String function;
private String number2;
private String answer;

public CalculateTwoNumbersTests(
                                String num1, String function, String num2, String answer)
{
        this.number1 = num1;
        this.function = function;
        this.number2 = num2;
        this.answer = answer;
}
```

I need to pass the data for the tests as a set of parameters to a constructor. Then to access the data in the tests I need to store the parameters in class field variables.

So where does the data come from?

Data supplied by an @Parameters annotated method

```
@Parameters
public static Collection data() {
      return Arrays.asList(
                      new Object[][] {
                                    { "1", "plus", "1", "2" },
                                    { "2", "times", "2", "4" },
                                    { "5", "divide", "2", "2.5" },
                                    { "10", "minus", "4", "6" },
                      }
              );
}
```

I create a method in the test class (here I called it data()). I annotate this method with the @Parameters annotation from JUnit. This method returns a collection of Arrays. Each Array has the data that I will use as the parameters.

In the above example I create a multidimensional array of objects “new Object[][]” which I convert to a list of arrays. The test runner will call the method annotated by @Parameters and for each array in the list, will pass the values through to the test constructor as parameters.

I have kept things simple in the example above by creating an array of data in the code. I could just as easily have had the data() method read a file and construct the data array from a file, e.g. A CSV file or XML file (you will see this in a few pages time).

A Data Driven Test

This is all the new magic. Thereafter our tests are as they have always been:

```
@BeforeClass
static public void startServer() throws IOException{
        sm = new SeleniumManager();
        sm.start("http://www.compendiumdev.co.uk");
        selenium = sm.getSelenium();
}

@Test
public void test_calculate_two_values(){
        calculate = new CalculateForm(selenium);
        calculate.open();

        calculate.setNumber1(this.number1);
        calculate.setFunction(this.function);
        calculate.setNumber2(this.number2);

        calculate = calculate.doCalculation();

        assertEquals(this.answer,calculate.getAnswer());
}

@AfterClass
static public void stopServer(){
        sm.stop();
}
```

Here you can see the @Test annotated method using the field variables that were instantiated by the constructor, so they have the values that were passed in from our data method.

Running a Data Driven Test

Running this test will show a slightly different JUnit test display:

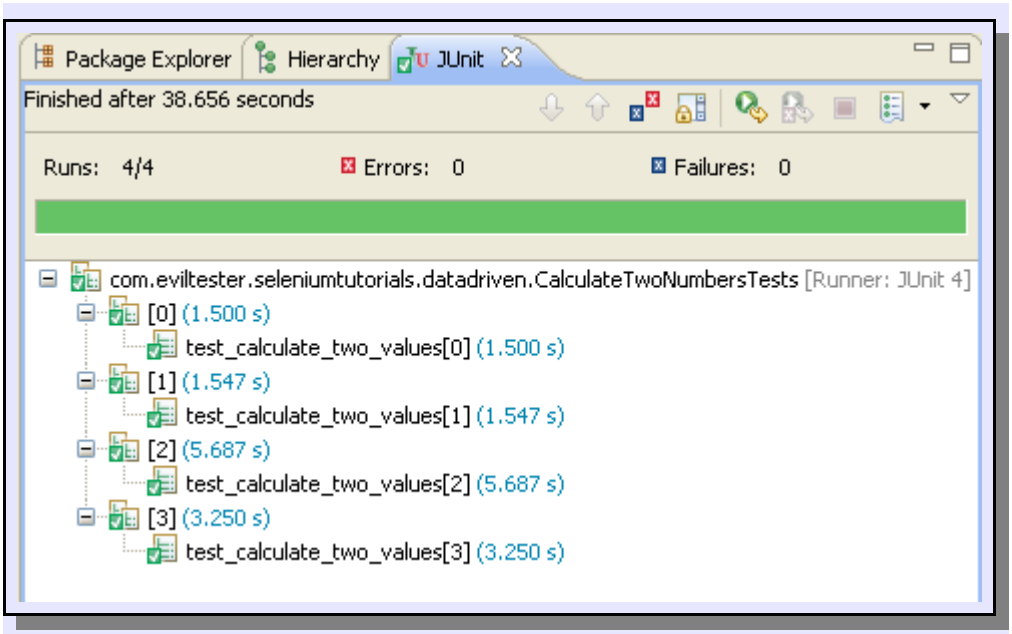


Figure 31.2 : The Parameterized test results

Here you can see that the name of the test is displayed, with the index of the parameter object array used as parameters in the constructor.

So test_calculate_two_values[0] is the first array of data in our Object[][] i.e. { "1", "plus", "1", "2" }

The full code for the test

```
package com.eviltester.seleniumtutorials.datadriven;
import static org.junit.Assert.assertEquals;
import java.io.IOException;
import java.util.Arrays;
import java.util.Collection;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;
import org.junit.runners.Parameterized.Parameters;
import com.eviltester.seleniumtutorials.datadriven.pageObjectModel.CalculateForm;
import com.eviltester.seleniumutils.SeleniumManager;
import com.thoughtworks.selenium.Selenium;

@RunWith(Parameterized.class)
public class CalculateTwoNumbersTests {

        static SeleniumManager sm;
        static Selenium selenium;
        static CalculateForm calculate;
        private String number1;
        private String function;
        private String number2;
        private String answer;

    public CalculateTwoNumbersTests(
                        String num1, String function, String num2, String answer){
        this.number1 = num1;
        this.function = function;
        this.number2 = num2;
        this.answer = answer;
    }

    @Parameters
    public static Collection data() {
        return Arrays.asList( new Object[][] { { "1", "plus", "1", "2" },
                                               { "2", "times", "2", "4" },
                                               { "5", "divide", "2", "2.5" },
                                               { "10", "minus", "4", "6" }, });}

        @BeforeClass
        static public void startServer() throws IOException{
                sm = new SeleniumManager();
                sm.start("http://www.compendiumdev.co.uk");
                selenium = sm.getSelenium();
        }

        @Test
        public void test_calculate_two_values(){
                calculate = new CalculateForm(selenium);
                calculate.open();
                calculate.setNumber1(this.number1);
                calculate.setFunction(this.function);
                calculate.setNumber2(this.number2);
                calculate = calculate.doCalculation();
                assertEquals(this.answer,calculate.getAnswer());
        }

        @AfterClass
        static public void stopServer(){
                sm.stop();
        }
}
```

You can see the new imports that I had to add for JUnit 4 parameterization functionality.

Multiple Tests

It is possible to have more than one @Test annotated method in the parameterized test class. e.g. I could add another (this test checks the calculation process by using the answer in the calculation):

```
@Test
public void test_reverse_calculate_two_values(){
	calculate = new CalculateForm(selenium);
	calculate.open();

	calculate.setNumber1(this.answer);

	if(this.function.compareTo("plus")==0)
		calculate.setFunction("minus");

	if(this.function.compareTo("minus")==0)
		calculate.setFunction("plus");

	if(this.function.compareTo("divide")==0)
		calculate.setFunction("times");

	if(this.function.compareTo("times")==0)
		calculate.setFunction("divide");

	calculate.setNumber2(this.number2);

	calculate = calculate.doCalculation();

	assertEquals(this.number1,calculate.getAnswer());
}
```

This would be run with the same parameters.

If you include an @Test method in a parameterized test class which does not use the parameters it will still be run as many times as there are items in the List.

Reading Data From Tab Delimited Files

To show the basic functioning of data driven tests our data was supplied from a hard coded collection.

For ease of maintenance some people prefer to keep the data separate from the code. So in this next example we will see how to load in the data from a tab delimited file.

The benefit of a tab delimited file is that it is very easy to write a method to read and parse the file without worrying about adding additional libraries to our tests.

So I will need to create a readTabDelimFile method which, when given a file name, returns the data as a collection of String Arrays. A very quick google search revealed a simple example of reading a tab delimited file http://progzoo.net/ht05IO/320TDF.xml so I simply customize their code to match our needs:

```
public static Collection<String[]> readTabDelimFile(String filename)
                    throws NumberFormatException, IOException {

	ArrayList<String[]> lines = new ArrayList<String[]>();
	BufferedReader fh = new BufferedReader(new FileReader(filename));
	String s;

	while ((s=fh.readLine())!=null){
		String f[] = s.split("\t");
		lines.add(f);
	}
	fh.close();
	return lines;
}
```

Then we have to make the @Parameterized method read the data from a tab Delimited file.

```
@Parameters
public static Collection data() throws NumberFormatException, IOException {
	return readTabDelimFile("resources\\datafiles\\calculate2numbers.tab");
}
```

Then I have to create the data file. Which I do by creating a folder called resources at the top level of my project, with a datafiles sub-folder beneath that.

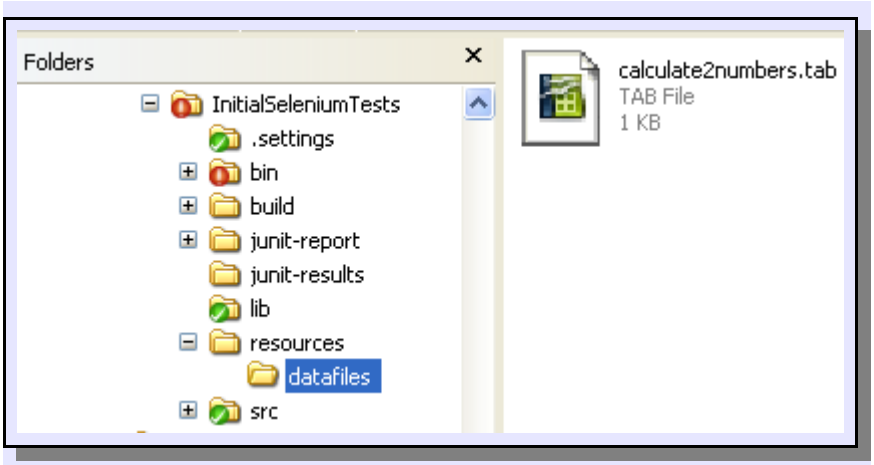


Figure 31.3 : Resources folder structure

I create the tab delimited file in a text editor with each line representing a full line of data, and with each value separated from the next with a [tab] character.

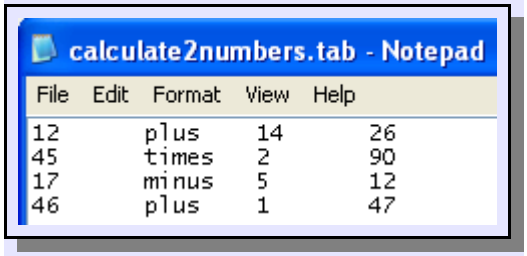


Figure 31.4 : Using Notepad to edit a tab separated file

Note:

The data file is in the resources\datafiles folder, the code for data driven from a file is contained in the test class CalculateTwoNumbersTestsTabFile.java in package com.eviltester.seleniumtutorials.chap31.datadriven

General Hints

In the section above I tried to give an overview of the type of approach I use when trying to expand my tests.

1. I get the basics working (array provided by code)
2. Identify what I want to do next (use tab separated files)
3. Find simple code on the internet (find simple code examples on the internet to reuse and

build on)

4. Get the code working then customise to do what I need

If I were now to expand the test to use a spreadsheet, I would start looking for Java libraries to use for reading spreadsheets. Get those libraries working with simple examples using System.out.println to print the values from the spreadsheet and only once I was comfortable that I understood the library would I incorporate it into my test.

> **Note:**
>
> **An alternative test framework called TestNG has much better support for data driven testing and in the results shows the actual parameters passed through to the test. While it is easy to move from JUnit to TestNG, this beginners guide will only cover JUnit. For more information on TestNG you can visit http://testng.org/doc/index.html**

Related Reading:

- http://weblogs.java.net/blog/johnsmart/archive/2009/11/28/data-driven-tests-junit-4-and-excel John Ferguson Smart has a good example of how to use Excel as the data source for the data driven tests.
- http://junit.org/apidocs/org/junit/runners/Parameterized.html the official documentation of the parameterized class
- http://ourcraft.wordpress.com/2008/08/27/writing-a-parameterized-junit-test/ a good description from Daniel Meyer of how to understand the parameterized test class

Chapter 32: Screen Capture on Failure

The last time we saw screen capture was when we inherited from the SeleneseTestCase object back on page 74 in "Chapter 7 - Use Attach Source to see the Selenium Driver Source Code" in section "Creating Screenshots on Failure with SeleneseTestCase".

Now we will learn to do it with the JUnit TestWatchman functionality so that we can recover some of the functionality that we lost when we went on our own and stopped using the helper Object.

I have left this chapter close to the end of the book because it contains the most complicated code that I will cover.

Note: to follow this code you need an up to date version of JUnit. If your version of Eclipse does not have a high enough version then you need to add a recent version of JUnit to your project.

- Download the JUnit jar from the JUnit website.

- http://github.com/KentBeck/junit/downloads

- Add this jar file to somewhere in your workspace e.g. a \lib or \libs folder

- Amend the Java Build Path in Eclipse from the project properties and on the 'Libraries' tab [Add External Jars] and point to the JUnit jar you downloaded.

- Remove any other JUnit references from the build path.

This code also builds on the data driven test knowledge that we gained in the previous chapter.

Basically I will create a test, which uses all the browsers that support screen capture functionality under Selenium, do something that fails, and write the screenshots to a folder.

So before you see the complete listing I will go through the important details.

First, I have made this a data driven test

```
@Parameters
public static Collection testData(){
	return Arrays.asList(new Object[][]{ {"firefox"},
	                                     {"chrome"},
	                                     {"iexplore"},
	                                     {"googlechrome"},
	                                     {"iexploreproxy"}});
}

public ScreenCaptureOnFailureWithWatchmanTests(String currentBrowser){
	this.currentBrowser = currentBrowser;
}
```

In this case my "data" is the browser name, so it demonstrates one way of making your tests run on multiple browsers. This is not the way I would recommend to you, but for the purposes of illustrating screenshot taking, this seemed like a convenient approach to take.

My constructor simply stores the browser name we want to use, and I use this in the test when I start the server and open the browser to view the home page, http://www.compendiumdev.co.uk, the test then checks for the wrong title to make the test fail.

```
@Test
public void captureScreenshots(){
	startServerAndOpenRoot(currentBrowser);
	assertEquals(WRONG_TITLE,selenium.getTitle());
}
```

We need to have a quick understanding of JUnit 4 rules.

JUnit 4 allows us to easily customize the behaviour of the test runner locally by the use of rules.

So if I add the following into my test class:

```
@Rule
public MethodRule watchman= new TestWatchman() {

	@Override
	public void failed(Throwable e, FrameworkMethod method) {
		captureScreenShots(	screenShotsFolder,
					currentBrowser,
				method.getName() + "_" + e.getClass().getSimpleName());
		stopSelenium();
	}

	@Override
	public void succeeded(FrameworkMethod method) {
		stopSelenium();
	}
};
```

I have created a custom rule, with a TestWatchman which allows me to augment the functionality of the test runner. In this case I am adding some additional functionality for the 'failed' event of the test runner and the succeeded event.

Here, if the test fails, our failed method above will override the JUnit test runner rule and will call our captureScreenShots method (which I will explain later). I pass to the captureScreenShots method the name of the @Test annotated method ("method.getName") which failed, and the name of the error which caused the test to fail ("e.getClass().getSimpleName()");

I also override the successful test ending, and in this case all I do is call the stopSelenium local method.

And that was it, that was all I had to do to trap the failure event on test running. Everything else in my test exists to support the screenshots.

I also have to create a folder hierarchy to store the screenshots in. I have chosen to create a main /screenshots folder and beneath it, a date based folder so that every test run has a folder to store its tests in e.g.:

/screenshots/2010-05-10-13-22

Then in the date based folder I store the screenshots and prefix each screenshot with the name of the browser in use.

So at the end of a test there would be a number of files:

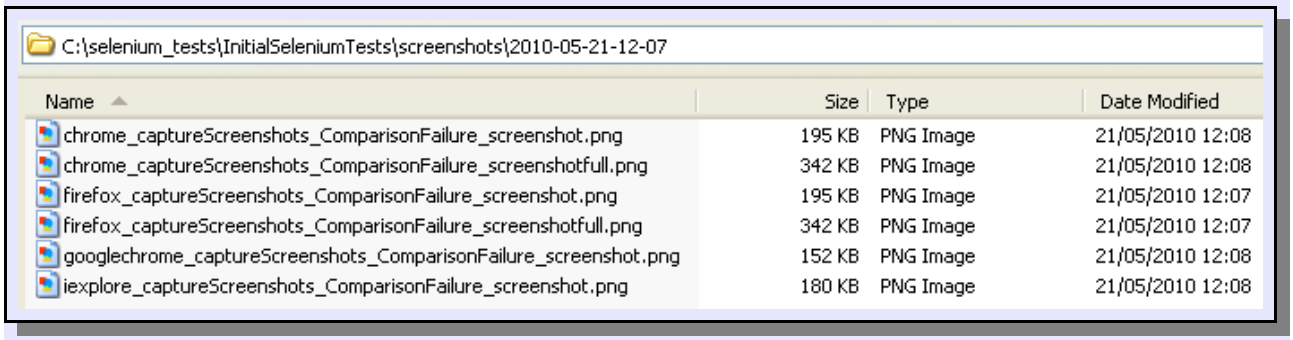


Figure 32.1 : List of files generated by the test

I have a private method called in the @BeforeClass start up method to create the folder structure:

```
static private String createNowfolder() {
        String screenShotsFolder =
                System.getProperty("user.dir") +
                File.separator + "screenshots" + File.separator;
        SimpleDateFormat sdfmth = new SimpleDateFormat("yyyy-MM-dd-HH-mm");
        Calendar cal = Calendar.getInstance();
        screenShotsFolder = screenShotsFolder + sdfmth.format(cal.getTime());
        new File(screenShotsFolder).mkdirs();
        return screenShotsFolder + File.separator;
}
```

Then there is the important captureScreenShots method which captures the screenshot and saves it to a file.

There are two capture screenshot methods available from Selenium.

- captureScreenshot
  - captures the entire screen, browser agnostic
- captureEntirePageScreenshot
  - captures the full page within the browser, only works on a subset of browsers: iexploreproxy, firefox, chrome

```
private void captureScreenShots(     String screenShotsFolder,
                                     String browser,
                                     String fileNameAppend) {

        selenium.windowMaximize();

        try{ Thread.sleep(1000);} catch (Exception e) {}

        String filename =     screenShotsFolder + browser + "_" +
                              fileNameAppend + "_screenshot.png";

        selenium.captureScreenshot(filename);

        try{ selenium.captureEntirePageScreenshot(
                       filename.replace(".png","full.png"),""); }catch(Exception e){}
}
```

In this method I first maximize the browser window so that it fills the screen. I do this because the captureScreenshot method captures the full screen, not just the browser details.

Because browsers might take a little time to maximize the screen and render it properly, I add a pause in the test using the "Thread.sleep(1000)" command. I wrap this in a try catch block to ignore the exception that Thread.sleep might throw.

I then build the filename path for the general screenshot.

For the captureEntirePageScreenshot I wrap it in a try catch block. This way, when an unsupported browser is used and an exception is thrown, it will be caught and ignored and no screenshot will be produced.

So the complete test listing is:

```
package com.eviltester.seleniumtutorials.chap32.screenCapture;

import static org.junit.Assert.*;
import java.io.File;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Arrays;
```

```
import java.util.Calendar;
import java.util.Collection;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Test;
import org.openqa.selenium.server.SeleniumServer;
import com.thoughtworks.selenium.DefaultSelenium;
import com.thoughtworks.selenium.Selenium;

// for the @Rule annotation
import org.junit.rules.MethodRule;
import org.junit.Rule;
import org.junit.rules.TestWatchman;
import org.junit.runners.model.FrameworkMethod;

import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;
import org.junit.runners.Parameterized.Parameters;

@RunWith(Parameterized.class)
public class ScreenCaptureOnFailureWithWatchmanTests {

        static SeleniumServer server;
        static Selenium selenium;
        static String screenShotsFolder;
        String currentBrowser;

        final String WRONG_TITLE =
                "WRONG_TITLESoftware Testing Essays, Book Reviews and Information";

        @BeforeClass
        public static void startServer() throws Exception{
                server = new SeleniumServer();
                server.start();
                screenShotsFolder = createNowfolder();
        }

        @Rule
        public MethodRule watchman= new TestWatchman() {

                @Override
                public void failed(Throwable e, FrameworkMethod method) {
                        captureScreenShots(   screenShotsFolder,
                                              currentBrowser,
                                method.getName() + "_" + e.getClass().getSimpleName());
                        stopSelenium();
                }

                @Override
                public void succeeded(FrameworkMethod method) {
                        stopSelenium();
                }
        };

        public ScreenCaptureOnFailureWithWatchmanTests(String currentBrowser){
                this.currentBrowser = currentBrowser;
        }

        @Parameters
        public static Collection testData(){
                return Arrays.asList(new Object[][]{{"firefox"},{"chrome"},
                                                    {"iexplore"},{"googlechrome"}});
        }

        @Test
        public void captureScreenshots(){
                        startServerAndOpenRoot(currentBrowser);
                        assertEquals(WRONG_TITLE,selenium.getTitle());
        }

        private void captureScreenShots(String screenShotsFolder,
                                        String browser, String fileNameAppend) {
                selenium.windowMaximize();

                try { Thread.sleep(1000); } catch (Exception e) {}

                String filename = screenShotsFolder + browser + "_" +
```

```
                                fileNameAppend + "_screenshot.png";

                selenium.captureScreenshot(filename);

                try{ selenium.captureEntirePageScreenshot(
                        filename.replace(".png","full.png"),"");   }catch(Exception e){}
        }

        private void startServerAndOpenRoot(String browser) {
                selenium = new DefaultSelenium("localhost",
                                server.getPort(), "*" + browser,
                                "http://www.compendiumdev.co.uk");
                selenium.start();
                selenium.open("/");
        }

        static private String createNowfolder() {
                String screenShotsFolder = System.getProperty("user.dir") +
                                File.separator + "screenshots" + File.separator;

                SimpleDateFormat sdfmth = new SimpleDateFormat("yyyy-MM-dd-HH-mm");
                Calendar cal = Calendar.getInstance();

                screenShotsFolder = screenShotsFolder + sdfmth.format(cal.getTime());

                new File(screenShotsFolder).mkdirs();
                return screenShotsFolder + File.separator;
        }

        public void stopSelenium(){
                selenium.close();
                selenium.stop();
        }

        @AfterClass
        public static void stopServer(){
                server.stop();
        }
}
```

Related Reading:

- http://kentbeck.github.com/junit/javadoc/latest/org/junit/rules/TestWatchman.html The official documentation for the TestWatchman rules
- http://blog.mycila.com/2009/11/writing-your-own-junit-extensions-using.html A blog post by Mathieu Carbou describing how to use the @Rule annotation.
- http://www.sonatype.com/people/2009/10/selenium-part-4/ Patrick Lightbody describes the creation of an alternative JUnit 4 test runner to capture screenshots.

Chapter 33: Run the tests on multiple browsers

There are a lot of ways that I could setup the tests in order to run on multiple browsers, I have listed 3 below. This is not an exhaustive list, just the most obvious candidates:

- Method 1: Multiple browsers with data driven tests
- Method 2: Property Files
- Method 3: System Properties

We covered method 1 in the previous chapter "Chapter 32: Screen Capture on Failure".

Browser Codes

Selenium supports a number of browser codes which we can use when creating a selenium instance

```
selenium = new DefaultSelenium("localhost", 4444, "*iexplore",
"http://www.compendiumdev.co.uk");
```

Starting selenium with an incorrect browser code will force Selenium to give an error message, which also lists the supported browsers.

There are nuances associated with the different codes but we really don't need to understand very much about them.

If you want to use a browser then try the main code for it and see if the tests work:

i.e.

- *firefox Mozilla Firefox
- *iexplore Internet Explorer
- *googlechrome Google Chrome
- *konqueror Linux Konqueror
- *safari Apple Safari
- *opera Opera

If your tests fail to work then try a code which bypasses some of the browser security controls and gives Selenium more leeway with its security bypassing controls. Most of these codes have been retained for backwards compatibility, but sometimes a test that fails to run under the main code can run with one of the earlier codes. Even if you are using the most up to date version of Selenium. It really depends on your local desktop environment, the browser versions installed and the security protocols on your desktop machine:

For Firefox:

- *firefoxproxy
- *pifirefox
- *chrome
- *firefoxchrome

For Internet Explorer:

- *iexploreproxy
- *piiexplore
- *iehta - *you should never really have to use iehta though*

For Safari:

- *safariproxy

Custom is used for browsers that are not supported out of the box by Selenium. I have never needed to use this option. It is explained in the official documentation if you really need to use a custom browser.

- *custom

If you want to understand the reasons for the different codes then the official documentation is a useful place to start. http://seleniumhq.org/docs/05_selenium_rc.html

I will not repeat the various explanations in here because they are well explained in the official documentation and, to be honest, you don't really need to know them in order to use Selenium.

Property Files

A property file is a simple text file. e.g.

```
# the browser to run tests on
browser = *iexplore
```

Any line beginning with # is a comment, any other line is a property. Where the name of the property is to the left of the "=" and the property value is to the right.

Java has a utility class for reading these property files which makes them a very simple method for controlling the configuration of a Java application without re-compiling the application.

One very common way that people have of setting up the Selenium configuration is through property files.

e.g. In our DefaultSelenium call we have a number of items which might be useful to have in a properties file

```
selenium = new DefaultSelenium("localhost", 4444, "*iexplore",
"http://www.compendiumdev.co.uk");
```

The host location, port and browser are all obvious candidates for being put in a properties file rather than into our code.

So we might construct a property file like the following:

```
# the browser to run tests on
browser = *iexplore
host = localhost
port = 4444
```

We save the property file in our root directory for the project, in my case "C:\selenium_tests\InitialSeleniumTests" and although we can name it anything we want, the general convention is to call it <something>.properties so I named it "selenium.properties"

Now we just have to amend our SeleniumManager class to make use of a properties file.

I add the following code to SeleniumManager.java

```
String host;
String port;
String browser;

private void readProperties(){

	Properties props;
	props = new Properties();

	String filePath = "selenium.properties";

	try {
		props.load(new FileInputStream(filePath));
	} catch (FileNotFoundException e) {
		System.out.println("No Properties file found");
		e.printStackTrace();
	} catch (IOException e) {
		System.out.println("Error loading properties file");
		e.printStackTrace();
	}

	this.browser = props.getProperty("browser","*firefox");
	this.host = props.getProperty("host","localhost");
	this.port = props.getProperty("port","4444");
}
```

I added three variables to store the host, port and browser.

Then a new method to read in the values from the properties file.

I create a Properties object, which is the java.utils class for handling property files.

I use the .load method on the properties file to load the properties file.

Then I get the property with the "getProperty" method. The "getProperty" method takes two parameters, the first is the key of the property, the second is the default value if the property is not found.

So we always have default values, even if the property file can't be read, or hasn't been created.

All I have to do then, is update the SeleniumManager .start method to use these field variables.

```
public void start(String baseURL) throws IOException {

	String shutdownCommand =
		"http://%s:%s/selenium-server/driver/?cmd=shutDownSeleniumServer";

	readProperties();

	String stopSeleniumCommand = String.format(shutdownCommand,host,port);

	try {
		seleniumServer = new SeleniumServer();
		seleniumServer.start();
	} catch (java.net.BindException bE){
		System.out.println("could not bind - carrying on");
		if( runHTTPCommand(stopSeleniumCommand)){
			try {
				seleniumServer = new SeleniumServer();
				seleniumServer.start();
			} catch (Exception e) {
				throw new IllegalStateException(
			"Could not stop existing server on blocked port " + port, e);
			}
		}
	}
	catch (Exception e) {
		throw new IllegalStateException("Can't start selenium server", e);
	}

	selenium = new DefaultSelenium(host, new Integer(port), browser, baseURL);
	selenium.start();
}
```

Because we introduced the SeleniumManager class into our tests, we only have to make the changes once, and all the tests that use the SeleniumManager will now be configured by the properties file.

Note:

The code for this version of the SeleniumManager can be found in com.eviltester.seleniumutils as SeleniumManager03PropertiesFile.java

System Properties

An alternative properties mechanism is to use the system properties.

We can set these in our Ant file and in Hudson.

Reading these through Java is even easier than the properties file:

```
private static final String SELENIUM_PORT = "selenium.port";
private static final String SELENIUM_HOST = "selenium.host";
private static final String SELENIUM_BROWSER = "selenium.browser";

public void readSystemProperties() {
	Properties systemProperties = System.getProperties();

	if(systemProperties.containsKey(SELENIUM_BROWSER))
		this.browser = systemProperties.getProperty(SELENIUM_BROWSER);

	if(systemProperties.containsKey(SELENIUM_HOST))
		this.host = systemProperties.getProperty(SELENIUM_HOST);

	if(systemProperties.containsKey(SELENIUM_PORT))
		this.port = systemProperties.getProperty(SELENIUM_PORT);
}
```

Because I will allow system properties to override those set in the properties file I don't need to add any code to handle defaults, as the defaults are handled in readProperties();

So all that remains in SeleniumManager now is to add a call to this method, as shown in this code extract:

```
public void start(String baseURL) throws IOException {

	String shutdownCommand =
		"http://%s:%s/selenium-server/driver/?cmd=shutDownSeleniumServer";

	readProperties();		// get the properties from a file
	readSystemProperties();	// system properties override the properties
...
```

I can set the system properties in the Ant file as the following extract shows:

```
<!-- setup a default browser property, overwrite with -DBROWSER=iexplore etc.-->
<property name="BROWSER" value="firefox" />

<target name="run_param_junit_tests">
	<junit printsummary="yes" fork="yes"
		errorproperty="junit.error" failureproperty="junit.error">

		<sysproperty key="selenium.browser" value="${BROWSER}"/>

		<classpath refid="junit.class.path" />
		<formatter type="xml" />

		<test todir="${testresults}"
		name="com.eviltester.seleniumtutorials.chap22.MultiBrowserExampleTest" />
	</junit>
```

```
        <antcall target="report-junit"/>
        <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
</target>
```

In this extract I create a local property called BROWSER with a default value, and in the run_param_junit_tests target to set the system property "selenium.browser" that will be picked up by the "systemProperties.getProperty(SELENIUM_BROWSER);" command.

This allows us to override the property in our Hudson build so that we can setup multiple Hudson builds, each of which run the tests on a different browser.

We add the properties in Hudson to the "Advanced" fields of the build section:

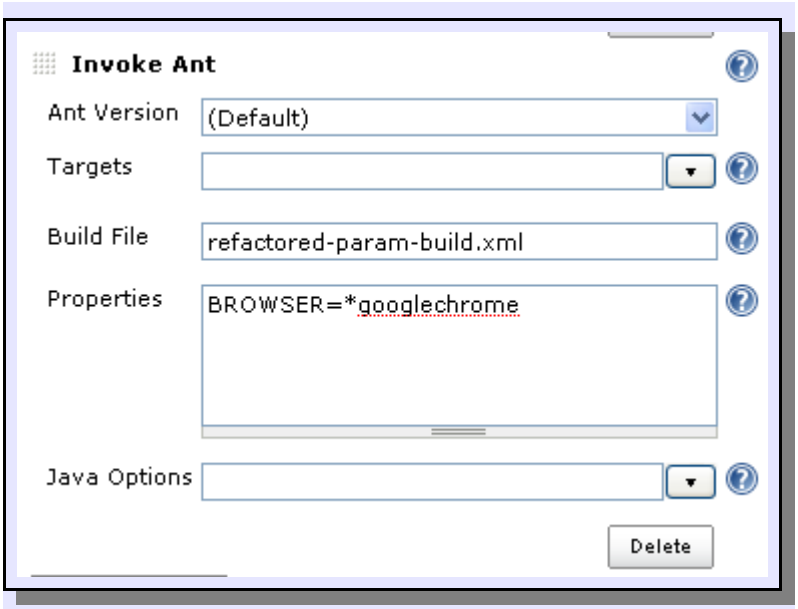


Figure 33.1 : Override the system properties in Hudson

The properties field uses the same text format as our properties file. Note that the properties are case sensitive, so since I used <property name="BROWSER" value="firefox" /> in the Ant file I have to use BROWSER in the properties section.

If I run this build setup then the tests will run under Google Chrome browser. If I remove the properties above then it will use the default setup in the Ant file "firefox".

If I run the tests directly within Eclipse, the system properties will not be set so the selenium.properties file will be used to define the parameters.

Note:

See the refactored-param-build.xml file for example usage.

The SeleniumManager class with this code added is in package com.eviltester.seleniumutils as SeleniumManager04SystemProperties.java

Chapter 34: JavaScript And Dynamic HTML Testing Redux

Or... a little more about JavaScript testing.

For some reason, testing Ajax enabled websites, or dynamic HTML websites can cause initial concern for testers learning automation.

I don't think that this is due to the complexity of using Selenium to test it, really you only have to learn one new command (waitForCondition).

The concern seems to stem from the investigation required to work out what the Ajax or dynamic HTML does and how to identify the requisite states for when the Ajax processing has finished.

Example Dynamic HTML Page

You saw in Chapter 20 Using JavaScript with Selenium the example dynamic HTML page.
http://www.compendiumdev.co.uk/selenium/basic_ajax.html

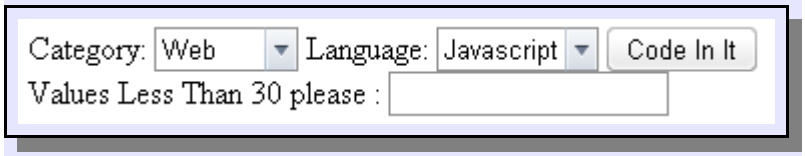


Figure 34.1 : Dynamic HTML form

When the user selects one of the Category values, the 'Language' dropdown is repopulated with a relevant set of languages by making a call to the server and then updating the dropdown with the results passed back from the Server. This client/server interaction is represented on screen by the display of an animated GIF to the left of the "Code in It" button.

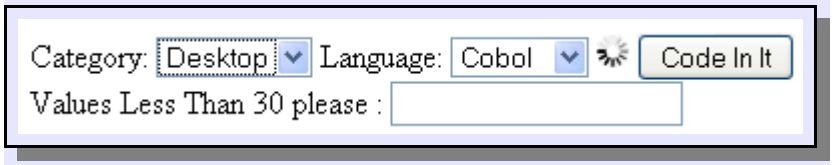


Figure 34.2 : Client/Server interaction animated GIF

So if we write a test, which selects the Category entry and then checks for a particular language being displayed in the Language dropdown. The test will fail because the language drop down will not have been changed.

```
@Test(expected=org.junit.ComparisonFailure.class)
public void chooseACategoryAndCheckTheLanguageThrowsExceptionDueToNoWait(){
        ajaxPage.selectCategory("Server");
        assertEquals("Cobol",ajaxPage.getLanguage());

        ajaxPage.selectCategory("Web");
        assertEquals("Javascript",ajaxPage.getLanguage());

        ajaxPage.selectCategory("Desktop");
        assertEquals("C++",ajaxPage.getLanguage());
}
```

We can't use a WaitForPageToLoad because the page does not reload, JavaScript is being used to

refresh the entry.

We have to use the WaitForCondition method.

> **Note:**
>
> **In the above test, note that I used the optional (expected='...') notation. This allows me to write tests which 'fail' but do not show as failures in the running of the test because I have told the test runner to expect the test to fail and throw an exception.**

First Model the behaviour in the test

The first thing I will do is model the behaviour I expect in the test.

```
@Test
public void chooseACategoryAndCheckTheLanguage(){

        ajaxPage.selectCategory("Server");
        ajaxPage.waitForAjaxSymbolToGoAway();

        assertEquals("Cobol",ajaxPage.getLanguage());

        ajaxPage.selectCategory("Web");
        ajaxPage.waitForAjaxSymbolToGoAway();

        assertEquals("Javascript",ajaxPage.getLanguage());

        ajaxPage.selectCategory("Desktop");
        ajaxPage.waitForAjaxSymbolToGoAway();

        assertEquals("C++",ajaxPage.getLanguage());
}
```

All I have to do now, is write the waitForAjaxSymbolToGoAway in my page object model and my test should pass.

Easy.

Code the check in the page object model

The Selenium WaitForCondition method takes two parameters:

- A JavaScript statement to evaluate
- A Timeout for how long should we wait for the condition to return true

The WaitForCondition basically loops, evaluating the supplied JavaScript, until it returns true, or the timeout occurs.

The first hard part then, about WaitForCondition, is learning enough JavaScript to be able to make use of it.

The second is understanding the functioning of the page.

First understand the page

Before we try and figure out what JavaScript we need we have to have a look at the page and see what happens.

By examining the source-code for the basic_ajax.html page I can see the image in the page:

```
<span id="ajaxBusy" style="display: none;"><img src="ajax-loader.gif"/></span>
```

By default the image will not display, because of the CSS style values use “display: none;”.

So, I need to write a wait condition which checks the style value of the 'ajaxBusy' item.

A quick google search later for “javascript check if display none” and I have the basic information that I need “style.*display* == *'none'*”

- for more information visit http://www.dustindiaz.com/seven-togglers/

We have already seen that to interact with the page with JavaScript in Selenium that I have to prefix the access commands with “`window.document.`”

So I can write a method in my BasicAjaxPage.java page object as follows:

```
public void waitForAjaxSymbolToGoAway() {
        selenium.waitForCondition(
                "window.document.getElementById('ajaxBusy').style.display=='none'",
                "4000");

}
```

Which gets the element with the id “ajaxBusy” and checks that the display value in the CSS style is “none”.

The JavaScript that we need to use waitForCondition doesn't really need to go much beyond basic accessor styles and you can usually find examples via a quick search.

My test now passes.

Summary

It could be very easy to put yourself off testing at this level. Because you do have to know a bit of JavaScript. You do have to know a bit of CSS. You do have to be able to analyse the application page and work out what is happening.

But as you saw, you don't need *much* knowledge. You just need to be able to figure out what you want to do, have the skills to find the information, and then translate it into tests.

This will come with practice after you try to automate a few dynamic HTML or Ajax pages.

Your developers should be able to help write the conditions, and the easy way to learn is to work closely with the development team.

Note:

The tests and page object model for this chapter can be found as BasicAjaxPage.java and MoreJavaScriptTests.java in the package com.eviltester.seleniumtutorials.chap34.javascript.

Chapter 35: Cookie Handling

Cookies are small files which are stored on your computer which web sites use to store client specific information e.g. Session identifiers, or tracking information.

Chances are, any website which can remember who you are when you visit, without you telling it, has placed a cookie on your computer which it accesses when you next visit the site.

For more information on cookies, visit the Wikipedia entry for "HTTP Cookie"
http://en.wikipedia.org/wiki/HTTP_cookie

Cookies in the example application

The search page http://www.compendiumdev.co.uk/selenium/search.php uses cookies to track:

- previous search terms
- time and date of the last visit
- number of times visited

It creates the following cookies via JavaScript:

- seleniumSimplifiedSearchNumVisits to count the number of visits to the page
- seleniumSimplifiedSearchLastVisit to store the time and date of the last visit
- seleniumSimplifiedLastSearch to store the value of the last search

How to see cookies on a web page?

If you visit the search page above then you can see the cookies for yourself using Firefox.

Firefox by Default

Right click and choose "view page info" from the popup menu.

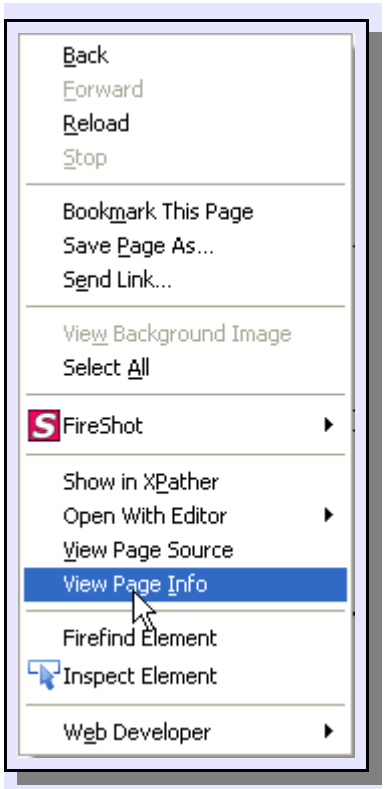


Figure 35.1 : View Page Info

On the page info dialog, click the "Details" button.

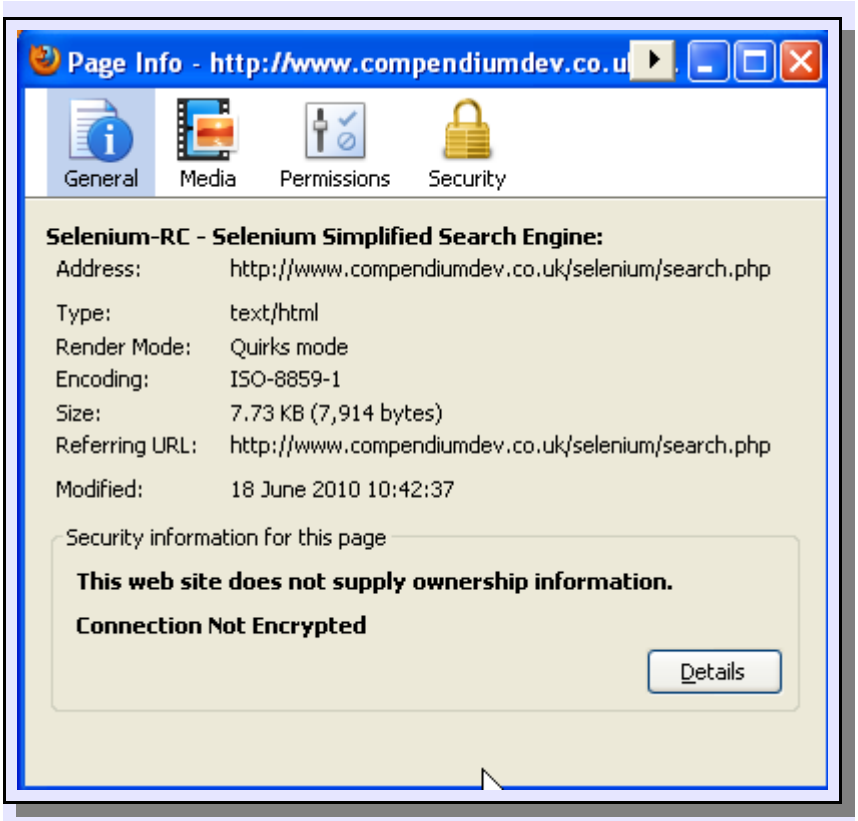


Figure 35.2 : Page Info Dialog shows basic page information

On the Page Info Details dialog, click the “View Cookies” button.

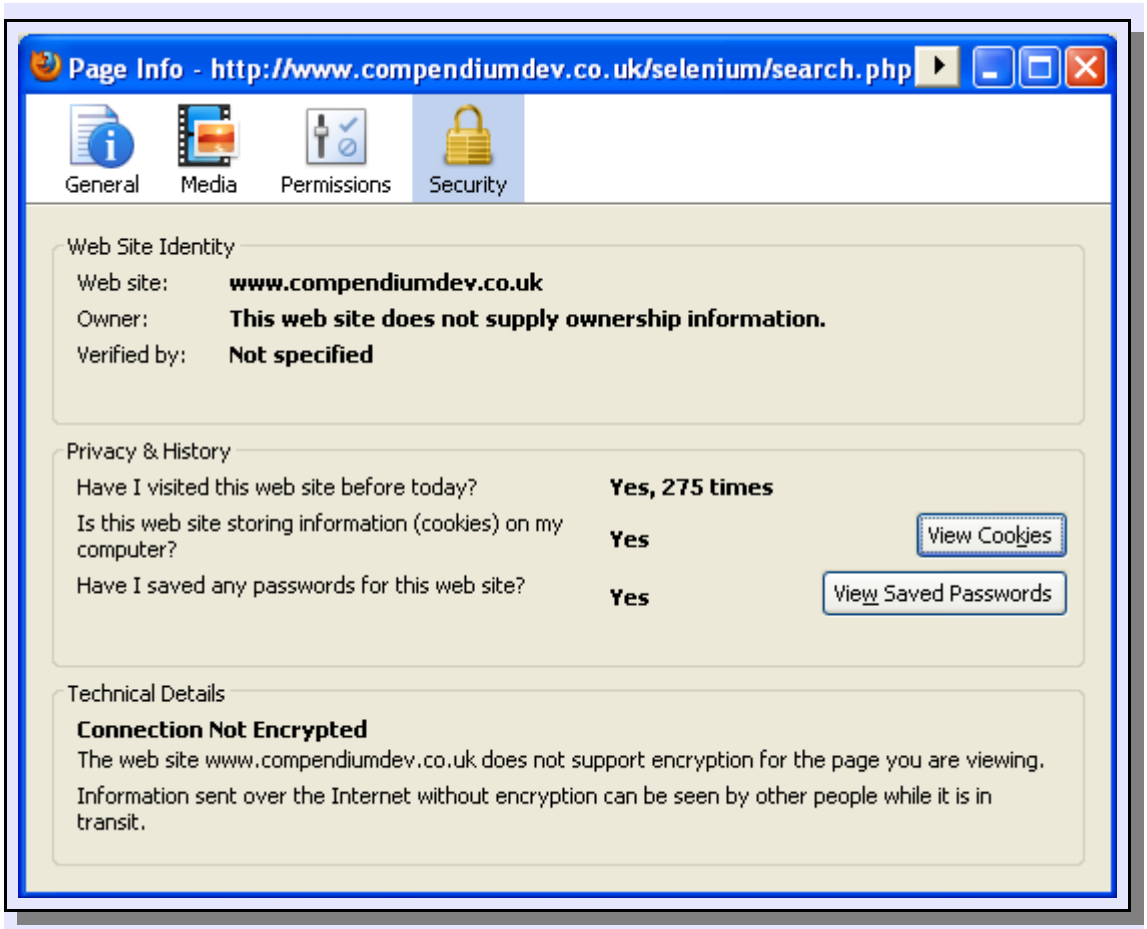


Figure 35.3 : Page Info dialog showing details allows you to view the cookies

You then see the cookies management dialog. Here you can see the cookie details and remove them.

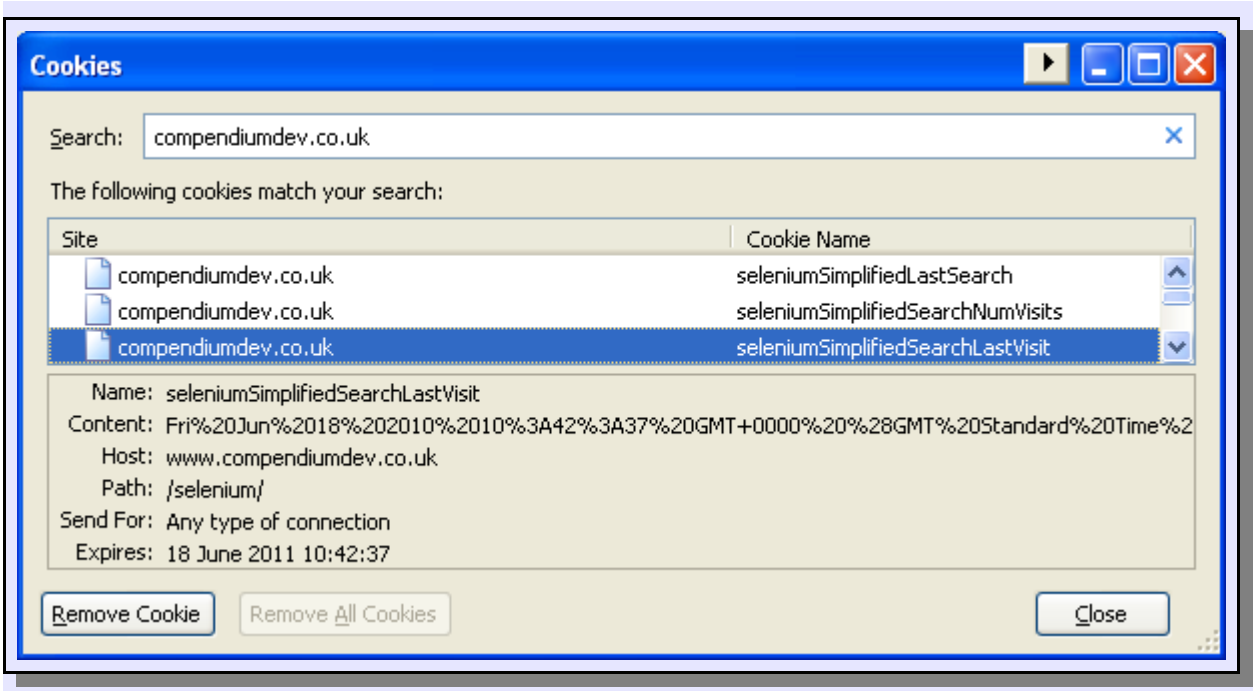


Figure 35.4 : Cookies Management Dialog

Firecookie - Firebug Plugin

To have more control over the cookies, you could use a Firebug plugin in Firefox called Firecookie by Jan Odvarko.

https://addons.mozilla.org/en-US/firefox/addon/6683/

This allows you to Edit the cookie as well as delete it. When we are testing with cookies, the ability to edit them comes in particularly handy. Editing them is less important for automation but I still tend to use the plugin than rely on the default cookie management.

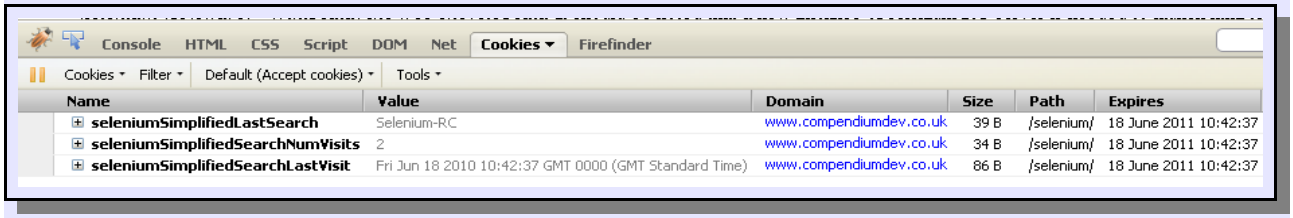


Figure 35.5 :Firecookie plugin for amending and viewing cookies

By right clicking on the cookie name you can choose to “Edit” cookie from the popup menu.

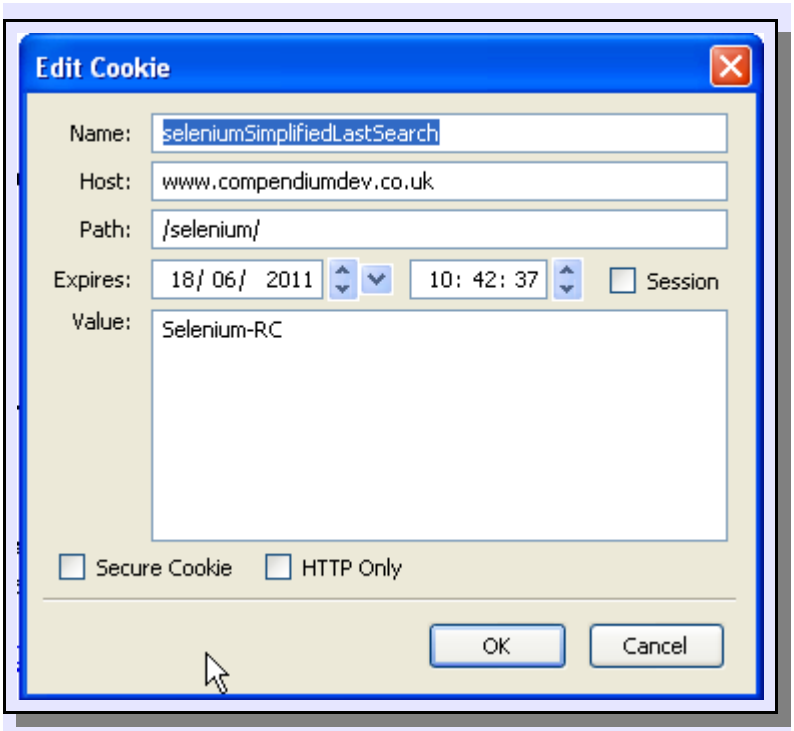


Figure 35.6 : Edit cookie in firecookie

Here you can edit the details of the cookies.

Selenium Cookie Commands

Selenium provides the following commands for working with cookies:

- deleteAllVisibleCookies();
  - delete all the cookies for the current page
- isCookiePresent("<cookie_name>");
  - return true if a particular cookie name is present on the page
- createCookie(...<cookie_details>);
  - create a cookie for the current page
- getCookie();
  - returns all the cookies for the current page
- getCookieByName("<cookie_name>");
- deleteCookie("<cookie_name>", "<options>");
  - delete a named cookie,
  - the options string takes a comma separated list of options:
    - path = the path under the specified domain which the cookie was created for
      - e.g. path=/selenium/
    - domain = the domain used to create the cookie

- e.g. domain=compendiumdev.co.uk

- "recurse=true" :try all sub-domains of the current domain
- Note: I have found certain variations of using the deleteCookie method incompatible between browers. You will see these later in the chapter.

Example Code For Cookie Testing

Basic Class

I create a basic class to add the following tests in. This will reuse the SeleniumManager and the SearchPage object that we created in previous chapters.

```
package com.eviltester.seleniumtutorials.chap35.cookies;

import static org.junit.Assert.*;
import java.io.IOException;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Test;
import com.eviltester.seleniumtutorials.refactored.pageObjectModel.SearchPage;
import com.eviltester.seleniumutils.SeleniumManager;
import com.thoughtworks.selenium.Selenium;

public class CookieTests {
	static SeleniumManager sm;
	static Selenium selenium;
	SearchPage searchPage;

	@BeforeClass
	static public void startServer() throws IOException{
		 sm = new SeleniumManager();
		 sm.start("http://www.compendiumdev.co.uk");
		 selenium = sm.getSelenium();
	}

	@AfterClass
	static public void stopServer(){
		sm.stop();
	}
}
```

Check Cookie Creation using deleteAllVisibleCookies and isCookiePresent

To the above class I will add a test that checks for cookie creation.

This test uses the deleteAllVisibleCookies and isCookiePresent methods to check that cookies get created.

I will first visit the page, and delete all the cookies. This way I know that I am starting from a clean position.

I will then open the page and check that two cookies get created on page visitation. Then I will do a search and check that the third cookie gets created.

```
@Test
public void checkCookiesGetCreated() {
	searchPage = new SearchPage(selenium);
	searchPage.open();

	selenium.deleteAllVisibleCookies();

	assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");
```

```
        assertCookieDoesNotExist("seleniumSimplifiedSearchLastVisit");
        assertCookieDoesNotExist("seleniumSimplifiedLastSearch");

        searchPage.open();
        assertCookieExists("seleniumSimplifiedSearchNumVisits");
        assertCookieExists("seleniumSimplifiedSearchLastVisit");
        assertCookieDoesNotExist("seleniumSimplifiedLastSearch");

        searchPage.typeSearchTerm("selenium");
        searchPage.clickSearchButton();
        assertCookieExists("seleniumSimplifiedSearchNumVisits");
        assertCookieExists("seleniumSimplifiedSearchLastVisit");
        assertCookieExists("seleniumSimplifiedLastSearch");
}

private void assertCookieDoesNotExist(String cookieName) {
        String cookieMessage = "We deleted all cookies, %s should not exist";

        assertFalse(   String.format(cookieMessage,cookieName),
                       selenium.isCookiePresent(cookieName));
}

private void assertCookieExists(String cookieName) {
        String cookieMessage = "We added cookie %s, it should exist";

        assertTrue(    String.format(cookieMessage,cookieName),
                       selenium.isCookiePresent(cookieName));
}
```

I created the private helper methods to remove repeated code in the test and help make the test itself more readable. If I find that I use methods like this across multiple tests then I would probably refactor them into a new class to make them available to all the tests, rather that creating them as private methods in each test class.

Get Cookie values using getCookie and getCookieByName

getCookie

The getCookie method returns a single string with all the cookies for the current page.

After running a searchPage.open() command the getCookie() method for the search page would return a string like the following:

```
seleniumSimplifiedSearchLastVisit=Fri%20Jun%2018%2012%3A10%3A04%20UTC
%202010; seleniumSimplifiedSearchNumVisits=1
```

This string has all the cookies separated by ";".

And you can see that each cookie has the format:

```
<cookie name>=<cookie value>
```

Cookie values are urlencoded (http://en.wikipedia.org/wiki/Percent-encoding)

getCookie is not likely to be a method that you use very frequently as you can get the cookies directly using the getCookieByName, but it can prove useful when you check to see if any cookies you did not expect to be added are on the page.

The following example test shows one way of parsing the results of the getCookie string:

```
@Test
public void parseResultsOfGetCookieExplored() throws UnsupportedEncodingException {
        searchPage = new SearchPage(selenium);
        searchPage.open();

        // split the cookie string so that each entry in the array
```

```
        // is a cookie name=value
        String []cookies = selenium.getCookie().split(";");

        for (int i = 0; i < cookies.length; i++) {
                // split the cookie=name value into name[0] and value[1] entries
                String []cookieNameValue = cookies[i].split("=");

                // remove extra spaces and encoding on the name and value
                String cookieName = cookieNameValue[0].trim();
                String cookieValue =
                                URLDecoder.decode(cookieNameValue[1].trim(),"UTF-8");

                // compare against the 'byName' methods
                assertTrue("expected to find cookie by name " + cookieName,
                        selenium.isCookiePresent(cookieName));
                assertEquals(cookieValue,
                        URLDecoder.decode(selenium.getCookieByName(cookieName,"UTF-8"));
        }
}
```

In the above code we first split the returned value from getCookie so that the ';' separated values become entries in a java array.

We loop over the array and split each cookie on the "=" character to give us an array where the first ([0]) value in the array is the cookie name and the second ([1]) is the cookie name.

I have to .trim() the returned values to remove any header or trailing whitespace characters. I decode both values to ensure that the comparisons are consistent.

We use the java.net.URLDecoder:

```
import java.net.URLDecoder;
```

Again, you probably won't use getCookie very much but it is worth knowing for those occasions where the cookies require parsing or decoding.

getCookieByName

You have already seen getCookieByName in action in the previous example test but I will now show it in the context of an actual test on the page.

```
@Test
public void getCookieByNameExplored() {

        String lastSearchTerm="";

        // open page and set to no cookies
        searchPage = new SearchPage(selenium);
        searchPage.open();
        selenium.deleteAllVisibleCookies();

        /* open the page 10 times and check cookie increments */
        for (int i = 1; i < 11; i++) {

                lastSearchTerm = "Selenium " + i;
                searchPage.typeSearchTerm(lastSearchTerm);
                searchPage.clickSearchButton();

                assertEquals(i + "",
                        selenium.getCookieByName("seleniumSimplifiedSearchNumVisits"));
                assertEquals(lastSearchTerm,
                        selenium.getCookieByName("seleniumSimplifiedLastSearch"));

                assertCookieExists("seleniumSimplifiedSearchLastVisit");
        }
}
```

In this test we open the page and delete all the cookies to get it in a known state. Then we loop around ten times and perform a search. Then on the resulting page we check that the number of visits has been incremented and that the last search term was stored correctly.

Note:

I used a shorthand way of allowing an integer to be compared with a string in the assertEquals statement:

***assertEquals*(i + "",**
***selenium*.getCookieByName("seleniumSimplifiedSearchNumVisits"));**

Here I converted the integer into a string by adding an empty string to it e.g.

I + ""

Horribly enough this is an old Visual Basic trick that I've carried through into other languages. It also helps avoid null pointer exceptions on string comparisons.

deleteCookie

Most of the time you will be able to use deleteCookie without passing through any options e.g.

```
selenium.deleteCookie("seleniumSimplifiedSearchNumVisits","");
```



Figure 35.7 : Cookie domain and path

If you need to get more specific then the following test illustrates some of the variations around using deleteCookie:

```
@Test
public void deleteCookieExplored() {
        searchPage = new SearchPage(selenium);
        searchPage.open();

        // simple delete works fine
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits","");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");

        // full domain and path work fine - you can see these in firecookie
        // issues getting this working in firefox, but works fine in IE
        if(sm.getBrowser().contains("ie")){
                searchPage.open();
                selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                        "domain=www.compendiumdev.co.uk, path=/selenium/");
                assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");
        }

        // delete example using just the domain
        if(sm.getBrowser().contains("ie")){
                searchPage.open();
                selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                        "domain=www.compendiumdev.co.uk");
                assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");
        }

        searchPage.open();
        // missing domain and having path will delete the cookie
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits", "path=/selenium/");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");

        // fail to delete when missing domain and getting path wrong
        searchPage.open();
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits", "path=/wrongpath/");
        assertCookieExists("seleniumSimplifiedSearchNumVisits");
```

```
        searchPage.open();
        // missing out the path and recurse succeed
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                "domain=www.compendiumdev.co.uk, recurse=true");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");

        // fail to delete when domain wrong even if path right
        searchPage.open();
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                "domain=compendiumdev.co.uk, path=/selenium/");
        assertCookieExists("seleniumSimplifiedSearchNumVisits");

        searchPage.open();
        // getting the domain wrong will pass if we recurse through subdomains
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                "domain=compendiumdev.co.uk, recurse=true");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");

        searchPage.open();
        // getting the domain wrong can pass if we recurse through subdomains,
        // path makes no difference
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                "domain=compendiumdev.co.uk, recurse=true, path=/selenium/");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");

        searchPage.open();
        // getting the domain wrong can pass if we recurse through subdomains
        // even if we get the path wrong
        selenium.deleteCookie("seleniumSimplifiedSearchNumVisits",
                "domain=compendiumdev.co.uk, recurse=true, path=/sele/");
        assertCookieDoesNotExist("seleniumSimplifiedSearchNumVisits");
    }
```

In the above example you can see the incompatibilities I had when running deleteCookie on different browsers. The base deleteCookie call with just the cookie name was portable across all browsers. The combination of domain, without the recurse parameter failed on Firefox, but passed on Internet Explorer.

> **Note:**
>
> **When your test code doesn't work, try it in different browsers, as you may have encountered a browser incompatibility problem.**
>
> **Keep the checks and method usages as simple as possible to avoid browser incompatibility issues, but not so simple that you can result in false positives in the test (i.e. a check passing when it should not)**

createCookie

If you need to create initial conditions for a test through cookies then you can use the createCookie command.

createCookie(<name value pair>,<options>);

- The name value pair, as we saw earlier from getCookies() has the format "name=value" where the value is URL encoded.
- Options, uses a comma separated list to have multiple options in a single string. If you choose not to use path and domain then the current page path and domain will be used.
    - path
        - the path under the domain for the cookie
        - e.g. path=/selenium/

- domain
  - the domain the cookie relates to
- max_age
  - a value in seconds for the cookie to be valid
  - e.g. max_age = 6000
    - 100 minutes
  - we can use max_age to create a session cookie by using a negative number
    - e.g. max_age=-200

createCookie does not urlencode our cookie values so we use the URLEncode.encode method from java.net.URLEncoder

```
import java.net.URLEncoder;

@Test
public void createCookieExplored() throws IOException {

        searchPage = new SearchPage(selenium);
        searchPage.open();
        selenium.deleteAllVisibleCookies();

        searchPage.open();
        assertCookieDoesNotExist("seleniumSimplifiedLastSearch");

        String lastSearch = "Search For This";
        String lastSearchAsCookieValue = URLEncoder.encode(lastSearch,"UTF-8");
        selenium.createCookie("seleniumSimplifiedLastSearch=" + lastSearchAsCookieValue,
                              "max_age=6000");

        searchPage.open();
        assertCookieExists("seleniumSimplifiedLastSearch");
        assertEquals(selenium.getCookieByName("seleniumSimplifiedLastSearch"),
                lastSearchAsCookieValue);
}
```

Note:

The code for this chapter can be found in class CookieTests.java in the package com.eviltester.seleniumtutorials.chap35.cookies

Chapter 36: The Future of Selenium

The Selenium tool suite is constantly being worked on. Having been embraced by Google as one of their automation tools, with many of the key figures in Selenium development working at Google, the future for Selenium looks rosy.

It can often seem daunting choosing between the many open-source software testing tools to know which ones will be supported and maintained over the years. The fact that Google works with it means that there will remain a talented set of programming and testing talent behind the tool.

Selenium is also used in spin off tools like the load testing tool from BrowserMob.com, which uses the selenium API as its scripting language from JavaScript to write the automated tests.

The current direction for Selenium, which should manifest as Selenium 2.0 merges the code bases for Selenium 1.0 and WebDriver.

Selenium takes the automation approach of driving the website through JavaScript with the Selenium RC server using Selenium Core to automate the site. This has some issues with security and cross browser scripting which is why there are so many ways of instantiating a browser session for the supported browsers. e.g. *iexplore, *firefox, *firefoxproxy, *iexploreproxy

WebDriver takes a different approach. It calls the browsers directly through their respective automation interfaces. This means that all tests run in the browser and have no security issues as it is the raw browser that takes all the actions.

WebDriver has a different API than Selenium. It is a far more Object Oriented interface and can seem cleaner. WebDriver also supports the use of HTMLUnit which means you can test websites without creating a 'real' browser so tests can run faster and more easily in a Continuous Integration environment.

For Selenium 2.0 the plan is to support both APIs:

- the Selenium API (large set of commands off the Selenium interface)
- the WebDriver API (object oriented browser automation based interface)

This means that you should still be able to write tests using the Selenium API, but have the tool drive the browser directly. This should see an end to some of the JavaScript injection issues and make the tests faster to run.

It also means that the WebDriver API can be used with the Selenium JavaScript injection approach, allowing WebDriver access to browsers which it doesn't yet have a native automation driver for.

All of this basically means that:

- Selenium is still being developed
- The Selenium API you have started learning with this text will still exist and be usable in Selenium 2.0

The time you have spent learning Selenium should be an investment in time and you can look forward to continuing to learn more in the future.

If you want to experiment with 2.0...

Since we have built the SeleniumManager abstraction class to remove the details of selenium for us, we can easily start using Selenium 2.0 by making a few changes to this class.

You can find the official guide to migrating from Selenium 1.x to Selenium 2.0 on the Selenium web site: http://seleniumhq.org/docs/09_webdriver.html#emulating-selenium-rc

This small section is designed to help you take the first steps towards experimenting with Selenium 2.0 using the basic frameworks that we have explored in this book.

So first, download the Selenium 2.0 client drivers from the selenium web site:

- http://code.google.com/p/selenium/downloads/list

You want to download the selenium-java-2.x.zip where x is the most recent version of the archive:

- e.g. Selenium-java-2.0a4.zip

Extract this folder to somewhere on your drive.

- I used c:\selenium2

In your properties in Eclipse for the test project in the Libraries tab, [Add External Class Folder...] and choose the folder you extracted the archive to e.g.

- C:\selenium2\selenium-java-2.0a4

Now you only have a few changes to make to the Selenium Manager to allow you to use Selenium 2.0 instead of Selenium 1.x

- Add a new field in SeleniumManager called useSeleniumTwo

```
public class SeleniumManager {

	SeleniumServer seleniumServer=null;
	Selenium selenium=null;

	String host;
	String port;
	String browser;
	private boolean useSeleniumTwo=true;

...
```

    - Manually change this from true to false depending on whether you want to use Selenium 2.x or not.
        - You can easily expand the class to have some helper methods to do this. See the SeleniumManager.java for an example

- Add code to return a WebDriverBackedSelenium instance instead of a Selenium 1.x instance

```
public void start(String baseURL) throws IOException {

	readProperties(); // get the properties from a file
	readSystemProperties(); // allow system properties to override properties file

	if(useSeleniumTwo){
		WebDriver driver = new FirefoxDriver();
		selenium = new WebDriverBackedSelenium(driver, baseURL);
		return;
	}
...
```

- Use Eclipse to fix the include errors

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebDriverBackedSelenium;
import org.openqa.selenium.firefox.FirefoxDriver;
```

    - The above includes should be added by Eclipse automatically as you fix the errors in the code, or you can add them manually.

The above is the minimum amount of code required to use the Firefox driver for Selenium 2.0 to have all tests which use the SeleniumManager running under Selenium 2.0

So now if you run any of the tests that you have created which use the SeleniumManager then they should run using Selenium 2.x instead of Selenium 1.x

You will find that not all tests run because there will be some subtle differences required when using Selenium 2.x, but hopefully the vast majority of the tests should run and pass successfully.

Selenium 2.x is still being worked on, but even now you can see that the migration path from Selenium 1.x to Selenium 2.x should not take an awful lot of work.

Note:

You can find more comprehensive code for using Selenium 2.0 in the SeleniumManager.java class.

It is commented out by default so as not to require everyone to amend the properties to reference Selenium 2.0

Chapter 37: Structuring the tests and code

I will use this chapter to tie up all the loose ends, to demonstrate what a 'production ready' version of the previous chapters might look like.

In previous chapters we have created a disparate set of tests which mostly exist to demonstrate the capabilities of Selenium. In this chapter we will:

- throw away the 'tests' which don't test anything,
- restructure the code as if in a production environment, to separate tests from page object models
- refactor code to have constants and manageable XPaths
- create a base class to derive our tests from
- tidy the Ant builds
- setup Hudson

This is the chapter you are most likely to refer to for creating the test suites that you use on your projects.

Throughout this chapter I will describe the decisions taken, explain the justification for the decisions and discuss alternatives that we could have pursued.

The decisions taken will be pragmatic, as you would take decisions in the real world. Which means that they are not abstract theoretical decisions. They are decisions which, while they work, involved some compromise, so as part of discussing the alternatives we will explore those compromises.

The code for this chapter is in a new project. You can download the project code, and add it to your own subversion repository from:

http://www.compendiumdev.co.uk/selenium/finalChapterSource.zip

Note: You will need to copy in selenium-server.jar to the lib folder to get the same set of files as from the xp-dev server (this was removed from the zip file due to the size of the file).

Or use it directly from the open subversion server provided by XP-dev (http://www.xp-dev.com/)

http://svn2.xp-dev.com/svn/seleniumsimplified/trunk/FinalSeleniumTests

I will use the above subversion link later when I setup Hudson to run the tests.

Java project folder and package structures

In Java, we logically organise our source-code using packages, we physically organise our source-code using folders.

There are many different ways of organizing a Java program physically. I have chosen to adopt the standard directory layout as described by the Maven project.

http://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html

Maven is a software project management tool which can help with organising project builds and

dependencies, although we haven't used it in this book, more and more people are using Maven so using the standard directory layout has the advantages that:

- people are more likely to have familiarity with it
- moving to Maven later should be easier
- you don't really have to debate too much about a subjective decision and can adopt an approach based on lessons learned by previous experienced Java developers

The basic folder structure for our project using these maven guidelines is:

- <project name>
    - src
        - main
            - java
                - ...<package structure represented here>
        - test
            - java
                - ...<package structure represented here>
            - resources

In addition, because we are not using maven, I need a folder to store those libraries which my project has a dependency upon, i.e. Junit4.x and the selenium server. So I create a lib directory at the root level of the project. Lib is not mentioned in the maven documentation because maven itself manages these dependencies and so they are stored in a local repository outside of the direct file management of the project.

        - lib
            - /lib is not mentioned in the maven structure but I have chosen to put it at the root level of the folder

I left all my properties files and build files at the root directory.

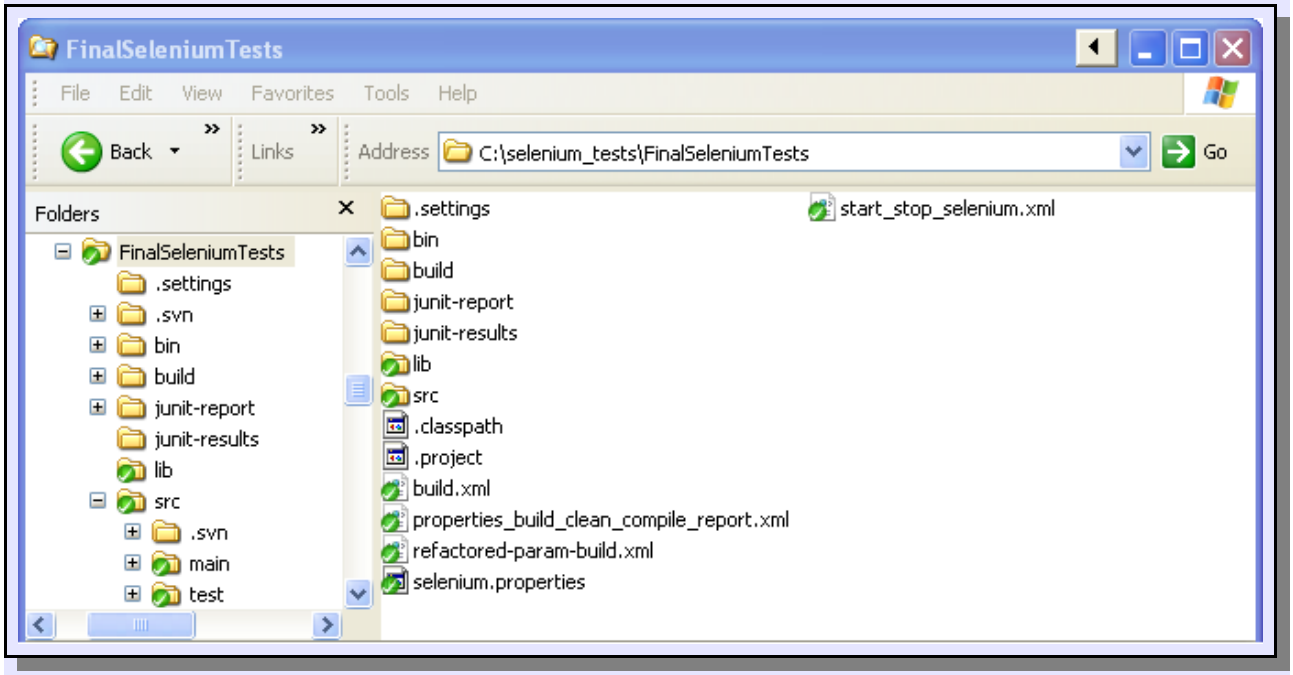


Figure 37.1 : New Folder structure based on Maven guidelines

During the previous chapters, the physical layout was not a big issue because:

- we were learning and did not need a lot of rules and formalities getting in the way
- our Ant files individually selected the classes to run
- we didn't really make a distinction between 'tests' and 'test code'
- we were not trying to fit in with a production project so we could do what we wanted
- while we did run the tests through Ant and Hudson, our main method for running the tests was directly in Eclipse

Now though:

- we want to be able to fit in with production quality projects, and learn 'professional' practices
- we want to separate 'tests' from 'test code'
- we want to make the Ant files easier to maintain and not individually add test classes

It is very easy to discuss and argue about the 'proper' structure to use for your tests. I made a simple decision to separate my 'tests' from my 'code' (code being all the java classes that abstract away the system). I think this makes it easier for me to make amendments because I know which folder branch to go into for the different parts of my code.

If I find myself doing a lot of maintenance work in the test branch then I need to rethink my abstraction strategy as I may have written my tests at the wrong level of abstraction. Ideally my tests should remain unchanged after a system change. If the 'test conditions' are still valid, I want most of my changes to be in the abstraction layer. This is not a hard and fast rule as it may not be worth the time building up an abstraction layer which removes all rework.

Our 'tests' are those classes which have @Test annotations within them. Our 'test code' covers our

Selenium Manager, and our Page Object Models. These are classes with 'real code' which theoretically we might choose to release as a separate .jar file for re-use on other projects. We actually want to maintain a separation in how we think about this code because, if we have built our abstraction layer correctly, when the app changes, only the abstraction layer changes – not the tests.

Also by separating out the 'tests' from 'test code', in the Ant files we can run all the classes in the src\test folder by using a fileset.

Changing what goes into Version Control

In previous chapters I was lazy about what went into version control and included the Eclipse property files:

- \.settings
- .classpath
- .project

This seemed a convenient 'poor practice' to use, because it stopped you having to configure the project for your particular environment.

But really, we should not have these configuration elements under version control as they can be unique to everyone's machine.

Configuring Source-code Folders in Eclipse

By removing these, when you import the project into Eclipse, you need to add the following folders as 'source folders' to allow you to run the tests from within Eclipse:

- src\main\java
- src\test\java

To do this:

1. select the source-code folder in the Package Explorer e.g. src\main\java
2. right-click mouse and select “build path\ use as source folder”
3. the folder will then be shown as the root of a package hierarchy

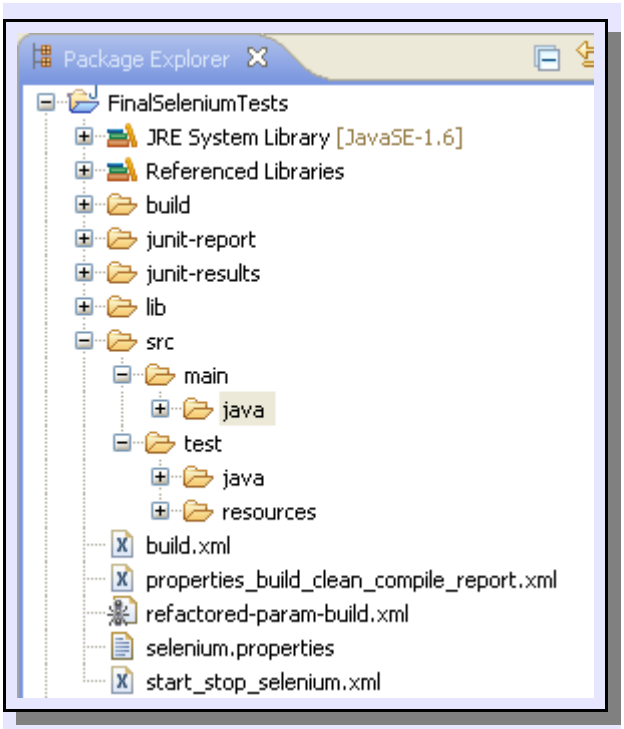


Figure 37.2 : Step One: Select Folder in Package Explorer

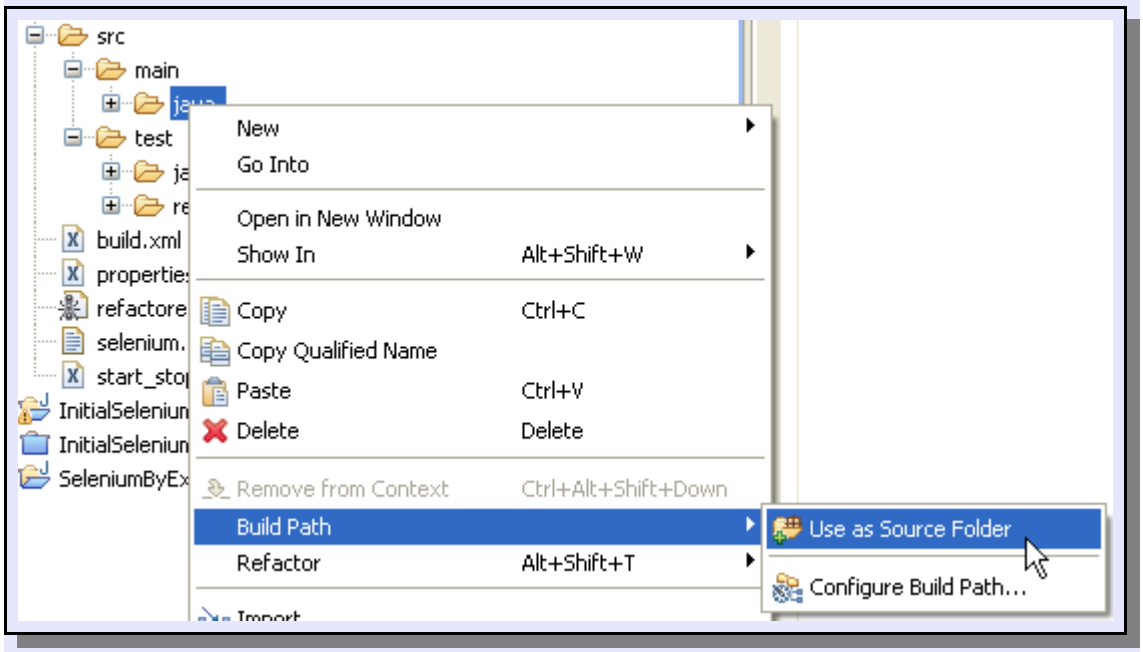


Figure 37.3 : Select "Use as Source Folder"

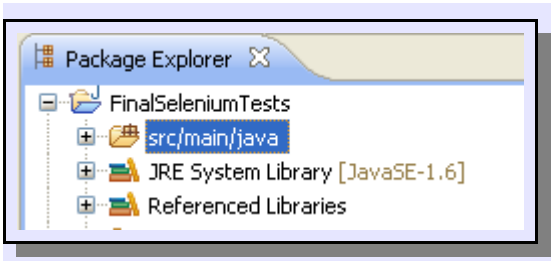


Figure 37.4 : Folder as source folder

You can see all the source folders listed in the "Java Build Path" "Source" properties by selecting "build path \ configure build path" from the right-click menu.

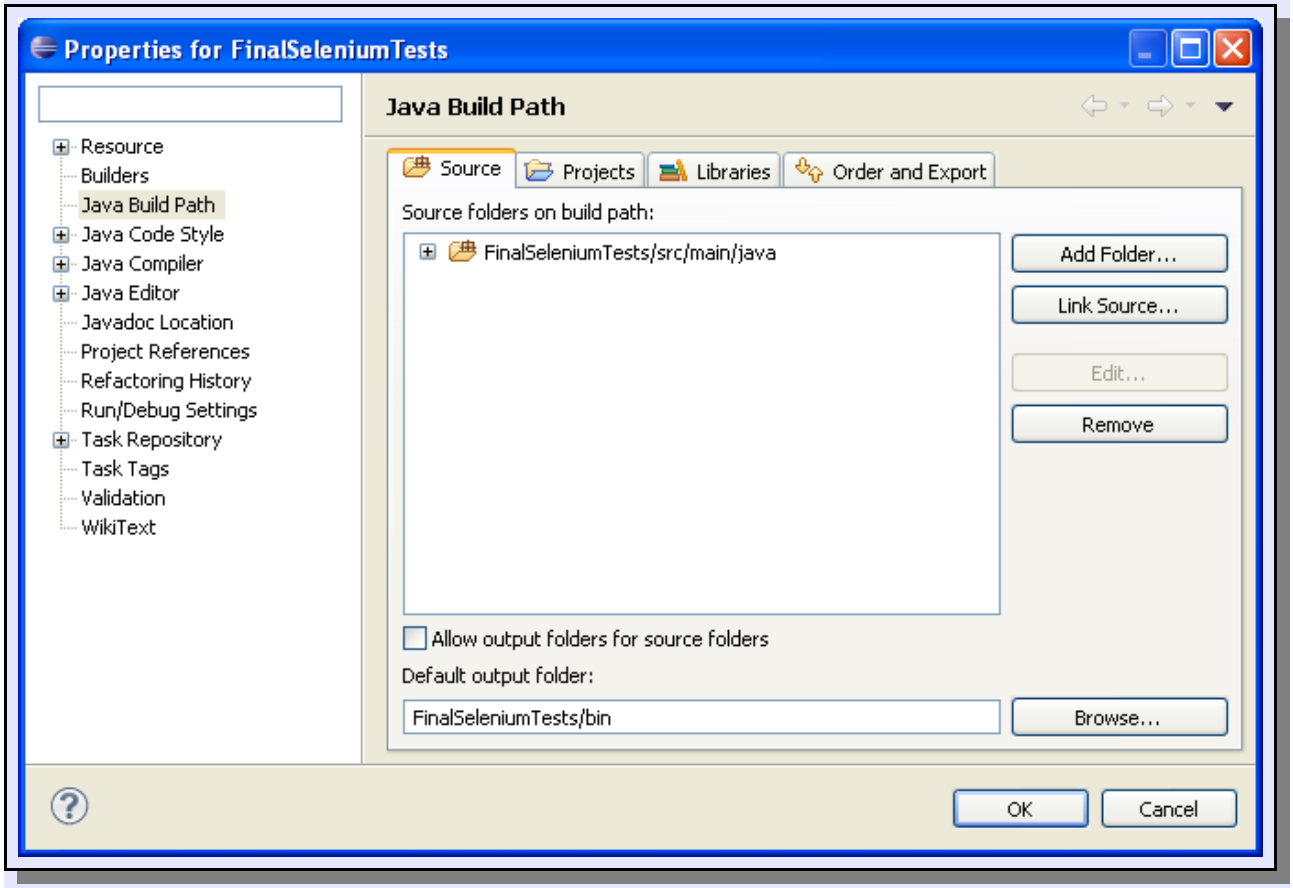


Figure 37.5 : Manage the source folders in the project properties dialog

Add libraries

I also added the libraries in the \lib folder using the "Add External JARs" function. As we have done in previous chapters.

Re-organise the Classes and Packages

Because I built the tests in the previous chapters over time, they were organised in a very adhoc fashion.

I will move all the tests around, and into the appropriate source folders.

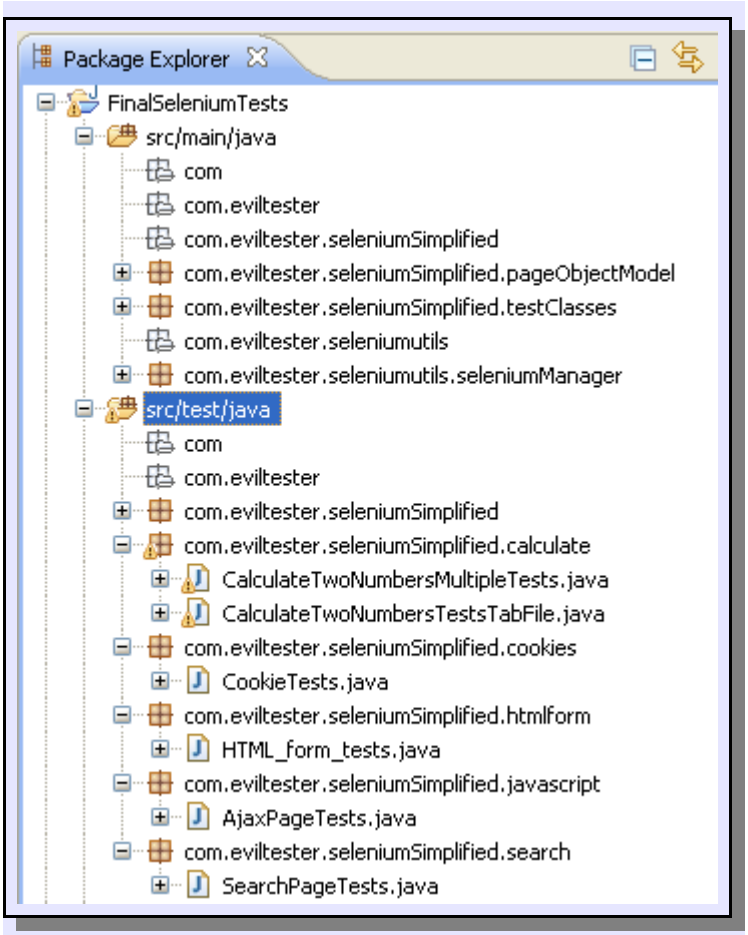


Figure 37.6 : Reorganised Package Structure

You can see that I have logically organised all my tests into classes in packages for the areas under test.

Even if the package only contains one class – as many of them do since we only have a very few tests. I still create packages to order them. I find that this:

- makes it easier to see coverage at a high level
- makes it easier to add new test classes
- makes it easier to find tests when I have to maintain them over time

You can also see that I have split the 'test code' from the 'tests'. All tests are in src/test/java and all test code is in src/main/java.

I have organised the 'test code' so that all classes related to the Selenium Manager are in a seleniumManager package, base classes for tests are in their own package 'testClasses' and all page object models are in another package.

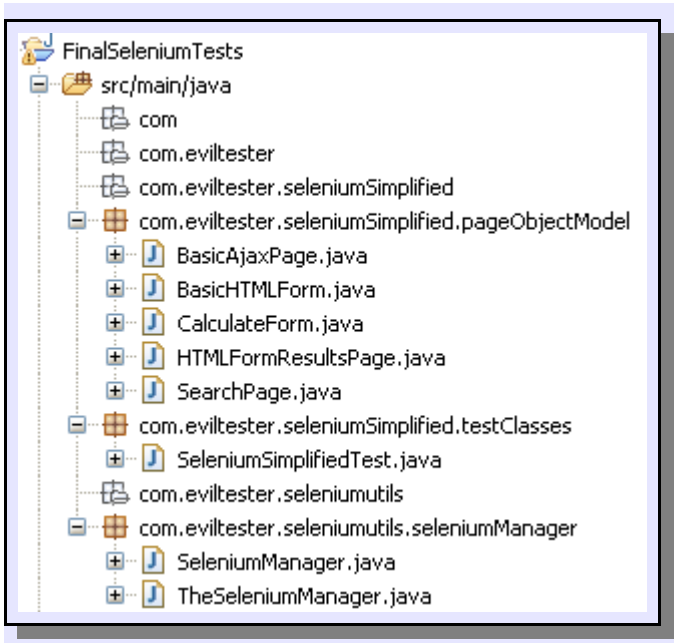


Figure 37.7 : Test Code Packages

I will explain SeleniumSimplifiedTest.java and TheSeleniumManager.java in the next section.

Share the Server among tests and create a base test class

In the earlier chapters, our tests used the SeleniumManager class to make it easier to manage the SeleniumServer, and our test classes start and stop the server within the class.

I made some changes to the SeleniumManager:

- to allow me to reuse the same SeleniumServer between tests.
- to allow me to reuse the same Selenium session between test classes

Since all our tests work on the same domain, if I change the selenium manager to use the same server instance with all the tests, and use the same Selenium instance for all tests that work with the same base URL, there should be a considerable speed up in the test execution speed.

Since all of our tests share the same structure I will create a class for them all to inherit from. This effectively brings us back to the mechanism that we saw at the very start of this book with the SeleneseTestCase, but this time it is completely under our control.

Share the Server using a Singleton

To have all the tests use the same Selenium server I only allow access to SeleniumManager via a singleton. In this way only one instantiated object of SeleniumManager is in use.

To implement this, I introduce a TheSeleniumManager class:

```
public class TheSeleniumManager {

        private volatile static SeleniumManager instance = null;

        private TheSeleniumManager() {
        }

        public static SeleniumManager getSeleniumManager() {
```

```
                if(instance == null) {
                        synchronized(SeleniumManager.class){
                                if(instance == null) {
                                        instance = new SeleniumManager();
                                        instance.useSeleniumOne();
                                        instance.resetProperties();
                                }
                            }
                        }
                        return instance;
                }
...
```

All of the above complicated code mans that to get access to SeleniumManager, you have to call the static method on TheSeleniumManager and if no selenium manager has been created, then create one, and once created, it will be reused.

This code was adapted from the Java Singleton code on Wikipedia where you can learn more about Singletons in general. (http://en.wikipedia.org/wiki/Singleton_pattern)

So in my test I would use it as follows:

```
sm = TheSeleniumManager.getSeleniumManager().start("http://www.compendiumdev.co.uk");
```

I made further amendments in the Selenium Manager code so that calls to .start() reuse the same selenium server and Selenium instance unless the base URL changes. We will look at this code later.

Create a Base Class

This effectively brings us back to the mechanism that we saw at the very start of this book with the SeleneseTestCase, but this time the definition of the class is completely under our control and our reasons for using it are completely our own.

My justifications for introducing a base class now:

- having written a lot of tests I can see the commonalities between our tests
- I would like to have the screenshots being taken on error that we saw in one of our tests using the JUnit TestWatchman
- it will simplify my test classes

The Initial Base Class I created looked like this:

```
package com.eviltester.seleniumSimplified.testClasses;

import java.io.IOException;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Rule;
import org.junit.rules.MethodRule;
import org.junit.rules.TestWatchman;
import org.junit.runners.model.FrameworkMethod;
import com.eviltester.seleniumutils.seleniumManager.SeleniumManager;
import com.eviltester.seleniumutils.seleniumManager.TheSeleniumManager;
import com.thoughtworks.selenium.Selenium;

public class SeleniumSimplifiedTest {


        protected static SeleniumManager sm;
        protected static Selenium selenium;
        protected static String screenShotsFolder;
        protected static String currentBrowser;

        @BeforeClass
```

```
	static public void startServer() throws IOException{
		sm = TheSeleniumManager.getSeleniumManager().
			start("http://www.compendiumdev.co.uk");
		selenium = sm.getSelenium();
		screenShotsFolder = null;
		currentBrowser = sm.getBrowser().replace("*", "");
	}

	@Rule
	public MethodRule watchman= new TestWatchman() {

		@Override
		public void failed(Throwable e, FrameworkMethod method) {
			captureScreenShots(screenShotsFolder,
				currentBrowser,method.getName() + "_" +
				e.getClass().getSimpleName());
		}

		@Override
		public void succeeded(FrameworkMethod method) {
		}
	};

	private void captureScreenShots(String screenShotsFolder,
					String browser, String fileNameAppend) {
		// since captureScreenshot takes the screen, we should maximise the
		// browser before we take the screenshot
		selenium.windowMaximize();
		selenium.windowFocus();

		if(screenShotsFolder==null){
			screenShotsFolder = TheSeleniumManager.getScreenshotsFolder();
		}

		// give the browser a chance to maximise properly
		try { Thread.sleep(1000);     } catch (Exception e) {}

		String filename =     screenShotsFolder + browser + "_" +
					fileNameAppend + "_screenshot.png";

		try{ selenium.captureScreenshot(filename);   }catch(Exception e){}
		try{ selenium.captureEntirePageScreenshot(
				filename.replace(".png","full.png"),"");   }
			catch(Exception e){}
	}

	@AfterClass
	static public void closeItDown(){
			sm.stopSelenium();
	}
}
```

Most of this code should be completely familiar to you since we used it all in a previous test and all I have done is move the code into this base class.

You can see that I moved responsibility for managing the screenshot folder to the selenium manager singleton. By doing this I only have one screenshot folder per SeleniumManager instantiation.

This is the screenshot folder management code from TheSeleniumManager:

```
private static String screenshotsFolder = null;

public static String getScreenshotsFolder() {
	// only create the folder when needed
	if(screenshotsFolder==null){
		screenshotsFolder = createNowfolder();
	}

	return screenshotsFolder;
}

static private String createNowfolder() {
```

```
        // setup the directory structure to create screenshots into
        String screenShotsFolder = System.getProperty("user.dir") +
                                   File.separator +
                                   "screenshots" +
                                   File.separator;
        SimpleDateFormat sdfmth = new SimpleDateFormat("yyyy-MM-dd-HH-mm-ss");
        Calendar cal = Calendar.getInstance();
        screenShotsFolder = screenShotsFolder + sdfmth.format(cal.getTime());
        new File(screenShotsFolder).mkdirs();
        return screenShotsFolder + File.separator;
    }
```

When the path for the screenshots folder is requested by a call to getScreenshotsFolder, if the folder has not been created then it creates the folder (as we saw in previous chapters), and then returns the path to the test.

Because of the @AfterClass method stopping the selenium session after every test class, the tests can run quite slowly. We will look at a method for preventing this happening and taking advantage of the Selenium Manager to reuse the selenium session when we look at Test Suites.

To use this base class, our tests have to extend it. And this means we don't have to include any @BeforeClass or @AfterClass annotated methods in the test as the setup code from SeleniumSimplifiedTest.java will be used.

It is worth pointing out here though, that you can use the @BeforeClass annotation in tests which extend the SeleniumSimplifiedTest.java, as illustrated in SearchPageTests.java

e.g.

```
public class SearchPageTests extends SeleniumSimplifiedTest{

        static SearchPage searchPage;

        @BeforeClass
        static public void automateTestSetup() throws IOException{
                startSeleniumAndSearchForSeleniumRC();
        }

        static void startSeleniumAndSearchForSeleniumRC() {
                searchPage = new SearchPage(selenium);
                searchPage.open();
                searchPage.typeSearchTerm("Selenium-RC");
                searchPage.clickSearchButton();
        }

        @Test
        public void typeInASearchTermAndAssertThatHomePageTextIsPresent(){
                assertTrue(searchPage.isTextPresent(
                        "Selenium Remote-Control Selenium RC comes in two parts." +
                        " A server which automatically"));
                assertTrue(searchPage.isTextPresent(
                        "launches and kills browsers, and acts as" +
                        " a HTTP proxy for web requests from them."));
        }
...
```

Here, we extend SeleniumSimplifiedTest so the @BeforeClass annotated method within SeleniumSimplifiedTest to start the selenium session will be executed, then the @BeforeClass annotated method within SearchPageTests will be executed.

Get Ant Working

The main Ant file that I have created for this cut down project looks much the same as our previous Ant file:

```
<?xml version="1.0"?>

<!DOCTYPE project [
          <!ENTITY properties_build_clean_compile_report SYSTEM
                       "properties_build_clean_compile_report.xml">
]>

<project name="Run Selenium Tests" default="all_selenium_tests">

       &properties_build_clean_compile_report;

       <!-- setup a default browser property, can be overwritten
              with -DBROWSER=iexplore etc.-->
       <property name="BROWSER" value="*firefox" />

       <target name="run_param_junit_tests">
              <junit printsummary="yes" fork="yes" errorproperty="junit.error"
                      failureproperty="junit.error">
                     <sysproperty key="selenium.browser" value="${BROWSER}"/>

                     <classpath refid="junit.class.path" />
                     <formatter type="xml" />

                     <batchtest todir="${testresults}" >
                            <fileset dir="${testDir}" includes="**/*.java" />
                     </batchtest>
              </junit>

              <antcall target="report-junit"/>

              <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
       </target>

    <target name="all_selenium_tests" depends="compile"
                     description="The Main Target for running all tests">
       <antcall target="run_param_junit_tests"/>
    </target>
</project>
```

I removed the calls to start and stop the server, and removed the include for start_stop_selenium.xml as all our tests use the Selenium Manager.

I kept the all_selenium_tests target, simply to make it easy to add other targets in the Ant file, should I need to later. But I could just have easily put all the code inside the "run_param_junit_tests" target, into "all_selenium_tests", instead of calling it with an antcall element.

I have however amended this file to make it 'future proof' so that it will run all the tests that I add into the src/test/java folder structure:

```
<batchtest todir="${testresults}" >
        <fileset dir="${testDir}" includes="**/*.java" />
</batchtest>
```

Instead of having a series of <test ... /> entries, where each test was listed as a separate item:

```
<test todir="${testresults}"
       name="com.eviltester.seleniumtutorials.refactored.HTML_form_tests" />
```

The batchtest defines a group of tests to run where the set of files to use is defined by the fileset element. This fileset element is defined as including all *.java files no matter where in the testDir folder they occur.

The ${testDir} folder is defined in the properties_build_clean_compile_report.xml file:

```
<property name="testDir" location="src/test/java" />
```

You can find more information on fileset and batchtest in the official Ant documentation:

- batchtest - http://ant.apache.org/manual/Tasks/junit.html
- fileset - http://ant.apache.org/manual/dirtasks.html

By adding the tests as a batchtest it means that I can add more tests into the folder structure without having to amend the Ant files which execute the tests.

Make sure all tests are valid tests

A lot of the early tests were designed to illustrate different commands in Selenium. What we really want are tests which do something and then assert that the application has done it. So I reworked the tests to remove any tests which don't add any 'testing' value. e.g. Setting the text in a field without submitting the form or checking the results.

Create end to end tests where we assert after executing functionality

You can see the results of all my edits in the source code, but I took the view that all tests had to be functional end to end:

- We don't just enter text in a form field, and assert it. We have to submit the form we enter the text into and check the result.
- For Ajax, it was acceptable to test within the page itself as that was the functional boundary for the Ajax code.

Have all tests pass on multiple browsers

When I ran the tests under multiple browsers I started to see some tests failing. I took a pragmatic quick view of fixing this, as you can see in the HTML_form_tests.java for submitting a filename in a form.

```
@Test
public void test_submit_form_with_filename(){

	// skip this test if it is not chrome or firefox
	if(!"|chrome|".contains("|" + currentBrowser))
		return;

	htmlForm.typeFilename("C:\\Selenium\\readme.txt");
	HTMLFormResultsPage results = htmlForm.clickSubmitButton();
	assertTrue(results.hasFilename());
	assertEquals("readme.txt",results.getFilename());
}
```

This means that only when the chrome or firefox browser is in use, will the test be run. This means that when running the test under iexplore, no test failures are reported, but it is also misleading because it suggests that we have tested this functionality under iexplore, when we haven't.

This may be a practical solution for a short period of time in a production system, but clearly if left for too long could make people think we are regularly testing things that we are not.

I suppose I could change the name of this test to make it a bit more obvious, e.g.

```
test_submit_form_with_filename_only_valid_for_chrome_or_firefox
```

But this still is not a very good solution to the problem.

JUnit does not provide a simple method to categorise tests, but the best solution would be to only include the test if the browser is valid.

Later in the chapter, after I have explained Suites, I will show how to group tests into suites to run them on different browsers.

Locators as constants or methods

In order to make tests maintainable we want to avoid repeated code, or make it possible to make a change in a single place.

Locators as constants

To help with this, I very often refactor Locators and other string literals into constants in my page object models.

e.g. Using the "refactor \ extract constant..." I can easily convert string literals in my code to constants at the head of my code. (this example from BasicAjaxPage.java)

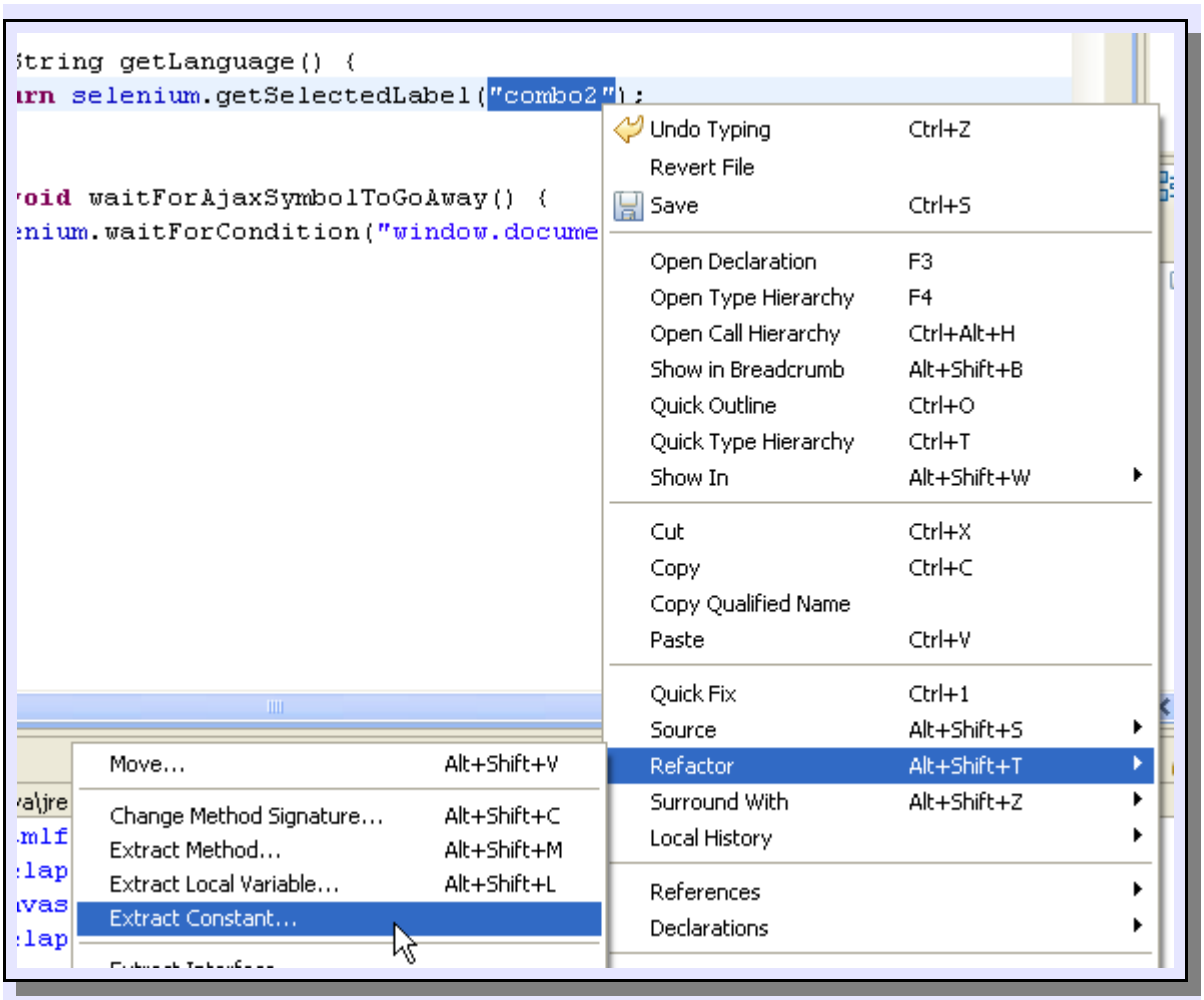


Figure 37.8 : Refactoring a string to a constant

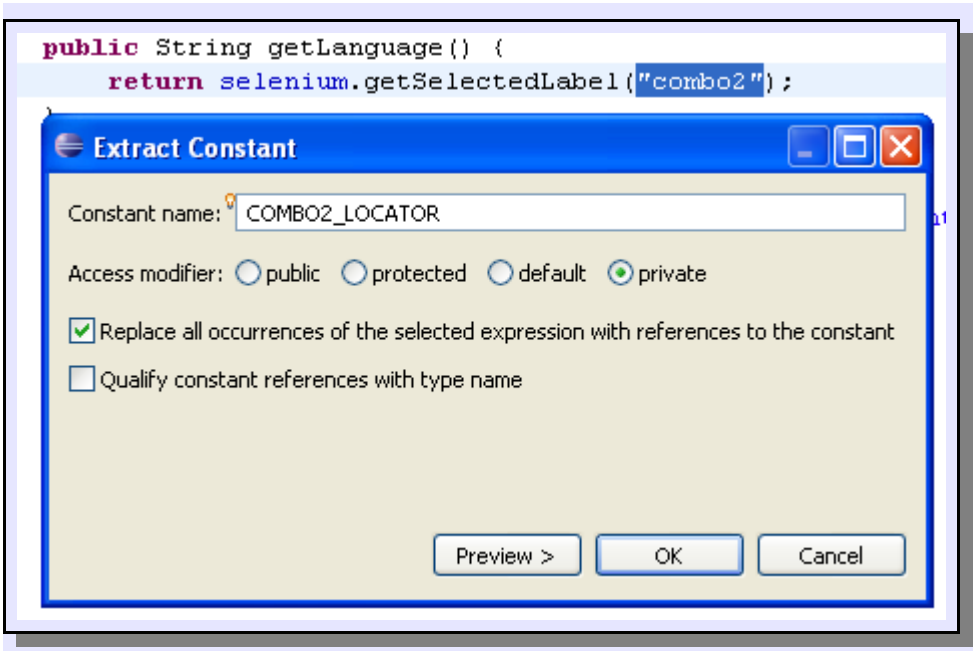


Figure 37.9 : provide a name for the extracted constant

This refactoring has the effect of generating the following code:

```
private static final String COMBO2_LOCATOR = "combo2";

...

public String getLanguage() {
        return selenium.getSelectedLabel(COMBO2_LOCATOR);
}
```

This means that I can easily amend the locator if the application changes, and I can easily switch between using XPath or CSS locators, simply by amending the text in the constant.

I prefer to initially add the constants in the Page Object source code itself, although some people seem to like creating property files.

My preference would be to add it into the code, unless that proved difficult for some reason. If I ever had to have one XPath for IE and one for Firefox then I might choose to move the strings into a property file so I could more easily configure the tests.

But I try to keep my coding approach simple. And the simplest thing to start with was a constant in my code.

Locators as methods

I often find that I write methods to generate locators. I find this useful because web applications often generate HTML code programmatically so ids might be generated automatically with consistent formats. This is the case for the HTMLFormResultsPage.java class.

I initially started refactoring my string literals into the form:

```
private static final String FILENAME_VALUE_ID = "_valuefilename";
private static final String FILENAME_ID = "_filename";
```

But since the results had a common pattern I took a dynamic approach to XPath and ID creation.

In this case I created a method to return the ValueID and one for the normal ID:

```
private String getValueID(String fieldName) {
```

```
        return ID_PREFIX + VALUE_PREFIX + fieldName;
}

private String getID(String fieldName) {
        return ID_PREFIX + fieldName;
}
```

To make this work I have two common constants:

```
private static final String ID_PREFIX = "_";
private static final String VALUE_PREFIX = "value";
```

And then I create a constant for each field name I am interested in:

```
private static final String FN_FILENAME = "filename";
```

I then use these methods and constants in my main methods:

```
public boolean hasFilename() {
        return selenium.isElementPresent(getID(FN_FILENAME));
}
```

This is a more dynamic way of refactoring literals out of code. You can see it in action in HTMLFormResultsPage.java . I do this frequently when the page I'm testing has an algorithm for using Ids, this allows me to refactor the page even further. It does mean that if the form changes I may have more work to do to put in exceptions to the algorithm, but when it works it means I can build the abstraction layer quickly.

Get Hudson Working

Since all my code is in subversion, I will be using Hudson to checkout the most up to date version of the tests and run it. I will setup multiple Hudson builds so that the tests run on multiple browsers by passing in the browser as a parameter to the build. I will also have each build trigger the next build.

The fields I have to fill in the build config are:

Field Name	**Value**
Project Name	xp-dev Final Selenium Simplified IExplore
Description	Runs the most up to date final tests from xp-dev svn on Iexplore
Subversion	Yes
Repository URL	http://svn2.xp-dev.com/svn/seleniumsimplified/trunk/FinalSeleniumTests
Build – Ant Version	(Default)
Build – Properties	BROWSER = *iexplore
Post-build Actions	Publish JUnit test result report: Yes
Test report XMLs	**/junit-results/*.xml

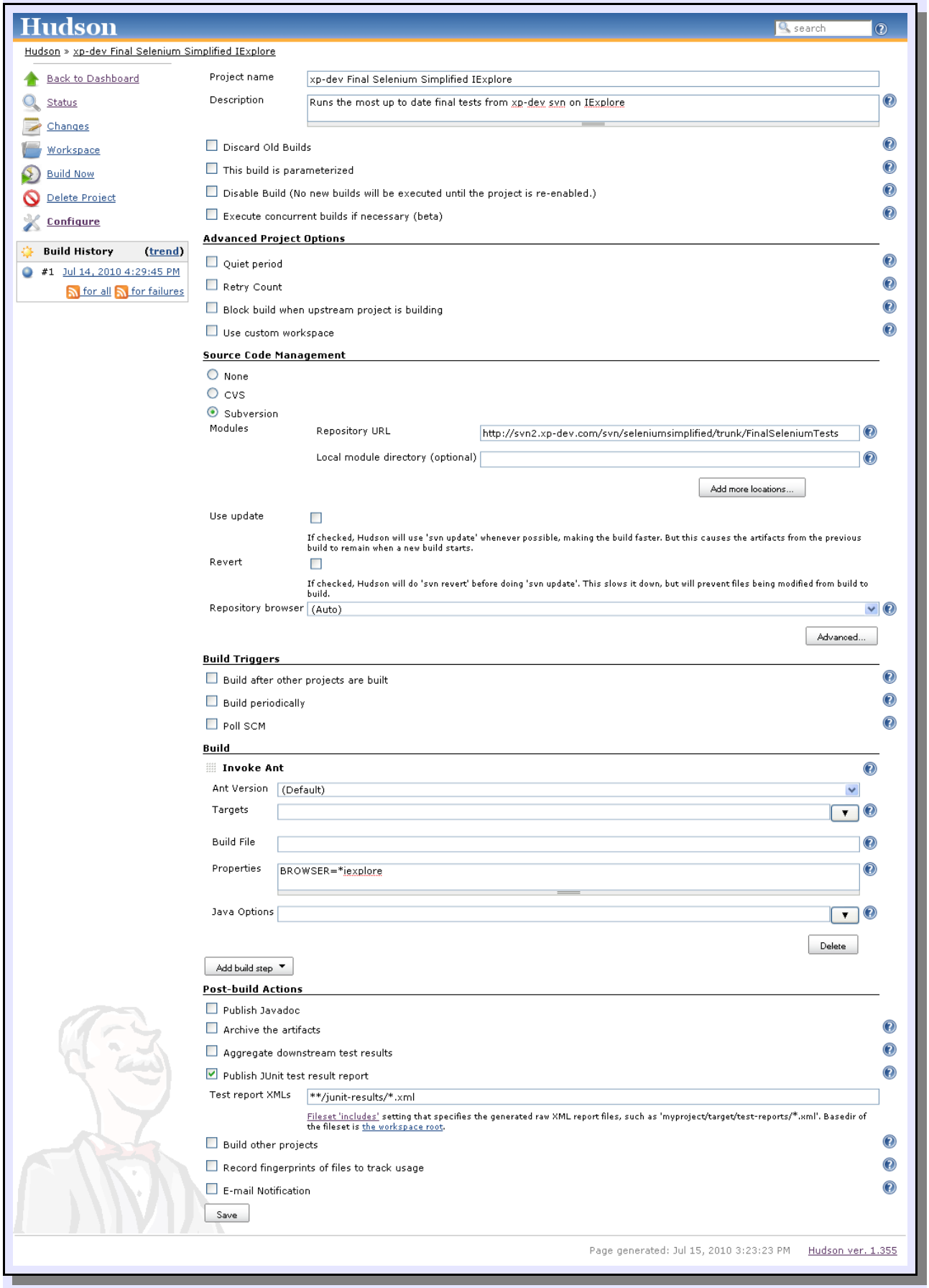


Figure 37.10 : Hudson config to run tests from svn in Internet Explorer

To create another build to run the tests under firefox, I copy the project name to the clipboard.

Then from the main Hudson dashboard I create a "New Job", selecting the "Copy existing job" option, pasting in the project name I want to copy into this field.

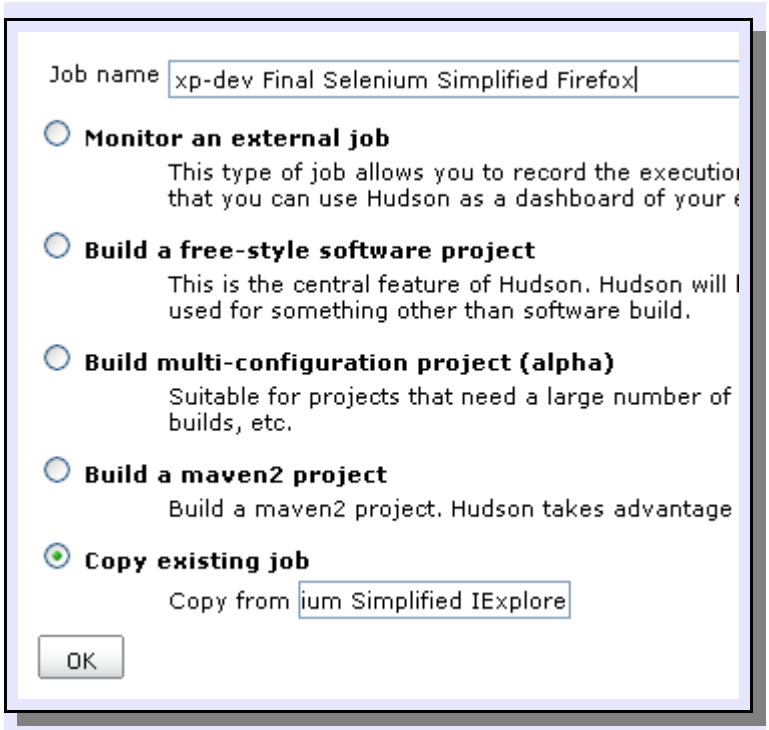


Figure 37.11 : Copy from existing job

After I press [OK] In the configuration edit for this new job. I have to amend the Build Properties but pressing the [Advanced...] button in the Build section.

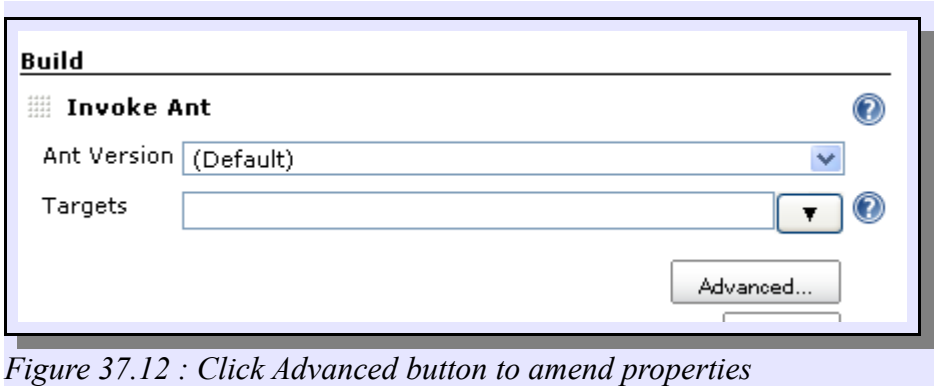


Figure 37.12 : Click Advanced button to amend properties

Amend the properties to read "BROWSER = *firefox"

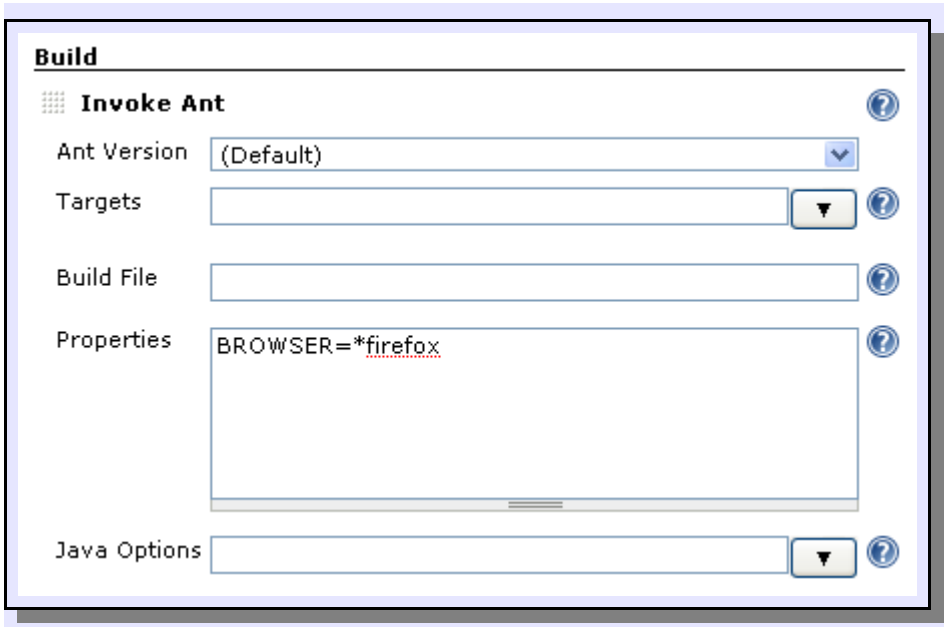


Figure 37.13 : Change browser in the properties

Then I need to make this build dependent upon the previous one. I do this by editing the Build Triggers. Selecting to "Build after other projects are built" and pasting in the name of the previous IExplore project.

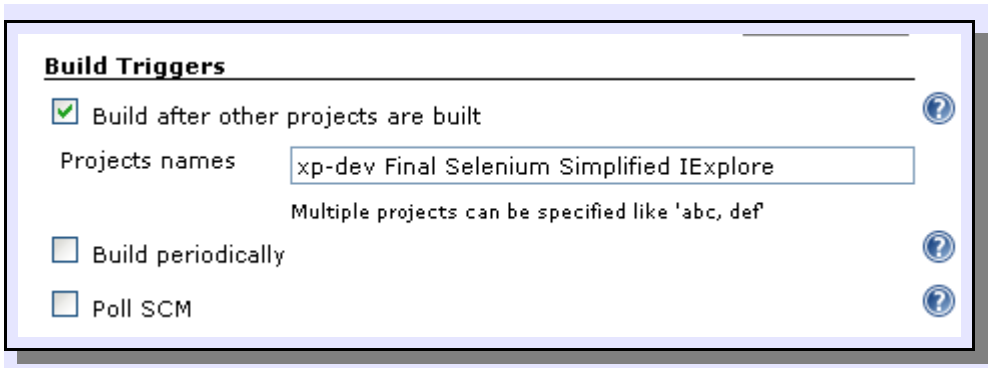


Figure 37.14 : Build after IExplore

Save this project config by clicking on the save button at the bottom.

Then go back to the main dashboard and edit the "xp-dev Final Selenium Simplified Iexplore" project.

If you scroll down to the "Post-build Actions" section, you will see that the "Build other projects" field has been selected.

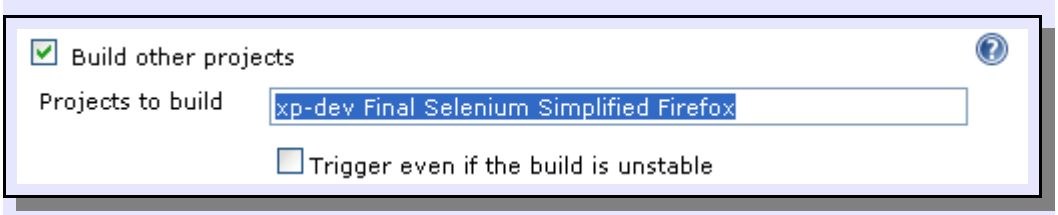


Figure 37.15 : Build other projects by default only runs if previous build is stable

You will also see that the "Trigger even if the build is unstable" checkbox has not been selected by default.

Because we want to check for cross browser, even if tests fail in the previous browser, we should check this box, and save the project.

If you run the IExplore project now, by selecting "Build Now" from the left hand menu. You should find that the tests run under Internet Explorer and under Firefox.

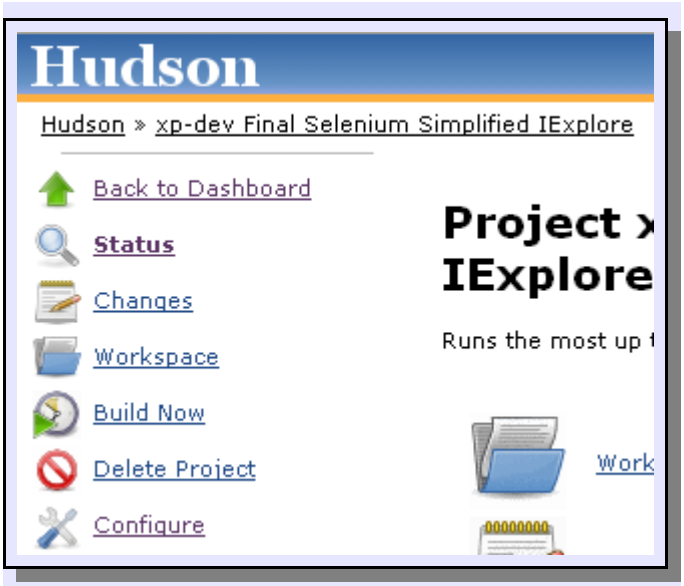


Figure 37.16 : Manually trigger a build by clicking on "Build Now"

Note also that each time the tests run the browser closes between tests.

We will fix this in the next section to make the test execution share the same Selenium session and make the tests run much faster.

Because we are downloading from an external subversion server and the Selenium-server.jar is quite a large file the builds may take some time to start to get all the files checked out.

You can speed up subsequent builds by amending the build configurations to "Use Update" under the "Source Code Management" options.

Speed up the test execution with a suite

With the setup currently explained, when we run the tests, the following happens:

- Test Class One:
  - start server,
  - start selenium session,
  - open browser,
  - run tests,
  - close browser,
  - stop session

- Test Class Two:
    - ... don't start server, but do all the same stuff as Test Class One
- Test Class Three:
    - ... don't start server, do all the same stuff as Test Class Two
    - stop server

This is quite slow because we don't reuse the browser instances or selenium sessions. Ideally what we want is:

- Start server
- Test Class One:
    - start selenium session,
    - open browser
    - run tests
- Test Class Two:
    - if I am using the same settings as Test Class One then reuse the session and browser
    - run tests
- Test Class Three
    - if I am using the same settings as Test Class One then reuse the session and browser
    - run tests
- close browser
- stop session
- stop server

We can achieve this if we introduce the notion of a test suite.

Very often when people use JUnit they refer to a suite of tests and the <batchtest /> construct in Ant as meaning the same thing. But unfortunately they are not. <batchtest /> is a convenient way of defining the tests to run but each test is run as a separate java program so a new SeleniumManager is started for each test, so in Ant we don't even get reuse of the SeleniumManager singleton.

If we run the tests from within Eclipse, by right clicking on a test folder and choosing to run as junit test, then a 'true' suite is created and a single SelenumManager is shared for all tests.

So at this point we will introduce JUnit suites.

You can find the suite below in the source-code in src/test/java in the com.eviltester.seleniumSimplified.testSuites package called TestSuite.java.

```
package com.eviltester.seleniumSimplified;

import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;

import com.eviltester.seleniumSimplified.calculate.CalculateTwoNumbersMultipleTests;
import com.eviltester.seleniumSimplified.calculate.CalculateTwoNumbersTestsTabFile;
```

```
import com.eviltester.seleniumSimplified.cookies.CookieTests;
import com.eviltester.seleniumSimplified.htmlform.HTML_form_tests;
import com.eviltester.seleniumSimplified.javascript.AjaxPageTests;
import com.eviltester.seleniumSimplified.search.SearchPageTests;
import com.eviltester.seleniumutils.seleniumManager.TheSeleniumManager;

@RunWith(Suite.class)
@Suite.SuiteClasses({
        CalculateTwoNumbersMultipleTests.class,
        CalculateTwoNumbersTestsTabFile.class,
        CookieTests.class,
        HTML_form_tests.class,
        SearchPageTests.class,
        AjaxPageTests.class
})

public class TestSuite {

}
```

A JUnit suite, using the JUnit 4 annotations does not need to have any code in the body of the testSuite class, and acts as a way of grouping the tests together to run as a single block.

If I right click on this class in Eclipse and run it, then it will run all the tests in all the classes imported via the @Suite annotation.

Since this suite of tests is a class in the src/test/java folder it would be run by our existing Ant scripts so I need to amend our Ant script to exclude this Suite from our list of files to execute as a batch:

```
<batchtest todir="${testresults}" >
        <fileset dir="${testDir}" includes="**/*.java"
excludes="**/testSuites/*.java"/>
</batchtest>
```

Here I have added an exclusion for all java files in the testSuites folder, so if I adopt the convention of having all my suite related files in this folder then they will not run.

Then I can create multiple suite files, but not have to worry about the tests running multiple times by the refactored-param-build.xml build file.

I will create a separate build file to run the suite.

You can find this as suite-build.xml in the code distribution for this chapter. This build file has a few minor differences to the <junit ... /> section in the file:

```
<junit printsummary="yes" fork="yes" errorproperty="junit.error"
                failureproperty="junit.error">
        <sysproperty key="selenium.browser" value="${BROWSER}"/>
        <sysproperty key="selenium.stopAfterSuite" value="TRUE"/>

                <classpath refid="junit.class.path" />
                <formatter type="xml" />

                <test todir="${testresults}"
                        name="com.eviltester.seleniumSimplified.TestSuite" />
</junit>
```

You can see that I have added a new property “selenium.stopAfterSuite” and I have changed the list of tests to run to run the individual test Suite.

So now we have to amend the rest of the code to take advantage of this system property.

I first of all amend the test suite class to have code.

```
public class testSuite {
```

```
	@BeforeClass
	public static void run_before_suite(){
		// If I want to run the suite from within Eclipse (and not use ant)
		// and I want to have the tests run quickly I can set the
		// system property to control the Selenium Manager interaction here
		//System.setProperty("selenium.stopAfterSuite", "TRUE");
	}

	@AfterClass
	public static void run_after_suite(){
		TheSeleniumManager.getSeleniumManager().stopSelenium();
	}
}
```

Here I have create an @AfterClass annotated method called run_after_suite. We have already seen that we can 'nest' @BeforeClass and @AfterClass annotations, so this annotation runs at the end of the testSuite class which means that it runs at the end of the suite. So if we stopSelenium here then we don't have to stop it at the end of each test.

I have to amend the @AfterClass in SeleniumSimplifiedTest to only stop the test if the property "selenium.stopAfterSuite" is not set to "TRUE"

```
@AfterClass
static public void closeItDown(){

	// default to closing after test
	boolean closeAfterTest = true;

	// unless we set the system property selenium.stopAfterSuite=TRUE
	if(System.getProperties().containsKey("selenium.stopAfterSuite")){
		if(System.getProperty("selenium.stopAfterSuite").
				equalsIgnoreCase("TRUE")){
			closeAfterTest = false;
		}
	}

	if(closeAfterTest)
		sm.stopSelenium();
}
```

Here you can see that I have amended the closeItDown method in SeleniumSimplifiedTest to check for the presence of the property, if it is present and set to "TRUE" then we do not stopSelenium at the end of each test.

We can take advantage of this within Eclipse, by amending the Suite class so that the run_before_suite method sets the property.

```
	@BeforeClass
	public static void run_before_suite(){
		// If I want to run the suite from within Eclipse (and not use ant)
		// and I want to have the tests run quickly I can set the
		// system property to control the Selenium Manager interaction here
		System.setProperty("selenium.stopAfterSuite", "TRUE");
	}
```

If I enable this line and right click and run the testSuite.java within Eclipse then all the tests will run and they will share the same selenium session and browser.

I can take advantage of this in my Hudson build by amending the "Build File" option in the Advanced section of "Invoke Ant" under the "Build" section and having it use the "suite-build.xml" file instead of the default one.

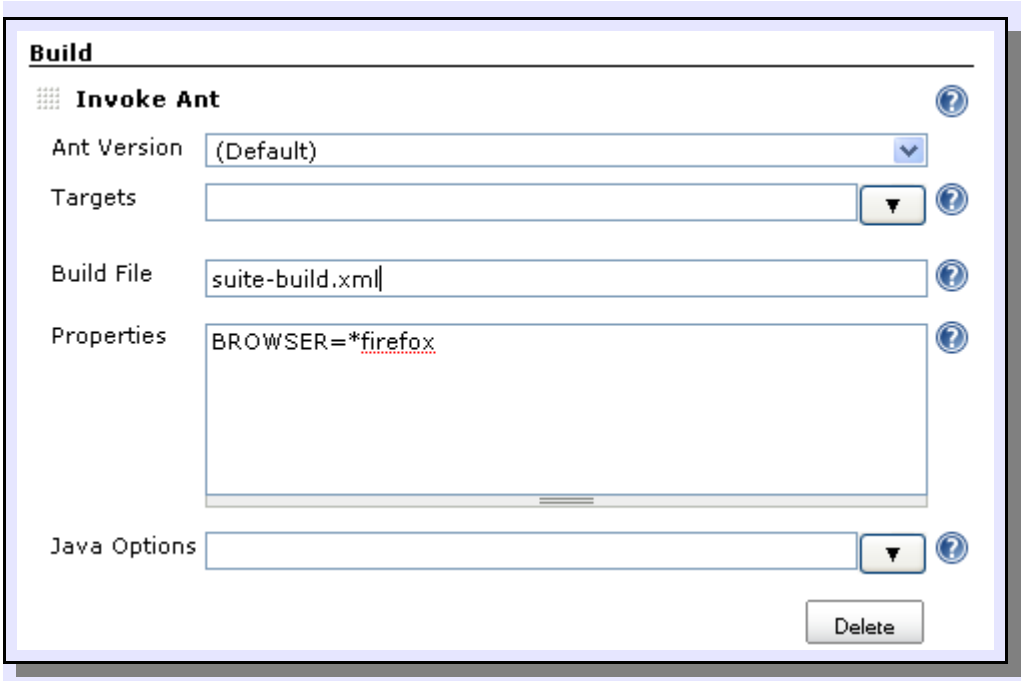


Figure 37.17 : Use suite-build.xml instead of default build file

I amend all the Hudson projects in this way so that they all use the suite instead of the individual files.

The test run is now taking full advantage of all the features we have built into the SeleniumManager:

- the Selenium Manager is being shared across all tests,
- all screenshot errors will be saved into the same folder
- the selenium server is shared for all tests
- the selenium session is shared for all tests
- the same browser is shared for all tests

You can also see that because we abstracted away all of the details of managing Selenium into the SeleniumSimplifiedTest.java base class, and the SeleniumManager set of classes. None of our tests had to change when we changed the architecture behind the test execution.

Running Different Suites for Different Browsers

If JUnit supported groups of tests then we could simply create a group of tests for all browsers, sub groups for specific browsers and then in a suite say, run all the tests for these groups.

JUnit does not support a group notation.

But we can use the Suite notation to build groups by nesting suites.

Nested Suites

We saw that a basic Suite in JUnit does not have any code in the class:

```
@RunWith(Suite.class)
```

```
@Suite.SuiteClasses({
	CalculateTwoNumbersMultipleTests.class,
	CalculateTwoNumbersTestsTabFile.class,
	CookieTests.class,
	HTML_form_tests.class,
	SearchPageTests.class,
	AjaxPageTests.class
})

public class TestSuite {
}
```

I am going to start viewing suites with no code as a group. And Suites with an @BeforeClass and @AfterClass as Suites.

With this view I can create a suite around the above 'group' as follows:

```
@RunWith(Suite.class)
@Suite.SuiteClasses({
	TestSuite.class,
})

public class AWrapperTestSuite {

	@AfterClass
	public static void run_after_suite(){
		TheSeleniumManager.getSeleniumManager().stopSelenium();
	}
}
```

Here I have nested one suite within another. It is perfectly possible for me to have @BeforeClass and @AfterClass within the 'group' suites as well and have sub level setup and tear down processes. But in our current model I only want one level to issue the .stopSelenium() command so I am creating these wrapper Suites.

So, how do we use this to run tests for specific browsers only on those browsers?

Browser Specific Groups

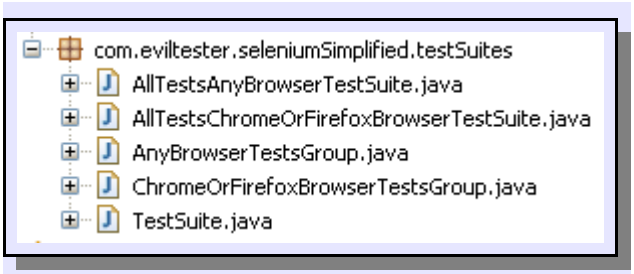


Figure 37.18 : Browser Specific Groups

I create suites with no method stopping Selenium as groups in the testSuite package e.g.

- AnyBrowserTestGroup.java
- ChromeOrFirefoxBrowserTestsGroup.java

Both of these are suites with empty class code e.g.

AnyBrowserTestGroup.java:

```
@RunWith(Suite.class)
@Suite.SuiteClasses({
	CalculateTwoNumbersMultipleTests.class,
	CalculateTwoNumbersTestsTabFile.class,
	CookieTests.class,
	HTML_form_tests.class,
	SearchPageTests.class,
```

```
        AjaxPageTests.class
})

public class AnyBrowserTestsGroup {
}
```

ChromeOrFirefoxBrowserTestsGroup.java:

```
@RunWith(Suite.class)
@Suite.SuiteClasses({
        Chrome_only_form_tests.class
})

public class ChromeOrFirefoxBrowserTestsGroup {
}
```

To support this I did a refactoring to move the test_submit_form_with_filename test into its own class called Chrome_only_form_tests.

Then I created suites which nest the above group suites. e.g.

```
package com.eviltester.seleniumSimplified.testSuites;

import org.junit.AfterClass;
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import com.eviltester.seleniumutils.seleniumManager.TheSeleniumManager;

@RunWith(Suite.class)
@Suite.SuiteClasses({
        AnyBrowserTestsGroup.class,
        ChromeOrFirefoxBrowserTestsGroup.class
})
public class AllTestsChromeOrFirefoxBrowserTestSuite {

        @AfterClass
        public static void run_after_suite(){
                TheSeleniumManager.getSeleniumManager().stopSelenium();
        }
}
```

This "AllTestsChromeOrFirefoxBrowserTestSuite" nests both Group suites as it will run all the tests.

To create a suite that excludes the Chrome test I just remove "ChromeOrFirefoxBrowserTestGroup.class" from the list of SuiteClasses.

Create a conditional Ant File

One last thing left to do; support this in Ant.

I create a conditional-suite-build.xml as a separate file. I am creating a separate file because you can then compare it with suite-build.xml

The changes are illustrated by the listing below:

```
<target name="run_param_junit_tests">

   <condition property="selenium.suitename"

value="com.eviltester.seleniumSimplified.testSuites.AllTestsChromeOrFirefoxBrowserTestS
uite"
        else="com.eviltester.seleniumSimplified.testSuites.AllTestsAnyBrowserTestSuite">

        <or>
                <equals arg1="${BROWSER}" arg2="*chrome"/>
```

```
            <contains string="${BROWSER}" substring="firefox"/>
        </or>
    </condition>

        <junit printsummary="yes" fork="yes" errorproperty="junit.error"
                    failureproperty="junit.error">

            <sysproperty key="selenium.browser" value="${BROWSER}"/>
            <sysproperty key="selenium.stopAfterSuite" value="TRUE"/>

            <classpath refid="junit.class.path" />
            <formatter type="xml" />

            <test todir="${testresults}" name="${selenium.suitename}" />

        </junit>

        <antcall target="report-junit"/>
        <fail if="junit.error" message="Selenium test(s) failed.  See reports!"/>
</target>
```

I have added a <condition ... /> block before the <junit .../> block.

This sets property “selenium.suitename” to the value “com.eviltester.seleniumSimplified.testSuites.AllTestsChromeOrFirefoxBrowserTestSuite” (which is the package path of the suite to run only if the browser is Chrome or Firefox), only if the conditions within the <or .../> block are true.

If the <or .../> block is not true (e.g. if *iexplore) then it sets the property to the path of the AllTestsAnyBrowserTestSuite.

In the <or .../> block I check if the BROWSER property equals *chrome, or if it contains the string “firefox”. If either of these are true, then it is valid to use the Chrome or Firefox test suite.

The other change was to use the property in the <test .../> element:

```
<test todir="${testresults}" name="${selenium.suitename}" />
```

By using this type of structure in the Ant file I can have a single Ant file which conditionally chooses which suite to run, based on the browser I am using.

To call this from the command line I would use:

```
ant -DBROWSER=*chrome -f conditional-suite-build.xml
```

And simply replace the browser with the name of the browser I want.

You already know how to amend the Hudson build to use this, just amend the advanced build properties to use a specific build file and pass in the Browser property.

In this way, we don't have to have misleading tests listed in our test run, and we only run the tests which are valid for that browser.

Related Reading:

- The Ant Condition task - http://ant.apache.org/manual/Tasks/condition.html
- The Conditions available - http://ant.apache.org/manual/Tasks/conditions.html

Get Selenium 2.0 working

In the previous chapter I described at a high level the changes required to the Selenium Manager to start moving over to Selenium 2.0

In the updated version for this chapter, you can switch over to a SeleniumBacked driver by amending TheSeleniumManager.java to useSeleniumTwo instead of useSeleniumOne in the getSeleniumManager method.

You will undoubtedly find some tests failing when you switch to Selenium 2.0 But by having this switch in your abstraction layer you can continue to monitor the progress of the backward compatibility of Selenium 2.0 before making a decision to move over to Selenium 2.0 and WebDriver.

Ways of using what we have built

In this final project we have a number of ways of working with our tests.

From Eclipse I can:

- amend selenium.properties to control the browser used to run the tests
- run an entire suite of tests quickly by amending the testSuite class to set the system property which causes session reuse
- run individual tests, which use the selenium.properties browser, but also close the browser and session when they are finished

From the command line:

- run the tests using Ant and the default browser in the Ant file by typing “ant”
- by using “ant -DBROWSER=*googlechrome” I can run the tests with a browser of my choosing
- by using “ant -f suite-build.xml” I can run the suite of tests quickly with the browser defined in the Ant file
- by using “ant -f suite-build.xml -DBROWSER=*googlechrome” I can run the suite of tests quickly with the browser defined in the command line

From Hudson:

- trigger builds in sequence to run the suite of tests against multiple browsers and track all the test results over time in Hudson itself.

This supports different ways of working.

I use the Eclipse method when I am writing and debugging tests. I initially test for cross browser compatibility when writing the tests by amending the properties file and rerunning the tests by right clicking from Eclipse.

Before doing a commit to subversion of any changes I made to the tests, I run the tests from the command line with Ant directly. I may choose to run the test for cross browser compatibility.

After checking the changes made to subversion, an automated build in Hudson will kick in to checkout the tests and run them on the test server to make sure that the tests run on a machine other than my own.

When you can do this, you have a production ready test setup.

Exercises For the reader:

- Refactor the BasicHTMLForm.java page object class so that instead of repeated code like
  String.format(CHECKBOXES_LOCATOR,checkBoxIDNumber)
  the code instead calls a method like getCheckBoxLocator(checkBoxIDNumber)
- If you look at the code in HTMLFormResultsPage.java where methods are used to generate locator values rather than constants. There is still a lot of code following the same pattern, e.g.

```
public String getDropDownValue() {
 if(hasDropDown()){
 return selenium.getText(getValueID(FN_DROPDOWN));
 }else{
 return selenium.getText(getNoValueLocator(FN_DROPDOWN));
 }
}
```

  Can you refactor the code so that this code pattern only appears once in the code?

  Hint. One way to do this would involve the above code looking like:

```
public String getDropDownValue() {
 return getFieldNameValue(hasDropDown(),FN_DROPDOWN);
}
```

  Does this make the code easier to read and maintain?

- Create a new Hudson build so that the tests run in Google's Chrome browser after the Firefox run.
- Create a new Hudson build so that the tests run in Firefox Chrome mode browser after the Google Chrome run.
- Change all your Hudson builds to use the conditional-suite-build.xml
  - Can you see the number of tests changing between browers?
  - 1 more test should be run for *chrome, and *firefox
- Change the code so that you can switch between Selenium 1.x and Selenium 2.x by changing a system property in the selenium.properties file and in the Ant files.

Chapter 38: In Closing

If you have worked your way through the book to this section then you should now have a good grasp on the basics creating production ready tests in Java using Selenium-RC with the Selenium 1.0 interface.

I know that readers have approached this book with very different levels of experience with programming, automated testing in general, and Selenium.

While I have tried to explain everything as well as I can, I still value feedback from readers to help me improve future versions of this text.

There is still a lot to learn, but you should have enough knowledge that you don't get phased by the prospect of learning more on your own.

This chapter will suggest some avenues for additional learning, but the key way of learning Selenium is to use it.

So:

- write lots of tests,
- refactor your tests,
- keep learning java,
- keep an eye on new updates to selenium,
- read the Selenium source code

Additional Reading

For Selenium:

- read the official documentation on the Selenium website http://seleniumhq.org/docs/
- subscribe to the email feed for the official Selenium Users group http://groups.google.com/group/selenium-users
- subscribe to the RSS feed at the Selenium Stack exchange http://area51.stackexchange.com/proposals/4693/selenium

For Java:

- Books:
  - Agile Java: Crafting Code with Test-Driven Development by Jeff Langr
    - An excellent learning guide to Java which teaches Java using Test Driven development.
  - Implementation Patterns by Kent Beck
    - An overview of effective Java coding which is a tremendous confidence boost to beginners to Java that keeping your code simple is a perfectly valid coding style used by very experienced developers.
  - Effective Java by Joshua Bloch

        - An overview of effective Java practices which speeds up your learning tremendously.
    - Growing Object-oriented Software – guided by tests by Steve Freeman and Nat Pryce
        - An excellent overview of abstraction, refactoring and automated unit testing.

Videos:

- Watch the Eclipse and Java tutorials http://eclipsetutorial.sourceforge.net/
- Watch Google Tech Talks on Java and Selenium
- Watch the videos from previous Google Test Automation Conference

Web Searches:

- I use Google all the time to search for answers to my queries typically of the form "<thing I don't know> java example"

Subscribe to the following blogs:

- http://element34.ca
    - Adam Goucher's Selenium blog
- http://seleniumhq.wordpress.com/
    - The Official Selenium Blog
- http://eviltester.com
    - The author's testing blog which contains Selenium posts
- http://www.theautomatedtester.co.uk/
    - David Burns' testing blog. Author of "Selenium 1.0 Testing Tools: Beginner's Guide"
- http://seleniumexamples.com/blog/
    - Examples and tips for selenium
- http://testingreflections.com/
    - A Testing blogs aggregator where you will find many posts on testing and Selenium.

Use the mindmap:

- Use the mindmap in "Appendix A - Selenium API MindMap" as a summary of the commands and various categories the commands fit into.

Download and study the source-code for this book:

- Follow the instructions in Chapter "Play along at home" and download the source-code:
    - http://www.compendiumdev.co.uk/selenium/InitialSeleniumTests_JUNIT.zip
- Install a subversion client and checkout the refactored source-code from:
    - http://svn2.xp-dev.com/svn/seleniumsimplified/trunk/FinalSeleniumTests

Appendix A - Selenium API MindMap

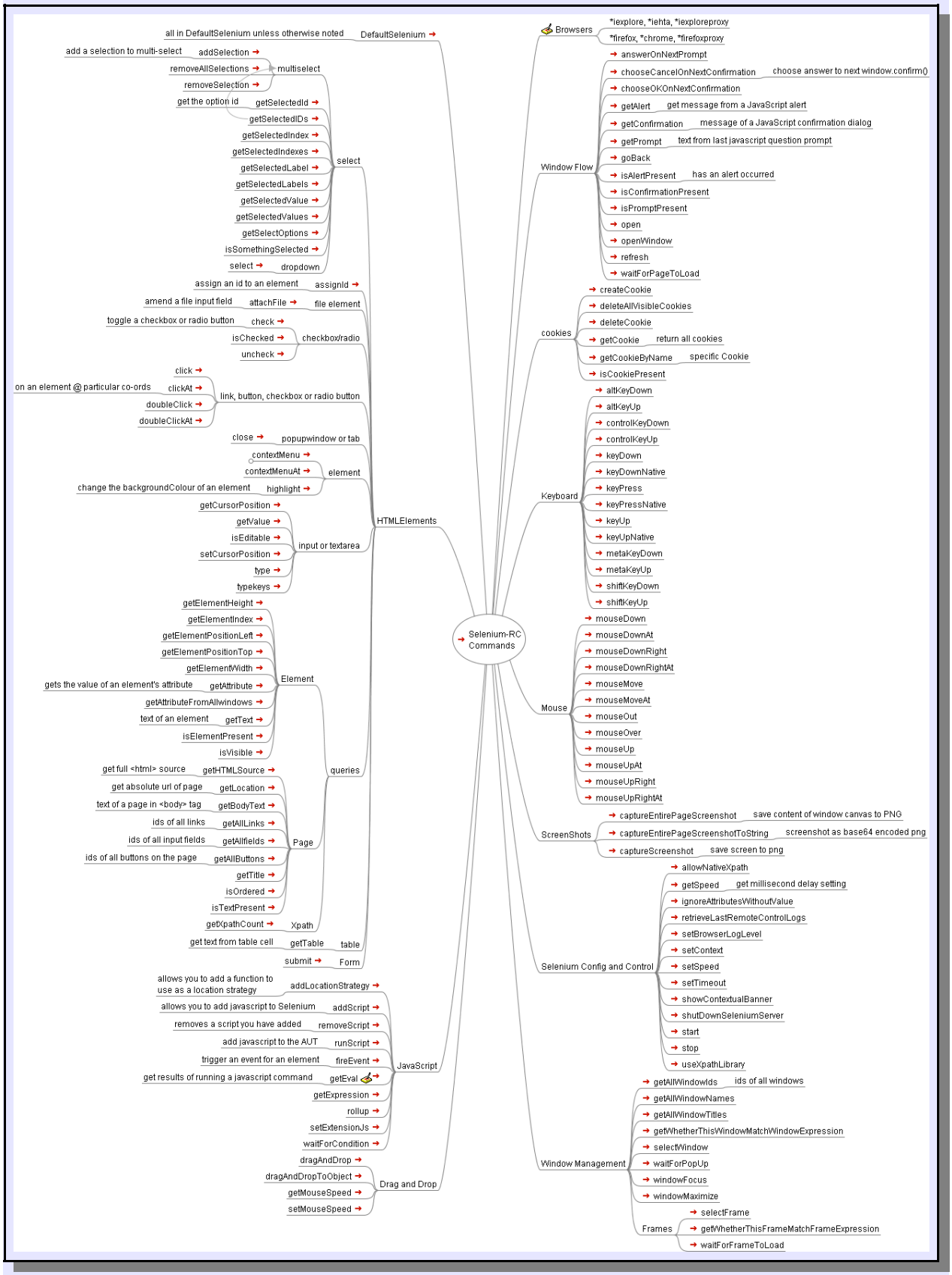


Figure 1 : Categorised Listing of the Selenium Commands

Appendix B - Playing along at home

You can download all the source code for the examples in the book:

- http://www.compendiumdev.co.uk/selenium/InitialSeleniumTests_JUNIT.zip

Unarchive this zip file to your Eclipse workspace.

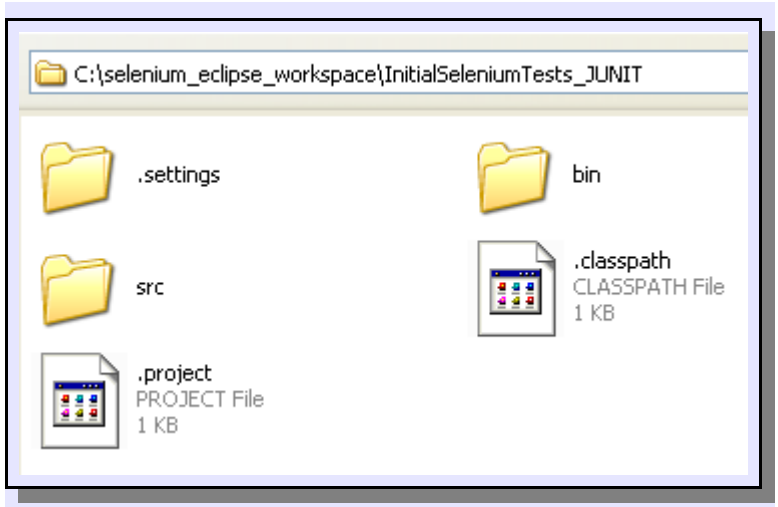


Figure 1 : Unarchived Eclipse Project

Import this project into Eclipse, by selecting "Import" from the File Menu.

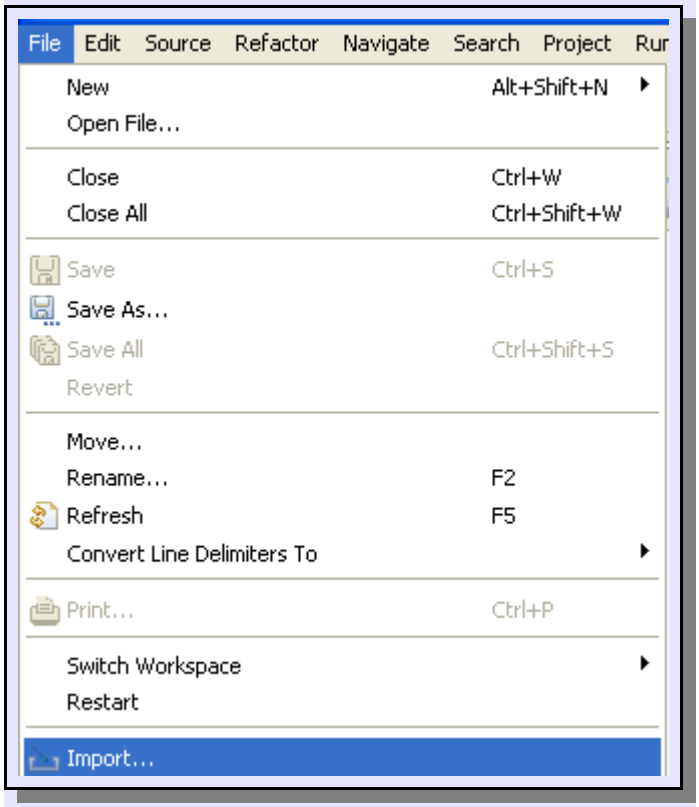


Figure 2 : Select File\Import

Choose “Existing Projects into Workspace” from the “General” section. And Press the [Next >] button on the Import dialog.

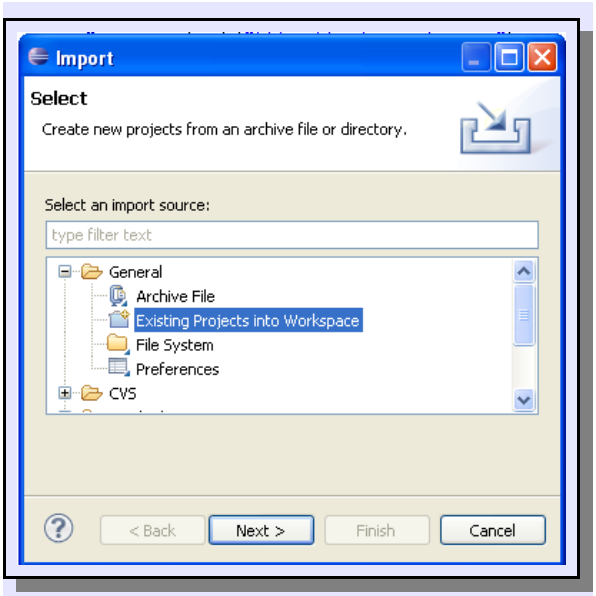


Figure 3 : Import Existing Projects Into Workspace Dialog

Browse to the root directory that you just created when you unzipped the archive. In my case this was “C:\selenium_eclipse_workspace\InitialSeleniumTests_JUNIT”. Make sure that the InitialSeleniumTests_JUNIT project is selected in the “Projects” list.

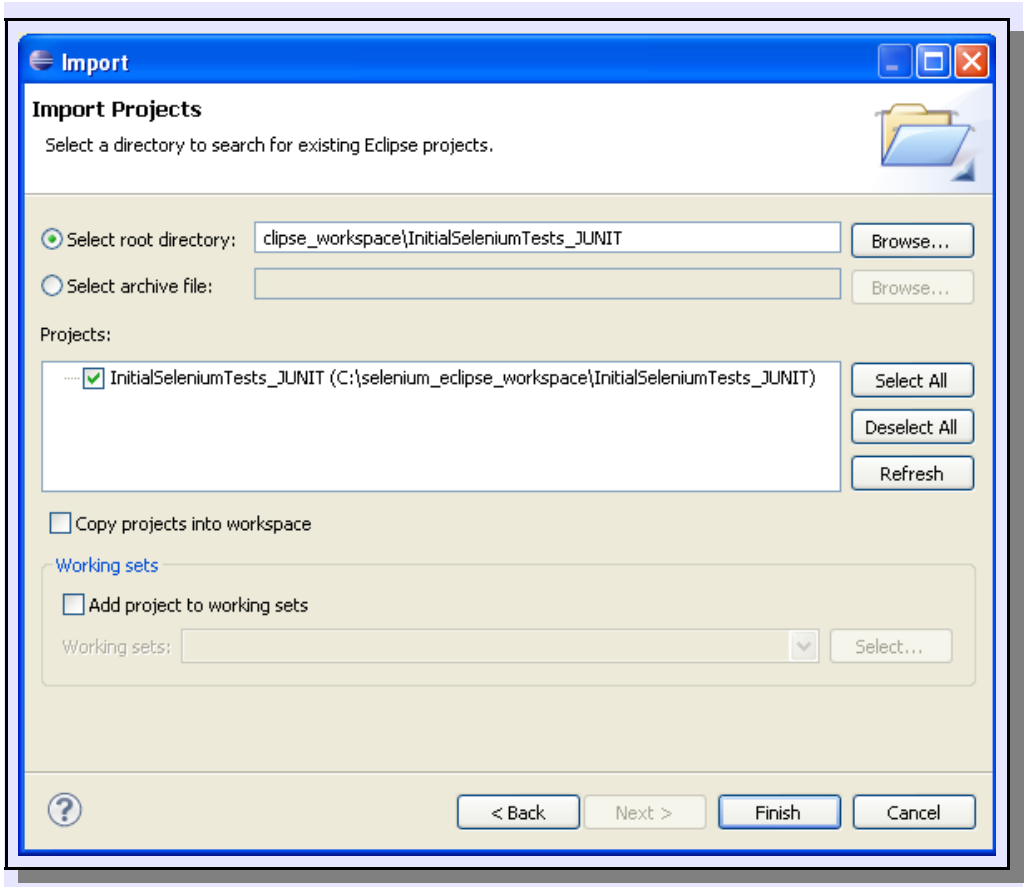


Figure 4 : Select Root Directory of project to import

After clicking finish you should see the imported project in the Package Explorer.

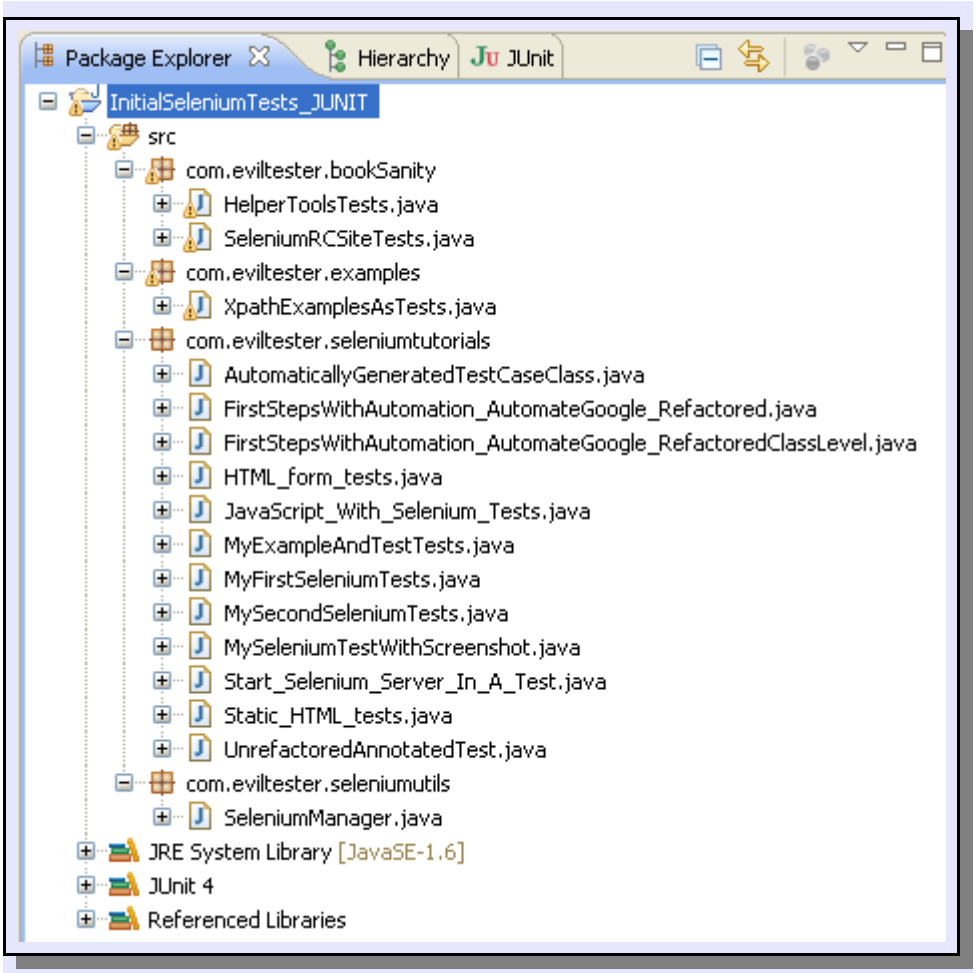


Figure 5 : Imported Project In Package Explorer

The only thing you may have to do is amend the project properties to point to the locations of the Selenium libraries, and you will only have to do this if you have installed the Selenium Jars in a different location from those used in the tutorial.

You can amend the properties by right clicking on the project, and selecting "Properties" from the context menu, then Amending the location of the libraries in the Java Build Path.

This was initially described in "Chapter 6: Create a JUnit test Using the JUnit export from Selenium-IDE" in the Section "Resolve Import Errors"

This project contains all the code listed in the book and additional code that I use to test the integrity of the links listed in the text which mention external tools.

Index

Alphabetical Index

A
Abstraction....66, 238, 254, 271
Acceptance Criteria....111
Annotations....
  Annotations....78
  @After....128, 131
  @AfterClass....128, 134
  @Before....128
  @BeforeClass....128, 134, 237
  @Ignore....106
  @Parameterized....279
  @Parameters....275, 279
  @Test....78
Ant....202, 318
Ant....
  Build.xml....207
  Hudson....219
  Installing....203
  Running....211
AssertEquals....171
AssertFalse....183
Assertion....73, 119
Assertions....112
Asserts....70p., 75
AssertTrue....112, 115, 183
B
Base Class....316
Base URL....16
Browser codes....286
C
Capture PlayBack 14
Command....17
Commands....
  AddScript....195
  AddSelection....168
  AssignId....187
  AttachFile....165
  CaptureEntirePageScreenshot....74, 283, 317
  CaptureScreenshot....74, 283, 317
  Check....165p., 171
  Click....63, 107, 162, 166, 185
  CreateCookie....298, 303
  DeleteAllVisibleCookies....298p.
  DeleteCookie....298, 302
  FireEvent....193

GetAlert....193
GetAttribute....187
GetBodyText....188
GetCookie....298, 300
GetCookieByName....298, 300p.
GetEval....170, 194
GetHtmlSource....115, 188
GetSelectedId....172p.
GetSelectedIds....172p.
GetSelectedIndex....172p.
GetSelectedIndexes....172p.
GetSelectedLabel....172, 174
GetSelectedLabels....172, 174
GetSelectedValue....172, 174
GetSelectedValues....172, 174
GetSelectOptions....172, 175
GetText....180
GetTitle....120, 179
GetValue....120, 171pp.
GetXPathCount....117, 183
GoBack....189
Highlight....189
IsAlertPresent....193
IsChecked....171
IsCookiePresent....298p.
IsElementPresent....182
IsSomethingSelected....172p.
IsTextPresent....111, 182
Open....63, 102, 162
RemoveAllSelections....168
RemoveSelection....168
RunScript....194
Select....170
Start....83
Stop....83
Submit....162
Type....63, 103, 162
TypeKeys....163
Uncheck....165, 167, 172
URLEncode....304
WaitForCondition....293
WaitForPageToLoad....63, 108, 162, 185
Console....22, 47
Cookies....294
CSS Selector....150
D
Data Driven Tests....274
Debug As....106
DefaultSelenium....197, 201
Domain Level Abstractions....272

Downloadable Source-code....9, 340
E
Chapter 5:Eclipse....12, 27
Attach Source....56
Automatic refactoring....124
Automatically Fix Errors....81
Breakpoint....48, 62
Code completion....82, 102
Configure the build path....41
Create a New Class....35
Debug mode....49
Debug perspective....89
Debugging....113
Fix error automatically....40
Line numbers....73
Package explorer....64
Selenium IDE....52
Environment Variables....204
F
File Field....164
FireBug....92, 103
Firecookie....297
Firefinder....150
Firefox....13
Firewall....45
H
Hidden Field Values....170
HTML....142, 158
HTML Suites....24
Hudson....202, 219, 323
I
Inspect Element....103
J
Java....11, 19, 31, 59
Build path....41
Classes....61
Constructor....66, 75
Escape characters....163
Exception....61
Exception Handling....198
Import....40, 61
Inheritance....61
Install Java jdk....203
Integer.valueOf....176
Object....62
Package declaration....39, 60
Package structures....308
Private....66
Public void....62
Random numbers....176

Static....134p.
String.format....152
System.out.println....174
.jar....63
JavaScript....170, 187, 191, 291
JUnit....33, 42, 45, 47, 62, 65, 136, 274, 316
JUnit custom rules....282
JUnit suites....328
L
Locator....106
Locators....321
M
Multiple browsers....320
MyFirstSeleniumTests.java....59
P
Page object....243
Page Object Models....230, 271
Page Objects....239, 264
Password Fields....163
Powertoy....
Open Command Window Here....22
Property Files....287
R
Refactoring....123, 133, 231, 239, 266, 271
Regular expressions....182
RemoteControlConfiguration....200
S
Scheduling a build....227
Screen Capture....281
Screenshots....74
Selenium 2.0....305, 334
Selenium IDE....11, 13, 17, 24, 38, 160
Selenium Remote Control....
Browser codes....61, 286
Cookie Handling....294
CSS locators....151
Custom configurations....200
DefaultSelenium....80, 84
Installing....20
JavaScript....191
Locators....321
Multiple browsers....286, 320
Running....22
Screen Capture....281
SeleneseTestCase....56, 61, 63, 65, 77
Selenium-RC....12
Start Selenium Programmatically....196
Stop the Server....23p.
Troubleshooting....88
Upgrading....139

XPath....149
SeleniumManager....232
SeleniumServer....197, 201
Singleton....315
Stack trace....72
Subversion....202, 228
Syntactic sugar....231
System Properties....289
T
Tab Delimited Files....278
Target....17
TestWatchman....282, 316
TextArea Fields....163
V
Value....17
Verify....68, 71
Version Control....311
W
WebDriver....305
X
XPath....96, 105, 117, 119, 144, 183
XPath Functions....147
XPath optimisation....148
XPather....92, 95, 184

About the Author

Alan Richardson has been involved with testing his entire professional career. Over his career he has done a lot of automation, using a lot of different tools. He has also done a lot of programming in a lot of different languages.

At the time of writing, he has used Selenium for over 3 years.

He writes tests in Java because he generally works on applications that are written in Java. This allows developers to help with the automation. But Java is also his current application programming language of choice.

In addition to test automation, Alan has performed test management, test consultancy and hands on testing roles.

He remains passionate about testing and committed to improving his hands on skills in: exploratory testing, technical testing and test automation.

He has presented at conferences world wide and maintains a blog at http://www.eviltester.com